

AN ELECTRONIC PURSE (in Z/Eves)
Specification, Refinement, and Proof
(version 0.3)

by

Leo Freitas
Jim Woodcock

University of York
Department of Computer Science
High Integrity Systems Engineering (HISE) Group

AN ELECTRONIC PURSE (in Z/Eves) Specification, Refinement, and Proof (version 0.3)

by

Leo Freitas
Jim Woodcock

Carbon copy of formal text in
Oxford University Computing Laboratory
Technical Monograph PRG-126
ISBN 0-902928-41-4

May–September 2006

University of York
Department of Computer Science (HISE Group)
Heslington Road
Heslington YO10 5DD
England

Copyright © 2006 Grand Challenge GC6

University of York
Department of Computer Science (HISE Group)
Heslington Road
Heslington YO10 5DD
England

email: leo@cs.york.ac.uk
jim@cs.york.ac.uk

Contents

I	Specification	1
1	Introduction	2
II	Models	3
2	SPs	4
3	$\mathcal{A}$ model: security policy	5
3.1	The abstract state	5
3.1.1	A theory for purse names	5
3.1.2	Finite space of numbers	7
3.1.3	Transfer details	8
3.1.4	Abstract world	9
3.2	Security properties	9
3.2.1	Authentic purses	9
3.2.2	Sufficient funds	9
3.3	Secure operations	9
3.3.1	Abstract inputs and outputs	9
3.3.2	Abstract ignore	11
3.3.3	Transfer	11
3.4	Abstract initial state	13
3.5	Abstract instantiation	13
3.6	Summary	14
4	$\mathcal{B}$ model, single purse, operations	16
4.1	Status	16
4.2	Message details	16
4.2.1	Start message counter party details	16
4.2.2	Payment log message details	16
4.3	Clear exception log validation	17
4.4	Messages	17
4.5	A concrete purse	17
4.6	Invisible operations	18
4.6.1	Increase purse	18
4.6.2	Exception logging	18
4.6.3	Abort purse	18
4.7	Value transfer operations	19
4.7.1	StartFromPurse	21
4.7.2	StartToPurse	22
4.7.3	<i>ReqPurse</i>	24
4.7.4	ValPurse	24
4.7.5	AckPurse	24
4.8	Exception logging operations	25
4.8.1	ReadExceptionLogPurse	25
4.8.2	ClearExceptionLogPurse	25
4.9	Summary	27

5	$\mathcal{B}$ model, promoted world	29
5.1	The world	29
5.2	Auxiliary definitions	29
5.2.1	Added information about finiteness	30
5.2.2	<i>AuxWorld</i> does not add constraints	33
5.3	Constraints on the ether	34
5.4	Framing schema	41
5.5	<i>Ignore</i> , <i>Increase</i> and <i>Abort</i>	42
5.6	Promoted operations	42
5.6.1	Value transfer operations	42
5.6.2	Exception log operations	43
5.7	Operations at the world level only	44
5.7.1	Exception Log clear authorisation	44
5.8	The complete protocol	44
5.9	Summary	45
6	$\mathcal{B}$ model, initialisation and finalisation	47
6.1	Summing values	47
6.2	Initialisation	47
6.3	Finalisation	49
6.4	Summary	50
7	$\mathcal{C}$ model: implementation	52
7.1	Concrete World State	52
7.2	Framing Schema	52
7.3	<i>Ignore</i> , <i>Increase</i> and <i>Abort</i>	53
7.4	Promoted operations	53
7.4.1	Value transfer operations	53
7.4.2	Exception log operations	54
7.5	Operations at the world level only	54
7.5.1	Exception Log clear authorisation	54
7.6	Initial state	54
7.7	Finalisation	55
7.8	Summary	56
8	Model consistency proofs	58
8.1	Introduction	58
8.2	$\mathcal{A}$ model consistency proofs	58
8.2.1	Existence of initial abstract state	58
8.2.2	Totality of abstract operations	58
8.3	Totality of between model for purses	62
8.4	Precondition of between model purses (Chapter 4)	63
8.5	$\mathcal{B}$ model consistency proofs (Chapter 5)	65
8.5.1	Existence of between initial state	65
8.5.2	Totality between operations	65
8.5.3	Promoted operations without <i>Ignore/Abort</i>	67
8.5.4	Operations at the world level only	79
8.6	$\mathcal{C}$ model consistency proofs	80
8.6.1	Existence of concrete initial state	80
8.6.2	$\mathcal{C}$ promotion operation precondition	80
8.6.3	Totality of concrete operations	80
8.6.4	Separating the branches of promoted operations	81
8.6.5	Operations at the world level only	85
8.7	Summary	86

III	First refinement: $\mathcal{A}$ to $\mathcal{B}$	91
IV	First refinement: $\mathcal{A}$ to $\mathcal{B}$	92
9	Refinement Proof Rules	93
10	$\mathcal{A}$ to $\mathcal{B}$ retrieve relation	94
10.1	Extended theory for <i>sumValue</i>	94
10.2	Retrieve state	94
10.2.1	Exposing <i>chosenLost</i>	94
10.2.2	Hiding <i>chosenLost</i>	95
10.2.3	Exposing <i>pdThis</i>	95
10.2.4	Partition	98
10.2.5	Quantified forms	100
10.2.6	The full Retrieve state relation	101
10.3	Retrieve inputs	101
10.4	Retrieve outputs	101
10.5	Summary	102
11	$\mathcal{A}$ to $\mathcal{B}$ initialisation proof	104
11.1	Proof obligations	104
11.2	Proof of initial inputs	104
11.3	Summary	105
12	$\mathcal{A}$ to $\mathcal{B}$ finalisation proof	107
12.1	Proof obligations	107
12.2	Output proofs	107
12.3	State proof	107
12.3.1	Case <i>AbFinState</i>	107
12.3.2	Case <i>Rab</i>	108
12.3.3	Actual state related theorem	108
12.4	Summary	109
13	$\mathcal{A}$ to $\mathcal{B}$ applicability proofs	111
13.1	Summary	112
14	$\mathcal{A}$ to $\mathcal{B}$ lemmas	114
14.1	Introduction	114
14.2	Lemma ‘multiple refinement’	114
14.3	Lemma ‘ignore’: separating the branches	114
14.4	Lemma ‘deterministic’: simplifying the <i>Okay</i> branch	115
14.4.1	Choosing an input	115
14.4.2	Cutting in $\Delta ConPurse$	115
14.4.3	Exposing <i>chosenLost</i> and <i>pdThis</i>	116
14.4.4	The existence of <i>AbWorld</i>	118
14.4.5	Statement of lemma ‘deterministic’	119
14.5	Lemma ‘lost unchanged’	119
14.5.1	Proof	119
14.5.2	Sufficient conditions for invoking lemma ‘lost unchanged’	120
14.6	Lemma ‘ <i>AbIgnore</i> ’: Operations that refine <i>AbIgnore</i>	120
14.7	<i>Ignore</i> refines <i>AbIgnore</i>	122
14.7.1	Invoking lemma ‘deterministic’	122
14.7.2	check-operation-ignore	122
14.8	<i>Abort</i> refines <i>AbIgnore</i>	123
14.8.1	Invoking lemma ‘deterministic’	123
14.8.2	exists-pd	123
14.8.3	Three cases	124
14.8.4	exists-chosenLost	126
14.8.5	check-operation-ignore	128

14.9	Lemma ‘abort backward’: operations that first abort	128
14.10	Summary of lemmas	129
14.11	Summary	130
15	<i>Increase</i>	132
15.1	Summary	133
16	<i>StartFrom</i>	135
16.1	Summary	136
17	<i>StartTo</i>	138
17.1	Summary	139
18	<i>Req</i>	141
18.1	Summary	142
19	<i>Val</i>	144
19.1	Summary	145
20	<i>Ack</i>	147
20.1	Summary	148
21	<i>ReadExceptionLog</i>	150
21.1	Summary	151
22	<i>ClearExceptionLog</i>	153
22.1	Summary	154
23	<i>AuthoriseExLogClear</i>	156
23.1	Summary	157
24	<i>Archive</i>	159
24.1	Summary	160
V	Second refinement: $\mathcal{B}$ to $\mathcal{C}$	162
25	$\mathcal{B}$ to $\mathcal{C}$ rules	163
25.1	Security of the implementation	163
25.2	Forwards rules proof obligations	163
25.2.1	Retrieve	163
25.2.2	Initialisation	163
25.2.3	Finalisation	164
25.2.4	Applicability	164
25.2.5	Correctness	164
25.3	Summary	165
26	<i>Rbc</i>	167
26.1	Retrieve state	167
26.2	Summary	168
27	Initialisation, Finalisation, and Applicability	170
27.1	Initialisation proof	170
27.2	Finalisation proof	170
27.3	Applicability proofs	170
27.4	Summary	171

28 $\mathcal{B}$ to $\mathcal{C}$ lemmas	173
28.1 Specialising the proof rules	173
28.2 Correctness of <i>CIgnore</i>	173
28.3 Correctness of a branch of the operation	174
28.3.1 Theorems for each branch from <i>BOpOkay</i> operation	174
28.3.2 Choosing <i>BetweenWorld'</i>	176
28.3.3 Case <i>BOpOkay</i>	176
28.3.4 Case <i>Rbc'</i>	176
28.3.5 Case ‘obey constraints’	177
28.3.6 Summary of <i>ConOkay</i> proof obligation	177
28.4 Correctness of <i>CIncrease</i>	177
28.5 Correctness of <i>CAbort</i>	177
28.6 Lemma ‘logs unchanged’	179
28.7 Lemma ‘abort forward’: operations that first abort	180
28.8 Summary	181
29 Correctness proofs	183
29.1 Summary	184
30 Summary	186
 VI Appendices	 187
A Lemmas	188
A.1 Lemma ‘deterministic’	188
A.2 Lemma ‘constraint’	188
A.3 Lemma ‘logs unchanged’	188
A.4 Lemma ‘compose backward’	188
A.5 Lemma ‘compose forward’	188
A.6 Lemma ‘promoted composition’	188
B Auxiliary toolkit definitions	189
B.1 Total abstract balance	189
B.2 Total lost value	189
B.3 Summary	191
C SPs	193
C.1 Abstract model SPs	193
C.1.1 No value creation	193
C.1.2 All value accounted	193
C.1.3 Authentic purses	193
C.2 Concrete model SPs	194
C.3 SPs and the models	194
C.4 Summary	195
 VII Proofs	 197
D Chapter 3 proofs	198
D.1 A theory for purse names	198
D.2 Finite space of numbers	199
D.3 Transfer details	200
D.4 Security properties	201
D.5 Secure operations	201
D.6 Summary	206

E	Chapter 4 proofs	207
E.1	Message details	207
E.2	Value transfer operations	207
E.2.1	<i>StartFrom</i>	208
E.2.2	<i>StartTo</i>	209
E.2.3	<i>ReqPurseOkay</i>	210
E.2.4	<i>ValPurse</i>	211
E.2.5	<i>AckPurse</i>	211
E.3	Exception logging operations	211
E.3.1	<i>ReadExceptionLogPurse</i>	211
E.3.2	<i>ClearExceptionLogPurse</i>	211
E.4	Summary	212
F	Chapter 5 proofs	213
F.1	The world	213
F.2	<i>AuxWorld</i>	213
F.2.1	Finiteness properties	214
F.2.2	<i>AuxWorld</i> does not add constraints	218
F.3	Constraints on the ether	223
F.4	Framing schema	226
F.5	Operations at the world level only	226
F.6	Summary	227
G	Chapter 6 proofs	228
G.1	Summing values	228
G.2	Initialisation	228
G.3	Finalisation	231
G.4	Summary	232
H	Chapter 7 proofs	233
H.1	Summary	234
I	Chapter 8 proofs	235
I.1	Abstract model consistency	235
I.2	Totality of between model for purses	242
I.3	Precondition of between model purses (Chapter 4)	244
I.4	$\mathcal{B}$ consistency proofs	255
I.5	$\mathcal{C}$ consistency proofs	316
I.6	Summary	325
J	Chapter 10 proofs	326
J.1	Retrieve state	326
J.2	Partition	329
J.2.1	Quantified forms	333
J.3	Summary	335
K	Chapter 11 proofs	336
K.1	Summary	337
L	Chapter 12 proofs	338
L.1	Output proofs	338
L.2	State proofs	338
L.3	Summary	341
M	Chapter 14 proofs	342
M.1	Summary	343
N	Chapter 15 proofs	344
N.1	Summary	345

O Chapter 16 proofs	346
O.1 Summary	347
P Chapter 17 proofs	348
P.1 Summary	349
Q Chapter 18 proofs	350
Q.1 Summary	351
R Chapter 19 proofs	352
R.1 Summary	353
S Chapter 20 proofs	354
S.1 Summary	355
T Chapter 21 proofs	356
T.1 Summary	357
U Chapter 22 proofs	358
U.1 Summary	359
V Chapter 23 proofs	360
V.1 Summary	361
W Chapter 24 proofs	362
W.1 Summary	363
X Chapter 27 proofs	364
X.1 Initialisation proof	364
X.2 Finalisation proof	364
X.3 Summary	366
Y Chapter 28 proofs	367
Y.1 Correctness of <i>CIgnore</i>	367
Y.2 Branching	374
Y.3 Correctness of a branch of the operation	374
Z Chapter 29 proofs	376
Z.1 Summary	377

List of Figures

List of Tables

3.1	Summary of original declarations	14
3.2	Summary of original declarations	15
3.3	Summary of added declarations	15
3.4	Summary of added theorems	15
3.5	Summary of Z/Eves rulees	15
4.1	Bindings changed to alphabetical rather than declaration order	21
4.2	Summary of original declarations	27
4.3	Summary of original declarations	27
4.4	Summary of added declarations	27
4.5	Summary of added theorems	28
4.6	Summary of Z/Eves rulees	28
5.1	Summary of original declarations	45
5.2	Summary of original declarations	45
5.3	Summary of added declarations	45
5.4	Summary of added theorems	46
5.5	Summary of Z/Eves rulees	46
6.1	Summary of original declarations	50
6.2	Summary of original declarations	50
6.3	Summary of added declarations	50
6.4	Summary of added theorems	51
6.5	Summary of Z/Eves rulees	51
7.1	Summary of original declarations	56
7.2	Summary of original declarations	56
7.3	Summary of added declarations	56
7.4	Summary of added theorems	57
7.5	Summary of Z/Eves rulees	57
8.1	Preconditions for $\mathcal{A}$ operations (see Chapter 3)	86
8.2	Preconditions for $\mathcal{B}$ operations (see Chapter 4)	86
8.3	Preconditions for promoted $\mathcal{B}$ operations (see Chapter 5)	87
8.4	(cont.) Preconditions for promoted $\mathcal{B}$ operations (see Chapter 5)	88
8.5	(Preconditions for $\mathcal{C}$ operations (see Chapter 7)	89
8.6	Summary of original declarations	89
8.7	Summary of original declarations	90
8.8	Summary of added declarations	90
8.9	Summary of added theorems	90
8.10	Summary of Z/Eves rulees	90
10.1	Summary of original declarations	102
10.2	Summary of original declarations	102
10.3	Summary of added declarations	102
10.4	Summary of added theorems	103
10.5	Summary of Z/Eves rulees	103
11.1	Summary of original declarations	105

11.2	Summary of original declarations	105
11.3	Summary of added declarations	105
11.4	Summary of added theorems	106
11.5	Summary of Z/Eves rulees	106
12.1	Summary of original declarations	109
12.2	Summary of original declarations	109
12.3	Summary of added declarations	109
12.4	Summary of added theorems	110
12.5	Summary of Z/Eves rulees	110
13.1	Summary of original declarations	112
13.2	Summary of original declarations	112
13.3	Summary of added declarations	112
13.4	Summary of added theorems	113
13.5	Summary of Z/Eves rulees	113
14.1	Summary of original declarations	130
14.2	Summary of original declarations	130
14.3	Summary of added declarations	130
14.4	Summary of added theorems	131
14.5	Summary of Z/Eves rulees	131
15.1	Summary of original declarations	133
15.2	Summary of original declarations	133
15.3	Summary of added declarations	133
15.4	Summary of added theorems	134
15.5	Summary of Z/Eves rulees	134
16.1	Summary of original declarations	136
16.2	Summary of original declarations	136
16.3	Summary of added declarations	136
16.4	Summary of added theorems	137
16.5	Summary of Z/Eves rulees	137
17.1	Summary of original declarations	139
17.2	Summary of original declarations	139
17.3	Summary of added declarations	139
17.4	Summary of added theorems	140
17.5	Summary of Z/Eves rulees	140
18.1	Summary of original declarations	142
18.2	Summary of original declarations	142
18.3	Summary of added declarations	142
18.4	Summary of added theorems	143
18.5	Summary of Z/Eves rulees	143
19.1	Summary of original declarations	145
19.2	Summary of original declarations	145
19.3	Summary of added declarations	145
19.4	Summary of added theorems	146
19.5	Summary of Z/Eves rulees	146
20.1	Summary of original declarations	148
20.2	Summary of original declarations	148
20.3	Summary of added declarations	148
20.4	Summary of added theorems	149
20.5	Summary of Z/Eves rulees	149
21.1	Summary of original declarations	151
21.2	Summary of original declarations	151

21.3	Summary of added declarations	151
21.4	Summary of added theorems	152
21.5	Summary of Z/Eves rulees	152
22.1	Summary of original declarations	154
22.2	Summary of original declarations	154
22.3	Summary of added declarations	154
22.4	Summary of added theorems	155
22.5	Summary of Z/Eves rulees	155
23.1	Summary of original declarations	157
23.2	Summary of original declarations	157
23.3	Summary of added declarations	157
23.4	Summary of added theorems	158
23.5	Summary of Z/Eves rulees	158
24.1	Summary of original declarations	160
24.2	Summary of original declarations	160
24.3	Summary of added declarations	160
24.4	Summary of added theorems	161
24.5	Summary of Z/Eves rulees	161
25.1	Summary of original declarations	165
25.2	Summary of original declarations	165
25.3	Summary of added declarations	165
25.4	Summary of added theorems	166
25.5	Summary of Z/Eves rulees	166
26.1	Summary of original declarations	168
26.2	Summary of original declarations	168
26.3	Summary of added declarations	168
26.4	Summary of added theorems	169
26.5	Summary of Z/Eves rulees	169
27.1	Summary of original declarations	171
27.2	Summary of original declarations	171
27.3	Summary of added declarations	171
27.4	Summary of added theorems	172
27.5	Summary of Z/Eves rulees	172
28.1	Summary of original declarations	181
28.2	Summary of original declarations	181
28.3	Summary of added declarations	181
28.4	Summary of added theorems	182
28.5	Summary of Z/Eves rulees	182
29.1	Summary of original declarations	184
29.2	Summary of original declarations	184
29.3	Summary of added declarations	184
29.4	Summary of added theorems	185
29.5	Summary of Z/Eves rulees	185
B.1	Summary of original declarations	191
B.2	Summary of original declarations	191
B.3	Summary of added declarations	191
B.4	Summary of added theorems	192
B.5	Summary of Z/Eves rulees	192
C.1	Summary of original declarations	195
C.2	Summary of original declarations	195
C.3	Summary of added declarations	195

C.4	Summary of added theorems	196
C.5	Summary of Z/Eves rulees	196
D.1	Summary of proofs	206
E.1	Summary of proofs	212
F.1	Summary of proofs	227
G.1	Summary of proofs	232
H.1	Summary of proofs	234
I.1	Summary of proofs	325
J.1	Summary of proofs	335
K.1	Summary of proofs	337
L.1	Summary of proofs	341
M.1	Summary of proofs	343
N.1	Summary of proofs	345
O.1	Summary of proofs	347
P.1	Summary of proofs	349
Q.1	Summary of proofs	351
R.1	Summary of proofs	353
S.1	Summary of proofs	355
T.1	Summary of proofs	357
U.1	Summary of proofs	359
V.1	Summary of proofs	361
W.1	Summary of proofs	363
X.1	Summary of proofs	366
Z.1	Summary of proofs	377

Original declarations summary

Abbreviations

Given sets

3.1	Unique purse name	5
4.1	Exception log clearance code for <i>PayDetails</i>	17

Free types

3.1	Abstract input	9
3.2	Abstract output	10
4.1	Between world status type	16
4.2	Between world message type	17

Generic definitions

Axiomatic definitions

4.1	<i>PayDetails</i> exception log clearance code	17
5.1	Log book of payments	29
6.1	Summing values from purse payments	47
B.1	Total balance of abstract purses	189
B.2	Total lost of abstract purses	189

Schemas

3.1	Abstract purse	5
3.2	Purse transference details	8
3.3	Abstract world of purses	9
3.4	Authentic purses	9
3.5	Abstract purses have sufficient funds	9
3.6	Abstract operation signature	10
3.7	Abstract ignore operation	11
3.8	Abstract purse transference operation	11
3.9	Abstract purse secure operations signature	11
3.10	Successful secure transfer operation on abstract purses	12
3.11	Successful secure transfer operation with hidden details	12
3.12	Lost secure transfer operation on abstract purses	13
3.13	Lost secure transfer operation with hidden details	13
3.14	Total transfer operation for abstract purses	13
3.15	Abstract world initialisation state	13
3.16	Abstract and global initialisation input relationship	13
3.17	Global world of purses	13
3.18	Abstract finalisation state	14
3.19	Abstract and global finalisation output relationship	14
4.1	Counter party details	16

4.2	Payment details	16
4.3	Concrete purse definition	18
4.4	Concrete purse signature for increase operations	18
4.5	Concrete purse successful increase operation	18
4.6	Exception log security property for concrete purses	18
4.7	Concrete purse signature for aborted operations	18
4.8	Concrete purse successful abort operation	19
4.9	Concrete purse signature for start operations	19
4.10	Concrete purse signature for request operations	19
4.11	Concrete purse signature for value transfer operations	19
4.12	Concrete purse signature for acknowledgement operations	19
4.13	<i>startFrom</i> operation invariant	21
4.14	Successful <i>startFrom</i> operation over concrete purses	21
4.15	Successful <i>startFrom</i> operation with recovery from abortion	21
4.16	<i>startTo</i> operation invariant	22
4.17	Successful <i>startTo</i> operation over concrete purses	22
4.18	Successful <i>startTo</i> operation with recovery from abortion	23
4.19	Request operation invariant	24
4.20	Successful request operation over concrete purses	24
4.21	Value operation invariant	24
4.22	Successful value operation over concrete purses	24
4.23	Acknowledgement operation invariant	24
4.24	Successful ack operation over concrete purses	25
4.25	Successful read exception log over concrete purses	25
4.26	Successful read exception log with recovery from abortion	25
4.27	Concrete purse signature for clearing the exception log	25
4.28	Successful clear exception log over concrete purses	25
4.29	Successful clear exception log with recovery from abortion	26
5.1	Concrete world	29
5.2	Auxiliary definitions for the concrete world	29
5.3	Concrete world auxiliary definitions relationship	33
5.4	Complete between world definition	38
5.5	Between world promotion schema	41
5.6	Between world ignore operation	42
5.7	Between world increase operation	42
5.8	Between world abort operation	42
5.9	Between world value transfer <i>startFrom</i> complete operation	42
5.10	Between world value transfer <i>startTo</i> complete operation	42
5.11	Promoted <i>StartFrom</i> without <i>Ignore</i> and <i>Abort</i>	43
5.12	Promoted <i>StartTo</i> without <i>Ignore</i> and <i>Abort</i>	43
5.13	Between world value transfer request operation	43
5.14	Between world value transfer value operation	43
5.15	Between world value transfer acknowledgement operation	43
5.16	Between world read exception log complete operation	43
5.17	Between world clear exception log complete operation	43
5.18	Promoted <i>ReadExceptionLog</i> without <i>Ignore</i> and <i>Abort</i>	43
5.19	Promoted <i>ClearExceptionLog</i> without <i>Ignore</i> and <i>Abort</i>	43
5.20	Between world authorised exception log clear operation	44
5.21	Authorised between world log clear operation	44
5.22	Between world exception log archiving operation	44
5.23	Complete transfer protocol between concrete levels	44
6.1	Between world initial state	47
6.2	Between world initialisation input	49
6.3	Between world finalisation state	49
6.4	Between world finalisation output	49
7.1	Concrete world promotion	52
7.2	Concrete world promoted ignore operation	53
7.3	Concrete world promoted increase operation	53
7.4	Concrete world promoted abort operation	53

7.5	<i>ConWorld</i> promoted <i>startFrom</i> operation	53
7.6	<i>ConWorld</i> promoted <i>startTo</i> operation	53
7.7	<i>ConWorld</i> promoted request operation	53
7.8	<i>ConWorld</i> promoted value transfer operation	53
7.9	<i>ConWorld</i> promoted acknowledgement operation	53
7.10	<i>ConWorld</i> promoted read exception log operation	54
7.11	<i>ConWorld</i> promoted clear exception log operation	54
7.12	<i>ConWorld</i> promoted authorise exception log clear	54
7.13	Exception log archiving operation	54
7.14	<i>ConWorld</i> initialisation state	55
7.15	<i>ConWorld</i> finalisation state	55
10.1	Retrieve state of $\mathcal{A}$ and $\mathcal{B}$ models	94
10.2	Retrieve state of between and concrete models	94
10.3	Hiding <i>chosenLost</i>	95
10.4	Exposing <i>pdThis</i>	95
10.5	Other purses in the retrieve state	95
10.6	Retrieve state of $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ for <i>Okay</i> operations	95
10.7	Retrieve state of $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ for future <i>Lost</i> operations	96
10.8	Retrieve state of $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ for past <i>Lost</i> operations	97
10.9	Retrieve state of $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ for finalisation	98
10.10	Complete retrieve state for <i>Okay</i> operations	101
10.11	Complete retrieve state for future <i>Lost</i> operations	101
10.12	Complete retrieve state for past <i>Lost</i> operations	101
10.13	Complete retrieve state for finalisation operations	101
10.14	Complete retrieve state inputs	101
10.15	Complete retrieve state outputs	101
26.1	$\mathcal{B}$ and $\mathcal{C}$ (functional) retrieve relation	167
C.1	No value is created	193
C.2	All values are accounted for	193

Lemmas

10.1	Lemma ‘notLoggedAndIn’ (<i>C.12</i>)	100
10.2	Lemma ‘lost’ (<i>C.13</i>)	100
12.1	Case: <i>cut</i> on <i>AbWorld</i> for the <i>AbFinState</i> (12.3.1)	107
12.2	Case: <i>cut</i> on <i>AbWorld</i> for the <i>Rab</i> (12.3.2)	108
14.1	<i>AbWorld</i> is unique within a choice in <i>Betweenworld</i>	118

Theorems

5.1	Auxiliary world elements identity	30
5.2	<i>AuxWorld</i> do not add constraints to <i>ConWorld</i>	34
8.1	Existence of initial abstract state	58
8.2	$\mathcal{A}$ ignore operation is total	59
8.3	$\mathcal{A}$ transfer operation is total	59
8.4	Existence of a between world initial state	65
8.5	Existence of concrete initial state	80
10.1	Retrieve state partition	100
10.2	Retrieve state partition $\forall$ format	100
10.3	Retrieve state partition $\exists$ format	100
11.1	$\mathcal{B}$ world init. refines $\mathcal{A}$ world init.	104
11.2	$\mathcal{B}$ world init. refines $\mathcal{A}$ world init.	104
11.3	$\mathcal{B}$ and $\mathcal{A}$ world initial inputs	104
12.1	$\mathcal{B}$ world outputs refines $\mathcal{A}$ world outputs.	107
12.2	$\mathcal{B}$ world final. refines $\mathcal{A}$ world final.	108
14.1	Backward simulation correctness: <i>AbIgnore</i> $\sqsubseteq$ <i>Ignore</i>	122
14.2	Backward simulation correctness: <i>AbIgnore</i> $\sqsubseteq$ <i>Abort</i>	122
27.1	$\mathcal{B}$ to $\mathcal{C}$ initialisation	170

27.2 $\mathcal{B}$ to $\mathcal{C}$ finalisation	170
--	-----

Added declarations summary

Axiomatic definitions

3.1	Existence of distinct names	5
3.2	Properties of unique purse names	6
3.3	Upper bound for $\mathbb{N}$	7
3.4	Bounded $\mathbb{N}$	7
5.1	Possible values for <i>PayDetails</i> components	31
5.2	Possible values for <i>PayDetails</i> bindings	31
5.3	<i>PayDetails</i> elements for <i>from purse</i>	33
5.4	<i>PayDetails</i> elements for <i>to purse</i>	33
5.5	<i>epv</i> set of payment details via <i>from</i>	33
5.6	<i>epa</i> set of payment details via <i>from</i>	33
5.7	<i>epv</i> set of payment details via <i>to</i>	33
5.8	<i>eaTo</i> set of payment details via <i>to</i>	34

Schemas

5.1	Between world property <i>B1</i>	35
5.2	Between world property <i>B2</i>	35
5.3	Missing between world property for B3	35
5.4	Missing between world property for B3	35
5.5	Between world property <i>B3</i>	35
5.6	Missing between world property for B4	35
5.7	Missing between world property for B4	36
5.8	Between world property <i>B4</i>	36
5.9	Between world property <i>B5</i>	36
5.10	Between world property <i>B6</i>	36
5.11	Between world property <i>B7</i>	36
5.12	Between world property <i>B8</i>	37
5.13	Between world property <i>B9</i>	37
5.14	Between world property <i>B10</i>	37
5.15	Between world property <i>B11</i>	37
5.16	Between world property <i>B12</i>	37
5.17	Between world property <i>B13</i>	37
5.18	Between world property <i>B14</i>	37
5.19	Between world property <i>B15</i>	38
5.20	Between world property <i>B16</i>	38
7.1	Renamed <i>ConWorld</i> for Chapter 7	52
7.2	<i>ConWorld</i> promoted exception log clear	54
7.3	Renamed <i>AuxWorld</i> for Chapter 7	55
8.1	$\mathcal{A}$ operation signature	58
8.2	$\mathcal{A}$ secure operations signature	59
8.3	$\mathcal{A}$ secure transfer operations signature	59
8.4	$\mathcal{B}$ purse operations signature	62
8.5	$\mathcal{B}$ purse start operations signature	64
8.6	$\mathcal{B}$ operations signature	65
8.7	Promoted between world operations signature	68

8.8	Promoted <i>Increase</i> without <i>Ignore</i>	68
8.9	Promoted <i>Abort</i> without <i>Ignore</i>	68
8.10	Promoted <i>StartFrom</i> without <i>Ignore</i> and with <i>Abort</i>	69
8.11	Promoted <i>StartTo</i> without <i>Ignore</i> and with <i>Abort</i>	69
8.12	Promoted <i>Req</i> without <i>Ignore</i>	69
8.13	Promoted <i>Val</i> without <i>Ignore</i>	69
8.14	Promoted <i>Ack</i> without <i>Ignore</i>	69
8.15	Promoted <i>ReadExceptionLog</i> without <i>Ignore</i> and with <i>Abort</i>	69
8.16	Promoted <i>ClearExceptionLog</i> without <i>Ignore</i> and with <i>Abort</i>	69
8.17	Signature for promoted operations in $\mathcal{B}$, filtered from <i>Ignore</i>	71
8.18	Signature for promoted <i>Abort</i> , filtering <i>Ignore</i>	72
8.19	Signature for promoted <i>StartFrom</i> , filtering <i>Ignore</i>	73
8.20	Signature for promoted <i>StartTo</i> , filtering <i>Ignore</i>	74
8.21	Signature for promoted <i>Req</i> , filtering <i>Ignore</i>	76
8.22	Signature for promoted <i>Val</i> , filtering <i>Ignore</i>	76
8.23	Signature for promoted <i>Ack</i> , filtering <i>Ignore</i>	77
8.24	Signature for promoted <i>ReadExceptionLog</i> , filtering <i>Ignore</i>	78
8.25	Signature for promoted <i>ClearExceptionLog</i> , filtering <i>Ignore</i>	78
8.26	$\mathcal{C}$ operations signature	80
8.27	$\mathcal{C}$ promoted operations signature	80
8.28	Promoted <i>CIncrease</i> without <i>CIgnore</i>	81
8.29	Promoted <i>CAbort</i> without <i>CIgnore</i>	81
8.30	Promoted <i>CStartFrom</i> without <i>CIgnore</i> and with <i>CAbort</i>	82
8.31	Promoted <i>CStartTo</i> without <i>CIgnore</i> and with <i>CAbort</i>	82
8.32	Promoted <i>CReq</i> without <i>CIgnore</i>	82
8.33	Promoted <i>CVal</i> without <i>CIgnore</i>	82
8.34	Promoted <i>CAck</i> without <i>CIgnore</i>	82
8.35	Promoted <i>CReadExceptionLog</i> without <i>CIgnore</i> and with <i>CAbort</i>	82
8.36	Promoted <i>ClearExceptionLog</i> without <i>CIgnore</i> and with <i>CAbort</i>	82
8.37	Promoted <i>CAuthorisedExLogClear</i> without <i>CIgnore</i>	82
8.38	Signature for promoted operations in $\mathcal{C}$, filtered from <i>CIgnore</i>	82
8.39	Signature for promoted <i>CIncrease</i> , filtering <i>CIgnore</i>	83
8.40	Signature for promoted <i>CAbort</i> , filtering <i>CIgnore</i>	83
8.41	Signature for promoted <i>CStartFrom</i> , filtering <i>CIgnore</i>	83
8.42	Signature for promoted <i>CStartFrom</i> , filtering <i>CIgnore</i>	83
8.43	Signature for promoted <i>CReq</i> , filtering <i>CIgnore</i>	84
8.44	Signature for promoted <i>CVal</i> , filtering <i>CIgnore</i>	84
8.45	Signature for promoted <i>CAck</i> , filtering <i>CIgnore</i>	84
8.46	Signature for promoted <i>CReadExceptionLog</i> , filtering <i>CIgnore</i>	85
8.47	Signature for promoted <i>CClearExceptionLog</i> , filtering <i>CIgnore</i>	85
28.1	<i>Abort</i> $\sqsubseteq$ <i>CAbort</i> correctness signature	173
28.2	<i>Req</i> $\sqsubseteq$ <i>CReq</i> signature	174

Lemmas

3.1	There exists at least one structured name	6
3.2	There exists two different names	6
3.3	Given a name, there exists another distinct name	7
4.1	It is always possible to find a payment details	17
5.1	<i>PayDetails</i> space is finite	31
5.2	<i>PayDetails</i> bindings are within <i>PayDetails</i> space	31
5.3	<i>PayDetails</i> bindings are finite	31
5.4	<i>PayDetails</i> bindings encompass the whole of <i>PayDetails</i>	32
5.5	<i>PayDetails</i> is a finite set of its bindings	32
5.6	<i>PayDetails</i> fixed <i>from</i> is finite	32
5.7	<i>PayDetails</i> fixed <i>to</i> is finite	32
5.8	<i>PayDetails</i> <i>from</i> purse from dom is finite	32
5.9	<i>PayDetails</i> <i>to</i> purse from dom is finite	32
5.10	<i>PayDetailsFromDomEPRPurse</i> is finite	34

5.11	<i>epa</i> set of payment details is finite via <i>from</i>	34
5.12	<i>epv</i> set of payment details is finite via <i>to</i>	34
5.13	<i>eaTo</i> set of payment details is finite via <i>to</i>	34
5.14	Between world simple expansion	38
5.15	Between <i>AuxWorld</i> expansion	39
5.16	Between world full expansion	40
8.1	Abstract transfer okay PRE <i>abAuthPurse</i> maximal type	59
8.2	Abstract transfer okay PRE <i>abAuthPurse</i> maximal type	60
8.3	Abstract transfer okay PRE <i>AbWorld</i> type	60
8.4	Abstract transfer okay PRE $\triangleleft$ equivalence	60
8.5	Abstract transfer okay PRE <i>Authentic</i>	60
8.6	Abstract transfer okay PRE <i>from?</i> $\oplus$ equivalence	61
8.7	Abstract transfer okay PRE <i>to?</i> $\oplus$ equivalence	61
8.8	Promoted <i>AuxWorld</i> type	68
8.9	Promoted <i>BetweenWorld</i> type	68
8.10	$\mathcal{B}$ <i>fromLogged</i> extraction (<i>AuxWorld</i>)	69
8.11	$\mathcal{B}$ <i>toLogged</i> extraction (<i>AuxWorld</i>)	69
8.12	$\mathcal{B}$ <i>authenticFrom</i> extraction (<i>AuxWorld</i>)	70
8.13	$\mathcal{B}$ <i>authenticTo</i> extraction (<i>AuxWorld</i>)	70
8.14	$\mathcal{B}$ <i>toInEpv</i> extraction (<i>AuxWorld</i>)	70
8.15	$\mathcal{B}$ <i>toInEapayee</i> extraction (<i>AuxWorld</i>)	70
8.16	$\mathcal{B}$ <i>fromInEpr</i> extraction (<i>AuxWorld</i>)	70
8.17	$\mathcal{B}$ <i>fromInEpa</i> extraction (<i>AuxWorld</i>)	70
8.18	$\mathcal{B}$ <i>allLogs</i> extraction (<i>AuxWorld</i>)	70
8.19	$\mathcal{B}$ <i>definitelyLost</i> extraction (<i>AuxWorld</i>)	70
8.20	$\mathcal{B}$ <i>maybeLost</i> extraction (<i>AuxWorld</i>)	70
8.21	$\mathcal{B}$ <i>archive</i> extraction (<i>AuxWorld</i>)	70
8.22	Promoted <i>conAuthPurse</i> is injective	71
8.23	Promoted <i>conAuthPurse</i> updated is injective	71
8.24	Promoted <i>AuxWorld</i> type	72
8.25	Promoted <i>BetweenWorld</i> type for <i>Abort</i>	72
8.26	Promoted <i>AuxWorld</i> type for <i>StartFrom</i>	74
8.27	Promoted <i>BetweenWorld</i> type for <i>StartFrom</i>	74
8.28	Promoted <i>AuxWorld</i> type for <i>StartTo</i>	74
8.29	Promoted <i>BetweenWorld</i> type for <i>StartTo</i>	75
8.30	Promoted <i>AuxWorld</i> type for <i>Req</i>	77
8.31	Promoted <i>BetweenWorld</i> type for <i>Req</i>	77
8.32	Promoted <i>AuxWorld</i> type for <i>Val</i>	77
8.33	Promoted <i>BetweenWorld</i> type for <i>Val</i>	77
8.34	Promoted <i>AuxWorld</i> type for <i>Ack</i>	77
8.35	Promoted <i>BetweenWorld</i> type for <i>Ack</i>	78
8.36	Promoted <i>AuxWorld</i> type for <i>ClearExceptionLog</i>	79
8.37	Promoted <i>BetweenWorld</i> type for <i>ClearExceptionLog</i>	79
8.38	<i>conAuthPurse</i> injectivity and finiteness upon $\oplus$	83
10.1	<i>maybeLost</i> \ <i>chosenLost</i> member purses $\in \text{dom } \textit{conAuthPurse}$	95
10.2	<i>chosenLost</i> member purses $\in \text{dom } \textit{conAuthPurse}$	96
10.3	<i>definitelyLost</i> member purses $\in \text{dom } \textit{conAuthPurse}$	97
10.4	$\mathcal{B}$ <i>fromLogged</i> extraction	99
10.5	$\mathcal{B}$ <i>toLogged</i> extraction	99
10.6	$\mathcal{B}$ <i>authenticFrom</i> extraction	99
10.7	$\mathcal{B}$ <i>authenticTo</i> extraction	99
10.8	$\mathcal{B}$ <i>toInEpv</i> extraction	99
10.9	$\mathcal{B}$ <i>toInEapayee</i> extraction	99
10.10	$\mathcal{B}$ <i>fromInEpr</i> extraction	99
10.11	$\mathcal{B}$ <i>fromInEpa</i> extraction	99
10.12	$\mathcal{B}$ <i>allLogs</i> extraction	99
10.13	$\mathcal{B}$ <i>definitelyLost</i> extraction	100
10.14	$\mathcal{B}$ <i>maybeLost</i> extraction	100
10.15	$\mathcal{B}$ <i>archive</i> extraction	100

12.1	<i>maybeLost</i> is a finite set	108
27.1	Concrete <i>AuxWorld</i> equivalence at finalisation	170
28.1	<i>Abort</i> $\sqsubseteq$ <i>CAabort</i> correct <i>AuxWorld</i> type	173
28.2	<i>Abort</i> $\sqsubseteq$ <i>CAabort</i> correct <i>BetweenWorld</i> type	174

Theorems

3.1	<i>NAT</i> definition is consistent	8
8.1	$\mathcal{A}$ operation	58
8.2	$\mathcal{A}$ secure operation precondition	59
8.3	Abstract transfer okay TD precondition	62
8.4	Abstract transfer okay TD precondition	62
8.5	Increase purse okay is total	62
8.6	Abort purse okay is total	62
8.7	Request purse precondition	63
8.8	Value purse precondition	63
8.9	Value purse precondition	63
8.10	Read exception log purse precondition	63
8.11	Clear exception log purse precondition	63
8.12	Complete read exception log precondition	63
8.13	Complete clear exception log precondition	63
8.14	<i>StartFromPurseEafromOkay</i> purse precondition (avoids <i>Abort</i>)	64
8.15	<i>StartToPurseEafromOkay</i> purse precondition (avoids <i>Abort</i>)	65
8.16	<i>StartFromPurseOkay</i> precondition (considers <i>Abort</i>)	65
8.17	<i>StartToPurseOkay</i> precondition (considers <i>Abort</i>)	65
8.18	$\mathcal{B}$ ignore operation is total	66
8.19	$\mathcal{B}$ increase operation is total	66
8.20	$\mathcal{B}$ abort operation is total	66
8.21	$\mathcal{B}$ <i>startFrom</i> operation is total	66
8.22	$\mathcal{B}$ <i>startTo</i> operation is total	66
8.23	$\mathcal{B}$ request operation is total	66
8.24	$\mathcal{B}$ value operation is total	66
8.25	$\mathcal{B}$ acknowledgement operation is total	66
8.26	$\mathcal{B}$ read exception log operation is total	67
8.27	$\mathcal{B}$ clear exception log operation is total	67
8.28	$\mathcal{B}$ authorisation to clear exception log is total	67
8.29	$\mathcal{B}$ log archiving is total	67
8.30	Promoted between world schema is total	68
8.31	<i>Increase</i> promotion PRE without <i>Ignore</i>	71
8.32	<i>Abort</i> promotion PRE without <i>Ignore</i>	73
8.33	<i>StartFrom</i> promotion PRE without <i>Ignore</i> and <i>Abort</i>	75
8.34	<i>StartFrom</i> promotion PRE without <i>Ignore</i> and with <i>Abort</i>	75
8.35	<i>StartTo</i> promotion PRE without <i>Ignore</i> and <i>Abort</i>	75
8.36	<i>StartTo</i> promotion PRE without <i>Ignore</i> and with <i>Abort</i>	75
8.37	<i>Req</i> promotion PRE without <i>Ignore</i> and <i>Abort</i>	78
8.38	<i>Val</i> promotion PRE without <i>Ignore</i> and <i>Abort</i>	78
8.39	<i>Ack</i> promotion PRE without <i>Ignore</i> and <i>Abort</i>	78
8.40	<i>ReadExceptionLog</i> promotion PRE — <i>Ignore</i> — <i>Abort</i>	79
8.41	<i>ReadExceptionLog</i> promotion PRE — <i>Ignore</i> + <i>Abort</i>	79
8.42	<i>ClearExceptionLog</i> promotion PRE — <i>Ignore</i> — <i>Abort</i>	79
8.43	<i>ClearExceptionLog</i> promotion PRE — <i>Ignore</i> + <i>Abort</i>	79
8.44	<i>AuthoriseExLogClear</i> PRE without <i>Ignore</i>	79
8.45	$\mathcal{C}$ promotion schema precondition	80
8.46	$\mathcal{C}$ Ignore is Total	80
8.47	$\mathcal{C}$ Increase is Total	80
8.48	$\mathcal{C}$ Abort is Total	80
8.49	$\mathcal{C}$ StartFrom is Total	80
8.50	$\mathcal{C}$ StartTo is Total	81
8.51	$\mathcal{C}$ Req is Total	81

8.52	$\mathcal{C}$ Val is Total	81
8.53	$\mathcal{C}$ Ack is Total	81
8.54	$\mathcal{C}$ ReadExceptionLog is Total	81
8.55	$\mathcal{C}$ ClearExceptionLog is Total	81
8.56	$\mathcal{C}$ AuthoriseExceptionLogClear is Total	81
8.57	$\mathcal{C}$ Authentic is Total	81
8.58	$\mathcal{C}Increase$ promotion PRE without $\mathcal{C}Ignore$	83
8.59	$\mathcal{C}Abort$ promotion PRE without $\mathcal{C}Ignore$	83
8.60	$\mathcal{C}StartFrom$ promotion PRE without $\mathcal{C}Ignore$ and $\mathcal{C}Abort$	84
8.61	$\mathcal{C}StartTo$ promotion PRE without $\mathcal{C}Ignore$ and $\mathcal{C}Abort$	84
8.62	$\mathcal{C}Req$ promotion PRE without $\mathcal{C}Ignore$	84
8.63	$\mathcal{C}Val$ promotion PRE without $\mathcal{C}Ignore$	84
8.64	$\mathcal{C}Ack$ promotion PRE without $\mathcal{C}Ignore$	84
8.65	$\mathcal{C}ReadExceptionLogOkay \ \Phi$ PRE without $\mathcal{C}Ignore$ and $\mathcal{C}Abort$	85
8.66	$\mathcal{C}ClearExceptionLogOkay \ \Phi$ PRE without $\mathcal{C}Ignore$ and $\mathcal{C}Abort$	85
8.67	$\mathcal{C}AuthoriseExLogClear$ PRE without $\mathcal{C}Ignore$	85
28.1	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}Ignore$ correct	173
28.2	$\mathcal{B}$ $\mathcal{C}$ correctness for $Abort$	174
28.3	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}StartFrom$ correct	174
28.4	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}StartTo$ correct	174
28.5	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}Req$ correct	174
28.6	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}Val$ correct	175
28.7	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}Ack$ correct	175
28.8	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}ReadExceptionLog$ correct	175
28.9	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}ClearExceptionLog$ correct	175
28.10	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}AuthoriseExLogClear$ correct	175
28.11	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}Archive$ correct	175
28.12	$\mathcal{B}$ to $\mathcal{C}$ $\mathcal{C}Increase$ correct	175
28.13	$\mathcal{B}$ to $\mathcal{C}$ branching is correct	175

Axiomatic consistency lemmas

3.1	$NAMES$ definition existential consistency	6
3.2	$NAMES$ definition universal consistency	6
3.3	NAT boundary consistency	7
3.4	NAT definition consistency	7
5.1	$Logbook$ consistency	29

Z/Eves housekeeping

Z Toolkit extension

3.1	Finite set weakening	6
5.1	Improved version of toolkit rule <i>crossFinite</i>	31
5.2	Subset of finite set is finite	31
8.1	Partial function element	67
8.2	Overridden partial function containment	67
8.3	Overridden partial function pointwise equivalence	67

Assumption Rules

3.1	<i>MAX_NAT</i> maximal type	7
3.2	<i>MAX_NAT</i> type	7
3.3	Abstract input transfer constructor $\leftrightarrow$ maximal type	10
3.4	Abstract input transfer constructor relational type	10
3.5	Abstract input transfer constructor partial function type	10
3.6	Abstract input transfer constructor partial injection type	10
4.1	<i>startFrom MESSAGE</i> relation type	20
4.2	<i>startFrom MESSAGE</i> partial function type	20
4.3	<i>startFrom MESSAGE</i> partial injection type	20
4.4	<i>startTo MESSAGE</i> relation type	22
4.5	<i>startTo MESSAGE</i> partial function type	22
4.6	<i>startTo MESSAGE</i> partial injection type	22
5.1	<i>PayDetails</i> bindings maximal $\leftrightarrow$ type	32
5.2	<i>PayDetails</i> bindings maximal $\leftrightarrow$ type	32
6.1	<i>req MESSAGE</i> relational type	48
6.2	<i>req MESSAGE</i> partial function type	48
6.3	<i>req MESSAGE</i> relational non-maximal type	48
6.4	<i>req MESSAGE</i> partial injection type	48

Rewriting Rules

3.1	Every structured name belongs to the original <i>NAME</i>	6
3.2	<i>MAX_NAT</i> is a natural number (LHS)	7
3.3	<i>MAX_NAT</i> is a natural number (RHS)	8
3.4	<i>NAT</i> equivalence	8
3.5	Abstract input transfer constructor result type	10
3.6	Abstract input transfer constructor is a total function	10
3.7	Abstract input transfer constructor inverse type	10
3.8	Expanded abstract purse <i>balance</i> type	11
3.9	Expanded abstract purse <i>lost</i> type	11
3.10	Theta instantiation idiom for abstract purses	11
3.11	Authentic <i>from?</i> purse <i>TransferOkay</i> mu equivalence	11
3.12	Authentic <i>to?</i> purse <i>TransferOkay</i> mu equivalence	12
3.13	Authentic <i>from?</i> purse <i>TransferLost</i> mu equivalence	12
6.1	<i>req MESSAGE</i> inverse maximal type	48
8.1	μ - θ <i>PayDetails</i> expressions equivalence in <i>startFrom</i>	64

8.2	μ - θ <i>PayDetails</i> expressions equivalence in <i>startTo</i>	64
8.3	Request message of <i>PayDetails</i> is a message	65
12.1	Expanded abstract purse <i>balance</i> type	108

Forward Rules

3.1	<i>TransferDetails</i> <i>value</i> type	8
3.2	<i>TransferDetails</i> binding maximal type	8
3.3	<i>TransferDetails</i> binding type	8
3.4	<i>TransferDetails</i> binding bembership	8
3.5	Authentic <i>name</i> type	9
4.1	<i>CounterPartyDetails</i> <i>name</i> type	19
4.2	<i>CounterPartyDetails</i> <i>value</i> upper bound	19
4.3	<i>CounterPartyDetails</i> <i>value</i> lower bound	20
4.4	<i>CounterPartyDetails</i> <i>nextSeqNo</i> upper bound	20
4.5	<i>CounterPartyDetails</i> <i>nextSeqNo</i> lower bound	20
4.6	<i>CounterPartyDetails</i> binding type	20
4.7	<i>CounterPartyDetails</i> binding membership	20
5.1	<i>AuxWorld</i> <i>authenticFrom</i> is finite	32
5.2	<i>AuxWorld</i> <i>authenticFrom</i> is finite	32
6.1	Payment details <i>from</i> purse type	47
6.2	Payment details <i>to</i> purse type	48
6.3	Payment details <i>value</i> type	48
6.4	<i>PayDetails</i> binding type	48
8.1	<i>ConPurse</i> <i>nextSeqNo</i> maximal type	58

Abstract

This document is a formalisation in the Z/Eves theorem prover of Oxford’s PRG number 126, which formally specifies the *Mondex Electronic Purse*. We took a conservative approach, which considers a carbon-copy of the original Z specification. Apart from new definitions for the auxiliary toolkit in Appendix B, we added some implicit invariants about finiteness at a few points in the between world (see Chapter 5), and theorems discharging such addition are included.

We expect with this effort to contribute to the verified software repository of the *Grand Challenge in Computer Research* on dependable systems evolution (GC6). With the mechanisation we could find bugs in the original specification, and spot informal or missing arguments in some proofs. This work is relevant as it checks this very important piece of formal specification, which had been certified with ITSEC Level E6, the highest standard for software development in the UK.

Other efforts have been done in this direction, and a workshop on the formalisation of Mondex has been set out. We also want to compare the results of different mechanisations in order to understand the subtleties between different tools and their formalisms. As we provide a faithful representation of the original purse, we believe it will be possible to underpin possible semantic losses while translating from the different formalisms used, hence open ground for discussion and debate about appropriateness of formalisms, as well as adequacy and scalability of their tools.

Part I

Specification

Chapter 1

Introduction

Part II

Models

Chapter 2

Security Properties

Chapter 3

A model: security policy

A positive side-effect for being faithful to the original source is that we do not need to worry much about understanding the protocol. As it happens, we learned it by osmosis!

3.1 The abstract state

Original schema 3.1 (Abstract purse)

$$AbPurse \triangleq [balance, lost : \mathbb{N}]$$

□

3.1.1 A theory for purse names

In the definition of the various structures that follows in Mondex, a unique purse *NAME* often appears. Another situation that often appears is that sets involving names must be finite. Nevertheless, nothing about the structure of *NAME* is given. Thus, such assumed finiteness properties cannot be proved (see *AuxWorld* in Chapter 5, *BetwFinState* in Chapter 6, and many others throughout the PRG).

Given set 3.1 (Unique purse name)

$$[NAME]$$

□

For instance, one of those finiteness assumptions comes from expressions such as

$$\begin{aligned} PayDetails &\triangleq [from, to : NAME; \dots \mid from \neq to] \\ name : NAME &\vdash \{pd : PayDetails \mid pd.from = name\} \in \mathbb{F} PayDetails \end{aligned}$$

Because this set comprehension expression represents a set of bindings, even if the *from* purse is fixed to a *name*, we could have an infinite number of *to* purses this *from* purse is associated with. Therefore, we must introduce further structuring for the unique purse names in order for the hidden finiteness assumptions used throughout the PRG to hold. Luckily, they are fairly obvious and abstract. We call it *NAMES* and it could be defined as

$$NAMES \in \mathbb{F} NAME$$

Nevertheless, names are mostly use for characterising payment details between two distinct (unique) purses. In fact, in Section 10.2.5, the existentially quantified equivalence of the retrieve relation partition is provable only if there exists at least two different names.

Taking this into account, let us define at least two different known names. This is also useful in order to prove the definition of *NAMES* is consistent.

Added axiomatic Definition 3.1 (Existence of distinct names)

$$\left| \begin{array}{l} name1, name2 : NAME \\ \hline \langle\langle \text{disabled rule } dExistsDistinctNames \rangle\rangle \\ \neg name1 = name2 \end{array} \right|$$

□

Next, we ensure that the definition of *NAMES* we are about to introduce is consistent.

Axiomatic consistency lemma 3.1 (*NAMES* definition existential consistency)

theorem tNamesExistentialConsistency
 $\exists NAMES : \mathbb{F}_1 \text{ NAME} \bullet \exists n1, n2 : NAMES \bullet \neg n1 = n2$

□

Axiomatic consistency lemma 3.2 (*NAMES* definition universal consistency)

theorem tNamesUniversalConsistency
 $\exists NAMES : \mathbb{F}_1 \text{ NAME} \bullet \forall n1 : NAMES \bullet \exists n2 : NAMES \bullet \neg n1 = n2$

□

After proving those two theorems, we can then safely axiomatically define *NAMES*.

Added axiomatic Definition 3.2 (Properties of unique purse names)

$NAMES : \mathbb{F}_1 \text{ NAME}$	$\langle\langle \text{disabled } dNamesElem \rangle\rangle$ $\exists n1, n2 : NAMES \bullet \neg n1 = n2$ $\langle\langle \text{disabled } dNamesDistinctElem \rangle\rangle$ $\forall n1 : NAMES \bullet \exists n2 : NAMES \bullet \neg n1 = n2$
-------------------------------------	---

□

With this structure model of names, we represent the finiteness assumptions made throughout the PRG, as well as guarantee that payment transaction are total.

Automation

Rewriting rule 3.1 (Every structured name belongs to the original *NAME*)

theorem rule rInNames
 $n \in NAMES \Rightarrow n \in NAME$

□

Toolkit extension 3.1 (Finite set weakening)

theorem rule rFiniteMember $[X]$
 $A \in \mathbb{F} X \Rightarrow A \in \mathbb{P} X$

□

The next theorem is an obvious conclusion from the declaration of *NAMES* using $\mathbb{F}_1$.

Added lemma 3.1 (There exists at least one structured name)

theorem lExistsName
 $\exists name : NAMES \bullet true$

□

The next theorem is important to prove the finiteness (and existence) of *PayDetails*, as at least two different names are required. That is, we must have at least one distinct originator and receptor for any authentic payment transaction to take place.

Added lemma 3.2 (There exists two different names)

theorem lExistsDifferentNames
 $\exists n1, n2 : NAMES \bullet \neg n1 = n2$

□

Thus, structured names make payment transactions total, hence always allows a payment to take place. Similarly, we sometimes know one of the names and only assert the existence of the next one.

Added lemma 3.3 (Given a name, there exists another distinct name)

theorem lExistsDifferentNameFromGivenName
 $\forall n1 : NAMES \bullet \exists n2 : NAMES \bullet \neg n1 = n2$

□

3.1.2 Finite space of numbers

Similarly, in order to make schemas finite as required, we must also add a finite space of numbers by defining a non-empty range over $\mathbb{N}$, which is bounded by a MAX_NAT value defined as

Axiomatic consistency lemma 3.3 (NAT boundary consistency)

theorem tNATBoundaryConsistency
 $\exists MAX_NAT : \mathbb{N} \bullet true$

□

Axiomatic consistency lemma 3.4 (NAT definition consistency)

theorem tNATDefinitionConsistency
 $\exists NAT : \mathbb{P}(0 \dots MAX_NAT) \bullet true$

□

Added axiomatic Definition 3.3 (Upper bound for $\mathbb{N}$)

| $MAX_NAT : \mathbb{N}$

□

Added axiomatic Definition 3.4 (Bounded $\mathbb{N}$)

$NAT == 0 \dots MAX_NAT$

□

Automation

Next, we add some trivial automation rules about these data types in order to improve automation levels.

Assumption rule 3.1 (MAX_NAT maximal type)

theorem grule gMaxNatMaxType
 $MAX_NAT \in \mathbb{Z}$

□

Assumption rule 3.2 (MAX_NAT type)

theorem grule gMaxNatNatType
 $MAX_NAT \in \mathbb{N}$

□

Rewriting rule 3.2 (MAX_NAT is a natural number (LHS))

theorem rule rMaxNatType
 $MAX_NAT \geq 0$

□

Rewriting rule 3.3 (MAX_NAT is a natural number (RHS))

theorem rule rMaxNatType2
 $0 \leq MAX_NAT$

□

This last rule is given just for the sake of pattern matching on different scenarios, hence making rules application smoother.

After that, we do the same for NAT .

Added theorem 3.1 (NAT definition is consistent)

theorem tNATConsistency
 $\exists n : NAT \bullet true$

□

Rewriting rule 3.4 (NAT equivalence)

theorem rule rInNAT
 $x \in NAT \Leftrightarrow x \in \mathbb{Z} \wedge x \geq 0 \wedge x \leq MAX_NAT$

□

3.1.3 Transfer details

Now, with the definition of the finite elements, we redefine *TransferDetails* to be finite. Note that because we leave *AbPurse* components unbound (as $\mathbb{N}$), no additional proof obligations when values are added to the *balance* or *lost* fields are generated. We do not change *AbPurse* to be bounded because there is no requirement that it should be finite.

Original schema 3.2 (Purse transference details)

$TransferDetails$ $from, to : NAMES$ $value : NAT$
--

□

Automation

Forward rule 3.1 ($TransferDetails$ value type)

theorem frule fTransferDetailsValueType
 $\forall TransferDetails \bullet value \in NAT$

□

Forward rule 3.2 ($TransferDetails$ binding maximal type)

theorem frule fTransferDetailsMemberMaxType
 $x \in TransferDetails \Rightarrow x \in \langle from : NAME; to : NAME; value : \mathbb{Z} \rangle$

□

Forward rule 3.3 ($TransferDetails$ binding type)

theorem frule fTransferDetailsMember
 $x \in TransferDetails \Rightarrow x \in \langle from : NAME; to : NAME; value : \mathbb{N} \rangle$

□

Forward rule 3.4 ($TransferDetails$ binding membership)

theorem frule fTransferDetailsInSetMember
 $x \in \langle from : NAMES; to : NAMES; value : NAT \rangle \Rightarrow x \in TransferDetails$

□

3.1.4 Abstract world

Original schema 3.3 (Abstract world of purses)

<i>AbWorld</i>	_____
$abAuthPurse : NAMES \leftrightarrow AbPurse$	
<i>Partial finiteness functionality factored to the predicate part for Z/Eves automation purposes only.</i>	
$abAuthPurse \in NAMES \leftrightarrow AbPurse$	

□

3.2 Security properties

3.2.1 Authentic purses

This is the same as subsection 2.2.3 (p.12) from PRG-126.

Original schema 3.4 (Authentic purses)

<i>Authentic</i>	_____
<i>AbWorld</i>	
$name? : NAMES$	
$name? \in \text{dom } abAuthPurse$	

□

Automation

Forward rule 3.5 (Authentic *name* type)

theorem frule fAuthenticNameType
 $\forall \text{Authentic} \bullet name? \in \text{dom } abAuthPurse$

□

3.2.2 Sufficient funds

This is the same as subsection 2.2.4 (p.12) from PRG-126.

Original schema 3.5 (Abstract purses have sufficient funds)

<i>SufficientFundsProperty</i>	_____
<i>AbWorld</i>	
<i>TransferDetails?</i>	
$Authentic[from?/name?]$	
$value? \leq (abAuthPurse from?).balance$	

□

3.3 Secure operations

3.3.1 Abstract inputs and outputs

Free type 3.1 (Abstract input)

$AIN ::= aNullIn \mid transfer\langle\langle TransferDetails \rangle\rangle$

□

Free type 3.2 (Abstract output)

$AOUT ::= aNullOut$

□

Original schema 3.6 (Abstract operation signature)

$ \begin{array}{l} \text{AbOp} \\ \hline \Delta AbWorld \\ a? : AIN \\ a! : AOUT \\ \hline a! = aNullOut \end{array} $
--

□

Automation

Assumption rule 3.3 (Abstract input transfer constructor $\leftrightarrow$ maximal type)

theorem grule gAINTrasferPFunMaxType
 $transfer \in \langle from : NAME; to : NAME; value : \mathbb{Z} \rangle \leftrightarrow AIN$

□

Assumption rule 3.4 (Abstract input transfer constructor relational type)

theorem grule gAINTrasferRelType
 $transfer \in \langle from : NAME; to : NAME; value : \mathbb{N} \rangle \leftrightarrow AIN$

□

Assumption rule 3.5 (Abstract input transfer constructor partial function type)

theorem grule gAINTrasferPFunType
 $transfer \in \langle from : NAME; to : NAME; value : \mathbb{N} \rangle \rightarrow AIN$

□

Assumption rule 3.6 (Abstract input transfer constructor partial injection type)

theorem grule gAINTransferType
 $transfer \in \langle from : NAME; to : NAME; value : \mathbb{N} \rangle \rightarrowtail AIN$

□

Rewriting rule 3.5 (Abstract input transfer constructor result type)

theorem rule rAINTransferResult
 $\forall TransferDetails \bullet transfer \theta TransferDetails \in AIN$

□

Rewriting rule 3.6 (Abstract input transfer constructor is a total function)

theorem rule rAINTransferIsTotal
 $\forall TransferDetails \bullet \theta TransferDetails \in \text{dom } transfer$

□

Rewriting rule 3.7 (Abstract input transfer constructor inverse type)

theorem rule rAINTransferDomRanRel
 $\forall in : AIN \mid in \in \text{ran } transfer \bullet in \in \text{dom } (transfer \sim)$

□

3.3.2 Abstract ignore

Original schema 3.7 (Abstract ignore operation)

$AbIgnore$	_____
$AbOp$	
$abAuthPurse' = abAuthPurse$	

□

3.3.3 Transfer

Original schema 3.8 (Abstract purse transference operation)

$$AbPurseTransfer \hat{=} AbPurse \setminus (balance, lost)$$

□

Original schema 3.9 (Abstract purse secure operations signature)

$AbWorldSecureOp$	_____
$AbOp$	
$TransferDetails?$	
$a? \in \text{ran } transfer$	
$\theta TransferDetails? = (transfer \sim) a?$	
$\{from?, to?\} \triangleleft abAuthPurse' = \{from?, to?\} \triangleleft abAuthPurse$	

□

Automation for Transfer okay TD domain check

Rewriting rule 3.8 (Expanded abstract purse *balance* type)

theorem rule rAbPurseBalanceTypeOpen
 $\forall \text{purse} : NAMES \multimap AbPurse; \text{name} : NAMES \mid \text{name} \in \text{dom } \text{purse} \bullet$
 $(\text{purse } \text{name}).\text{balance} \in \mathbb{N}$

□

Rewriting rule 3.9 (Expanded abstract purse *lost* type)

theorem rule rAbPurseLostTypeOpen
 $\forall \text{purse} : NAMES \multimap AbPurse; \text{name} : NAMES \mid$
 $\text{name} \in \text{dom } \text{purse} \bullet (\text{purse } \text{name}).\text{lost} \in \mathbb{N}$

□

Rewriting rule 3.10 (Theta instantiation idiom for abstract purses)

theorem rule rThetaPurseInstantiated
 $\forall \text{purse} : NAMES \multimap AbPurse; \text{name} : NAMES \mid \text{name} \in \text{dom } \text{purse} \bullet$
 $\theta AbPurse[\text{balance} := (\text{purse } \text{name}).\text{balance},$
 $\text{lost} := (\text{purse } \text{name}).\text{lost}] = \text{purse } \text{name}$

□

Rewriting rule 3.11 (Authentic *from?* purse *TransferOkay* μ equivalence)

theorem rule lAbAuthPurseFromTransferOkayMuEquivalence
 $\forall AbWorldSecureOp \mid \text{Authentic}[\text{from?}/\text{name?}] \wedge \text{SufficientFundsProperty}$
 $\wedge \text{from?} \neq \text{to?} \bullet$
 $(\mu \Delta AbPurse \mid$
 $\theta AbPurse = abAuthPurse \text{ from?} \wedge$
 $\text{balance}' = \text{balance} - \text{value?} \wedge$
 $\text{lost}' = \text{lost} \wedge$
 $\exists AbPurseTransfer \bullet \theta AbPurse') =$
 $\theta AbPurse'[\text{balance}' := (abAuthPurse \text{ from?}).\text{balance} - \text{value?},$
 $\text{lost}' := (abAuthPurse \text{ from?}).\text{lost}]$

□

Rewriting rule 3.12 (Authentic *to?* purse *TransferOkay* mu equivalence)

theorem rule lAbAuthPurseToTransferOkayMuEquivalence
 $\forall AbWorldSecureOp \mid Authentic[to?/name?] \wedge SufficientFundsProperty$
 $\wedge from? \neq to? \bullet$
 $(\mu \Delta AbPurse \mid$
 $\theta AbPurse = abAuthPurse to? \wedge$
 $balance' = balance + value? \wedge$
 $lost' = lost \wedge$
 $\exists AbPurseTransfer \bullet \theta AbPurse') =$
 $\theta AbPurse'[balance' := (abAuthPurse to?).balance + value?,$
 $lost' := (abAuthPurse to?).lost]$

□

Rewriting rule 3.13 (Authentic *from?* purse *TransferLost* mu equivalence)

theorem rule lAbAuthPurseFromTransferLostMuEquivalence
 $\forall AbWorldSecureOp \mid Authentic[from?/name?] \wedge SufficientFundsProperty$
 $\wedge from? \neq to? \bullet$
 $(\mu \Delta AbPurse \mid$
 $\theta AbPurse = abAuthPurse from?$
 $\wedge balance' = balance - value?$
 $\wedge lost' = lost + value?$
 $\wedge \exists AbPurseTransfer \bullet \theta AbPurse') =$
 $\theta AbPurse'[balance' := (abAuthPurse from?).balance - value?,$
 $lost' := (abAuthPurse from?).lost + value?]$

□

Transfer okay TD

The component *abAuthPurse* is never directly initialised (as far as we could see). Nevertheless, the retrieve relation from Chapter 10 establishes that

$$\text{dom } abAuthPurse = \text{dom } conAuthPurse$$

and in Chapter 6, the range of *conAuthPurse*, but nothing is said about its domain. In these circumstances, as *AbWorldSecureOp* allows, some operation could certainly remove *from?* purse from the domain of *abAuthPurse'*, hence we need the $\Delta Authentic$ below.

Original schema 3.10 (Successful secure transfer operation on abstract purses)

$AbTransferOkayTD$ $AbWorldSecureOp$ $\Delta Authentic[from?/name?, from?/name?']$ $\Delta Authentic[to?/name?, to?/name?']$ $SufficientFundsProperty$ $from? \neq to?$ $abAuthPurse' from? = (\mu \Delta AbPurse \mid \theta AbPurse =$ $abAuthPurse from? \wedge balance' = balance - value? \wedge$ $lost' = lost \wedge \exists AbPurseTransfer \bullet \theta AbPurse')$ $abAuthPurse' to? = (\mu \Delta AbPurse \mid \theta AbPurse =$ $abAuthPurse to? \wedge balance' = balance + value? \wedge$ $lost' = lost \wedge \exists AbPurseTransfer \bullet \theta AbPurse')$
--

□

Original schema 3.11 (Successful secure transfer operation with hidden details)

$$AbTransferOkay \hat{=} AbTransferOkayTD \setminus (to?, from?, value?)$$

□

Transfer lost TD

We need the $\Delta Authentic$ here for $abAuthPurse' to?$, because although no changes take place, it assumes $to?$ belongs to the domain of $abAuthPurse'$.

Original schema 3.12 (Lost secure transfer operation on abstract purses)

$AbTransferLostTD$ $AbWorldSecureOp$ $\Delta Authentic[from?/name?, from?/name?']$ $\Delta Authentic[to?/name?, to?/name?']$ $SufficientFundsProperty$ $from? \neq to?$ $abAuthPurse' from? \in \{ \Delta AbPurse \mid$ $\quad \theta AbPurse = abAuthPurse from?$ $\quad \wedge balance' = balance - value?$ $\quad \wedge lost' = lost + value?$ $\quad \wedge \exists AbPurseTransfer$ $\quad \quad \bullet \theta AbPurse' \}$ $abAuthPurse' to? = abAuthPurse to?$	
---	--

□

Original schema 3.13 (Lost secure transfer operation with hidden details)

$$AbTransferLost \hat{=} AbTransferLostTD \setminus (to?, from?, value?)$$

□

Full transfer

Original schema 3.14 (Total transfer operation for abstract purses)

$$AbTransfer \hat{=} AbTransferOkay \vee AbTransferLost \vee AbIgnore$$

□

3.4 Abstract initial state

Original schema 3.15 (Abstract world initialisation state)

$$AbInitState \hat{=} AbWorld'$$

□

Original schema 3.16 (Abstract and global initialisation input relationship)

$$AbInitIn \hat{=} [a?, g? : AIN \mid a? = g?]$$

□

3.5 Abstract instantiation

Changed to use renaming to avoid duplicating the factoring of partial finiteness of the $abAuthPurse$ function.

Original schema 3.17 (Global world of purses)

$$GlobalWorld \hat{=} AbWorld[gAuthPurse/abAuthPurse]$$

□

Original schema 3.18 (Abstract finalisation state)

<i>AbFinState</i>
<i>AbWorld</i>
<i>GlobalWorld</i>
$gAuthPurse = abAuthPurse$

□

Original schema 3.19 (Abstract and global finalisation output relationship)

$$AbFinOut \triangleq [a!, g! : AOUT \mid a! = g!]$$

□

3.6 Summary

We have substituted *NAME* for *NAMES* and $\mathbb{N}$ for *NAT* at various places for the forementioned reasons. In here we provide the list where those changes in declarations took place:

- Chapter 3: *TransferDetails*, *AbWorld*, *Authentic*
- Chapter 4: *CounterPartyDetails*, *PayDetails*, *MESSAGE*, *ConPurse*
- Chapter 5: *Logbook*, *AuxWorld*, *PhiBOP*, *Ignore*, *AuthoriseExLogClearOkay*, *Archive*
- Chapter 6: *BetwInitIn*
- Chapter 7: *PhiCOP*, *CIgnore*, *CArchive*
- Chapter 8: *BetweenOpSig*, *ConOpSig*

Obviously, the theorems involved in these definitions are somewhat affected, but we tried to keep to the minimum possible. Moreover, because of the limit on the maximal values, we need added preconditions for the concrete world operations, precisely:

- Chapter 4: *StartFromPurseEafromOkay*, *StartToPurseEafromOkay*.
- Chapter 8: corresponding (4) theorems for the added change in Chapter 4.

Maybe *AbWorld.abAuthPurse* do no need *NAMES*. *ConWorld.conAuthPurse*, *PhiBOP.names?*, *Ignore.names?*, *AuthoriseExLogClearOkay.names?* need *NAMES* due to the relationship with *Logbook* and the *archive*.

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	1	1
Free types	2	2
Axiomatic definitions	0	0
Generic definitions	0	0
Schemas	19	19
Total	22	22

Table 3.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	0
Theorems	0	0
Total	0	0

Table 3.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	4	4
Schema	0	0
Total	4	4

Table 3.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	3	3
Theorems	1	1
Axiomatic consistency	4	4
Total	8	8

Table 3.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	1	1
Assumption rules	6	6
Rewriting rules	13	13
Forward rules	5	5
Total	25	25

Table 3.5: Summary of *Z/Eves* rulees

Chapter 4

β model, single purse, operations

4.1 Status

Free type 4.1 (Between world status type)

$$STATUS ::= eaFrom \mid eaTo \mid epr \mid epv \mid epa$$

□

4.2 Message details

4.2.1 Start message counter party details

As *CounterPartyDetails* appears in (μ -) equivalences and expressions related to *PayDetails*, we also change to be a finite schema.

Original schema 4.1 (Counter party details)

<i>CounterPartyDetails</i>	_____
<i>name</i> : NAMES	
<i>value</i> : NAT	
<i>nextSeqNo</i> : NAT	

□

4.2.2 Payment log message details

During the various definitions and theorems where *PayDetails* appear, it often happens that some sets of its bindings restricted in particular ways are finite. Unfortunately, that is not true because the space of names and possible values are not bounded. To amend that and make the claimed finiteness assumptions appearing in the PRG *true*, we need change the values to be bounded as *NAT*.

Original schema 4.2 (Payment details)

<i>PayDetails</i>	_____
<i>TransferDetails</i>	
<i>fromSeqNo, toSeqNo</i> : NAT	
<i>from</i> $\neq$ <i>to</i>	

□

There are two (wrong) assumptions made in the PRG: one implicit, and another explicit. The implicit one is that the set

$$\{ pd : PayDetails \mid pd.from = name \} \in \mathbb{F} PayDetails$$

of *PayDetails* bindings is finite. This true if, and only if, the available names are finite and the values are bounded. Otherwise, we could have payments from the same person *to* infinitely many other persons with infinitely many possible values. The explicit one is that the set

$$\text{purse} \in \text{NAME} \leftrightarrow \text{ConPurse} \Rightarrow \{pd : \text{PayDetails} \mid pd.\text{from} \in \text{dom } \text{purse}\} \in \mathbb{F} \text{PayDetails}$$

is finite. It is explicit because the PRG clearly mentions (p.40) for the definition of *toInEprv* being finite “because *conAuthPurse* is finite”, which is not *true*.

4.3 Clear exception log validation

Given set 4.1 (Exception log clearance code for *PayDetails*)

[*CLEAR*]

□

To prove the consistency of *image* below, we require that at least one payment details could be created, which is ensured by the next lemma.

Added lemma 4.1 (It is always possible to find a payment details)

theorem lExistsPayDetails
 $\exists pd : \text{PayDetails} \bullet \text{true}$

□

TODO: Add discharging axiomatic definition to make *image* an injection on some choice of $c \in \text{CLEAR}$.

Original axiomatic Definition 4.1 (*PayDetails* exception log clearance code)

$\mid \text{image} : \mathbb{P}_1 \text{PayDetails} \rightarrow \text{CLEAR}$

□

4.4 Messages

Free type 4.2 (Between world message type)

MESSAGE ::= *startFrom*⟨⟨*CounterPartyDetails*⟩⟩
 $\mid$ *startTo*⟨⟨*CounterPartyDetails*⟩⟩
 $\mid$ *readExceptionLog*
 $\mid$ *req*⟨⟨*PayDetails*⟩⟩
 $\mid$ *val*⟨⟨*PayDetails*⟩⟩
 $\mid$ *ack*⟨⟨*PayDetails*⟩⟩
 $\mid$ *exceptionLogResult*⟨⟨*NAMES* × *PayDetails*⟩⟩
 $\mid$ *exceptionLogClear*⟨⟨*NAMES* × *CLEAR*⟩⟩
 $\mid$ $\perp$

□

4.5 A concrete purse

Although there are no finiteness restrictions on expressions involving *ConPurse*, it appears in expressions updating *PayDetails*, which includes finiteness restrictions. Thus, differently from *AbPurse*, we need to impose restrictions on the elements of *ConPurse* affected by *PayDetails*.

Original schema 4.3 (Concrete purse definition)

$\frac{\text{ConPurse}}{\text{balance} : \mathbb{N}$ $\text{exLog} : \mathbb{P} \text{ PayDetails}$ $\text{name} : \text{NAMES}$ $\text{nextSeqNo} : \text{NAT}$ $\text{pdAuth} : \text{PayDetails}$ $\text{status} : \text{STATUS}$
$\forall \text{pd} : \text{exLog} \bullet \text{name} \in \{ \text{pd.from}, \text{pd.to} \}$ $\text{status} = \text{epr} \Rightarrow \text{name} = \text{pdAuth.from}$ $\quad \wedge \text{pdAuth.value} \leq \text{balance}$ $\quad \wedge \text{pdAuth.fromSeqNo} < \text{nextSeqNo}$ $\text{status} = \text{epv} \Rightarrow \text{pdAuth.toSeqNo} < \text{nextSeqNo}$ $\text{status} = \text{epa} \Rightarrow \text{pdAuth.fromSeqNo} < \text{nextSeqNo}$

□

4.6 Invisible operations

4.6.1 Increase purse

Original schema 4.4 (Concrete purse signature for increase operations)

$$\text{ConPurseIncrease} \hat{=} \text{ConPurse} \setminus (\text{nextSeqNo})$$

□

Original schema 4.5 (Concrete purse successful increase operation)

$\frac{\text{IncreasePurseOkay}}{\Delta \text{ConPurse}}$ $m?, m! : \text{MESSAGE}$
$\Xi \text{ConPurseIncrease}$ $\text{nextSeqNo}' \geq \text{nextSeqNo}$ $m! = \perp$

□

4.6.2 Exception logging

This is the same as subsection 2.3.1 (p.13) from PRG-126.

Original schema 4.6 (Exception log security property for concrete purses)

$\frac{\text{LogIfNecessary}}{\Delta \text{ConPurse}}$
$\text{exLog}' = \text{exLog} \cup (\text{if } \text{status} \in \{ \text{epv}, \text{epa} \} \text{ then } \{ \text{pdAuth} \} \text{ else } \emptyset)$

□

4.6.3 Abort purse

Original schema 4.7 (Concrete purse signature for aborted operations)

$$\text{ConPurseAbort} \hat{=} \text{ConPurse} \setminus (\text{nextSeqNo}, \text{exLog}, \text{pdAuth}, \text{status})$$

□

Original schema 4.8 (Concrete purse successful abort operation)

AbortPurseOkay $\Delta \text{ConPurse}$ $m?, m! : \text{MESSAGE}$
$\Xi \text{ConPurseAbort}$ LogIfNecessary $\text{status}' = \text{eaFrom}$ $\text{nextSeqNo}' \geq \text{nextSeqNo}$

□

4.7 Value transfer operations

Original schema 4.9 (Concrete purse signature for start operations)

$$\text{ConPurseStart} \hat{=} \text{ConPurse} \setminus (\text{nextSeqNo}, \text{pdAuth}, \text{status})$$

□

Original schema 4.10 (Concrete purse signature for request operations)

$$\text{ConPurseReq} \hat{=} \text{ConPurse} \setminus (\text{balance}, \text{status})$$

□

Original schema 4.11 (Concrete purse signature for value transfer operations)

$$\text{ConPurseVal} \hat{=} \text{ConPurse} \setminus (\text{balance}, \text{status})$$

□

Original schema 4.12 (Concrete purse signature for acknowledgement operations)

$$\text{ConPurseAck} \hat{=} \text{ConPurse} \setminus (\text{status}, \text{pdAuth})$$

□

Automation for *StartFromPurse*

The following 3 rules are needed for discharging goals involving the types of elements from counter party details schema. They appear because in the definite description of

$$\text{StartFromPurseEafromOkay}$$

we make direct use of the fields of *cpd*.

Forward rule 4.1 (*CounterPartyDetails* name type)

theorem frule fCounterPartyDetailsNameType
 $x \in \text{CounterPartyDetails} \Rightarrow x.\text{name} \in \text{NAMES}$

□

Forward rule 4.2 (*CounterPartyDetails* value upper bound)

theorem frule fCounterPartyDetailsValueUpperBound
 $x \in \text{CounterPartyDetails} \Rightarrow x.\text{value} \leq \text{MAX_NAT}$

□

Forward rule 4.3 (*CounterPartyDetails value lower bound*)

theorem frule fCounterPartyDetailsValueLowerBound
 $x \in \text{CounterPartyDetails} \Rightarrow 0 \leq x.\text{value}$

□

Forward rule 4.4 (*CounterPartyDetails nextSeqNo upper bound*)

theorem frule fCounterPartyDetailsNextSeqNoUpperBound
 $x \in \text{CounterPartyDetails} \Rightarrow x.\text{nextSeqNo} \leq \text{MAX_NAT}$

□

Forward rule 4.5 (*CounterPartyDetails nextSeqNo lower bound*)

theorem frule fCounterPartyDetailsNextSeqNoLowerBound
 $x \in \text{CounterPartyDetails} \Rightarrow 0 \leq x.\text{nextSeqNo}$

□

The following 5 rules are needed for discharging goals involving

$\text{startFrom} \sim m?$

The relationship between them is explained in the corresponding proof scripts below.

Forward rule 4.6 (*CounterPartyDetails binding type*)

theorem frule fCounterPartyDetailsMember
 $x \in \text{CounterPartyDetails} \Rightarrow x \in \langle \text{name} : \text{NAME}; \text{nextSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle$

□

Forward rule 4.7 (*CounterPartyDetails binding membership*)

theorem frule fCounterPartyDetailsInSetMember
 $x \in \langle \text{name} : \text{NAMES}; \text{nextSeqNo} : \text{NAT}; \text{value} : \text{NAT} \rangle \Rightarrow x \in \text{CounterPartyDetails}$

□

Assumption rule 4.1 (*startFrom MESSAGE relation type*)

theorem grule gMESSAGEStartFromRelType
 $\text{startFrom} \in \langle \text{name} : \text{NAME}; \text{nextSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle \leftrightarrow \text{MESSAGE}$

□

Assumption rule 4.2 (*startFrom MESSAGE partial function type*)

theorem grule gMESSAGEStartFromPFunType
 $\text{startFrom} \in \langle \text{name} : \text{NAME}; \text{nextSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle \rightharpoonup \text{MESSAGE}$

□

Assumption rule 4.3 (*startFrom MESSAGE partial injection type*)

theorem grule gMESSAGEStartFromPInjType
 $\text{startFrom} \in \langle \text{name} : \text{NAME}; \text{nextSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle \mapsto \text{MESSAGE}$

□

Interestingly, during the experiments with an old version of **Z/Eves** in an old machine, we had the last theorem as

$\text{startFrom} \in \langle \text{name} : \text{NAME}; \text{value} : \mathbb{N}; \text{nextSeqNo} : \mathbb{N} \rangle \mapsto \text{MESSAGE}$

which should be irrelevant, but the old prover complained that “You may only make binding sets of an already-declared schema type” (error code: **NoBindingSetAlphabet**). The solution is trivial: the earlier version requires that binding variable names appears in alphabetical order, rather than in declaration order. Similar changes in the order of binding variable names are needed in:

Chapter	Binding	Theorem
4	<i>CounterPartyDetails</i>	<i>gMESSAGEStartToPInjType</i>
5	<i>PayDetails</i>	<i>lPayDetailsIsFinite</i>
6	<i>PayDetails</i>	<i>fPayDetailsMember</i>
6	<i>PayDetails</i>	<i>gMESSAGEReqRelType</i>
6	<i>PayDetails</i>	<i>gMESSAGEReqPFunType</i>
6	<i>PayDetails</i>	<i>gMESSAGEReqPInjType</i>

Table 4.1: Bindings changed to alphabetical rather than declaration order

4.7.1 StartFromPurse

Maybe a schema inclusion with the interface is better for **Z/Eves** automation. Let us see.

Original schema 4.13 (*startFrom* operation invariant)

ValidStartFrom ConPurse $m? : \text{MESSAGE}$ $\text{cpd} : \text{CounterPartyDetails}$
$m? \in \text{ran } \text{startFrom}$ $\text{cpd} = (\text{startFrom } \sim) m?$ $\text{cpd.name} \neq \text{name}$ $\text{cpd.value} \leq \text{balance}$

□

Original schema 4.14 (Successful *startFrom* operation over concrete purses)

$\text{StartFromPurseEafromOkay}$ $\Delta \text{ConPurse}$ $m?, m! : \text{MESSAGE}$ $\text{cpd} : \text{CounterPartyDetails}$
ValidStartFrom $\text{status} = \text{eaFrom}$ $\exists \text{ConPurseStart}$ <i>Due to the limit on natural number values</i> <i>we must make sure we are not at the limit</i> $\text{nextSeqNo} < \text{MAX_NAT}$ $\text{nextSeqNo}' > \text{nextSeqNo}$ $\text{pdAuth}' = (\mu \text{PayDetails} \mid$ $\quad \text{from} = \text{name}$ $\quad \wedge \text{to} = \text{cpd.name}$ $\quad \wedge \text{value} = \text{cpd.value}$ $\quad \wedge \text{fromSeqNo} = \text{nextSeqNo}$ $\quad \wedge \text{toSeqNo} = \text{cpd.nextSeqNo})$ $\text{status}' = \text{epr}$ $m! = \perp$

□

Original schema 4.15 (Successful *startFrom* operation with recovery from abortion)

$$\text{StartFromPurseOkay} \hat{=} \text{AbortPurseOkay} \circ \text{StartFromPurseEafromOkay} \setminus (\text{cpd})$$

□

4.7.2 StartToPurse

Automation for *ValidStartTo*

These theorems are similar to those for *startFrom* in 4.7

Assumption rule 4.4 (*startTo MESSAGE relation type*)

theorem grule gMESSAGEStartToRelType
 $startTo \in \langle name : NAME; nextSeqNo : \mathbb{N}; value : \mathbb{N} \rangle \leftrightarrow MESSAGE$

□

Assumption rule 4.5 (*startTo MESSAGE partial function type*)

theorem grule gMESSAGEStartToPFunType
 $startTo \in \langle name : NAME; nextSeqNo : \mathbb{N}; value : \mathbb{N} \rangle \rightarrow MESSAGE$

□

Assumption rule 4.6 (*startTo MESSAGE partial injection type*)

theorem grule gMESSAGEStartToPInjType
 $startTo \in \langle name : NAME; nextSeqNo : \mathbb{N}; value : \mathbb{N} \rangle \rightarrowtail MESSAGE$

□

StartToPurse definition

Original schema 4.16 (*startTo operation invariant*)

$ValidStartTo$ $ConPurse$ $m? : MESSAGE$ $cpd : CounterPartyDetails$
$m? \in \text{ran } startTo$ $cpd = (startTo \sim) m?$ $cpd.name \neq name$

□

Original schema 4.17 (Successful *startTo* operation over concrete purses)

$StartToPurseEafromOkay$ $\Delta ConPurse$ $m?, m! : MESSAGE$ $cpd : CounterPartyDetails$
$ValidStartTo$ $status = eaFrom$ $\exists ConPurseStart$ <i>Due to the limit on natural number values we must make sure we are not at the limit</i> $nextSeqNo < MAX_NAT$ $nextSeqNo' > nextSeqNo$ $pdAuth' = (\mu PayDetails \mid$ $\quad to = name$ $\quad \wedge from = cpd.name$ $\quad \wedge value = cpd.value$ $\quad \wedge toSeqNo = nextSeqNo$ $\quad \wedge fromSeqNo = cpd.nextSeqNo)$ $status' = epv$ $m! = reqpdAuth'$

□

Original schema 4.18 (Successful *startTo* operation with recovery from abortion)

$$StartToPurseOkay \hat{=} AbortPurseOkay \mathbin{\text{\texttt{;}}} StartToPurseEafromOkay \setminus (cpd)$$

□

4.7.3 ReqPurse

Original schema 4.19 (Request operation invariant)

<i>AuthenticReqMessage</i>	_____
<i>ConPurse</i> $m? : MESSAGE$	
$m? = req\ pdAuth$	

□

Original schema 4.20 (Successful request operation over concrete purses)

<i>ReqPurseOkay</i>	_____
$\Delta ConPurse$ $m?, m! : MESSAGE$	
<i>AuthenticReqMessage</i> $status = epr$	
$\exists ConPurseReq$	
$balance' = balance - pdAuth.value$ $status' = epa$	
$m! = val\ pdAuth$	

□

4.7.4 ValPurse

Original schema 4.21 (Value operation invariant)

<i>AuthenticValMessage</i>	_____
<i>ConPurse</i> $m? : MESSAGE$	
$m? = val\ pdAuth$	

□

Original schema 4.22 (Successful value operation over concrete purses)

<i>ValPurseOkay</i>	_____
$\Delta ConPurse$ $m?, m! : MESSAGE$	
<i>AuthenticValMessage</i> $status = epv$	
$\exists ConPurseVal$	
$balance' = balance + pdAuth.value$ $status' = eaTo$	
$m! = ack\ pdAuth$	

□

4.7.5 AckPurse

Original schema 4.23 (Acknowledgement operation invariant)

<i>AuthenticAckMessage</i>	_____
<i>ConPurse</i> <i>m?</i> : <i>MESSAGE</i>	
<i>m?</i> = <i>ack pdAuth</i>	

□

Original schema 4.24 (Successful ack operation over concrete purses)

<i>AckPurseOkay</i>	_____
Δ <i>ConPurse</i> <i>m?</i> , <i>m!</i> : <i>MESSAGE</i>	
<i>AuthenticAckMessage</i> <i>status</i> = <i>epa</i>	
$\exists$ <i>ConPurseAck</i> <i>status'</i> = <i>eaFrom</i> <i>m!</i> = $\perp$	

□

4.8 Exception logging operations

4.8.1 ReadExceptionLogPurse

Original schema 4.25 (Successful read exception log over concrete purses)

<i>ReadExceptionLogPurseEafromOkay</i>	_____
$\exists$ <i>ConPurse</i> <i>m?</i> , <i>m!</i> : <i>MESSAGE</i>	
<i>m?</i> = <i>readExceptionLog</i> <i>status</i> = <i>eaFrom</i> <i>m!</i> $\in \{ \perp \} \cup \{ ld : exLog' \bullet exceptionLogResult(name, ld) \}$	

□

Original schema 4.26 (Successful read exception log with recovery from abortion)

$$ReadExceptionLogPurseOkay \hat{=} AbortPurseOkay \circ ReadExceptionLogPurseEafromOkay$$

□

4.8.2 ClearExceptionLogPurse

Original schema 4.27 (Concrete purse signature for clearing the exception log)

$$ConPurseClear \hat{=} ConPurse \setminus (exLog)$$

□

Original schema 4.28 (Successful clear exception log over concrete purses)

$\frac{\text{ClearExceptionLogPurseEafromOkay}}{\Delta \text{ConPurse}} \frac{m?, m! : \text{MESSAGE}}{m! = \perp}$	
$\frac{\text{exLog} \neq \emptyset}{m? = \text{exceptionLogClear}(\text{name}, \text{image exLog})}$	
$\text{status} = \text{eaFrom}$	
$\Xi \text{ConPurseClear}$	
$\text{exLog}' = \emptyset$	
$m! = \perp$	

□

Original schema 4.29 (Successful clear exception log with recovery from abortion)

$$\text{ClearExceptionLogPurseOkay} \hat{=} \text{AbortPurseOkay} \circ \text{ClearExceptionLogPurseEafromOkay}$$

□

4.9 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	1	2
Free types	2	4
Axiomatic definitions	1	1
Generic definitions	0	0
Schemas	29	48
Total	33	55

Table 4.2: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	0
Theorems	0	0
Total	0	0

Table 4.3: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	4
Schema	0	0
Total	0	4

Table 4.4: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	1	4
Theorems	0	1
Axiomatic consistency	0	4
Total	1	9

Table 4.5: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	1
Assumption rules	6	12
Rewriting rules	0	13
Forward rules	7	12
Total	13	38

Table 4.6: Summary of *Z/Eves* rulees

Chapter 5

$\mathcal{B}$ model, promoted world

5.1 The world

Axiomatic consistency lemma 5.1 (*Logbook consistency*)

theorem tLogbookConsistency
 $\exists \text{Logbook} : \mathbb{P}(\text{NAMES} \leftrightarrow \text{PayDetails}) \bullet$
 $\text{Logbook} = \mathbb{P}(\{ \text{PayDetails} \bullet \text{from} \mapsto \theta \text{PayDetails} \} \cup$
 $\{ \text{PayDetails} \bullet \text{to} \mapsto \theta \text{PayDetails} \})$

□

Original axiomatic Definition 5.1 (Log book of payments)

$\text{Logbook} : \mathbb{P}(\text{NAMES} \leftrightarrow \text{PayDetails})$ $\langle\langle \text{disabled rule } d\text{Logbook} \rangle\rangle$ $\text{Logbook} = \mathbb{P}(\{ \text{PayDetails} \bullet \text{from} \mapsto \theta \text{PayDetails} \} \cup$ $\{ \text{PayDetails} \bullet \text{to} \mapsto \theta \text{PayDetails} \})$
--

□

Original schema 5.1 (Concrete world)

ConWorld $\text{conAuthPurse} : \text{NAMES} \rightsquigarrow \text{ConPurse}$ $\text{ether} : \mathbb{P} \text{ MESSAGE}$ $\text{archive} : \text{Logbook}$
$\forall n : \text{dom } \text{conAuthPurse} \bullet (\text{conAuthPurse } n). \text{name} = n$ $\forall nld : \text{archive} \bullet \text{first } nld \in \text{dom } \text{conAuthPurse}$

□

5.2 Auxiliary definitions

Now, we leave the definition of *AuxWorld* without mentioning any finiteness properties, but prove it later as a consequence of the involved sets being restricted to the domain of the finite injection *conAuthPurse* from *ConWorld*.

Original schema 5.2 (Auxiliary definitions for the concrete world)

AuxWorld

ConWorld

$allLogs : NAMES \leftrightarrow PayDetails$
 $authenticFrom, authenticTo : \mathbb{P} PayDetails$
 $fromLogged, toLogged : \mathbb{P} PayDetails$
 $toInEpr, toInEpayee, fromInEpr, fromInEpa : \mathbb{P} PayDetails$
 $definitelyLost : \mathbb{P} PayDetails$
 $maybeLost : \mathbb{P} PayDetails$

Finiteness of components left out for better automation.

They are proved as theorems instead, because conAuthPurse is finite anyway.

$toInEpr \in \mathbb{P} PayDetails$
 $toInEpayee \in \mathbb{P} PayDetails$
 $fromInEpr \in \mathbb{P} PayDetails$
 $fromInEpa \in \mathbb{P} PayDetails$
 $maybeLost \in \mathbb{P} PayDetails$

$fromLogged = \{ pd : authenticFrom \mid pd.from \mapsto pd \in allLogs \}$
 $toLogged = \{ pd : authenticTo \mid pd.to \mapsto pd \in allLogs \}$

See comment above on the definition of ConWorld

$authenticFrom$
 $= \{ pd : PayDetails \mid pd.from \in \text{dom } conAuthPurse \}$
 $authenticTo$
 $= \{ pd : PayDetails \mid pd.to \in \text{dom } conAuthPurse \}$

$toInEpr = \{ pd : authenticTo \mid$
 $(conAuthPursepd.to).status = epr$
 $\wedge (conAuthPursepd.to).pdAuth = pd \}$

$toInEpayee = \{ pd : authenticTo \mid$
 $(conAuthPursepd.to).status = epa$
 $\wedge (conAuthPursepd.to).pdAuth = pd \}$

$fromInEpr = \{ pd : authenticFrom \mid$
 $(conAuthPursepd.from).status = epr$
 $\wedge (conAuthPursepd.from).pdAuth = pd \}$

$fromInEpa = \{ pd : authenticFrom \mid$
 $(conAuthPursepd.from).status = epa$
 $\wedge (conAuthPursepd.from).pdAuth = pd \}$

$allLogs = archive \cup$
 $\{ n : \text{dom } conAuthPurse; pd : PayDetails \mid pd \in (conAuthPurse n).exLog \}$

$definitelyLost = toLogged \cap (fromLogged \cup fromInEpa)$

$maybeLost = (fromInEpa \cup fromLogged) \cap toInEpr$

□

Original theorem 5.1 (Auxiliary world elements identity)

theorem tAuxWorldIdentity

$\forall AuxWorld \bullet definitelyLost \cup maybeLost =$
 $(fromInEpa \cup fromLogged) \cap (toInEpr \cup toLogged)$

□

5.2.1 Added information about finiteness

From the original document, it was left implicit that *authenticFrom* and *authenticTo* were finite as well. Thus, we explicitly prove that both *authenticFrom* and *authenticTo* are finite, something missing from the original definition. Before adding the necessary forward rules, we need some extra lemmas about the finiteness of particular *PayDetails* sets.

Extending the toolkit

Toolkit extension 5.1 (Improved version of toolkit rule *crossFinite*)

theorem rule rCrossFinite

$$A \times B \in \mathbb{F}(C \times D) \Leftrightarrow \\ A = \{\} \vee B = \{\} \vee (A \in \mathbb{F} C \wedge B \in \mathbb{F} D)$$

□

Toolkit extension 5.2 (Subset of finite set is finite)

theorem lFinsetSubset

$$X \in \mathbb{P} Y \wedge Y \in \mathbb{F} Z \Rightarrow X \in \mathbb{F} Z$$

□

PayDetails bindings are finite

To prove these lemmas, we need to establish a basic theory for *PayDetails* finiteness first. As we have already included finite types in *PayDetails*, it becomes possible to prove a more basic fact needed for the lemmas mentioned above. That is, the set of bindings *PayDetails* represent is itself finite

$$PayDetails \in \mathbb{F} \langle from : NAME; to : NAME; value : \mathbb{Z}; fromSeqNo : \mathbb{Z}; toSeqNo : \mathbb{Z} \rangle$$

To prove this basic lemma, we need to establish some facts about finiteness in general, they are given below. Firstly, let us define the space under concern as the cross product of *PayDetails* components shaped as

$$((from, to), (value, (fromSeqNo, toSeqNo)))$$

Added axiomatic Definition 5.1 (Possible values for *PayDetails* components)

$$PayDetailsSpace == (NAMES \times NAMES) \times (NAT \times (NAT \times NAT))$$

□

Added axiomatic Definition 5.2 (Possible values for *PayDetails* bindings)

$$PayDetailsBindings == \\ \{ PayDetails \bullet (((from, to), (value, (fromSeqNo, toSeqNo))), \theta PayDetails) \}$$

□

Added lemma 5.1 (*PayDetails* space is finite)

theorem lPayDetailsSpaceFiniteMaxType

$$PayDetailsSpace \in \mathbb{F} ((NAME \times NAME) \times (\mathbb{Z} \times (\mathbb{Z} \times \mathbb{Z})))$$

□

Added lemma 5.2 (*PayDetails* bindings are within *PayDetails* space)

theorem lPayDetailsBindingsSubsetPayDetailsSpace

$$\text{dom } PayDetailsBindings \in \mathbb{P} PayDetailsSpace$$

□

Added lemma 5.3 (*PayDetails* bindings are finite)

theorem lPayDetailsBindingsFiniteMaxType

$$\text{dom } PayDetailsBindings \in \mathbb{F} ((NAME \times NAME) \times (\mathbb{Z} \times (\mathbb{Z} \times \mathbb{Z})))$$

□

Assumption rule 5.1 (*PayDetails* bindings maximal $\leftrightarrow$ type)

theorem grule gPayDetailsBindingsPfun
 $PayDetailsBindings \in ((NAME \times NAME) \times (\mathbb{Z} \times (\mathbb{Z} \times \mathbb{Z}))) \leftrightarrow PayDetails$

□

Assumption rule 5.2 (*PayDetails* bindings maximal $\nleftrightarrow$ type)

theorem grule gPayDetailsBindingsFfun
 $PayDetailsBindings \in ((NAME \times NAME) \times (\mathbb{Z} \times (\mathbb{Z} \times \mathbb{Z}))) \nleftrightarrow PayDetails$

□

Added lemma 5.4 (*PayDetails* bindings encompass the whole of *PayDetails*)

theorem rule lPayDetailsBindinsRange
 $\text{ran } PayDetailsBindings = PayDetails$

□

Added lemma 5.5 (*PayDetails* is a finite set of its bindings)

theorem grule lPayDetailsIsFinite
 $PayDetails \in \mathbb{F} \langle from : NAME; fromSeqNo : \mathbb{Z}; to : NAME; toSeqNo : \mathbb{Z}; value : \mathbb{Z} \rangle$

□

PayDetails sets that we are interested in are finite

Added lemma 5.6 (*PayDetails* fixed from is finite)

theorem rule lPayDetailsFromPurseNameFinite
 $\forall name : NAMES \bullet \{ pd : PayDetails \mid pd.from = name \} \in \mathbb{F} PayDetails$

□

Added lemma 5.7 (*PayDetails* fixed to is finite)

theorem rule lPayDetailsToPurseNameFinite
 $\forall name : NAMES \bullet \{ pd : PayDetails \mid pd.to = name \} \in \mathbb{F} PayDetails$

□

Added lemma 5.8 (*PayDetails* from purse from dom is finite)

theorem rule lPayDetailsFromPurseDomainFinite
 $\forall purse : NAMES \nleftrightarrow ConPurse \bullet$
 $\{ pd : PayDetails \mid pd.from \in \text{dom } purse \} \in \mathbb{F} PayDetails$

□

Added lemma 5.9 (*PayDetails* to purse from dom is finite)

theorem rule lPayDetailsToPurseDomainFinite
 $\forall purse : NAMES \nleftrightarrow ConPurse \bullet$
 $\{ pd : PayDetails \mid pd.to \in \text{dom } purse \} \in \mathbb{F} PayDetails$

□

Now, from the definition of *AuxWorld*, the necessary theorems about its components is trivial.

Forward rule 5.1 (*AuxWorld* authenticFrom is finite)

theorem frule fAuxWorldAuthenticFromIsFinite
 $\forall AuxWorld \bullet authenticFrom \in \mathbb{F} PayDetails$

□

Forward rule 5.2 (*AuxWorld* authenticTo is finite)

theorem frule fAuxWorldAuthenticToIsFinite
 $\forall AuxWorld \bullet authenticTo \in \mathbb{F} PayDetails$

□

5.2.2 *AuxWorld* does not add constraints

Original schema 5.3 (Concrete world auxiliary definitions relationship)

$$NewVariables \hat{=} \exists ConWorld \bullet AuxWorld$$

□

For the next theorem, because of the way finiteness is encoded in Mondex, we need additional lemmas. Furthermore, because **Z/Eves** does not accept using complex expressions (with schema binding selection within set comprehension) in the middle of a proof, we need to cover these finiteness properties through axiomatic definitions.

Firstly, we add those payments related to the domain of authentic purses.

Added axiomatic Definition 5.3 (*PayDetails* elements for *from* purse)

$$\begin{array}{|l} \hline PayDetailsFromDomPurse : \mathbb{P} PayDetails \\ \hline \langle\langle rule \ dPayDetailsFromDomPurse \rangle\rangle \\ \forall purse : NAMES \rightsquigarrow ConPurse \bullet PayDetailsFromDomPurse = \\ \quad \{ pd : PayDetails \mid pd.from \in \text{dom } purse \} \end{array}$$

□

TODO: Add consistency lambda-terms here! See Z/Eves user's guide p.22 (3.1.3).

Added axiomatic Definition 5.4 (*PayDetails* elements for *to* purse)

$$\begin{array}{|l} \hline PayDetailsToDomPurse : \mathbb{P} PayDetails \\ \hline \langle\langle rule \ dPayDetailsToDomPurse \rangle\rangle \\ \forall purse : NAMES \rightsquigarrow ConPurse \bullet PayDetailsToDomPurse = \\ \quad \{ pd : PayDetails \mid pd.to \in \text{dom } purse \} \end{array}$$

□

Next, we add those payments filtered accordingly.

Added axiomatic Definition 5.5 (*epr* set of payment details via *from*)

$$\begin{array}{|l} \hline PayDetailsFromDomEPRPurse : \mathbb{P} PayDetails \\ \hline \langle\langle rule \ dPayDetailsFromDomEPRPurse \rangle\rangle \\ \forall purse : NAMES \rightsquigarrow ConPurse \bullet PayDetailsFromDomEPRPurse = \\ \quad \{ pd : \{ pd1 : PayDetails \mid pd1.from \in \text{dom } purse \} \mid \\ \quad \quad (purse \ pd.from).status = epr \\ \quad \quad \wedge (purse \ pd.from).pdAuth = pd \} \end{array}$$

□

Added axiomatic Definition 5.6 (*epa* set of payment details via *from*)

$$\begin{array}{|l} \hline PayDetailsFromDomEPAPurse : \mathbb{P} PayDetails \\ \hline \langle\langle rule \ dPayDetailsFromDomEPAPurse \rangle\rangle \\ \forall purse : NAMES \rightsquigarrow ConPurse \bullet PayDetailsFromDomEPAPurse = \\ \quad \{ pd : \{ pd1 : PayDetails \mid pd1.from \in \text{dom } purse \} \mid \\ \quad \quad (purse \ pd.from).status = epa \\ \quad \quad \wedge (purse \ pd.from).pdAuth = pd \} \end{array}$$

□

Added axiomatic Definition 5.7 (*epv* set of payment details via *to*)

$$\begin{array}{|l} \hline PayDetailsToDomEPVPurse : \mathbb{P} PayDetails \\ \hline \langle\langle rule \ dPayDetailsToDomEPVPurse \rangle\rangle \\ \forall purse : NAMES \rightsquigarrow ConPurse \bullet PayDetailsToDomEPVPurse = \\ \quad \{ pd : \{ pd1 : PayDetails \mid pd1.to \in \text{dom } purse \} \mid \\ \quad \quad (purse \ pd.to).status = epv \wedge \\ \quad \quad (purse \ pd.to).pdAuth = pd \} \end{array}$$

□

Added axiomatic Definition 5.8 (*eaTo set of payment details via to*)

$$\begin{array}{|l}
\text{PayDetailsToDomEATOPurse} : \mathbb{P} \text{ PayDetails} \\
\hline
\langle\langle \text{rule } d\text{PayDetailsToDomEATOPurse} \rangle\rangle \\
\forall \text{ purse} : \text{NAMES} \rightsquigarrow \text{ConPurse} \bullet \text{PayDetailsToDomEATOPurse} = \\
\quad \{ \text{pd} : \{ \text{pd1} : \text{PayDetails} \mid \text{pd1.to} \in \text{dom purse} \} \mid \\
\quad \quad (\text{purse pd.to}).\text{status} = \text{eaTo} \wedge \\
\quad \quad (\text{purse pd.to}).\text{pdAuth} = \text{pd} \}
\end{array}$$

□

After that, we add the necessary lemmas

Added lemma 5.10 (*PayDetailsFromDomEPRPurse is finite*)

$$\begin{array}{l}
\text{theorem rule lPayDetailsFromPurseEPRIIsFinite} \\
\forall \text{ purse} : \text{NAMES} \rightsquigarrow \text{ConPurse} \bullet \text{PayDetailsFromDomEPRPurse} \in \mathbb{F} \text{ PayDetails}
\end{array}$$

□

Added lemma 5.11 (*epa set of payment details is finite via from*)

$$\begin{array}{l}
\text{theorem rule lPayDetailsFromPurseEPAIsFinite} \\
\forall \text{ purse} : \text{NAMES} \rightsquigarrow \text{ConPurse} \bullet \text{PayDetailsFromDomEPAPurse} \in \mathbb{F} \text{ PayDetails}
\end{array}$$

□

Added lemma 5.12 (*epv set of payment details is finite via to*)

$$\begin{array}{l}
\text{theorem rule lPayDetailsToPurseEPVIsFinite} \\
\forall \text{ purse} : \text{NAMES} \rightsquigarrow \text{ConPurse} \bullet \text{PayDetailsToDomEPVPurse} \in \mathbb{F} \text{ PayDetails}
\end{array}$$

□

Added lemma 5.13 (*eaTo set of payment details is finite via to*)

$$\begin{array}{l}
\text{theorem rule lPayDetailsToPurseEATOIsFinite} \\
\forall \text{ purse} : \text{NAMES} \rightsquigarrow \text{ConPurse} \bullet \text{PayDetailsToDomEATOPurse} \in \mathbb{F} \text{ PayDetails}
\end{array}$$

□

And finally, we prove the original Mondex theorem.

Original theorem 5.2 (*AuxWorld do not add constraints to ConWorld*)

$$\begin{array}{l}
\text{theorem tAuxWorldDoesNotAddConstraints} \\
\forall \text{ ConWorld} \bullet \exists_1 \text{ NewVariables} \bullet \text{AuxWorld}
\end{array}$$

□

5.3 Constraints on the ether

Due to the sheer size and complexity of the generated domain check proof, we decided to define *BetweenWorld* by layering its properties via schema inclusion. This allows not only a smoother but also much faster proof. Essentially, we are factoring each case of the original proof with schemas.

This is also a good idea because the assumptions immediately available for each domain check case from the original *BetweenWorld* are given in sequence according to the declared schema predicates. Otherwise, if we had tried to prove the monolithic version, we would need a plethora of auxiliary lemmas.

This layering also shows the interrelationship between different properties, and what predicates they depend upon. In the end, we just combine all properties to form the complete *BetweenWorld*. This layering idea comes from previous experience with a ZRC proof the a refinement model checking algorithm.

Added schema 5.1 (Between world property $B1$)

<i>BetweenWorldB1</i>	_____
<i>AuxWorld</i>	_____
$\forall pd : PayDetails \mid req\ pd \in ether \bullet pd \in authenticTo$	

□

Added schema 5.2 (Between world property $B2$)

<i>BetweenWorldB2</i>	_____
<i>AuxWorld</i>	_____
<i>BetweenWorldB1</i>	_____
$\forall pd : PayDetails \mid req\ pd \in ether \bullet$ $pd.toSeqNo < (conAuthPurse\ pd.to).nextSeqNo$	

□

Added schema 5.3 (Missing between world property for $B3$)

<i>BetweenWorldNEW1</i>	_____
<i>AuxWorld</i>	_____
$\forall pd : PayDetails \mid val\ pd \in ether \bullet pd \in authenticTo$	

□

Added schema 5.4 (Missing between world property for $B3$)

<i>BetweenWorldNEW2</i>	_____
<i>AuxWorld</i>	_____
$\forall pd : PayDetails \mid val\ pd \in ether \bullet pd \in authenticFrom$	

□

For property $B3$, the original purse was missing additional information about the authenticity of *val* messages in the *ether* for *to* and *from* purses. This did not happen for *rel* messages, as property $B1$ shows. Moreover, we include the new properties in the predicate part to have a uniform signature across *BetweenWorld* properties. Although not strictly necessary, it helps **Z/Eves** while performing reduction or automatically applying forward (*frules*) rules.

Added schema 5.5 (Between world property $B3$)

<i>BetweenWorldB3</i>	_____
<i>AuxWorld</i>	_____
<i>BetweenWorldNEW1</i>	_____
<i>BetweenWorldNEW2</i>	_____
$\forall pd : PayDetails \mid val\ pd \in ether \bullet$ $pd.toSeqNo < (conAuthPurse\ pd.to).nextSeqNo$ $\wedge pd.fromSeqNo < (conAuthPurse\ pd.from).nextSeqNo$	

□

Added schema 5.6 (Missing between world property for $B4$)

<i>BetweenWorldNEW3</i>	_____
<i>AuxWorld</i>	_____
$\forall pd : PayDetails \mid ack\ pd \in ether \bullet pd \in authenticTo$	

□

Added schema 5.7 (Missing between world property for B4)

<i>BetweenWorldNEW4</i>
<i>AuxWorld</i>
$\forall pd : PayDetails \mid ack\ pd \in ether \bullet pd \in authenticFrom$

□

Similarly, for property *B4*, we need to include the authenticity of *ack* messages in the *ether* for *to* and *from* purses.

Added schema 5.8 (Between world property *B4*)

<i>BetweenWorldB4</i>
<i>AuxWorld</i>
<i>BetweenWorldNEW3</i>
<i>BetweenWorldNEW4</i>
$\forall pd : PayDetails \mid ack\ pd \in ether \bullet$ $pd.toSeqNo < (conAuthPurse\ pd.to).nextSeqNo$ $\wedge pd.fromSeqNo < (conAuthPurse\ pd.from).nextSeqNo$

□

For *B5* nothing else is needed because the *from* purses in *fromLogged* are already authentic, as given by its definition in *AuxWorld*.

Added schema 5.9 (Between world property *B5*)

<i>BetweenWorldB5</i>
<i>AuxWorld</i>
$\forall pd : fromLogged \bullet$ $pd.fromSeqNo < (conAuthPurse\ pd.from).nextSeqNo$

□

Similarly, for *B6* *B7* and *B8*, the *to* purses in *toLogged* and *from* purses in *fromLogged* are already authentic as well.

Added schema 5.10 (Between world property *B6*)

<i>BetweenWorldB6</i>
<i>AuxWorld</i>
$\forall pd : toLogged \bullet pd.toSeqNo < (conAuthPurse\ pd.to).nextSeqNo$

□

Added schema 5.11 (Between world property *B7*)

<i>BetweenWorldB7</i>
<i>AuxWorld</i>
$\forall pd : fromLogged \mid$ $(conAuthPurse\ pd.from).status \in \{epr, epa\} \bullet$ $pd.fromSeqNo$ $< (conAuthPurse\ pd.from).pdAuth.fromSeqNo$

□

Added schema 5.12 (Between world property B8)

<i>BetweenWorldB8</i>
<i>AuxWorld</i>
$\forall pd : toLogged \mid (conAuthPurse\ pd.to).status \in \{epv, eaTo\} \bullet$ $pd.toSeqNo < (conAuthPurse\ pd.to).pdAuth.toSeqNo$

□

Added schema 5.13 (Between world property B9)

<i>BetweenWorldB9</i>
<i>AuxWorld</i>
$\forall pd : fromInEpr \bullet \text{disjoint } (\{val\ pd, ack\ pd\}, ether)$

□

Added schema 5.14 (Between world property B10)

<i>BetweenWorldB10</i>
<i>AuxWorld</i>
$\forall pd : PayDetails \bullet$ $(req\ pd \in ether \wedge ack\ pd \notin ether)$ $\Leftrightarrow (pd \in toInEpr \cup toLogged)$

□

Added schema 5.15 (Between world property B11)

<i>BetweenWorldB11</i>
<i>AuxWorld</i>
$\forall pd : PayDetails \mid val\ pd \in ether \wedge pd \in toInEpr \bullet$ $pd \in fromInEpa \cup fromLogged$

□

Added schema 5.16 (Between world property B12)

<i>BetweenWorldB12</i>
<i>AuxWorld</i>
$\forall pd : fromInEpa \cup fromLogged \bullet req\ pd \in ether$

□

Added schema 5.17 (Between world property B13)

<i>BetweenWorldB13</i>
<i>AuxWorld</i>
$toLogged \in \mathbb{F}\ PayDetails$

□

Added schema 5.18 (Between world property B14)

<i>BetweenWorldB14</i>
<i>AuxWorld</i>
$\forall pd : exceptionLogResult \sim (\mid ether \mid) \bullet pd \in allLogs$

□

Added schema 5.19 (Between world property $B15$)

<i>BetweenWorldB15</i>	
<i>AuxWorld</i>	
$\forall pds : \mathbb{P}_1 \text{ PayDetails}; \text{ name} : \text{NAMES} \mid$ $\text{exceptionLogClear}(\text{ name}, \text{ image } pds) \in \text{ ether} \bullet$ $\{\text{ name}\} \times pds \subseteq \text{ archive}$	

□

Added schema 5.20 (Between world property $B16$)

<i>BetweenWorldB16</i>	
<i>AuxWorld</i>	
$\forall pd : \text{ fromLogged} \cup \text{ toLogged} \bullet \text{ req } pd \in \text{ ether}$	

□

Original schema 5.4 (Complete between world definition)

$$\begin{aligned} \text{BetweenWorld} \quad \hat{=} \quad & \text{BetweenWorldB1} \wedge \text{BetweenWorldB2} \wedge \\ & \text{BetweenWorldB3} \wedge \text{BetweenWorldB4} \wedge \\ & \text{BetweenWorldB5} \wedge \text{BetweenWorldB6} \wedge \\ & \text{BetweenWorldB7} \wedge \text{BetweenWorldB8} \wedge \\ & \text{BetweenWorldB9} \wedge \text{BetweenWorldB10} \wedge \\ & \text{BetweenWorldB11} \wedge \text{BetweenWorldB12} \wedge \\ & \text{BetweenWorldB13} \wedge \text{BetweenWorldB14} \wedge \\ & \text{BetweenWorldB15} \wedge \text{BetweenWorldB16} \end{aligned}$$

□

Expanded *BetweenWorld*

Cannot be a rule because of quantifiers in the goal.

Added lemma 5.14 (Between world simple expansion)

theorem lBetweenWorldExpansion
 $\forall \text{ BetweenWorld} \bullet \quad \text{BetweenWorld components}$
 $\text{AuxWorld} \wedge$
 $\text{BetweenWorld invariant}$
 $(\forall pd : \text{ PayDetails} \mid \text{ req } pd \in \text{ ether} \bullet pd \in \text{ authenticTo})$

$$\begin{aligned}
& \wedge (\forall pd : PayDetails \mid req\ pd \in ether \bullet \\
& \quad pd.toSeqNo < (conAuthPurse\ pd.to).nextSeqNo) \\
& \wedge (\forall pd : PayDetails \mid val\ pd \in ether \bullet pd \in authenticTo) \\
& \wedge (\forall pd : PayDetails \mid val\ pd \in ether \bullet pd \in authenticFrom) \\
& \wedge (\forall pd : PayDetails \mid val\ pd \in ether \bullet \\
& \quad pd.toSeqNo < (conAuthPurse\ pd.to).nextSeqNo \\
& \quad \wedge pd.fromSeqNo < (conAuthPurse\ pd.from).nextSeqNo) \\
& \wedge (\forall pd : PayDetails \mid ack\ pd \in ether \bullet pd \in authenticTo) \\
& \wedge (\forall pd : PayDetails \mid ack\ pd \in ether \bullet pd \in authenticFrom) \\
& \wedge (\forall pd : PayDetails \mid ack\ pd \in ether \bullet \\
& \quad pd.toSeqNo < (conAuthPurse\ pd.to).nextSeqNo \\
& \quad \wedge pd.fromSeqNo < (conAuthPurse\ pd.from).nextSeqNo) \\
& \wedge (\forall pd : fromLogged \bullet \\
& \quad pd.fromSeqNo < (conAuthPurse\ pd.from).nextSeqNo) \\
& \wedge (\forall pd : toLogged \bullet pd.toSeqNo < (conAuthPurse\ pd.to).nextSeqNo) \\
& \wedge (\forall pd : fromLogged \mid \\
& \quad (conAuthPurse\ pd.from).status \in \{epr, epa\} \bullet \\
& \quad pd.fromSeqNo \\
& \quad < (conAuthPurse\ pd.from).pdAuth.fromSeqNo) \\
& \wedge (\forall pd : toLogged \mid (conAuthPurse\ pd.to).status \in \{epv, eaTo\} \bullet \\
& \quad pd.toSeqNo < (conAuthPurse\ pd.to).pdAuth.toSeqNo) \\
& \wedge (\forall pd : fromInEpr \bullet disjoint(\{val\ pd, ack\ pd\}, ether)) \\
& \wedge (\forall pd : PayDetails \bullet \\
& \quad (req\ pd \in ether \wedge ack\ pd \notin ether) \\
& \quad \Leftrightarrow (pd \in toInEpr \cup toLogged)) \\
& \wedge (\forall pd : PayDetails \mid val\ pd \in ether \wedge pd \in toInEpr \bullet \\
& \quad pd \in fromInEpa \cup fromLogged) \\
& \wedge (\forall pd : fromInEpa \cup fromLogged \bullet req\ pd \in ether) \\
& \wedge (toLogged \in \mathbb{F}\ PayDetails) \\
& \wedge (\forall pd : exceptionLogResult \sim (\mid ether \mid) \bullet pd \in allLogs) \\
& \wedge (\forall pds : \mathbb{P}_1\ PayDetails; name : NAMES \mid \\
& \quad exceptionLogClear(name, image\ pds) \in ether \bullet \\
& \quad \{name\} \times pds \subseteq archive) \\
& \wedge (\forall pd : fromLogged \cup toLogged \bullet req\ pd \in ether)
\end{aligned}$$

□

Unfortunately, if *AuxWorld* elements or invariant are needed in proofs, the next theorem is required. That is because *AuxWorld* is blown away by **Z/Eves** when the closed *BetweenWorld* appears early on the list of assumptions.

Added lemma 5.15 (Between *AuxWorld* expansion)

theorem lBetweenWorldAuxWorldExpansion

AuxWorld components

$$\begin{aligned}
& \forall BetweenWorld \bullet ConWorld \wedge allLogs \in NAMES \leftrightarrow PayDetails \wedge \\
& \quad authenticFrom \in \mathbb{P}\ PayDetails \wedge authenticTo \in \mathbb{P}\ PayDetails \wedge \\
& \quad fromLogged \in \mathbb{P}\ PayDetails \wedge toLogged \in \mathbb{P}\ PayDetails \wedge \\
& \quad toInEpr \in \mathbb{P}\ PayDetails \wedge toInEpayee \in \mathbb{P}\ PayDetails \wedge \\
& \quad fromInEpr \in \mathbb{P}\ PayDetails \wedge fromInEpa \in \mathbb{P}\ PayDetails \wedge \\
& \quad definitelyLost \in \mathbb{P}\ PayDetails \wedge maybeLost \in \mathbb{P}\ PayDetails \wedge
\end{aligned}$$

AuxWorld invariant

$$toInEpr \in \mathbb{F}\ PayDetails \wedge$$

$$\begin{aligned}
toInEapayee &\in \mathbb{F} PayDetails \wedge fromInEpr \in \mathbb{F} PayDetails \wedge \\
fromInEpa &\in \mathbb{F} PayDetails \wedge maybeLost \in \mathbb{F} PayDetails \wedge \\
authenticFrom &\in \mathbb{F} PayDetails \wedge authenticTo \in \mathbb{F} PayDetails \wedge \\
fromLogged &= \{ pd : authenticFrom \mid pd.from \mapsto pd \in allLogs \} \wedge \\
toLogged &= \{ pd : authenticTo \mid pd.to \mapsto pd \in allLogs \} \wedge \\
authenticFrom &= \{ pd : PayDetails \mid pd.from \in \text{dom } conAuthPurse \} \wedge \\
authenticTo &= \{ pd : PayDetails \mid pd.to \in \text{dom } conAuthPurse \} \wedge \\
toInEpr &= \{ pd : authenticTo \mid (conAuthPurse pd.to).status = epr \\
&\quad \wedge (conAuthPurse pd.to).pdAuth = pd \} \wedge \\
toInEpayee &= \{ pd : authenticTo \mid (conAuthPurse pd.to).status = eaTo \\
&\quad \wedge (conAuthPurse pd.to).pdAuth = pd \} \wedge \\
fromInEpr &= \{ pd : authenticFrom \mid (conAuthPurse pd.from).status = epr \\
&\quad \wedge (conAuthPurse pd.from).pdAuth = pd \} \wedge \\
fromInEpa &= \{ pd : authenticFrom \mid (conAuthPurse pd.from).status = epa \\
&\quad \wedge (conAuthPurse pd.from).pdAuth = pd \} \wedge \\
allLogs &= archive \cup \\
&\quad \{ n : \text{dom } conAuthPurse; pd : PayDetails \mid pd \in (conAuthPurse n).exLog \} \wedge \\
definitelyLost &= toLogged \cap (fromLogged \cup fromInEpa) \wedge \\
maybeLost &= (fromInEpa \cup fromLogged) \cap toInEpr \wedge \\
\textbf{BetweenWorld invariant} & \\
&(\forall pd : PayDetails \mid req pd \in ether \bullet pd \in authenticTo) \wedge \\
&(\forall pd : PayDetails \mid req pd \in ether \bullet pd.toSeqNo < (conAuthPurse pd.to).nextSeqNo) \\
&\wedge (\forall pd : PayDetails \mid val pd \in ether \bullet pd \in authenticTo) \wedge \\
&(\forall pd : PayDetails \mid val pd \in ether \bullet pd \in authenticFrom) \wedge \\
&(\forall pd : PayDetails \mid val pd \in ether \bullet pd.toSeqNo < (conAuthPurse pd.to).nextSeqNo \\
&\quad \wedge pd.fromSeqNo < (conAuthPurse pd.from).nextSeqNo) \wedge \\
&(\forall pd : PayDetails \mid ack pd \in ether \bullet pd \in authenticTo) \wedge \\
&(\forall pd : PayDetails \mid ack pd \in ether \bullet pd \in authenticFrom) \wedge \\
&(\forall pd : PayDetails \mid ack pd \in ether \bullet pd.toSeqNo < (conAuthPurse pd.to).nextSeqNo \\
&\quad \wedge pd.fromSeqNo < (conAuthPurse pd.from).nextSeqNo) \wedge \\
&(\forall pd : fromLogged \bullet pd.fromSeqNo < (conAuthPurse pd.from).nextSeqNo) \wedge \\
&(\forall pd : toLogged \bullet pd.toSeqNo < (conAuthPurse pd.to).nextSeqNo) \wedge \\
&(\forall pd : fromLogged \mid (conAuthPurse pd.from).status \in \{epr, epa\} \bullet \\
&\quad pd.fromSeqNo < (conAuthPurse pd.from).pdAuth.fromSeqNo) \wedge \\
&(\forall pd : toLogged \mid (conAuthPurse pd.to).status \in \{epv, eaTo\} \bullet \\
&\quad pd.toSeqNo < (conAuthPurse pd.to).pdAuth.toSeqNo) \wedge \\
&(\forall pd : fromInEpr \bullet \text{disjoint}(\{val pd, ack pd\}, ether)) \wedge \\
&(\forall pd : PayDetails \bullet (req pd \in ether \wedge ack pd \notin ether) \Leftrightarrow (pd \in toInEpr \cup toLogged)) \\
&\wedge (\forall pd : PayDetails \mid val pd \in ether \wedge pd \in toInEpr \bullet \\
&\quad pd \in fromInEpa \cup fromLogged) \wedge \\
&(\forall pd : fromInEpa \cup fromLogged \bullet req pd \in ether) \wedge (toLogged \in \mathbb{F} PayDetails) \wedge \\
&(\forall pd : exceptionLogResult \sim (\mid ether \mid) \bullet pd \in allLogs) \wedge \\
&(\forall pds : \mathbb{P}_1 PayDetails; name : NAMES \mid \\
&\quad exceptionLogClear(name, image pds) \in ether \bullet \\
&\quad \{name\} \times pds \subseteq archive) \wedge \\
&(\forall pd : fromLogged \cup toLogged \bullet req pd \in ether)
\end{aligned}$$

□

These theorems would be equivalent to having individual forward rules (frule), one for each invariant property and schema component. (see Chapter 10 for where it is used).

Added lemma 5.16 (Between world full expansion)

theorem lBetweenWorldFullExpansion

ConWorld components and invariant

$$\begin{aligned}
\forall \text{BetweenWorld} \bullet conAuthPurse \in NAMES \rightsquigarrow ConPurse \wedge ether \in \mathbb{P} MESSAGE \wedge \\
archive \in Logbook \wedge (\forall n : \text{dom } conAuthPurse \bullet (conAuthPurse n).name = n) \wedge \\
(\forall nld : archive \bullet first nld \in \text{dom } conAuthPurse) \wedge
\end{aligned}$$

AuxWorld components and invariant

$$allLogs \in NAMES \Leftrightarrow PayDetails \wedge authenticFrom \in \mathbb{P} PayDetails \wedge$$

$$\begin{aligned}
& \text{authenticTo} \in \mathbb{P} \text{ PayDetails} \wedge \text{fromLogged} \in \mathbb{P} \text{ PayDetails} \wedge \\
& \text{toLogged} \in \mathbb{P} \text{ PayDetails} \wedge \text{toInEpr} \in \mathbb{P} \text{ PayDetails} \wedge \\
& \text{toInEapayee} \in \mathbb{P} \text{ PayDetails} \wedge \text{fromInEpr} \in \mathbb{P} \text{ PayDetails} \wedge \\
& \text{fromInEpa} \in \mathbb{P} \text{ PayDetails} \wedge \text{definitelyLost} \in \mathbb{P} \text{ PayDetails} \wedge \\
& \text{maybeLost} \in \mathbb{P} \text{ PayDetails} \wedge \text{toInEpr} \in \mathbb{F} \text{ PayDetails} \wedge \\
& \text{toInEapayee} \in \mathbb{F} \text{ PayDetails} \wedge \text{fromInEpr} \in \mathbb{F} \text{ PayDetails} \wedge \\
& \text{fromInEpa} \in \mathbb{F} \text{ PayDetails} \wedge \text{maybeLost} \in \mathbb{F} \text{ PayDetails} \wedge \\
& \text{authenticFrom} \in \mathbb{F} \text{ PayDetails} \wedge \text{authenticTo} \in \mathbb{F} \text{ PayDetails} \wedge \\
& \text{fromLogged} = \{ pd : \text{authenticFrom} \mid pd.\text{from} \mapsto pd \in \text{allLogs} \} \wedge \\
& \text{toLogged} = \{ pd : \text{authenticTo} \mid pd.\text{to} \mapsto pd \in \text{allLogs} \} \wedge \\
& \text{authenticFrom} = \{ pd : \text{PayDetails} \mid pd.\text{from} \in \text{dom conAuthPurse} \} \wedge \\
& \text{authenticTo} = \{ pd : \text{PayDetails} \mid pd.\text{to} \in \text{dom conAuthPurse} \} \wedge \\
& \text{toInEpr} = \{ pd : \text{authenticTo} \mid (\text{conAuthPurse} pd.\text{to}).\text{status} = \text{epv} \\
& \quad \wedge (\text{conAuthPurse} pd.\text{to}).pdAuth = pd \} \wedge \\
& \text{toInEapayee} = \{ pd : \text{authenticTo} \mid (\text{conAuthPurse} pd.\text{to}).\text{status} = \text{eaTo} \\
& \quad \wedge (\text{conAuthPurse} pd.\text{to}).pdAuth = pd \} \wedge \\
& \text{fromInEpr} = \{ pd : \text{authenticFrom} \mid (\text{conAuthPurse} pd.\text{from}).\text{status} = \text{epr} \\
& \quad \wedge (\text{conAuthPurse} pd.\text{from}).pdAuth = pd \} \wedge \\
& \text{fromInEpa} = \{ pd : \text{authenticFrom} \mid (\text{conAuthPurse} pd.\text{from}).\text{status} = \text{epa} \\
& \quad \wedge (\text{conAuthPurse} pd.\text{from}).pdAuth = pd \} \wedge \\
& \text{allLogs} = \text{archive} \cup \\
& \quad \{ n : \text{dom conAuthPurse}; pd : \text{PayDetails} \mid pd \in (\text{conAuthPurse } n).\text{exLog} \} \wedge \\
& \text{definitelyLost} = \text{toLogged} \cap (\text{fromLogged} \cup \text{fromInEpa}) \wedge \\
& \text{maybeLost} = (\text{fromInEpa} \cup \text{fromLogged}) \cap \text{toInEpr} \wedge \\
& \textbf{BetweenWorld invariant} \\
& (\forall pd : \text{PayDetails} \mid \text{req } pd \in \text{ether} \bullet pd \in \text{authenticTo}) \wedge \\
& (\forall pd : \text{PayDetails} \mid \text{req } pd \in \text{ether} \bullet pd.\text{toSeqNo} < (\text{conAuthPurse } pd.\text{to}).\text{nextSeqNo}) \\
& \wedge (\forall pd : \text{PayDetails} \mid \text{val } pd \in \text{ether} \bullet pd \in \text{authenticTo}) \wedge \\
& (\forall pd : \text{PayDetails} \mid \text{val } pd \in \text{ether} \bullet pd \in \text{authenticFrom}) \wedge \\
& (\forall pd : \text{PayDetails} \mid \text{val } pd \in \text{ether} \bullet pd.\text{toSeqNo} < (\text{conAuthPurse } pd.\text{to}).\text{nextSeqNo} \\
& \quad \wedge pd.\text{fromSeqNo} < (\text{conAuthPurse } pd.\text{from}).\text{nextSeqNo}) \wedge \\
& (\forall pd : \text{PayDetails} \mid \text{ack } pd \in \text{ether} \bullet pd \in \text{authenticTo}) \wedge \\
& (\forall pd : \text{PayDetails} \mid \text{ack } pd \in \text{ether} \bullet pd \in \text{authenticFrom}) \wedge \\
& (\forall pd : \text{PayDetails} \mid \text{ack } pd \in \text{ether} \bullet pd.\text{toSeqNo} < (\text{conAuthPurse } pd.\text{to}).\text{nextSeqNo} \\
& \quad \wedge pd.\text{fromSeqNo} < (\text{conAuthPurse } pd.\text{from}).\text{nextSeqNo}) \wedge \\
& (\forall pd : \text{fromLogged} \bullet pd.\text{fromSeqNo} < (\text{conAuthPurse } pd.\text{from}).\text{nextSeqNo}) \wedge \\
& (\forall pd : \text{toLogged} \bullet pd.\text{toSeqNo} < (\text{conAuthPurse } pd.\text{to}).\text{nextSeqNo}) \wedge \\
& (\forall pd : \text{fromLogged} \mid (\text{conAuthPurse } pd.\text{from}).\text{status} \in \{\text{epr}, \text{epa}\} \bullet \\
& \quad pd.\text{fromSeqNo} < (\text{conAuthPurse } pd.\text{from}).pdAuth.\text{fromSeqNo}) \wedge \\
& (\forall pd : \text{toLogged} \mid (\text{conAuthPurse } pd.\text{to}).\text{status} \in \{\text{epv}, \text{eaTo}\} \bullet \\
& \quad pd.\text{toSeqNo} < (\text{conAuthPurse } pd.\text{to}).pdAuth.\text{toSeqNo}) \wedge \\
& (\forall pd : \text{fromInEpr} \bullet \text{disjoint}(\{\text{val } pd, \text{ack } pd\}, \text{ether})) \wedge \\
& (\forall pd : \text{PayDetails} \bullet (\text{req } pd \in \text{ether} \wedge \text{ack } pd \notin \text{ether}) \Leftrightarrow (pd \in \text{toInEpr} \cup \text{toLogged})) \\
& \wedge (\forall pd : \text{PayDetails} \mid \text{val } pd \in \text{ether} \wedge pd \in \text{toInEpr} \bullet \\
& \quad pd \in \text{fromInEpa} \cup \text{fromLogged}) \wedge \\
& (\forall pd : \text{fromInEpa} \cup \text{fromLogged} \bullet \text{req } pd \in \text{ether}) \wedge (\text{toLogged} \in \mathbb{F} \text{ PayDetails}) \wedge \\
& (\forall pd : \text{exceptionLogResult} \sim (\mid \text{ether} \mid) \bullet pd \in \text{allLogs}) \wedge \\
& (\forall pds : \mathbb{P}_1 \text{ PayDetails}; \text{name} : \text{NAMES} \mid \\
& \quad \text{exceptionLogClear}(\text{name}, \text{image pds}) \in \text{ether} \bullet \\
& \quad \{\text{name}\} \times \text{pds} \subseteq \text{archive}) \wedge \\
& (\forall pd : \text{fromLogged} \cup \text{toLogged} \bullet \text{req } pd \in \text{ether})
\end{aligned}$$

□

5.4 Framing schema

We could have used ΦBOp , but this creates a naming compatibility between the **Z/Eves** GUI and **Z/Eves** x-emacs interfaces.

Original schema 5.5 (Between world promotion schema)

ΦBOp $\Delta BetweenWorld$ $\Delta ConPurse$ $m?, m! : MESSAGE$ $name? : NAMES$
$m? \in ether$ $name? \in \text{dom } conAuthPurse$ $\theta ConPurse = conAuthPurse \text{ name?}$ $conAuthPurse' = conAuthPurse \oplus \{name? \mapsto \theta ConPurse'\}$ $archive' = archive$ $ether' = ether \cup \{m!\}$

□

5.5 Ignore, Increase and Abort

Original schema 5.6 (Between world ignore operation)

$$Ignore \hat{=} [\exists BetweenWorld; name? : NAMES; m?, m! : MESSAGE \mid m! = \perp]$$

□

Original schema 5.7 (Between world increase operation)

$$Increase \hat{=} Ignore \vee (\exists \Delta ConPurse \bullet \Phi BOp \wedge IncreasePurseOkay)$$

□

Original schema 5.8 (Between world abort operation)

$$Abort \hat{=} Ignore \vee (\exists \Delta ConPurse \bullet AbortPurseOkay \wedge [\Phi BOp \mid m! = \perp])$$

□

5.6 Promoted operations

5.6.1 Value transfer operations

Original schema 5.9 (Between world value transfer *startFrom* complete operation)

$$StartFrom \hat{=} Ignore \vee Abort \vee (\exists \Delta ConPurse \bullet \Phi BOp \wedge StartFromPurseOkay)$$

□

Original schema 5.10 (Between world value transfer *startTo* complete operation)

$$StartTo \hat{=} Ignore \vee Abort \vee (\exists \Delta ConPurse \bullet \Phi BOp \wedge StartToPurseOkay)$$

□

The next two operations are used in lemmas from Section 14.3.

Original schema 5.11 (Promoted *StartFrom* without *Ignore* and *Abort*)

$$\begin{aligned} \text{StartFromEafromOkay} &\hat{=} \exists \Delta \text{ConPurse} \bullet \\ &\quad \text{PhiBOp} \wedge \text{StartFromPurseEafromOkay} \end{aligned}$$

□

Original schema 5.12 (Promoted *StartTo* without *Ignore* and *Abort*)

$$\begin{aligned} \text{StartToEafromOkay} &\hat{=} \exists \Delta \text{ConPurse} \bullet \\ &\quad \text{PhiBOp} \wedge \text{StartToPurseEafromOkay} \end{aligned}$$

□

Original schema 5.13 (Between world value transfer request operation)

$$\text{Req} \hat{=} \text{Ignore} \vee (\exists \Delta \text{ConPurse} \bullet \text{PhiBOp} \wedge \text{ReqPurseOkay})$$

□

Original schema 5.14 (Between world value transfer value operation)

$$\text{Val} \hat{=} \text{Ignore} \vee (\exists \Delta \text{ConPurse} \bullet \text{PhiBOp} \wedge \text{ValPurseOkay})$$

□

Original schema 5.15 (Between world value transfer acknowledgement operation)

$$\text{Ack} \hat{=} \text{Ignore} \vee (\exists \Delta \text{ConPurse} \bullet \text{PhiBOp} \wedge \text{AckPurseOkay})$$

□

5.6.2 Exception log operations

Original schema 5.16 (Between world read exception log complete operation)

$$\begin{aligned} \text{ReadExceptionLog} &\hat{=} \text{Ignore} \\ &\quad \vee (\exists \Delta \text{ConPurse} \bullet \text{PhiBOp} \wedge \text{ReadExceptionLogPurseOkay}) \end{aligned}$$

□

Why considers abort twice?

Original schema 5.17 (Between world clear exception log complete operation)

$$\begin{aligned} \text{ClearExceptionLog} &\hat{=} \text{Ignore} \\ &\quad \vee \text{Abort} \\ &\quad \vee (\exists \Delta \text{ConPurse} \bullet \text{PhiBOp} \wedge \text{ClearExceptionLogPurseOkay}) \end{aligned}$$

□

The next two operations are used in lemmas from Section 14.3.

Original schema 5.18 (Promoted *ReadExceptionLog* without *Ignore* and *Abort*)

$$\begin{aligned} \text{ReadExceptionLogEafromOkay} &\hat{=} \exists \Delta \text{ConPurse} \bullet \\ &\quad \text{PhiBOp} \wedge \text{ReadExceptionLogPurseEafromOkay} \end{aligned}$$

□

Original schema 5.19 (Promoted *ClearExceptionLog* without *Ignore* and *Abort*)

$$\begin{aligned} \text{ClearExceptionLogEafromOkay} &\hat{=} \exists \Delta \text{ConPurse} \bullet \\ &\quad \text{PhiBOp} \wedge \text{ClearExceptionLogPurseEafromOkay} \end{aligned}$$

□

5.7 Operations at the world level only

5.7.1 Exception Log clear authorisation

Original schema 5.20 (Between world authorised exception log clear operation)

$ \begin{array}{l} \text{AuthoriseExLogClearOkay} \\ \hline \Delta \text{BetweenWorld} \\ m?, m! : \text{MESSAGE} \\ \text{name?} : \text{NAMES} \\ \hline \text{conAuthPurse}' = \text{conAuthPurse} \\ \exists pds : \mathbb{P}_1 \text{ PayDetails} \bullet \\ \quad \{ \text{name?} \} \times pds \subseteq \text{archive} \\ \quad \wedge m! = \text{exceptionLogClear}(\text{name?}, \text{image } pds) \\ \text{ether}' = \text{ether} \cup \{ m! \} \\ \text{archive} = \text{archive}' \end{array} $
--

□

Original schema 5.21 (Authorised between world log clear operation)

$$\text{AuthoriseExLogClear} \hat{=} \text{Ignore} \vee \text{AuthoriseExLogClearOkay}$$

□

Original schema 5.22 (Between world exception log archiving operation)

$ \begin{array}{l} \text{Archive} \\ \hline \Delta \text{BetweenWorld} \\ m?, m! : \text{MESSAGE} \\ \text{name?} : \text{NAMES} \\ \hline \text{conAuthPurse}' = \text{conAuthPurse} \\ \text{ether}' = \text{ether} \\ \text{archive} \subseteq \\ \quad \text{archive}' \subseteq \\ \quad \quad \text{archive} \cup \{ \text{log} : \text{NAMES} \times \text{PayDetails} \mid \\ \quad \quad \quad \text{exceptionLogResult } \text{log} \in \text{ether} \} \\ m! = \perp \end{array} $

□

5.8 The complete protocol

Original schema 5.23 (Complete transfer protocol between concrete levels)

$$\text{CompleteProtocol} \hat{=} \text{StartFrom} \circ \text{StartTo} \circ \text{Req} \circ \text{Val} \circ \text{Ack}$$

□

5.9 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	1	2
Generic definitions	0	0
Schemas	23	71
Total	24	79

Table 5.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	0
Theorems	2	2
Total	2	2

Table 5.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	8	12
Schema	20	20
Total	28	32

Table 5.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	16	20
Theorems	0	1
Axiomatic consistency	1	5
Total	17	26

Table 5.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	2	3
Assumption rules	2	14
Rewriting rules	0	13
Forward rules	2	14
Total	6	44

Table 5.5: Summary of *Z/Eves* rulees

Chapter 6

β model, initialisation and finalisation

6.1 Summing values

Originally from Appendix D, p.226.

Original axiomatic Definition 6.1 (Summing values from purse payments)

$$\begin{array}{l}
 \text{---} \\
 \text{sumValue} : \mathbb{F} \text{ PayDetails} \rightarrow \mathbb{N} \\
 \text{---} \\
 \langle\langle \text{rule } d\text{SumValueBase} \rangle\rangle \\
 \text{sumValue } \{ \} = 0 \\
 \langle\langle \text{rule } d\text{SumValueInduc} \rangle\rangle \\
 \forall pds : \mathbb{F} \text{ PayDetails}; \text{ PayDetails} \mid \theta \text{ PayDetails} \notin pds \bullet \\
 \quad \text{sumValue } (\{ \theta \text{ PayDetails} \} \cup pds) = \\
 \quad \quad \text{value} + \text{sumValue } pds
 \end{array}$$

□

6.2 Initialisation

Original schema 6.1 (Between world initial state)

$$\begin{array}{l}
 \text{---} \\
 \text{BetweenInitState} \\
 \text{BetweenWorld'} \\
 \text{---} \\
 \{ \text{readExceptionLog}, \perp \} \\
 \cup \\
 \bigcup \{ \text{cpd} : \text{CounterPartyDetails} \bullet \{ \text{startFrom cpd}, \text{startTo cpd} \} \} \\
 \subseteq \text{ether'}
 \end{array}$$

□

Automation for BetwInitIn

These rules are similar to those found in 4.7 for *CounterPartyDetails*.

Forward rule 6.1 (Payment details from purse type)

$$\begin{array}{l}
 \text{theorem frule fPayDetailsFromType} \\
 x \in \text{PayDetails} \Rightarrow x.\text{from} \in \text{NAMES}
 \end{array}$$

□

Forward rule 6.2 (Payment details to purse type)

theorem frule fPayDetailsToType
 $x \in \text{PayDetails} \Rightarrow x.to \in \text{NAMES}$

□

Forward rule 6.3 (Payment details value type)

theorem frule fPayDetailsValueType
 $x \in \text{PayDetails} \Rightarrow x.value \in \text{NAT}$

□

These rules are similar to those found in 4.7 for *startFrom* and *startTo*, but for schema *CounterPartyDetails* rather than *PayDetails*. Nevertheless, as *PayDetails* has an invariant, we cannot have an *frule* like *fCounterPartyDetailsInSetMember*. Thus, the proofs for the *grules* is slightly different at the end. Also, we need to include the relationship between *req* and *PayDetails* explicitly.

Forward rule 6.4 (PayDetails binding type)

theorem frule fPayDetailsMember
 $x \in \text{PayDetails} \Rightarrow x \in \langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{N};$
 $\text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle$

□

Assumption rule 6.1 (req MESSAGE relational type)

theorem grule gMESSAGEReqRelType
 $\text{req} \in \langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{N};$
 $\text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle \leftrightarrow \text{MESSAGE}$

□

Assumption rule 6.2 (req MESSAGE partial function type)

theorem grule gMESSAGEReqPFunType
 $\text{req} \in \langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{N};$
 $\text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle \rightarrow \text{MESSAGE}$

□

Assumption rule 6.3 (req MESSAGE relational non-maximal type)

theorem grule gMESSAGEReqPayDetailsRelType
 $\text{req} \in \text{PayDetails} \leftrightarrow \text{MESSAGE}$

□

Assumption rule 6.4 (req MESSAGE partial injection type)

theorem grule gMESSAGEReqPInjType
 $\text{req} \in \langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{N};$
 $\text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle \rightarrow \text{MESSAGE}$

□

The next rule is necessary to tell **Z/Eves** about the (non-maximal) type of the result of applying the given message to the inverse of the *req* message. Although this result is trivially obvious, it is quite hard to prove. A simple suggestion around this, would be to give a variable (with declared type) named *v* as

$$v \in \text{PayDetails} \wedge v = \text{req}^\sim$$

Rewriting rule 6.1 (req MESSAGE inverse maximal type)

theorem rule rMESSAGEReqInvMaxType
 $\forall m : \text{MESSAGE} \mid m \in \text{ran req} \bullet \text{req}^\sim m \in$
 $\langle \text{from} : \text{NAMES}; \text{fromSeqNo} : \text{NAT}; \text{to} : \text{NAMES}; \text{toSeqNo} : \text{NAT}; \text{value} : \text{NAT} \rangle$

□

BetwInitIn

In this schema the use of $(req \sim m?)$ directly is a bad idea for **Z/Eves** automation, as it incurs rather complex lemmas. To avoid this, one could simply declare a variable to hold such value. In this way, **Z/Eves** would know the type of this expression, the main problem appearing in the domain check. Nevertheless, for the sake of keeping to the original as much as possible, we left it unchanged.

Original schema 6.2 (Between world initialisation input)

$ \begin{array}{l} \text{BetwInitIn} \\ \hline g? : AIN \\ \\ m? : MESSAGE \\ name? : NAMES \\ \hline m? \in \text{ran } req \Rightarrow \\ \quad g? = \text{transfer}(\mu \text{ TransferDetails} \mid \\ \quad \quad from = ((req \sim m?).from \\ \quad \quad \wedge to = ((req \sim m?).to \\ \quad \quad \wedge value = ((req \sim m?).value) \\ \\ m? \notin \text{ran } req \Rightarrow g? = aNullIn \end{array} $
--

□

6.3 Finalisation

Original schema 6.3 (Between world finalisation state)

$ \begin{array}{l} \text{BetwFinState} \\ \hline \text{BetweenWorld} \\ \text{GlobalWorld} \\ \hline \text{dom } gAuthPurse = \text{dom } conAuthPurse \\ \forall name : \text{dom } conAuthPurse \bullet \\ \quad (gAuthPurse \text{ name}).balance = (conAuthPurse \text{ name}).balance \\ \quad \wedge (gAuthPurse \text{ name}).lost = \\ \quad \quad \text{sumValue}((\text{definitelyLost} \cup \text{maybeLost}) \\ \quad \quad \cap \{ld : \text{PayDetails} \mid ld.from = name\}) \end{array} $
--

□

Original schema 6.4 (Between world finalisation output)

$ \begin{array}{l} \text{BetwFinOut} \\ \hline g! : AOUT \\ m! : MESSAGE \\ \hline g! = aNullOut \end{array} $

□

6.4 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	1	3
Generic definitions	0	0
Schemas	4	75
Total	5	84

Table 6.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	0
Theorems	0	2
Total	0	2

Table 6.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	20
Total	0	32

Table 6.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	20
Theorems	0	1
Axiomatic consistency	0	5
Total	0	26

Table 6.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	3
Assumption rules	4	18
Rewriting rules	1	14
Forward rules	4	18
Total	9	53

Table 6.5: Summary of *Z/Eves* rulees

Chapter 7

$\mathcal{C}$ model: implementation

7.1 Concrete World State

The $\mathcal{C}$ world state has the same components as the $\mathcal{B}$ state; we decorate with a subscript zero to distinguish like-named $\mathcal{B}$ and $\mathcal{C}$ components.

Since $\Delta \text{ConWorld}_0$ has components dashed-then-subscripted, whereas we require subscripted-then-dashed, Mondex defines their own Δ and Ξ schemas. as

$$\begin{aligned}\Delta \text{ConWorld}_0 &\triangleq \text{ConWorld}_0 \wedge \text{ConWorld}_0' \\ \Xi \text{ConWorld}_0 &\triangleq [\Delta \text{ConWorld}_0 \mid \theta \text{ConWorld}_0 = \theta \text{ConWorld}_0']\end{aligned}$$

Nevertheless, there are two curious subtleties regarding this definition: (i) an oddity in the Z standard; and (ii) tool limitation. The curiosity in the Z standard is that in L^AT_EX $\mathbf{x}_0'$ (x_0') typesets exactly the same as $\mathbf{x}'_0$. For this there is an easy solution: $\mathbf{x}_0\sim'$ (x_0'). On the other hand, for **Z/Eves**, when decorated names are used in the way we need (x_0'), the pre expression does not work as expected, as it does not recognised that all subscribed-primed components need to be existentially quantified. This introduces a complication in the precondition calculation in Chapter 8 for all $\mathcal{C}$ model operations. Therefore, we decided to use variable renaming instead of subscripting here. It would be possible to keep the original, but this would incur in error prone preconditions done by hand for Chapter 8 and all refinement proofs. In the end, there is no semantic difference as this is just a “syntactic cosmetic surgery”.

Added schema 7.1 (Renamed *ConWorld* for Chapter 7)

$$\text{ConWorldCh7} \triangleq \text{ConWorld}[\text{conAuthPurseCh7}/\text{conAuthPurse}, \text{etherCh7}/\text{ether}, \text{archiveCh7}/\text{archive}]$$

□

7.2 Framing Schema

Original schema 7.1 (Concrete world promotion)

PhiCOp $\Delta \text{ConWorldCh7}$ $\Delta \text{ConPurse}$ $m?, m! : \text{MESSAGE}$ $\text{name?} : \text{NAMES}$
$m? \in \text{etherCh7}$ $\text{name?} \in \text{dom } \text{conAuthPurseCh7}$ $\theta \text{ConPurse} = \text{conAuthPurseCh7 } \text{name?}$ $\text{conAuthPurseCh7}' = \text{conAuthPurseCh7} \oplus \{ \text{name?} \mapsto \theta \text{ConPurse}' \}$ $\text{archiveCh7}' = \text{archiveCh7}$ $\text{etherCh7}' \subseteq \text{etherCh7} \cup \{ m! \}$

□

7.3 Ignore, Increase and Abort

In the original the next declarations were given in wrong $\text{\LaTeX}$ format. It did not followed Spivey's grammar which enforces one declaration per unboxed paragraph. That means we need a $\text{\texttt{begin\{zed\} end\{zed\}}}$ environment for each of definition, rather than one for all definitions.

Original schema 7.2 (Concrete world promoted ignore operation)

$$CIgnore \hat{=} [\exists ConWorldCh7; name? : NAMES; m?, m! : MESSAGE \mid m! = \perp]$$

□

Original schema 7.3 (Concrete world promoted increase operation)

$$CIncrease \hat{=} CIgnore \vee (\exists \Delta ConPurse \bullet PhiCOp \wedge IncreasePurseOkay)$$

□

Original schema 7.4 (Concrete world promoted abort operation)

$$CAbort \hat{=} CIgnore \vee (\exists \Delta ConPurse \bullet AbortPurseOkay \wedge [PhiCOp \mid m! = \perp])$$

□

7.4 Promoted operations

7.4.1 Value transfer operations

Original schema 7.5 (*ConWorld* promoted *startFrom* operation)

$$CStartFrom \hat{=} CIgnore \vee CAbort \vee (\exists \Delta ConPurse \bullet PhiCOp \wedge StartFromPurseOkay)$$

□

Original schema 7.6 (*ConWorld* promoted *startTo* operation)

$$CStartTo \hat{=} CIgnore \vee CAbort \vee (\exists \Delta ConPurse \bullet PhiCOp \wedge StartToPurseOkay)$$

□

Same $\text{\LaTeX}$ unboxed environment problem sorted here.

Original schema 7.7 (*ConWorld* promoted request operation)

$$CReq \hat{=} CIgnore \vee (\exists \Delta ConPurse \bullet PhiCOp \wedge ReqPurseOkay)$$

□

Original schema 7.8 (*ConWorld* promoted value transfer operation)

$$CVal \hat{=} CIgnore \vee (\exists \Delta ConPurse \bullet PhiCOp \wedge ValPurseOkay)$$

□

Original schema 7.9 (*ConWorld* promoted acknowledgement operation)

$$CAck \hat{=} CIgnore \vee (\exists \Delta ConPurse \bullet PhiCOp \wedge AckPurseOkay)$$

□

7.4.2 Exception log operations

Original schema 7.10 (*ConWorld* promoted read exception log operation)

$$\begin{aligned} CReadExceptionLog &\hat{=} CIgnore \\ &\vee (\exists \Delta ConPurse \bullet PhiCOp \wedge ReadExceptionLogPurseOkay) \end{aligned}$$

□

Original schema 7.11 (*ConWorld* promoted clear exception log operation)

$$\begin{aligned} CClearExceptionLog &\hat{=} CIgnore \\ &\vee CAbort \\ &\vee (\exists \Delta ConPurse \bullet PhiCOp \wedge ClearExceptionLogPurseOkay) \end{aligned}$$

□

7.5 Operations at the world level only

7.5.1 Exception Log clear authorisation

Added schema 7.2 (*ConWorld* promoted exception log clear)

$$\begin{aligned} CPhiCOpExpLogClearNEW &\hat{=} [PhiCOp \mid (\exists pds : \mathbb{P}_1 PayDetails \mid \\ &\quad \{ name? \} \times pds \subseteq archiveCh7 \bullet \\ &\quad m! = exceptionLogClear(name?, image pds))] \end{aligned}$$

□

Original schema 7.12 (*ConWorld* promoted authorise exception log clear)

$$CAuthoriseExLogClear \hat{=} CIgnore \vee (\exists \exists ConPurse \bullet CPhiCOpExpLogClearNEW)$$

□

The same L^AT_EX problems for subscripted names occurs here.

Original schema 7.13 (Exception log archiving operation)

$\begin{aligned} &CArchive \\ &\Delta ConWorldCh7 \\ &m?, m! : MESSAGE \\ &name? : NAMES \end{aligned}$ $\begin{aligned} &conAuthPurseCh7' = conAuthPurseCh7 \\ ðerCh7' \subseteq etherCh7 \\ &archiveCh7 \subseteq \\ &\quad archiveCh7' \subseteq \\ &\quad \quad archiveCh7' \cup \{ log : NAMES \times PayDetails \mid \\ &\quad \quad \quad exceptionLogResult log \in etherCh7 \} \\ &m! = \perp \end{aligned}$
--

□

7.6 Initial state

The same L^AT_EX problems for subscripted names occurs here.

Original schema 7.14 (*ConWorld* initialisation state)

$\frac{\text{ConInitState}}{\text{ConWorldCh7'}}$
$\begin{aligned} &\exists \text{BetweenWorld'} \mid \text{BetweenInitState} \bullet \\ &\quad \text{conAuthPurseCh7'} = \text{conAuthPurse'} \\ &\quad \wedge \text{archiveCh7'} = \text{archive'} \\ &\quad \wedge \{ \perp \} \subseteq \text{etherCh7'} \subseteq \text{ether'} \end{aligned}$

□

7.7 Finalisation

In here the same finiteness problem that occurs in *BetwFinState* (Section 6.3 on page 49) (Section 6.2 on page 53 on original document).

Added schema 7.3 (Renamed *AuxWorld* for Chapter 7)

$$\begin{aligned} \text{AuxWorldCh7} \triangleq & \text{AuxWorld}[\text{conAuthPurseCh7}/\text{conAuthPurse}, \\ & \text{etherCh7}/\text{ether}, \text{archiveCh7}/\text{archive}, \\ & \text{allLogsCh7}/\text{allLogs}, \text{authenticFromCh7}/\text{authenticFrom}, \\ & \text{authenticToCh7}/\text{authenticTo}, \text{fromLoggedCh7}/\text{fromLogged}, \\ & \text{toLoggedCh7}/\text{toLogged}, \text{toInEprCh7}/\text{toInEpr}, \\ & \text{toInEapayeeCh7}/\text{toInEapayee}, \text{fromInEprCh7}/\text{fromInEpr}, \\ & \text{fromInEpaCh7}/\text{fromInEpa}, \text{definitelyLostCh7}/\text{definitelyLost}, \\ & \text{maybeLostCh7}/\text{maybeLost}] \end{aligned}$$

□

Original schema 7.15 (*ConWorld* finalisation state)

$\frac{\text{ConFinState}}{\text{AuxWorldCh7} \quad \text{GlobalWorld}}$
$\text{dom } g\text{AuthPurse} = \text{dom } \text{conAuthPurseCh7}$ Obvious assumption that the set comprehension below in the set intersection is finite. $\begin{aligned} \forall \text{name} : \text{dom } \text{conAuthPurseCh7} \bullet & \quad (g\text{AuthPurse } \text{name}).\text{balance} \\ & = (\text{conAuthPurseCh7 } \text{name}).\text{balance} \\ \wedge (g\text{AuthPurse } \text{name}).\text{lost} = & \quad \text{sumValue}((\text{definitelyLostCh7} \cup \text{maybeLostCh7}) \\ & \quad \cap \{ \text{ld} : \text{PayDetails} \mid \text{ld.from} = \text{name} \}) \end{aligned}$

□

7.8 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	15	90
Total	15	99

Table 7.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	0
Theorems	0	2
Total	0	2

Table 7.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	3	23
Total	3	35

Table 7.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	20
Theorems	0	1
Axiomatic consistency	0	5
Total	0	26

Table 7.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	3
Assumption rules	0	18
Rewriting rules	0	14
Forward rules	0	18
Total	0	53

Table 7.5: Summary of *Z/Eves* rulees

Chapter 8

Model consistency proofs

We have included all the missing precondition calculations for all operations from Chapter 3 to Chapter 7. Those not included, for instance, *ConWorld* or *ConPurse*, is because they are just *true*.

8.1 Introduction

Due to the use of *NAT* in *ConPurse*, we add an additional (obvious) forward here for automation purposes

Forward rule 8.1 (*ConPurse nextSeqNo maximal type*)

theorem frule fConPurseNextSeqNoMaxType
 $\forall \text{ConPurse} \bullet \text{nextSeqNo} \in \mathbb{Z}$

□

8.2 $\mathcal{A}$ model consistency proofs

8.2.1 Existence of initial abstract state

Original theorem 8.1 (Existence of initial abstract state)

theorem tExistsInitialAbWorld
 $\exists \text{AbWorld}' \bullet \text{AbInitState}$

□

8.2.2 Totality of abstract operations

There are the operations from Chapter 3. For the operations to be total, we must assume not only the abstract operations signature.

Added schema 8.1 ($\mathcal{A}$ operation signature)

$\text{AbOpSig} \triangleq [\text{AbWorld}; a? : \text{AIN}]$

□

Added theorem 8.1 ($\mathcal{A}$ operation)

theorem tAbOpIsTotal
 $\forall \text{AbOpSig} \bullet \text{pre AbOp}$

□

Original theorem 8.2 ($\mathcal{A}$ ignore operation is total)

theorem tAbIgnoreIsTotal
 $\forall AbOpSig \bullet \text{pre } AbIgnore$

□

AbTransfer is total because of the disjunction with *AbIgnore*

Original theorem 8.3 ($\mathcal{A}$ transfer operation is total)

theorem tAbTransferIsTotal
 $\forall AbOpSig \bullet \text{pre } AbTransfer$

□

Nothing is in the purse about including the preconditions of the inner operations from *AbTransfer*.

$\text{pre } AbTransferOkay \wedge \text{pre } AbTransferOkayTD$
 $\text{pre } AbTransferLost \wedge \text{pre } AbTransferLostTD$

We include then here as well. For that we need to include the signature of secure operations as

Added schema 8.2 ($\mathcal{A}$ secure operations signature)

$AbWorldSecureOpSig \hat{=} [AbOpSig; TransferDetails?]$

□

Added theorem 8.2 ($\mathcal{A}$ secure operation precondition)

theorem tAbWorldSecureOpPRE
 $\forall AbWorldSecureOpSig \mid a? \in \text{ran } transfer \wedge$
 $\theta TransferDetails? = (transfer \sim) a? \bullet \text{pre } AbWorldSecureOp$

□

With the precondition for world secure operations, we can now tackle the inner transfer operations from the abstract world. Firstly, let us define the minimal signature for *AbTransferXXX* operations below.

Added schema 8.3 ($\mathcal{A}$ secure transfer operations signature)

$AbTransferOpSig$ $AbWorldSecureOpSig$
$a? \in \text{ran } transfer$ $\theta TransferDetails? = (transfer \sim) a?$

□

After that, in order to prove that

$\forall AbTransferOpSig \bullet \text{pre } AbTransferOkayTD$

a series of lemmas are required. They appear as negated assumptions in the goal after appropriate instantiation of *abAuthPurse'*.

Added lemma 8.1 (Abstract transfer okay PRE *abAuthPurse* maximal type)

theorem rule lAbTransferOkayPREAuthPurseUpdateMaxType
 $\forall AbWorld; TransferDetails? \mid$
 $Authentic[from?/name?]$
 $\wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty$
 $\wedge \neg from? = to? \bullet$
 $(\{(from?, \theta AbPurse[balance := -1 * value? + (abAuthPurse from?).balance,$
 $lost := (abAuthPurse from?).lost])\}$
 $\cup \{(to?, \theta AbPurse[balance := value? + (abAuthPurse to?).balance,$
 $lost := (abAuthPurse to?).lost])\}) \in$
 $\mathbb{P}(NAME \times \langle balance : \mathbb{Z}; lost : \mathbb{Z} \rangle)$

□

Added lemma 8.2 (Abstract transfer okay PRE $abAuthPurse$ maximal type)

theorem rule lAbTransferOkayPREAuthPurseMaxType
 $\forall AbWorld; TransferDetails? \mid$
 $Authentic[from?/name?]$
 $\wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty$
 $\wedge \neg from? = to? \bullet$
 $abAuthPurse \oplus$
 $(\{(from?, \theta AbPurse[balance := -1 * value? + (abAuthPurse from?).balance,$
 $lost := (abAuthPurse from?).lost])\}$
 $\cup \{(to?, \theta AbPurse[balance := value? + (abAuthPurse to?).balance,$
 $lost := (abAuthPurse to?).lost])\}) \in$
 $\mathbb{P}(NAME \times \langle balance : \mathbb{Z}; lost : \mathbb{Z} \rangle)$

□

Added lemma 8.3 (Abstract transfer okay PRE $AbWorld$ type)

theorem rule lAbTransferOkayPREAbWorld
 $\forall AbWorld; TransferDetails? \mid$
 $Authentic[from?/name?]$
 $\wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty$
 $\wedge \neg from? = to? \bullet$
 $AbWorld[abAuthPurse := abAuthPurse \oplus$
 $(\{(from?, \theta AbPurse[balance := -1 * value? +$
 $(abAuthPurse from?).balance,$
 $lost := (abAuthPurse from?).lost])\}$
 $\cup \{(to?, \theta AbPurse[balance := value? +$
 $(abAuthPurse to?).balance,$
 $lost := (abAuthPurse to?).lost])\})]$

□

Added lemma 8.4 (Abstract transfer okay PRE $\Leftarrow$ equivalence)

theorem rule lAbTransferOkayPRENDResEquiv
 $\forall AbWorld; TransferDetails? \mid Authentic[from?/name?]$
 $\wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty$
 $\wedge \neg from? = to? \bullet$
 $(\{from?\} \cup \{to?\}) \Leftarrow (abAuthPurse \oplus$
 $(\{(from?, \theta AbPurse[balance := -1 * value? +$
 $(abAuthPurse from?).balance, lost := (abAuthPurse from?).lost])\}$
 $\cup \{(to?, \theta AbPurse[balance := value? +$
 $(abAuthPurse to?).balance, lost := (abAuthPurse to?).lost])\}))$
 $= (\{from?\} \cup \{to?\}) \Leftarrow abAuthPurse$

□

Added lemma 8.5 (Abstract transfer okay PRE *Authentic*)

theorem lAbTransferOkayPREAuthentic
 $\forall AbWorld; TransferDetails? \mid$
 $Authentic[from?/name?]$
 $\wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty$
 $\wedge \neg from? = to? \bullet$
 $Authentic[abAuthPurse := abAuthPurse \oplus$
 $(\{(from?, \theta AbPurse[balance := \bar{1} * value?$
 $+ (abAuthPurse from?).balance,$
 $lost := (abAuthPurse from?).lost\})$
 $\cup \{(to?, \theta AbPurse[balance := value? +$
 $(abAuthPurse to?).balance,$
 $lost := (abAuthPurse to?).lost\})\}, from?/name?]$
 $\wedge Authentic[abAuthPurse := abAuthPurse \oplus$
 $(\{(from?, \theta AbPurse[balance := \bar{1} * value?$
 $+ (abAuthPurse from?).balance,$
 $lost := (abAuthPurse from?).lost\})$
 $\cup \{(to?, \theta AbPurse[balance := value? +$
 $(abAuthPurse to?).balance,$
 $lost := (abAuthPurse to?).lost\})\}, to?/name?]$

□

Added lemma 8.6 (Abstract transfer okay PRE *from? $\oplus$ equivalence*)

theorem lAbTransferOkayPREFromOPlusEquiv
 $\forall AbWorld; TransferDetails? \mid Authentic[from?/name?]$
 $\wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty$
 $\wedge \neg from? = to? \bullet$
 $(abAuthPurse \oplus (\{(from?,$
 $\theta AbPurse[balance := -1 * value? +$
 $(abAuthPurse from?).balance,$
 $lost := (abAuthPurse from?).lost\})$
 $\cup \{(to?, \theta AbPurse[balance := value? +$
 $(abAuthPurse to?).balance,$
 $lost := (abAuthPurse to?).lost\})\})) from? =$
 $\theta AbPurse[balance := -1 * value? +$
 $(abAuthPurse from?).balance,$
 $lost := (abAuthPurse from?).lost]$

□

Added lemma 8.7 (Abstract transfer okay PRE *to? $\oplus$ equivalence*)

theorem lAbTransferOkayPREToOPlusEquiv
 $\forall AbWorld; TransferDetails? \mid Authentic[from?/name?]$
 $\wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty$
 $\wedge \neg from? = to? \bullet$
 $(abAuthPurse \oplus (\{(from?,$
 $\theta AbPurse[balance := -1 * value? +$
 $(abAuthPurse from?).balance,$
 $lost := (abAuthPurse from?).lost\})$
 $\cup \{(to?, \theta AbPurse[balance := value? +$
 $(abAuthPurse to?).balance,$
 $lost := (abAuthPurse to?).lost\})\})) to? =$
 $\theta AbPurse[balance := value? +$
 $(abAuthPurse to?).balance,$
 $lost := (abAuthPurse to?).lost]$

□

Finally, by tackling those lemmas separately, we could come to the final precondition as

Added theorem 8.3 (Abstract transfer okay TD precondition)

theorem tAbTransferOkayTDPRE
 $\forall AbTransferOpSig \mid Authentic[from?/name?]$
 $\wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty$
 $\wedge from? \neq to? \bullet$
 $\text{pre } AbTransferOkayTD$

□

For the *AbTransferOkayTD* the calculation is similar as we just hide some elements. At last, we do the same for the *AbTransferLostTD* operation, which again similar to *AbTransferOkayTD*, but slightly simpler to prove.

Added theorem 8.4 (Abstract transfer okay TD precondition)

theorem tAbTransferLostTDPRE
 $\forall AbTransferOpSig \mid Authentic[from?/name?] \wedge Authentic[to?/name?]$
 $\wedge SufficientFundsProperty \wedge from? \neq to? \bullet \text{pre } AbTransferLostTD$

□

8.3 Totality of between model for purses

These are the operations from Chapter 4. Firstly, let us define the signature.

Added schema 8.4 ($\mathcal{B}$ purse operations signature)

$BetwPurseOpSig \hat{=} [ConPurse; m? : MESSAGE]$

□

Added theorem 8.5 (Increase purse okay is total)

theorem tIncreasePurseOkayIsTotal
 $\forall BetwPurseOpSig \bullet \text{pre } IncreasePurseOkay$

□

The number of cases in this proof is impressive. To make it worst, the tuple selection on *pd.from* and *pd.to* raises some trouble for **Z/Eves** at some predicates. This lead to a quite complex (surgical) choice of case analysis.

Added theorem 8.6 (Abort purse okay is total)

theorem tAbortPurseOkayIsTotal
 $\forall BetwPurseOpSig \mid name \in \{pdAuth.from, pdAuth.to\} \bullet \text{pre } AbortPurseOkay$

□

As a result, we found out that the operation is not total as expected, and conditions upon the authenticated payment details are required for abortion!

8.4 Precondition of between model purses (Chapter 4)

Simple operations

Added theorem 8.7 (Request purse precondition)

theorem tReqPurseOkayPRE
 $\forall \text{BetwPurseOpSig} \mid \text{status} = \text{epr}$
 $\wedge \text{AuthenticReqMessage} \bullet \text{pre ReqPurseOkay}$

□

Added theorem 8.8 (Value purse precondition)

theorem tValPurseOkayPRE
 $\forall \text{BetwPurseOpSig} \mid \text{status} = \text{epv}$
 $\wedge \text{AuthenticValMessage} \bullet \text{pre ValPurseOkay}$

□

Added theorem 8.9 (Value purse precondition)

theorem tAckPurseOkayPRE
 $\forall \text{BetwPurseOpSig} \mid \text{status} = \text{epa}$
 $\wedge \text{AuthenticAckMessage} \bullet \text{pre AckPurseOkay}$

□

Added theorem 8.10 (Read exception log purse precondition)

theorem tReadExceptionLogPurseEafromOkayPRE
 $\forall \text{BetwPurseOpSig} \mid \text{status} = \text{eaFrom}$
 $\wedge m? = \text{readExceptionLog} \bullet$
 $\text{pre ReadExceptionLogPurseEafromOkay}$

□

Added theorem 8.11 (Clear exception log purse precondition)

theorem tClearExceptionLogPurseEafromOkayPRE
 $\forall \text{BetwPurseOpSig} \mid \text{status} = \text{eaFrom} \wedge \text{exLog} \neq \emptyset$
 $\wedge m? = \text{exceptionLogClear}(\text{name}, \text{image exLog}) \bullet$
 $\text{pre ClearExceptionLogPurseEafromOkay}$

□

Added theorem 8.12 (Complete read exception log precondition)

theorem tReadExceptionLogEafromOkayPRE
 $\forall \text{BetwPurseOpSig} \mid m? = \text{readExceptionLog}$
 $\wedge \text{name} \in \{ \text{pdAuth.from}, \text{pdAuth.to} \} \bullet$
 $\text{pre ReadExceptionLogPurseOkay}$

Added theorem 8.13 (Complete clear exception log precondition)

theorem tClearExceptionLogEafromOkayPRE
 $\forall \text{BetwLog} \mid \text{exLog} \neq \emptyset$
 $\wedge m? = \text{exceptionLogClear}(\text{name},$
 $\quad \text{image}(\text{exLog} \cup (\text{if status} \in \{ \text{epv}, \text{epa} \}$
 $\quad \text{then } \{ \text{pdAuth} \} \text{ else } \emptyset)))$
 $\wedge \text{name} \in \{ \text{pdAuth.from}, \text{pdAuth.to} \}$
 $\bullet \text{pre ClearExceptionLogPurseOkay}$

Start (which are complex) operations

Added schema 8.5 ($\mathcal{B}$ purse start operations signature)

$$BetwPurseStartOpSig \hat{=} [BetwPurseOpSig; cpd : CounterPartyDetails]$$

□

Automation for *StartFromPurseEafromOkay* and *StartToPurseEafromOkay*

These automation rules are tricky to prove because support for definite description is rather poor (and badly documented) in **Z/Eves**. Luckily, because the definite descriptions are given pointwise, we can rewrite them as the corresponding binding of the θ expressions the pointwise μ represents. This is useful because **Z/Eves** have better automation support for bindings. Without this next lemma, the proofs involving definite description would not be proved. Moreover, because it is defined as an automatic rewriting rule (i.e., fired upon *rewrite* tactics). Therefore, the equality substitution takes place automatically, and pointwise μ proofs are amenable for full automation.

Rewriting rule 8.1 (μ - θ *PayDetails* expressions equivalence in *startFrom*)

theorem rule lStartFromMuPayDetailsEquivalence

$$\begin{aligned} &\forall name : NAMES; nextSeqNo : NAT; cpd : CounterPartyDetails \mid name \neq cpd.name \bullet \\ &(\mu m : \{PayDetails \mid \\ &\quad from = name \\ &\quad \wedge to = cpd.name \\ &\quad \wedge value = cpd.value \\ &\quad \wedge fromSeqNo = nextSeqNo \\ &\quad \wedge toSeqNo = cpd.nextSeqNo\}) = \\ &\quad \theta PayDetails[from := name, to := cpd.name, value := cpd.value, \\ &\quad \quad fromSeqNo := nextSeqNo, toSeqNo := cpd.nextSeqNo] \end{aligned}$$

□

It is hard because definite description have poor automation (and related laws) in general. Both in the Z standard (Sections 13.2.6.12 for the type rules and 15.2.5.8 for the semantics of μ) and its mathematical toolkit (Section B.8.3 for definition of $\mathbb{F}$ using μ), and Spivey's reference manual (p.111), offer no law that could aid automation of μ . The only possible sources we know are the **Z/Eves** Z toolkit, and some natural deduction laws from the CadiZ theorem prover and Woodcock's Z book (p.53).

Rewriting rule 8.2 (μ - θ *PayDetails* expressions equivalence in *startTo*)

theorem rule lStartToMuPayDetailsEquivalence

$$\begin{aligned} &\forall name : NAMES; nextSeqNo : NAT; cpd : CounterPartyDetails \mid name \neq cpd.name \bullet \\ &(\mu m : \{PayDetails \mid \\ &\quad to = name \\ &\quad \wedge from = cpd.name \\ &\quad \wedge value = cpd.value \\ &\quad \wedge toSeqNo = nextSeqNo \\ &\quad \wedge fromSeqNo = cpd.nextSeqNo\}) = \\ &\quad \theta PayDetails[from := cpd.name, to := name, value := cpd.value, \\ &\quad \quad fromSeqNo := cpd.nextSeqNo, toSeqNo := nextSeqNo] \end{aligned}$$

□

Added theorem 8.14 (*StartFromPurseEafromOkay* purse precondition (avoids *Abort*))

theorem tStartFromPurseEafromOkayPRE

$$\begin{aligned} &\forall BetwPurseStartOpSig \mid ValidStartFrom \\ &\quad \wedge status = eaFrom \wedge nextSeqNo < MAX_NAT \bullet \\ &\quad pre StartFromPurseEafromOkay \end{aligned}$$

□

Added theorem 8.15 (*StartToPurseEafromOkay* purse precondition (avoids *Abort*))

theorem tStartToPurseEafromOkayPRE
 $\forall \text{BetwPurseStartOpSig} \mid \text{ValidStartTo}$
 $\wedge \text{status} = \text{eaFrom} \wedge \text{nextSeqNo} < \text{MAX_NAT}$
 $\wedge \text{cpd.nextSeqNo} < \text{nextSeqNo} \bullet \text{pre StartToPurseEafromOkay}$

□

Added theorem 8.16 (*StartFromPurseOkay* precondition (considers *Abort*))

theorem tStartFromEafromOkayPRE
 $\forall \text{BetwPurseStartOpSig} \mid \text{ValidStartFrom}$
 $\wedge \text{name} \in \{ \text{pdAuth.from}, \text{pdAuth.to} \}$
 $\wedge \text{nextSeqNo} < \text{MAX_NAT}$
 $\bullet \text{pre StartFromPurseOkay}$

□

Rewriting rule 8.3 (Request message of *PayDetails* is a message)

theorem rule rReqStartToPayDetailsInMessage
 $\forall \text{name} : \text{NAMES}; \text{nextSeqNo} : \text{NAT}; \text{cpd} : \text{CounterPartyDetails} \mid$
 $\text{name} \neq \text{cpd.name} \bullet \text{req}\theta \text{PayDetails}[\text{from} := \text{cpd.name},$
 $\text{fromSeqNo} := \text{cpd.nextSeqNo}, \text{name}/\text{to},$
 $\text{nextSeqNo}/\text{toSeqNo},$
 $\text{value} := \text{cpd.value}] \in \text{MESSAGE}$

□

Added theorem 8.17 (*StartToPurseOkay* precondition (considers *Abort*))

theorem tStartToEafromOkayPRE
 $\forall \text{BetwPurseStartOpSig} \mid \text{ValidStartTo}$
 $\wedge \text{cpd.nextSeqNo} < \text{nextSeqNo}$
 $\wedge \text{name} \in \{ \text{pdAuth.from}, \text{pdAuth.to} \}$
 $\wedge \text{nextSeqNo} < \text{MAX_NAT}$
 $\bullet \text{pre StartToPurseOkay}$

□

8.5 $\mathcal{B}$ model consistency proofs (Chapter 5)

8.5.1 Existence of between initial state

Original theorem 8.4 (Existence of a between world initial state)

theorem tExistsInitialBetweenWorld
 $\exists \text{BetweenWorld}' \bullet \text{BetweenInitState}$

□

8.5.2 Totality between operations

In this section we divide the proofs according to the order operations appear in Chapter 5. For schemas *ConWorld*, *AuxWorld*, *NewVariables*, and *BetweenWorld* do not need precondition calculation as they do not mention dashed or output variables.

The signature for the precondition proofs contain the state invariant as well as input variable information. That is, the before variables involved in the operations we want to prove the precondition of.

Added schema 8.6 ($\mathcal{B}$ operations signature)

$\text{BetweenOpSig} \triangleq [\text{BetweenWorld}; m? : \text{MESSAGE}; \text{name}? : \text{NAMES}]$

□

Ignore, *Increase* **ad** *Abort*

The next proofs are related to the definitions from Section 5.5.

Added theorem 8.18 ($\mathcal{B}$ ignore operation is total)

theorem tBetwIgnoreIsTotal
 $\forall \textit{BetweenOpSig} \bullet \textit{pre Ignore}$

□

Added theorem 8.19 ($\mathcal{B}$ increase operation is total)

theorem tBetwIncreaseIsTotal
 $\forall \textit{BetweenOpSig} \bullet \textit{pre Increase}$

□

Added theorem 8.20 ($\mathcal{B}$ abort operation is total)

theorem tBetwAbortIsTotal
 $\forall \textit{BetweenOpSig} \bullet \textit{pre Abort}$

□

Promoted operations (with *Ignore/Abort* disjunct/composition)

In this section, we divide the proofs according to the declarations of Section 5.6 as they are given without considering the fact those preconditions are trivially *true* due to the disjunction with *Ignore*.

Value transfer operations

Added theorem 8.21 ($\mathcal{B}$ *startFrom* operation is total)

theorem tBetwStartFromIsTotal
 $\forall \textit{BetweenOpSig} \bullet \textit{pre StartFrom}$

□

Added theorem 8.22 ($\mathcal{B}$ *startTo* operation is total)

theorem tBetwStartToIsTotal
 $\forall \textit{BetweenOpSig} \bullet \textit{pre StartTo}$

□

Added theorem 8.23 ($\mathcal{B}$ request operation is total)

theorem tBetwReqIsTotal
 $\forall \textit{BetweenOpSig} \bullet \textit{pre Req}$

□

Added theorem 8.24 ($\mathcal{B}$ value operation is total)

theorem tBetwValIsTotal
 $\forall \textit{BetweenOpSig} \bullet \textit{pre Val}$

□

Added theorem 8.25 ($\mathcal{B}$ acknowledgement operation is total)

theorem tBetwAckIsTotal
 $\forall \textit{BetweenOpSig} \bullet \textit{pre Ack}$

□

Exception log operations

Added theorem 8.26 (**$\mathcal{B}$** read exception log operation is total)

theorem tBetwReadExceptionLogIsTotal
 $\forall \text{ BetweenOpSig} \bullet \text{pre ReadExceptionLog}$

□

Added theorem 8.27 (**$\mathcal{B}$** clear exception log operation is total)

theorem tBetwClearExceptionLogIsTotal
 $\forall \text{ BetweenOpSig} \bullet \text{pre ClearExceptionLog}$

□

Operations at the world level only

In this section, we divide the proofs according to the declarations of Section 5.7.

Exception Log clear authorisation

AuthoriseExLogClear is trivially true because of the disjunction with *Ignore*.

Added theorem 8.28 (**$\mathcal{B}$** authorisation to clear exception log is total)

theorem tBetwAuthoriseExLogClearIsTotal
 $\forall \text{ BetweenOpSig} \bullet \text{pre AuthoriseExLogClear}$

□

Added theorem 8.29 (**$\mathcal{B}$** log archiving is total)

theorem tBetwArchiveIsTotal
 $\forall \text{ BetweenOpSig} \bullet \text{pre Archive}$

□

8.5.3 Promoted operations without *Ignore/Abort*

In this section we consider the promoted operations precondition without taking into account the disjunction with *Ignore*. This exposes interesting facts about the requirements for promoted operations, which are not present in the PRG.

Instead, they are mentioned/explained in Chapter 14 in Section 14.3. As we need them for the precondition proofs, we must introduce here the definitions that were originally (mentioned but not given) there.

Theory about functional overriding

Leave general theorems disabled to avoid conflict (or loops) with other proofs. On the other hand, leave the specific theorem enabled to increase the levels of automation.

Toolkit extension 8.1 (Partial function element)

theorem disabled rule rPFunElement $[X, Y]$
 $\forall f : X \rightarrowtail Y; x : X; y : Y \mid x \in \text{dom } f \wedge y = f x \bullet (x, y) \in f$

□

Toolkit extension 8.2 (Overridden partial function containment)

theorem disabled rule rPFunSubsetOplusRel $[X, Y]$
 $\forall f, g : X \rightarrowtail Y \mid g \subseteq f \bullet f \oplus g = f \oplus (\text{dom } g \triangleleft f)$

□

Toolkit extension 8.3 (Overriden partial function pointwise equivalence)

theorem lPFunSubsetOplusUnitRel $[X, Y]$
 $\forall f : X \rightarrowtail Y; x : X; y : Y \mid x \in \text{dom } f \wedge y = f x \bullet f = f \oplus \{(x \mapsto y)\}$

□

Framing schema: *PhiBOP*

This proofs are related to the definitions from Section 5.4. Nothing is said about totality of the promotion schema for the between world. This in fact is quite a hard proof.

Added schema 8.7 (Promoted between world operations signature)

$$PhiBOPSig \hat{=} BetweenOpSig \wedge ConPurse$$

□

Added lemma 8.8 (Promoted *AuxWorld* type)

theorem lPromotedAuxWorldType

$$\begin{aligned} &\forall AuxWorld; ConPurse; m! : MESSAGE \mid name? \in \text{dom } conAuthPurse \\ &\quad \wedge \theta ConPurse = conAuthPurse \text{ name? } \bullet \\ &\quad AuxWorld[conAuthPurse := conAuthPurse \oplus \{ (name?, \theta ConPurse) \}, \\ &\quad \quad ether := ether \cup \{ m! \}] \end{aligned}$$

□

Added lemma 8.9 (Promoted *BetweenWorld* type)

theorem lPromotedBetweenWorldType

$$\begin{aligned} &\forall PhiBOPSig; m! : MESSAGE \mid m! \in ether \wedge \\ &\quad name? \in \text{dom } conAuthPurse \wedge \theta ConPurse = conAuthPurse \text{ name? } \bullet \\ &\quad BetweenWorld[conAuthPurse := conAuthPurse \oplus \{ (name?, \theta ConPurse) \}, \\ &\quad \quad ether := ether \cup \{ m! \}] \end{aligned}$$

□

Added theorem 8.30 (Promoted between world schema is total)

theorem tBetwPhiBOPPRE

$$\begin{aligned} &\forall PhiBOPSig \mid m? \in ether \wedge name? \in \text{dom } conAuthPurse \\ &\quad \wedge \theta ConPurse = conAuthPurse \text{ name? } \bullet \text{ pre } PhiBOP \end{aligned}$$

□

Separating the branches of promoted operations

Increase, Abort

Added schema 8.8 (Promoted *Increase* without *Ignore*)

$$IncreaseOkay \hat{=} \exists \Delta ConPurse \bullet PhiBOP \wedge IncreasePurseOkay$$

□

Added schema 8.9 (Promoted *Abort* without *Ignore*)

$$AbortOkay \hat{=} (\exists \Delta ConPurse \bullet AbortPurseOkay \wedge [PhiBOP \mid m! = \perp])$$

□

Value transfer and exception log operations

StartFrom and *StartTo* have already been defined without the *Ignore* disjunct and the *Abort* composition in Section 5.6.1 as *StartFromEafromOkay* and *StartToEafromOkay*. For these two, the names can be quite confusing and we summarise them here:

StartFromPurseEafromOkay	Ch 4	successful <i>startFrom</i> over <i>ConPurse</i> (no <i>Abort</i> or <i>Ignore</i>)
StartFromPurseOkay	Ch 4	<i>AbortPurseOkay</i> composed with <i>StartFromPurseEafromOkay</i> for recovery abortion
StartFromEafromOkay	Ch 5	promoted version of <i>StartFromPurseEafromOkay</i> without <i>Ignore</i> disjunct
StartFromOkay	Ch 8	promoted version of <i>StartFromPurseOkay</i> without <i>Ignore</i> and <i>Abort</i> disjuncts
StartFrom	Ch 5	promoted version of <i>StartFromPurseOkay</i> with <i>Ignore</i> and <i>Abort</i> disjuncts

The same applies for *StartTo*, *ReadExceptionLog*, and *ClearExceptionLog*.

Added schema 8.10 (Promoted *StartFrom* without *Ignore* and with *Abort*)

$$StartFromOkay \triangleq \exists \Delta ConPurse \bullet PhiBOP \wedge StartFromPurseOkay$$

□

Added schema 8.11 (Promoted *StartTo* without *Ignore* and with *Abort*)

$$StartToOkay \triangleq \exists \Delta ConPurse \bullet PhiBOP \wedge StartToPurseOkay$$

□

Added schema 8.12 (Promoted *Req* without *Ignore*)

$$ReqOkay \triangleq \exists \Delta ConPurse \bullet PhiBOP \wedge ReqPurseOkay$$

□

Added schema 8.13 (Promoted *Val* without *Ignore*)

$$ValOkay \triangleq \exists \Delta ConPurse \bullet PhiBOP \wedge ValPurseOkay$$

□

Added schema 8.14 (Promoted *Ack* without *Ignore*)

$$AckOkay \triangleq \exists \Delta ConPurse \bullet PhiBOP \wedge AckPurseOkay$$

□

Added schema 8.15 (Promoted *ReadExceptionLog* without *Ignore* and with *Abort*)

$$ReadExceptionLogOkay \triangleq \exists \Delta ConPurse \bullet PhiBOP \wedge ReadExceptionLogPurseOkay$$

□

Added schema 8.16 (Promoted *ClearExceptionLog* without *Ignore* and with *Abort*)

$$ClearExceptionLogOkay \triangleq \exists \Delta ConPurse \bullet PhiBOP \wedge ClearExceptionLogPurseOkay$$

□

Expanded *BetweenWorld*

In various proof obligations (as lemmas) for the proper instantiation of *BetweenWorld*, we need to expand the definitions from *AuxWorld*. Nevertheless, we do not want to clog the goals with unnecessary information from *AuxWorld*. Therefore, we introduce expansion theorems that expand each *AuxWorld* element individually. This is also used in Chapters 10, 14–24, and 28–29.

Added lemma 8.10 ($\mathcal{B}$ *fromLogged* extraction (*AuxWorld*))

$$\begin{aligned} &\textbf{theorem } lAuxWorldFromLoggedExpansion \\ &\quad \forall AuxWorld \bullet fromLogged = \{ pd : authenticFrom \mid pd.from \mapsto pd \in allLogs \} \end{aligned}$$

□

Added lemma 8.11 ($\mathcal{B}$ *toLogged* extraction (*AuxWorld*))

$$\begin{aligned} &\textbf{theorem } lAuxWorldToLoggedExpansion \\ &\quad \forall AuxWorld \bullet toLogged = \{ pd : authenticTo \mid pd.to \mapsto pd \in allLogs \} \end{aligned}$$

□

Added lemma 8.12 ($\mathcal{B}$ *authenticFrom* extraction (*AuxWorld*))

theorem lAuxWorldAuthenticFromExpansion
 $\forall \text{AuxWorld} \bullet \text{authenticFrom} = \{ pd : \text{PayDetails} \mid pd.\text{from} \in \text{dom } \text{conAuthPurse} \}$

□

Added lemma 8.13 ($\mathcal{B}$ *authenticTo* extraction (*AuxWorld*))

theorem lAuxWorldAuthenticToExpansion
 $\forall \text{AuxWorld} \bullet \text{authenticTo} = \{ pd : \text{PayDetails} \mid pd.\text{to} \in \text{dom } \text{conAuthPurse} \}$

□

Added lemma 8.14 ($\mathcal{B}$ *toInEpv* extraction (*AuxWorld*))

theorem lAuxWorldToInEpvExpansion
 $\forall \text{AuxWorld} \bullet \text{toInEpv} = \{ pd : \text{authenticTo} \mid$
 $(\text{conAuthPurse}pd.\text{to}).\text{status} = \text{epv} \wedge (\text{conAuthPurse}pd.\text{to}).pdAuth = pd \}$

□

Added lemma 8.15 ($\mathcal{B}$ *toInEapayee* extraction (*AuxWorld*))

theorem lAuxWorldToInEapayeeExpansion
 $\forall \text{AuxWorld} \bullet \text{toInEapayee} = \{ pd : \text{authenticTo} \mid$
 $(\text{conAuthPurse}pd.\text{to}).\text{status} = \text{eaTo} \wedge (\text{conAuthPurse}pd.\text{to}).pdAuth = pd \}$

□

Added lemma 8.16 ($\mathcal{B}$ *fromInEpr* extraction (*AuxWorld*))

theorem lAuxWorldFromInEprExpansion
 $\forall \text{AuxWorld} \bullet \text{fromInEpr} = \{ pd : \text{authenticFrom} \mid$
 $(\text{conAuthPurse}pd.\text{from}).\text{status} = \text{epr} \wedge (\text{conAuthPurse}pd.\text{from}).pdAuth = pd \}$

□

Added lemma 8.17 ($\mathcal{B}$ *fromInEpa* extraction (*AuxWorld*))

theorem lAuxWorldFromInEpaExpansion
 $\forall \text{AuxWorld} \bullet \text{fromInEpa} = \{ pd : \text{authenticFrom} \mid$
 $(\text{conAuthPurse}pd.\text{from}).\text{status} = \text{epa} \wedge (\text{conAuthPurse}pd.\text{from}).pdAuth = pd \}$

□

Added lemma 8.18 ($\mathcal{B}$ *allLogs* extraction (*AuxWorld*))

theorem lAuxWorldAllLogsExpansion
 $\forall \text{AuxWorld} \bullet \text{allLogs} = \text{archive} \cup$
 $\{ n : \text{dom } \text{conAuthPurse}; pd : \text{PayDetails} \mid pd \in (\text{conAuthPurse } n).\text{exLog} \}$

□

Added lemma 8.19 ($\mathcal{B}$ *definitelyLost* extraction (*AuxWorld*))

theorem lAuxWorldDefinitelyLostExpansion
 $\forall \text{AuxWorld} \bullet \text{definitelyLost} = \text{toLogged} \cap (\text{fromLogged} \cup \text{fromInEpa})$

□

Added lemma 8.20 ($\mathcal{B}$ *maybeLost* extraction (*AuxWorld*))

theorem lAuxWorldMaybeLostExpansion
 $\forall \text{AuxWorld} \bullet \text{maybeLost} = (\text{fromInEpa} \cup \text{fromLogged}) \cap \text{toInEpv}$

□

Added lemma 8.21 ($\mathcal{B}$ *archive* extraction (*AuxWorld*))

theorem lAuxWorldArchiveExpansion
 $\forall \text{AuxWorld} \bullet \text{archive} \in \mathbb{P}(\{ \text{PayDetails} \bullet \text{from} \mapsto \theta \text{PayDetails} \} \cup$
 $\{ \text{PayDetails} \bullet \text{to} \mapsto \theta \text{PayDetails} \})$

□

Promoted *IncreaseOkay* and *AbortOkay* precondition

For the filtered promoted operations, we need another signature that includes the precondition of *PhiBOP* itself (see theorem *tBetwPhiBOPPRE* in Section 8.5.3).

Added schema 8.17 (Signature for promoted operations in $\mathcal{B}$, filtered from *Ignore*)

$\frac{\text{PhiBOPBranchesSig}}{\text{PhiBOPSig}}$
$m? \in \text{ether}$ $\text{name?} \in \text{dom } \text{conAuthPurse}$ $\theta \text{ConPurse} = \text{conAuthPurse name?}$

□

Added theorem 8.31 (*Increase* promotion PRE without *Ignore*)

theorem *tBetwIncreaseOkayPRE*
 $\forall \text{PhiBOPBranchesSig} \mid \perp \in \text{ether} \bullet \text{pre } \text{IncreaseOkay}$

□

For the abort operation, there is a quite tricky issue. The promotion precondition requires that

$$\theta \text{ConPurse} = \text{conAuthPurse name?}$$

but the *AbortPurseOkay* operation requires that the $\theta \text{ConPurse}'$ to be included in $\text{conAuthPurse}'$ satisfy different constraints from $\theta \text{ConPurse}$, such as having a different status (*eaFrom*). Therefore, we need to capture this subtle precondition for the filtered version of promoted *Abort*. Also note that because conAuthPurse is an injection, and two different purses are being included for the same name? , we must also enforce the new purse is not already present, otherwise the injectivity is broken.

The promoted *Abort* requires that there exists a $\theta \text{ConPurse}'$, after the *AbortPurseOkay* operation has taken place. At this stage, *status* is set, and the promotion *PhiBOP* have already updated $\text{conAuthPurse}'$. The problem is that $\theta \text{ConPurse}'$ is not the same as $\theta \text{ConPurse}$, but it will be the one associated with name? . To take this into account, we need to ensure that the resulting $\text{conAuthPurse}'$ from *PhiBOP* is an injection:

$$\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ConPurse})\} \in \text{NAMES} \rightsquigarrow \text{ConPurse}$$

As *AbortPurseOkay* will change $\theta \text{ConPurse}'$, this result must not be present in conAuthPurse , hence the update keeps its injectivity. Furthermore, the information about injectivity is redundant, because of the invariant in *PhiBOPBranchesSig*. This is discharged by the next lemma

Added lemma 8.22 (Promoted conAuthPurse is injective)

theorem *rule lPromotedConAuthPurseInjectivity*
 $\forall \text{ConWorld}; \text{ConPurse}; \text{name?} : \text{NAMES} \mid \text{name?} \in \text{dom } \text{conAuthPurse}$
 $\wedge \theta \text{ConPurse} = \text{conAuthPurse name?} \bullet$
 $\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ConPurse})\} \in \text{NAMES} \rightsquigarrow \text{ConPurse}$

□

Do not use $\Delta \text{ConPurse}$ for automation purposes (might be ineffective as the pattern might not match directly anyway).

Added lemma 8.23 (Promoted conAuthPurse updated is injective)

theorem *rule lPromotedConAuthNewPurseInjectivity*
 $\forall \text{ConWorld}; \text{ConPurse}; \text{ConPurse}'; \text{name?} : \text{NAMES} \mid \text{name?} \in \text{dom } \text{conAuthPurse}$
 $\wedge \theta \text{ConPurse} = \text{conAuthPurse name?}$
 $\wedge \neg \theta \text{ConPurse}' \in \text{ran } (\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ConPurse})\})$
 $\bullet \text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ConPurse}')\} \in \text{NAMES} \rightsquigarrow \text{ConPurse}$

□

Finally, we can give the signature for *Abort* as

Added schema 8.18 (Signatture for promoted *Abort*, filtering *Ignore*)

$\Phi BOpAbortSig$ $\Phi BOpBranchesSig$	BetweenWorld for lPromotedBetweenWorldAbortType (B9-B16 but B13) $\perp \in ether$ conAuthPurse update for all related theorems $name \in \{pdAuth.from, pdAuth.to\}$ conAuthPurse oplus update in AuxWorld to keep injectivity cases for lPromotedAuxWorldAbortType $(\forall xEPV : PayDetails \mid xEPV.to \in \text{dom } conAuthPurse$ $\wedge (conAuthPurse xEPV.to).status = epv$ $\wedge (conAuthPurse xEPV.to).pdAuth = xEPV \bullet xEPV.to \neq name?)$ $(\forall xEATO : PayDetails \mid xEATO.to \in \text{dom } conAuthPurse$ $\wedge (conAuthPurse xEATO.to).status = eaTo$ $\wedge (conAuthPurse xEATO.to).pdAuth = xEATO \bullet xEATO.to \neq name?)$ $(\forall xEPR : PayDetails \mid xEPR.from \in \text{dom } conAuthPurse$ $\wedge (conAuthPurse xEPR.from).status = epr$ $\wedge (conAuthPurse xEPR.from).pdAuth = xEPR \bullet xEPR.from \neq name?)$ $(\forall xEPA : PayDetails \mid xEPA.from \in \text{dom } conAuthPurse$ $\wedge (conAuthPurse xEPA.from).status = epa$ $\wedge (conAuthPurse xEPA.from).pdAuth = xEPA \bullet xEPA.from \neq name?)$ Possible allLogs update in AuxWorld $pdAuth \notin exLog \Rightarrow (name?, pdAuth) \in archive$ Inejctivity type of conAuthPurse in lPromotedAuxWorldAbortType $\neg \theta ConPurse[exLog := exLog \cup$ $(\text{if } status = epv \vee status = epa \text{ then } \{pdAuth\} \text{ else } \{\}),$ $status := eaFrom] \in \text{ran}(conAuthPurse \oplus \{(name?, \theta ConPurse)\})$
---	---

□

Added lemma 8.24 (Promoted *AuxWorld* type)

theorem lPromotedAuxWorldAbortType
 $\forall \Phi BOpAbortSig \bullet$
if $(status = epv \vee status = epa)$ **then**
 $AuxWorld[conAuthPurse := conAuthPurse \oplus \{(name?,$
 $\theta ConPurse'[balance' := balance, exLog' := exLog \cup \{pdAuth\},$
 $name' := name, nextSeqNo' := nextSeqNo, pdAuth' := pdAuth,$
 $status' := eaFrom])\}, ether := ether \cup \{\perp\}]$
else
 $AuxWorld[conAuthPurse := conAuthPurse \oplus \{(name?,$
 $\theta ConPurse'[balance' := balance, exLog' := exLog, name' := name,$
 $nextSeqNo' := nextSeqNo, pdAuth' := pdAuth, status' := eaFrom])\},$
 $ether := ether \cup \{\perp\}]$

□

The additional condition that $\perp \in ether$ is important for B9 – B16 but B13.

Added lemma 8.25 (**Promoted** *BetweenWorld* type for *Abort*)

theorem lPromotedBetweenWorldAbortType
 $\forall \text{PhiBOpAbortSig} \bullet$
 if ($\text{status} = \text{epv} \vee \text{status} = \text{epa}$) then
 $\text{BetweenWorld}[\text{conAuthPurse} := \text{conAuthPurse} \oplus \{ (\text{name?},$
 $\theta \text{ConPurse}'[\text{balance}' := \text{balance}, \text{exLog}' := \text{exLog} \cup \{ \text{pdAuth} \},$
 $\text{name}' := \text{name}, \text{nextSeqNo}' := \text{nextSeqNo}, \text{pdAuth}' := \text{pdAuth},$
 $\text{status}' := \text{eaFrom}] \}, \text{ether} := \text{ether} \cup \{ \perp \}]$
 else
 $\text{BetweenWorld}[\text{conAuthPurse} := \text{conAuthPurse} \oplus \{ (\text{name?},$
 $\theta \text{ConPurse}'[\text{balance}' := \text{balance}, \text{exLog}' := \text{exLog}, \text{name}' := \text{name},$
 $\text{nextSeqNo}' := \text{nextSeqNo}, \text{pdAuth}' := \text{pdAuth}, \text{status}' := \text{eaFrom}] \},$
 $\text{ether} := \text{ether} \cup \{ \perp \}]$

□

Added theorem 8.32 (*Abort* promotion PRE without *Ignore*)

theorem tBetwAbortOkayPRE
 $\forall \text{PhiBOpAbortSig} \bullet \text{pre } \text{AbortOkay}$

□

Promoted *StartFromEafromOkay* and *StartToEafromOkay*

Added schema 8.19 (Signatture for promoted *StartFrom*, filtering *Ignore*)

$\text{PhiBOpStartFromSig}$ PhiBOpBranchesSig ValidStartFrom	
Original precondition of StartFrom $\text{status} = \text{eaFrom}$ $\text{nextSeqNo} < \text{MAX_NAT}$	
BetweenWorld for lPromotedBetweenWorldAbortType (B9-B16 but B13) $\perp \in \text{ether}$	
After conAuthPurse update, we need it for B7 $\forall \text{pd} : \text{PayDetails} \mid \text{pd.from} = \text{name?} \bullet \text{pd.fromSeqNo} < \text{nextSeqNo}$	
Used by fromInEpr for injectivity of AuxWorld $\text{name} \in \text{dom } \text{conAuthPurse} \Rightarrow \text{name} \neq \text{name?}$	
Inejctivity type of conAuthPurse in lPromotedAuxWorldStartFromType $\neg \theta \text{ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$ $\text{pdAuth} := \theta \text{PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$ $\text{to} := \text{cpd.name}, \text{toSeqNo} := \text{cpd.nextSeqNo},$ $\text{value} := \text{cpd.value}],$ $\text{status} := \text{epv}] \in \text{ran } (\text{conAuthPurse} \oplus \{ (\text{name?}, \theta \text{ConPurse}) \})$	

□

Added schema 8.20 (Signatture for promoted *StartTo*, filtering *Ignore*)

PhiBOpStartToSig PhiBOpBranchesSig ValidStartTo
Original precondition of <i>StartFrom</i> $\text{status} = \text{eaFrom}$ $\text{nextSeqNo} < \text{MAX_NAT}$ $\text{cpd.nextSeqNo} < \text{nextSeqNo}$
After <i>conAuthPurse</i> update, we need it for <i>B8</i> $\forall \text{pd} : \text{PayDetails} \mid \text{pd.to} = \text{name?} \bullet \text{pd.toSeqNo} < \text{nextSeqNo}$
Used by <i>toInEpr</i> for injectivity of <i>AuxWorld</i> $\text{name} \in \text{dom } \text{conAuthPurse} \Rightarrow \text{name} \neq \text{name?}$
To avoid repeating the complex <i>theta</i> expression we give it one-pointly quantified $(\exists \text{pd}' : \text{PayDetails} \mid \text{pd}' =$ $\quad \theta \text{PayDetails}[\text{from} := \text{cpd.name}, \text{fromSeqNo} := \text{cpd.nextSeqNo},$ $\quad \text{name}/\text{to}, \text{nextSeqNo}/\text{toSeqNo}, \text{value} := \text{cpd.value}] \bullet$
<i>B1</i>: authenticity of updated <i>pd</i> from req message That is, if the updated <i>pd</i> is not in the ether, for it to be authentic for <i>to</i> purses, its name must had been previously. $(\text{req } \text{pd}' \notin \text{ether} \Rightarrow \text{name} \in \text{dom } \text{conAuthPurse}) \wedge$
<i>B2</i>: validation of <i>nextSeqNo</i> of updated <i>pd</i> $(\text{req } \text{pd}' \notin \text{ether} \Rightarrow \text{nextSeqNo} < (\text{conAuthPurse } \text{name}).\text{nextSeqNo}) \wedge$
<i>B10</i>: validation of updated <i>pd</i> in ether $(\forall \text{pdOut} : \text{PayDetails} \mid \text{req } \text{pdOut} \notin \text{ether} \bullet \text{pdOut} \neq \text{pd}') \wedge$
Inejctivity type of <i>conAuthPurse</i> in <i>IPromotedAuxWorldStartToType</i> $\neg \theta \text{ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo}, \text{pdAuth} := \text{pd}', \text{status} := \text{epv}]$ $\in \text{ran}(\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ConPurse})\})$

□

Added lemma 8.26 (Promoted *AuxWorld* type for *StartFrom*)

theorem *IPromotedAuxWorldStartFromType*

$$\forall \text{PhiBOpStartFromSig} \bullet \text{AuxWorld}[\text{conAuthPurse} := \text{conAuthPurse}$$

$$\oplus \{(\text{name?}, \theta \text{ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name}, \text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}], \text{status} := \text{epv}]\}, \text{ether} := \text{ether} \cup \{\perp\}]$$

□

Added lemma 8.27 (Promoted *BetweenWorld* type for *StartFrom*)

theorem *IPromotedBetweenWorldStartFromType*

$$\forall \text{PhiBOpStartFromSig} \bullet \text{BetweenWorld}[\text{conAuthPurse} := \text{conAuthPurse}$$

$$\oplus \{(\text{name?}, \theta \text{ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name}, \text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}], \text{status} := \text{epv}]\}, \text{ether} := \text{ether} \cup \{\perp\}]$$

□

Added lemma 8.28 (Promoted *AuxWorld* type for *StartTo*)

theorem lPromotedAuxWorldStartToType
 $\forall \text{PhiBOPStartToSig} \bullet \text{AuxWorld}[\text{conAuthPurse} := \text{conAuthPurse}$
 $\oplus \{ (name?, \theta \text{ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$
 $\text{pdAuth} := \theta \text{PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$
 $\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name}/\text{to}, \text{nextSeqNo}/\text{toSeqNo},$
 $\text{value} := (\text{startTo} \sim m?).\text{value}], \text{status} := \text{epv}]) \},$
 $\text{ether} := \text{ether} \cup \{ \text{req} \theta \text{PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$
 $\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name}/\text{to}, \text{nextSeqNo}/\text{toSeqNo},$
 $\text{value} := (\text{startTo} \sim m?).\text{value}] \}$

□

Added lemma 8.29 (Promoted *BetweenWorld* type for *StartTo*)

theorem lPromotedBetweenWorldStartToType
 $\forall \text{PhiBOPStartToSig} \bullet \text{BetweenWorld}[\text{conAuthPurse} := \text{conAuthPurse}$
 $\oplus \{ (name?, \theta \text{ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$
 $\text{pdAuth} := \theta \text{PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$
 $\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name}/\text{to}, \text{nextSeqNo}/\text{toSeqNo},$
 $\text{value} := (\text{startTo} \sim m?).\text{value}], \text{status} := \text{epv}]) \},$
 $\text{ether} := \text{ether} \cup \{ \text{req} \theta \text{PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$
 $\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name}/\text{to}, \text{nextSeqNo}/\text{toSeqNo},$
 $\text{value} := (\text{startTo} \sim m?).\text{value}] \}$

□

Added theorem 8.33 (*StartFrom* promotion PRE without *Ignore* and *Abort*)

theorem tBetwStartFromEafromOkayPRE
 $\forall \text{PhiBOPStartFromSig} \bullet \text{pre } \text{StartFromEafromOkay}$

□

Added theorem 8.34 (*StartFrom* promotion PRE without *Ignore* and with *Abort*)

theorem tBetwStartFromOkayPRE
 $\forall \text{PhiBOPStartFromSig} \bullet \text{pre } \text{StartFromOkay}$

□

Added theorem 8.35 (*StartTo* promotion PRE without *Ignore* and *Abort*)

theorem tBetwStartToEafromOkayPRE
 $\forall \text{PhiBOPStartToSig} \bullet \text{pre } \text{StartToEafromOkay}$

□

Added theorem 8.36 (*StartTo* promotion PRE without *Ignore* and with *Abort*)

theorem tBetwStartToOkayPRE
 $\forall \text{PhiBOPStartToSig} \bullet \text{pre } \text{StartToOkay}$

□

Promoted *ReqOkay*, *ValOkay*, and *AckOkay* precondition

Added schema 8.21 (Signatture for promoted *Req*, filtering *Ignore*)

$\Phi BOpReqSig$ $\Phi BOpBranchesSig$ $AuthenticReqMessage$
Original precondition $status = epr$ BetweenWorld related $(val\ pdAuth \notin ether \Rightarrow$ B3 $pdAuth.to \in \text{dom } conAuthPurse$ $\wedge pdAuth.from \in \text{dom } conAuthPurse$ $\wedge pdAuth.toSeqNo < nextSeqNo$ $\wedge (pdAuth.to \neq name? \Rightarrow$ $pdAuth.toSeqNo < (conAuthPurse\ pdAuth.to).nextSeqNo)$ $\wedge (pdAuth.from \neq name? \Rightarrow$ $pdAuth.fromSeqNo < (conAuthPurse\ pdAuth.from).nextSeqNo)$ B9 $\wedge (\forall\ pdIn : fromInEpr \bullet pdIn \neq pdAuth)$ B11 $\wedge ((pdAuth.from, pdAuth) \notin archive \Rightarrow$ $pdAuth \in (conAuthPurse\ pdAuth.from).exLog)$ Used for conAuthPurse oplus update in AuxWorld to keep injectivity cases for fromInEpr $(\forall\ xEPR : PayDetails \mid xEPR.from \in \text{dom } conAuthPurse$ $\wedge (conAuthPurse\ xEPR.from).status = epr$ $\wedge (conAuthPurse\ xEPR.from).pdAuth = xEPR \bullet xEPR.from \neq name?)$ Injectivity of AuxWorld related to fromInEpa $\forall\ pd : PayDetails \mid pd.from \in \text{dom } conAuthPurse \bullet pd \neq pdAuth$ Injectivity of updated conAuthPurse in AuxWorld $\neg \theta ConPurse[balance := balance - pdAuth.value, name := pdAuth.from,$ $status := epr] \in \text{ran } (conAuthPurse \oplus \{ (name?, \theta ConPurse[status := epr]) \})$

□

Added schema 8.22 (Signatture for promoted *Val*, filtering *Ignore*)

$\Phi BOpValSig$ $\Phi BOpBranchesSig$ $AuthenticValMessage$
Original precondition $status = epv$ BetweenWorld B4 $ack\ pdAuth \notin ether \Rightarrow pdAuth.to \in \text{dom } conAuthPurse$ Used for conAuthPurse oplus update in AuxWorld to keep injectivity cases for lPromotedAuxWorldAbortType $(\forall\ xEPV : PayDetails \mid xEPV.to \in \text{dom } conAuthPurse$ $\wedge (conAuthPurse\ xEPV.to).status = epv$ $\wedge (conAuthPurse\ xEPV.to).pdAuth = xEPV \bullet xEPV.to \neq name?)$ Injectivity of AuxWorld related to toInEapayee $\forall\ pd : PayDetails \mid pd.to \in \text{dom } conAuthPurse \bullet pd \neq pdAuth$ Used is for inejctivity type of conAuthPurse in AuxWorld $\neg \theta ConPurse[balance := balance + pdAuth.value, status := eaTo]$ $\in \text{ran } (conAuthPurse \oplus \{ (name?, \theta ConPurse[status := epv]) \})$

□

Added schema 8.23 (Signatture for promoted *Ack*, filtering *Ignore*)

$\Phi BOpAckSig$ $\Phi BOpBranchesSig$ $AuthenticAckMessage$
Original precondition $status = epa$
Used for $conAuthPurse$ oplus update in $AuxWorld$ to keep injectivity cases for $IPromotedAuxWorldAbortType$ $(\forall xEPA : PayDetails \mid xEPA.from \in \text{dom } conAuthPurse$ $\wedge (conAuthPurse \ xEPA.from).status = epa$ $\wedge (conAuthPurse \ xEPA.from).pdAuth = xEPA \bullet xEPA.from \neq name?)$
Used is for inejectivity type of $conAuthPurse$ in $AuxWorld$ $\neg \theta ConPurse[status := eaFrom] \in \text{ran } (conAuthPurse \oplus$ $\{ (name?, \theta ConPurse[status := epa]) \})$

□

Added lemma 8.30 (Promoted *AuxWorld* type for *Req*)

theorem $IPromotedAuxWorldReqType$
 $\forall \Phi BOpReqSig \bullet AuxWorld[conAuthPurse := conAuthPurse \oplus \{ (name?,$
 $\theta ConPurse[balance := balance + ^{-1} * pdAuth.value,$
 $name := pdAuth.from, status := epa]) \},$
 $ether := ether \cup \{ val pdAuth \}]$

□

Added lemma 8.31 (Promoted *BetweenWorld* type for *Req*)

theorem $IPromotedBetweenWorldReqType$
 $\forall \Phi BOpReqSig \bullet BetweenWorld[conAuthPurse := conAuthPurse \oplus \{ (name?,$
 $\theta ConPurse[balance := balance + ^{-1} * pdAuth.value,$
 $name := pdAuth.from, status := epa]) \},$
 $ether := ether \cup \{ val pdAuth \}]$

□

Added lemma 8.32 (Promoted *AuxWorld* type for *Val*)

theorem $IPromotedAuxWorldValType$
 $\forall \Phi BOpValSig \bullet AuxWorld[conAuthPurse := conAuthPurse \oplus \{ (name?,$
 $\theta ConPurse[balance := balance + pdAuth.value, status := eaTo]) \},$
 $ether := ether \cup \{ ack pdAuth \}]$

□

Added lemma 8.33 (Promoted *BetweenWorld* type for *Val*)

theorem $IPromotedBetweenWorldValType$
 $\forall \Phi BOpValSig \bullet BetweenWorld[conAuthPurse := conAuthPurse \oplus \{ (name?,$
 $\theta ConPurse[balance := balance + pdAuth.value, status := eaTo]) \},$
 $ether := ether \cup \{ ack pdAuth \}]$

□

Added lemma 8.34 (Promoted *AuxWorld* type for *Ack*)

theorem $IPromotedAuxWorldAckType$
 $\forall \Phi BOpAckSig \bullet AuxWorld[conAuthPurse := conAuthPurse \oplus \{ (name?,$
 $\theta ConPurse[status := eaFrom]) \}, ether := ether \cup \{ \perp \}]$

□

Added lemma 8.35 (**Promoted** *BetweenWorld* type for *Ack*)

theorem lPromotedBetweenWorldAckType
 $\forall \text{PhiBOpAckSig} \bullet \text{BetweenWorld}[\text{conAuthPurse} := \text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ConPurse}[\text{status} := \text{eaFrom}])\}, \text{ether} := \text{ether} \cup \{\perp\}]$

□

Added theorem 8.37 (*Req* promotion PRE without *Ignore* and *Abort*)

theorem tBetwReqOkayPRE
 $\forall \text{PhiBOpReqSig} \bullet \text{pre ReqOkay}$

□

Added theorem 8.38 (*Val* promotion PRE without *Ignore* and *Abort*)

theorem tBetwValOkayPRE
 $\forall \text{PhiBOpValSig} \bullet \text{pre ValOkay}$

□

Added theorem 8.39 (*Ack* promotion PRE without *Ignore* and *Abort*)

theorem tBetwAckOkayPRE
 $\forall \text{PhiBOpAckSig} \bullet \text{pre AckOkay}$

□

Promoted *ReadExceptionLogEafromOkay* and *ClearExceptionLogEafromOkay*

Added schema 8.24 (Signatture for promoted *ReadExceptionLog*, filtering *Ignore*)

$\text{PhiBOpReadExceptionLogSig}$ PhiBOpBranchesSig
Original precondition $\text{status} = \text{eaFrom}$ $m? = \text{readExceptionLog}$ To allow using the general <i>BetweenWorld</i> instantiation In <i>PhiBOp</i> we have that $m!$ is within the ether, which is bot here $\perp \in \text{ether}$

□

Added schema 8.25 (Signatture for promoted *ClearExceptionLog*, filtering *Ignore*)

$\text{PhiBOpClearExceptionLogSig}$ PhiBOpBranchesSig
Original precondition $\text{status} = \text{eaFrom}$ $\text{exLog} \neq \emptyset$ $m? = \text{exceptionLogClear}(\text{name}, \text{image exLog})$ Injectivity property of <i>conAuthPurse</i> for allLogs $\forall \text{pd} : \text{PayDetails} \bullet \neg \text{pd} \in (\text{conAuthPurse name?}).\text{exLog}$ Used is for inejctivity type of <i>conAuthPurse</i> in <i>AuxWorld</i> $\neg \theta \text{ConPurse}[\text{exLog} := \{\}] \in \text{ran}(\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ConPurse})\})$

□

Added lemma 8.36 (Promoted *AuxWorld* type for *ClearExceptionLog*)

theorem lPromotedAuxWorldClearExceptionLogType
 $\forall \text{PhiBOPClearExceptionLogSig} \bullet \text{AuxWorld}[\text{conAuthPurse} := \text{conAuthPurse} \oplus$
 $\{ (name?, \theta \text{ConPurse}[exLog := \{\}, status := eaFrom]) \}, ether := ether \cup \{\perp\}]$

□

Added lemma 8.37 (Promoted *BetweenWorld* type for *ClearExceptionLog*)

theorem lPromotedBetweenWorldClearExceptionLogType
 $\forall \text{PhiBOPClearExceptionLogSig} \bullet \text{BetweenWorld}[\text{conAuthPurse} := \text{conAuthPurse} \oplus$
 $\{ (name?, \theta \text{ConPurse}[exLog := \{\}, status := eaFrom]) \}, ether := ether \cup \{\perp\}]$

□

Added theorem 8.40 (*ReadExceptionLog* promotion PRE – Ignore – Abort)

theorem tBetwReadExceptionLogEafromOkayPRE
 $\forall \text{PhiBOPReadExceptionLogSig} \bullet \text{pre ReadExceptionLogEafromOkay}$

□

Added theorem 8.41 (*ReadExceptionLog* promotion PRE – Ignore + Abort)

theorem tBetwReadExceptionLogOkayPRE
 $\forall \text{PhiBOPReadExceptionLogSig} \bullet \text{pre ReadExceptionLogOkay}$

□

Added theorem 8.42 (*ClearExceptionLog* promotion PRE – Ignore – Abort)

theorem tBetwClearExceptionLogEafromOkayPRE
 $\forall \text{PhiBOPClearExceptionLogSig} \bullet \text{pre ClearExceptionLogEafromOkay}$

□

Added theorem 8.43 (*ClearExceptionLog* promotion PRE – Ignore + Abort)

theorem tBetwClearExceptionLogOkayPRE
 $\forall \text{PhiBOPClearExceptionLogSig} \bullet \text{pre ClearExceptionLogOkay}$

□

8.5.4 Operations at the world level only

As before, we also separate the *Ignore* disjunct for *AuthoriseExLogClear*. As *AuthoriseExLogClear* has already been defined without *Ignore* in Section 5.7 as *AuthoriseExLogClearOkay*, we do not need to add it here. Instead we just prove its precondition. Also, as it is not a promoted operation, the signature is only of *BetweenWorld*.

Added theorem 8.44 (*AuthoriseExLogClear* PRE without *Ignore*)

theorem tBetwAuthoriseExLogClearOkayPRE
 $\forall \text{BetweenOpSig} \bullet \text{pre AuthoriseExLogClearOkay}$

□

The *Archive* operation does not disjoin with *Ignore* and is not related with *Abort*, so we do not need to worry about it.

8.6 $\mathcal{C}$ model consistency proofs

8.6.1 Existence of concrete initial state

Original theorem 8.5 (Existence of concrete initial state)

theorem tExistsInitialConWorld
 $\exists \text{ConWorldCh7}' \bullet \text{ConInitState}$

□

8.6.2 $\mathcal{C}$ promotion operation precondition

Added schema 8.26 ($\mathcal{C}$ operations signature)

$\text{ConOpSig} \triangleq [\text{ConWorldCh7}; \text{name?} : \text{NAMES}; \text{m?} : \text{MESSAGE}]$

□

Added schema 8.27 ($\mathcal{C}$ promoted operations signature)

$\text{PhiCOpSig} \triangleq \text{ConOpSig} \wedge \text{ConPurse}$

□

Added theorem 8.45 ($\mathcal{C}$ promotion schema precondition)

theorem tPhiCOPPRE
 $\forall \text{PhiCOpSig} \mid \text{name?} \in \text{dom conAuthPurseCh7}$
 $\wedge \text{m?} \in \text{etherCh7} \wedge$
 $\theta \text{ConPurse} = \text{conAuthPurseCh7 name?}$
 $\bullet \text{pre PhiCOp}$

□

8.6.3 Totality of concrete operations

Added theorem 8.46 ($\mathcal{C}$ Ignore is Total)

theorem tCIgnoreIsTotal
 $\forall \text{ConOpSig} \bullet \text{pre CIgnore}$

□

Added theorem 8.47 ($\mathcal{C}$ Increase is Total)

theorem tCIncreaseIsTotal
 $\forall \text{ConOpSig} \bullet \text{pre CIncrease}$

□

Added theorem 8.48 ($\mathcal{C}$ Abort is Total)

theorem tCAbortIsTotal
 $\forall \text{ConOpSig} \bullet \text{pre CAbort}$

□

Added theorem 8.49 ($\mathcal{C}$ StartFrom is Total)

theorem tCStartFromIsTotal
 $\forall \text{ConOpSig} \bullet \text{pre CStartFrom}$

□

Added theorem 8.50 ($\mathcal{C}$ StartTo is Total)

theorem tCStartToIsTotal
 $\forall \text{ ConOpSig} \bullet \text{pre } CStartTo$

□

Added theorem 8.51 ($\mathcal{C}$ Req is Total)

theorem tCReqIsTotal
 $\forall \text{ ConOpSig} \bullet \text{pre } CReq$

□

Added theorem 8.52 ($\mathcal{C}$ Val is Total)

theorem tCValIsTotal
 $\forall \text{ ConOpSig} \bullet \text{pre } CVal$

□

Added theorem 8.53 ($\mathcal{C}$ Ack is Total)

theorem tCAckIsTotal
 $\forall \text{ ConOpSig} \bullet \text{pre } CAck$

□

Added theorem 8.54 ($\mathcal{C}$ ReadExceptionLog is Total)

theorem tCReadExceptionLogIsTotal
 $\forall \text{ ConOpSig} \bullet \text{pre } CReadExceptionLog$

□

Added theorem 8.55 ($\mathcal{C}$ ClearExceptionLog is Total)

theorem tCClearExceptionLogIsTotal
 $\forall \text{ ConOpSig} \bullet \text{pre } CClearExceptionLog$

□

Added theorem 8.56 ($\mathcal{C}$ AuthoriseExceptionLogClear is Total)

theorem tCAuthoriseExLogClearIsTotal
 $\forall \text{ ConOpSig} \bullet \text{pre } CAuthoriseExLogClear$

□

Added theorem 8.57 ($\mathcal{C}$ Authentic is Total)

theorem tCArchiveIsTotal
 $\forall \text{ ConOpSig} \bullet \text{pre } CArchive$

□

8.6.4 Separating the branches of promoted operations

Increase, Abort

Added schema 8.28 (Promoted $CIncrease$ without $CIgnore$)

$CIncreaseOkay \hat{=} \exists \Delta ConPurse \bullet \Phi iCOp \wedge IncreasePurseOkay$

□

Added schema 8.29 (Promoted $CAbort$ without $CIgnore$)

$CAbortOkay \hat{=} (\exists \Delta ConPurse \bullet AbortPurseOkay \wedge [\Phi iCOp \mid m! = \perp])$

□

Value transfer and exception log operations

Added schema 8.30 (Promoted $CStartFrom$ without $CIgnore$ and with $CAbort$)

$$CStartFromOkay \triangleq \exists \Delta ConPurse \bullet PhiCOp \wedge StartFromPurseOkay$$

□

Added schema 8.31 (Promoted $CStartTo$ without $CIgnore$ and with $CAbort$)

$$CStartToOkay \triangleq \exists \Delta ConPurse \bullet PhiCOp \wedge StartToPurseOkay$$

□

Added schema 8.32 (Promoted $CReq$ without $CIgnore$)

$$CReqOkay \triangleq \exists \Delta ConPurse \bullet PhiCOp \wedge ReqPurseOkay$$

□

Added schema 8.33 (Promoted $CVal$ without $CIgnore$)

$$CValOkay \triangleq \exists \Delta ConPurse \bullet PhiCOp \wedge ValPurseOkay$$

□

Added schema 8.34 (Promoted $CAck$ without $CIgnore$)

$$CAckOkay \triangleq \exists \Delta ConPurse \bullet PhiCOp \wedge AckPurseOkay$$

□

Added schema 8.35 (Promoted $CReadExceptionLog$ without $CIgnore$ and with $CAbort$)

$$CReadExceptionLogOkay \triangleq \exists \Delta ConPurse \bullet PhiCOp \wedge ReadExceptionLogPurseOkay$$

□

Added schema 8.36 (Promoted $CClearExceptionLog$ without $CIgnore$ and with $CAbort$)

$$CClearExceptionLogOkay \triangleq \exists \Delta ConPurse \bullet PhiCOp \wedge ClearExceptionLogPurseOkay$$

□

Added schema 8.37 (Promoted $CAuthorisedExLogClear$ without $CIgnore$)

$$CAuthoriseExLogClearOkay \triangleq \exists \exists ConPurse \bullet CPhiCOpExpLogClearNEW$$

□

Promoted $IncreaseOkay$ and $AbortOkay$ precondition

For the filtered promoted operations, we need another signature that includes the precondition of $PhiBOP$ itself (see theorem $tBetwPhiBOPPRE$ in Section 8.6.2).

Added schema 8.38 (Signature for promoted operations in $\mathcal{C}$, filtered from $CIgnore$)

$PhiCOpBranchesSig$	_____
$PhiCOpSig$	_____
$m? \in etherCh7$	
$name? \in \text{dom } conAuthPurseCh7$	
$\theta ConPurse = conAuthPurseCh7 \text{ name?}$	

□

Added schema 8.39 (Signatture for promoted $CIncrease$, filtering $CIgnore$)

$\frac{PhiCOpIncreaseSig}{PhiCOpBranchesSig}$
$\perp \in etherCh7$

□

Added schema 8.40 (Signatture for promoted $CAbort$, filtering $CIgnore$)

$\frac{PhiCOpAbortSig}{PhiCOpBranchesSig}$
$\perp \in etherCh7$ $name \in \{pdAuth.from, pdAuth.to\}$

□

Added lemma 8.38 ($conAuthPurse$ injectivity and finiteness upon $\oplus$)

theorem rule lConAuthPurseInjFinite
 $\forall ConPurse; ConWorldCh7; name? : NAMES \mid$
 $name? \in \text{dom } conAuthPurseCh7$
 $\wedge \theta ConPurse = conAuthPurseCh7 \text{ name?} \bullet$
 $conAuthPurseCh7 \oplus \{(name?, \theta ConPurse)\} \in NAMES \rightsquigarrow ConPurse$

□

Added theorem 8.58 ($CIncrease$ promotion PRE without $CIgnore$)

theorem tConcIncreaseOkayPRE
 $\forall PhiCOpIncreaseSig \bullet \text{pre } CIncreaseOkay$

□

Added theorem 8.59 ($CAbort$ promotion PRE without $CIgnore$)

theorem tConcAbortOkayPRE
 $\forall PhiCOpAbortSig \bullet \text{pre } CAbortOkay$

□

Promoted $StartFromEafromOkay$ and $StartToEafromOkay$

Added schema 8.41 (Signatture for promoted $CStartFrom$, filtering $CIgnore$)

$\frac{PhiCOpStartFromSig}{PhiCOpBranchesSig}$
$ValidStartFrom$
$status = eaFrom$ $nextSeqNo < MAX_NAT$

□

Added schema 8.42 (Signatture for promoted $CStartFrom$, filtering $CIgnore$)

$\frac{PhiCOpStartToSig}{PhiCOpBranchesSig}$
$ValidStartTo$
$status = eaFrom$ $nextSeqNo < MAX_NAT$ $cpd.nextSeqNo < nextSeqNo$

□

Added theorem 8.60 (*CStartFrom* promotion PRE without *CIgnore* and *CAbort*)

theorem tConcStartFromOkayPRE
 $\forall \text{PhiCOpStartFromSig} \bullet \text{pre } CStartFromOkay$

□

Added theorem 8.61 (*CStartTo* promotion PRE without *CIgnore* and *CAbort*)

theorem tConcStartToOkayPRE
 $\forall \text{PhiCOpStartToSig} \bullet \text{pre } CStartToOkay$

□

Promoted *ReqOkay*, *ValOkay*, and *AckOkay* precondition

Added schema 8.43 (Signatture for promoted *CReq*, filtering *CIgnore*)

<i>PhiCOpReqSig</i>	_____
<i>PhiCOpBranchesSig</i>	
<i>AuthenticReqMessage</i>	
<i>status = epr</i>	

□

Added schema 8.44 (Signatture for promoted *CVal*, filtering *CIgnore*)

<i>PhiCOpValSig</i>	_____
<i>PhiCOpBranchesSig</i>	
<i>AuthenticValMessage</i>	
<i>status = epv</i>	

□

Added schema 8.45 (Signatture for promoted *CAck*, filtering *CIgnore*)

<i>PhiCOpAckSig</i>	_____
<i>PhiCOpBranchesSig</i>	
<i>AuthenticAckMessage</i>	
<i>status = epa</i>	

□

Added theorem 8.62 (*CReq* promotion PRE without *CIgnore*)

theorem tConcReqOkayPRE
 $\forall \text{PhiCOpReqSig} \bullet \text{pre } CReqOkay$

□

Added theorem 8.63 (*CVal* promotion PRE without *CIgnore*)

theorem tConcValOkayPRE
 $\forall \text{PhiCOpValSig} \bullet \text{pre } CValOkay$

□

Added theorem 8.64 (*CAck* promotion PRE without *CIgnore*)

theorem tConcAckOkayPRE
 $\forall \text{PhiCOpAckSig} \bullet \text{pre } CAckOkay$

□

Promoted *ReadExceptionLogEafromOkay* and *ClearExceptionLogEafromOkay*

Added schema 8.46 (Signatture for promoted *CReadExceptionLog*, filtering *CIgnore*)

<i>PhiCOpReadExceptionLogSig</i>	_____
<i>PhiCOpBranchesSig</i>	_____
<i>status</i> = <i>eaFrom</i>	
<i>m?</i> = <i>readExceptionLog</i>	

□

Added schema 8.47 (Signatture for promoted *CClearExceptionLog*, filtering *CIgnore*)

<i>PhiCOpClearExceptionLogSig</i>	_____
<i>PhiCOpBranchesSig</i>	_____
<i>status</i> = <i>eaFrom</i>	
<i>exLog</i> ≠ ∅	
<i>m?</i> = <i>exceptionLogClear</i> (<i>name</i> , <i>image exLog</i>)	

□

Added theorem 8.65 (*CReadExceptionLogOkay* Φ PRE without *CIgnore* and *CAbort*)

theorem tConcReadExceptionLogOkayPRE
 $\forall \text{PhiCOpReadExceptionLogSig} \bullet \text{pre } CReadExceptionLogOkay$

□

Added theorem 8.66 (*CClearExceptionLogOkay* Φ PRE without *CIgnore* and *CAbort*)

theorem tConcClearExceptionLogOkayPRE
 $\forall \text{PhiCOpClearExceptionLogSig} \bullet \text{pre } CClearExceptionLogOkay$

□

8.6.5 Operations at the world level only

Added theorem 8.67 (*CAuthoriseExLogClear* PRE without *CIgnore*)

theorem tConcAuthoriseExLogClearOkayPRE
 $\forall \text{PhiCOpSig} \bullet \text{pre } CAuthoriseExLogClearOkay$

□

8.7 Summary

Operation	Precondition
<i>AbOp</i>	<i>true</i>
<i>AbIgnore</i>	<i>true</i>
<i>AbWorldSecureOp</i>	$a? \in \text{ran } \textit{transfer} \wedge$ $\theta \textit{TransferDetails}? = (\textit{transfer} \sim) a?$
<i>AbTransfer</i> <i>AbTransferOkayTD</i>	$\textit{from}? \in \text{dom } \textit{abAuthPurse} \wedge \textit{to}? \in \text{dom } \textit{abAuthPurse}$ $\wedge \textit{value}? \leq (\textit{abAuthPurse } \textit{from}?).\textit{balance} \wedge \textit{from}? \neq \textit{to}?$
<i>AbTransferLostTD</i>	<i>pre AbTransferOkayTD</i>
<i>AbIgnore</i>	<i>true</i>
<i>AbTransfer</i>	<i>true</i>

Table 8.1: Preconditions for $\mathcal{A}$ operations (see Chapter 3)

Operation	Precondition
<i>IncreasePurseOkay</i>	<i>true</i>
<i>AbortPurseOkay</i>	$\textit{name} \in \{ \textit{pdAuth}.\textit{from}, \textit{pdAuth}.\textit{to} \}$
<i>StartFromPurseEafromOkay</i>	$m? \in \text{ran } \textit{startFrom} \wedge \textit{cpd} = (\textit{startFrom} \sim) m? \wedge$ $\textit{cpd}.\textit{name} \neq \textit{name} \wedge \textit{cpd}.\textit{value} \leq \textit{balance} \wedge$ $\textit{status} = \textit{eaFrom} \wedge \textit{nextSeqNo} < \textit{MAX_NAT}$
<i>StartFromPurseOkay</i>	<i>pre AbortPurseOkay</i> $\wedge m? \in \text{ran } \textit{startFrom} \wedge$ $\textit{cpd} = (\textit{startFrom} \sim) m? \wedge \textit{cpd}.\textit{name} \neq \textit{name} \wedge$ $\textit{cpd}.\textit{value} \leq \textit{balance} \wedge \textit{nextSeqNo} < \textit{MAX_NAT}$
<i>StartToPurseEafromOkay</i>	$\textit{ValidStartTo} \wedge \textit{status} = \textit{eaFrom}$ $\wedge \textit{nextSeqNo} < \textit{MAX_NAT}$ $\wedge \textit{cpd}.\textit{nextSeqNo} < \textit{nextSeqNo}$
<i>StartToPurseOkay</i>	<i>pre AbortPurseOkay</i> $\wedge m? \in \text{ran } \textit{startTo} \wedge$ $\textit{cpd} = (\textit{startTo} \sim) m? \wedge \textit{cpd}.\textit{name} \neq \textit{name} \wedge$ $\textit{cpd}.\textit{nextSeqNo} < \textit{nextSeqNo} \wedge \textit{nextSeqNo} < \textit{MAX_NAT}$
<i>ReqPurseOkay</i>	$\textit{status} = \textit{epv} \wedge m? = \textit{req } \textit{pdAuth}$
<i>ValPurseOkay</i>	$\textit{status} = \textit{epv} \wedge m? = \textit{val } \textit{pdAuth}$
<i>AckPurseOkay</i>	$\textit{status} = \textit{epa} \wedge m? = \textit{ack } \textit{pdAuth}$
<i>ReadExceptionLogPurseEafromOkay</i>	$\textit{status} = \textit{eaFrom} \wedge m? = \textit{readExceptionLog}$
<i>ReadExceptionLogPurseOkay</i>	<i>pre AbortPurseOkay</i> $\wedge m? = \textit{readExceptionLog}$
<i>ClearExceptionLogPurseEafromOkay</i>	$\textit{status} = \textit{eaFrom} \wedge \textit{exLog} \neq \emptyset$ $\wedge m? = \textit{exceptionLogClear}(\textit{name}, \textit{image } \textit{exLog})$
<i>ClearExceptionLogPurseOkay</i>	<i>pre AbortPurseOkay</i> $\wedge \textit{exLog} \neq \emptyset$ $\wedge m? = \textit{exceptionLogClear}(\textit{name},$ $\quad \textit{image } (\textit{exLog} \cup (\textbf{if } \textit{status} \in \{ \textit{epv}, \textit{epa} \}$ $\quad \quad \quad \textbf{then } \{ \textit{pdAuth} \} \textbf{ else } \emptyset)))$

Table 8.2: Preconditions for $\mathcal{B}$ operations (see Chapter 4)

For promoted operations without recovery from abortion (*i.e.*, sequentially composed with *AbortPurseOkay*), the preconditions are just the same as the promoted versions. That is:

$$\begin{aligned}
\textit{pre } \textit{StartFromEafromOkay} &\equiv \textit{pre } \textit{StartFromOkay} \\
\textit{pre } \textit{StartToEafromOkay} &\equiv \textit{pre } \textit{StartToOkay} \\
\textit{pre } \textit{ReadExceptionLogEafromOkay} &\equiv \textit{pre } \textit{ReadExceptionLogOkay} \\
\textit{pre } \textit{ClearExceptionLogEafromOkay} &\equiv \textit{pre } \textit{ClearExceptionLogOkay}
\end{aligned}$$

Operation		Precondition
<i>PhiBOp</i>		$m? \in \text{ether} \wedge \text{name?} \in \text{dom conAuthPurse}$ $\wedge \theta \text{ConPurse} = \text{conAuthPurse name?}$
<i>Ignore</i>		<i>true</i>
<i>Increase</i>	<i>Ignore</i> <i>IncreaseOkay</i>	<i>true</i> $\text{pre PhiBOp} \wedge \perp \in \text{ether}$
<i>Abort</i>	<i>Ignore</i> <i>AbortOkay</i>	<i>true</i> $\text{pre PhiBOp} \wedge \text{pre AbortPurseOkay} \wedge \perp \in \text{ether}$
<i>StartFrom</i>	<i>Ignore</i> <i>Abort</i> <i>StartFromOkay</i>	<i>true</i> <i>true</i> $\text{pre PhiBOp} \wedge \text{pre StartFromPurseEafromOkay} \wedge \perp \in \text{ether}$ $\wedge (\forall \text{pd} : \text{PayDetails} \mid \text{pd.from} = \text{name?} \bullet$ $\quad \text{pd.fromSeqNo} < \text{nextSeqNo})$ $\wedge \text{name} \in \text{dom conAuthPurse} \Rightarrow \text{name} \neq \text{name?}$ $\wedge \theta \text{ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$ $\quad \text{pdAuth} := \theta \text{PayDetails}[\text{name}/\text{from},$ $\quad \text{nextSeqNo}/\text{fromSeqNo}, \text{to} := \text{cpd.name},$ $\quad \text{toSeqNo} := \text{cpd.nextSeqNo}, \text{value} := \text{cpd.value}],$ $\text{status} := \text{epv}] \notin \text{ran conAuthPurse}$
<i>StartTo</i>	<i>Ignore</i> <i>Abort</i> <i>StartToOkay</i>	<i>true</i> <i>true</i> $\text{pre PhiBOp} \wedge \text{pre StartToPurseEafromOkay} \wedge$ $(\forall \text{pd} : \text{PayDetails} \mid \text{pd.to} = \text{name?} \bullet$ $\quad \text{pd.toSeqNo} < \text{nextSeqNo})$ $\wedge \text{name} \in \text{dom conAuthPurse} \Rightarrow \text{name} \neq \text{name?}$ $(\exists \text{pd}' : \text{PayDetails} \mid \text{pd}' = \theta \text{PayDetails}[\text{from} := \text{cpd.name},$ $\quad \text{fromSeqNo} := \text{cpd.nextSeqNo}, \text{name}/\text{to}, \text{value} := \text{cpd.value}$ $\quad \text{nextSeqNo}/\text{toSeqNo}] \bullet (\text{req pd}' \notin \text{ether} \Rightarrow$ $\quad \text{name} \in \text{dom conAuthPurse} \wedge$ $\quad \text{nextSeqNo} < (\text{conAuthPurse name}).\text{nextSeqNo}) \wedge$ $(\forall \text{pdOut} : \text{PayDetails} \mid \text{req pdOut} \notin \text{ether} \bullet \text{pdOut} \neq \text{pd}')$ $\wedge \theta \text{ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo}, \text{pdAuth} := \text{pd}',$ $\quad \text{status} := \text{epv}] \notin \text{ran conAuthPurse})$
<i>Req</i>	<i>Ignore</i> <i>ReqOkay</i>	<i>true</i> $\text{pre PhiBOp} \wedge \text{pre ReqPurseOkay} \wedge$ $(\text{val pdAuth} \notin \text{ether} \Rightarrow$ $\quad \text{pdAuth.to} \in \text{dom conAuthPurse}$ $\quad \wedge \text{pdAuth.from} \in \text{dom conAuthPurse}$ $\quad \wedge \text{pdAuth.toSeqNo} < \text{nextSeqNo}$ $\quad \wedge (\text{pdAuth.to} \neq \text{name?} \Rightarrow$ $\quad \quad \text{pdAuth.toSeqNo} < (\text{conAuthPurse pdAuth.to}).\text{nextSeqNo})$ $\quad \wedge (\text{pdAuth.from} \neq \text{name?} \Rightarrow$ $\quad \quad \text{pdAuth.fromSeqNo} < (\text{conAuthPurse pdAuth.from}).\text{nextSeqNo})$ $\quad \wedge (\forall \text{pdIn} : \text{fromInEpr} \bullet \text{pdIn} \neq \text{pdAuth})$ $\quad \wedge ((\text{pdAuth.from}, \text{pdAuth}) \notin \text{archive} \Rightarrow$ $\quad \quad \text{pdAuth} \in (\text{conAuthPurse pdAuth.from}).\text{exLog})) \wedge$ $(\forall \text{xEPR} : \text{PayDetails} \mid \text{xEPR.from} \in \text{dom conAuthPurse} \wedge$ $\quad (\text{conAuthPurse xEPR.from}).\text{status} = \text{epv} \wedge$ $\quad (\text{conAuthPurse xEPR.from}).\text{pdAuth} = \text{xEPR} \bullet$ $\quad \text{xEPR.from} \neq \text{name?}) \wedge$ $(\forall \text{pd} : \text{PayDetails} \mid \text{pd.from} \in \text{dom conAuthPurse} \bullet \text{pd} \neq \text{pdAuth})$ $\wedge \theta \text{ConPurse}[\text{balance} := \text{balance} - \text{pdAuth.value},$ $\quad \text{name} := \text{pdAuth.from}, \text{status} := \text{epa}] \notin \text{ran conAuthPurse}$

Table 8.3: Preconditions for promoted $\mathcal{B}$ operations (see Chapter 5)

Operation		Precondition
<i>Val</i>	<i>Ignore</i> <i>ValOkay</i>	$true$ $\text{pre } \text{PhiBOP} \wedge \text{pre } \text{ValPurseOkay} \wedge$ $(\text{ack } \text{pdAuth} \notin \text{ether} \Rightarrow \text{pdAuth.to} \in \text{dom } \text{conAuthPurse}) \wedge$ $(\forall xEPV : \text{PayDetails} \mid xEPV.to \in \text{dom } \text{conAuthPurse}$ $\quad \wedge (\text{conAuthPurse } xEPV.to).status = \text{epv}$ $\quad \wedge (\text{conAuthPurse } xEPV.to).pdAuth = xEPV \bullet xEPV.to \neq \text{name?})$ $\wedge (\forall \text{pd} : \text{PayDetails} \mid \text{pd.to} \in \text{dom } \text{conAuthPurse} \bullet \text{pd} \neq \text{pdAuth})$ $\wedge \theta \text{ConPurse}[\text{balance} := \text{balance} + \text{pdAuth.value}, \text{status} := \text{eaTo}] \notin$ $\text{ran } \text{conAuthPurse}$
<i>Ack</i>	<i>Ignore</i> <i>AckOkay</i>	$true$ $\text{pre } \text{PhiBOP} \wedge \text{pre } \text{AckPurseOkay} \wedge$ $(\forall xEPA : \text{PayDetails} \mid xEPA.from \in \text{dom } \text{conAuthPurse}$ $\quad \wedge (\text{conAuthPurse } xEPA.from).status = \text{epa}$ $\quad \wedge (\text{conAuthPurse } xEPA.from).pdAuth = xEPA \bullet$ $\quad \quad xEPA.from \neq \text{name?}) \wedge$ $\theta \text{ConPurse}[\text{status} := \text{eaFrom}] \notin \text{ran } \text{conAuthPurse}$
<i>Read</i> <i>Exception</i> <i>Log</i>	<i>Ignore</i> <i>Read</i> <i>Exception</i> <i>LogOkay</i>	$true$ $\text{pre } \text{PhiBOP} \wedge \text{pre } \text{ReadExceptionLogPurseEafromOkay} \wedge \perp \in \text{ether}$
<i>Clear</i> <i>Exception</i> <i>Log</i>	<i>Ignore</i> <i>Abort</i> <i>Clear</i> <i>Exception</i> <i>LogOkay</i>	$true$ $true$ $\text{pre } \text{PhiBOP} \wedge \text{pre } \text{ClearExceptionLogPurseEafromOkay} \wedge$ $(\forall \text{pd} : \text{PayDetails} \bullet \neg \text{pd} \in (\text{conAuthPurse } \text{name?}).\text{exLog})$ $\theta \text{ConPurse}[\text{exLog} := \{\}] \notin \text{ran } \text{conAuthPurse}$
<i>Authorise</i> <i>ExLog</i> <i>Clear</i>	<i>Ignore</i> <i>Authorise</i> <i>ExLog</i> <i>ClearOkay</i>	$true$ $true$
<i>Archive</i>		$true$

Table 8.4: (cont.) Preconditions for promoted $\mathcal{B}$ operations (see Chapter 5)

Operation		Precondition
<i>PhiCOp</i>		$m? \in \text{etherCh7} \wedge \text{name?} \in \text{dom conAuthPurseCh7} \wedge \theta \text{ConPurse} = \text{conAuthPurseCh7 name?}$
<i>CIgnore</i>		<i>true</i>
<i>CIncrease</i>	<i>CIgnore</i> <i>CIncreaseOkay</i>	<i>true</i> $\text{pre PhiCOp} \wedge \perp \in \text{ether}$
<i>CAbort</i>	<i>CIgnore</i> <i>CAbortOkay</i>	<i>true</i> $\text{pre PhiCOp} \wedge \text{pre AbortPurseOkay} \wedge \perp \in \text{etherCh7}$
<i>CStartFrom</i>	<i>CIgnore</i> <i>CAbort</i> <i>CStartFromOkay</i>	<i>true</i> <i>true</i> $\text{pre PhiCOp} \wedge \text{pre StartFromPurseEafromOkay} \wedge \dots$
<i>CStartTo</i>	<i>CIgnore</i> <i>CAbort</i> <i>CStartToOkay</i>	<i>true</i> <i>true</i> $\text{pre PhiCOp} \wedge \text{pre StartToPurseEafromOkay} \wedge \dots$
<i>CReq</i>	<i>CIgnore</i> <i>CReqOkay</i>	<i>true</i> $\text{pre PhiCOp} \wedge \text{pre ReqPurseOkay} \wedge \dots$
<i>CVal</i>	<i>CIgnore</i> <i>CValOkay</i>	<i>true</i> $\text{pre PhiCOp} \wedge \text{pre ValPurseOkay} \wedge \dots$
<i>CAck</i>	<i>CIgnore</i> <i>CAckOkay</i>	<i>true</i> $\text{pre PhiCOp} \wedge \text{pre AckPurseOkay} \wedge \dots$
<i>CRead</i> <i>Exception</i> <i>Log</i>	<i>CIgnore</i> <i>CRead</i> <i>Exception</i> <i>LogOkay</i>	<i>true</i> $\text{pre PhiCOp} \wedge \text{pre ReadExceptionLogPurseEafromOkay} \wedge \dots$
<i>CClear</i> <i>Exception</i> <i>Log</i>	<i>CIgnore</i> <i>CClear</i> <i>Exception</i> <i>LogOkay</i>	<i>true</i> $\text{pre PhiCOp} \wedge \text{pre ClearExceptionLogPurseEafromOkay} \wedge \dots$
<i>CAuthorise</i> <i>ExLog</i> <i>Clear</i>	<i>CIgnore</i> <i>CAuthorise</i> <i>ExLog</i> <i>ClearOkay</i>	<i>true</i> $\text{pre PhiCOp} \wedge \dots$
<i>CArchive</i>		<i>true</i>

Table 8.5: (Preconditions for $\mathcal{C}$ operations (see Chapter 7))

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	90
Total	0	99

Table 8.6: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	0
Theorems	5	7
Total	5	7

Table 8.7: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	47	70
Total	47	82

Table 8.8: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	38	58
Theorems	67	68
Axiomatic consistency	0	5
Total	105	131

Table 8.9: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	3	6
Assumption rules	0	18
Rewriting rules	3	17
Forward rules	1	19
Total	7	60

Table 8.10: Summary of *Z/Eves* rulees

Part III

First refinement: $\mathcal{A}$ to $\mathcal{B}$

Part IV

First refinement: $\mathcal{A}$ to $\mathcal{B}$

Chapter 9

Refinement Proof Rules

No formal material to be added.

Chapter 10

$\mathcal{A}$ to $\mathcal{B}$ retrieve relation

10.1 Extended theory for *sumValue*

TODO

10.2 Retrieve state

Original schema 10.1 (Retrieve state of $\mathcal{A}$ and $\mathcal{B}$ models)

<i>AbstractBetween</i>	_____
<i>AbWorld</i>	
<i>BetweenWorld</i>	
dom <i>abAuthPurse</i> = dom <i>conAuthPurse</i>	

□

10.2.1 Exposing *chosenLost*

Retrieve state

Here *chosenLost* is finite by definition but because it is a subset of *maybeLost*, but it is not given as one.

Original schema 10.2 (Retrieve state of between and concrete models)

<i>RabCl</i>	_____
<i>AbstractBetween</i>	
<i>chosenLost</i> : $\mathbb{P}$ <i>PayDetails</i>	
<i>chosenLost</i> $\in \mathbb{F}$ <i>PayDetails</i>	
<i>chosenLost</i> $\subseteq$ <i>maybeLost</i>	
$\forall$ <i>name</i> : dom <i>conAuthPurse</i> •	
$ \begin{aligned} & (abAuthPurse\ name).lost = \\ & \quad sumValue\ ((definitelyLost \cup chosenLost) \\ & \quad \cap \{ pd : PayDetails \mid pd.from = name \}) \end{aligned} $	
$ \begin{aligned} & \wedge (abAuthPurse\ name).balance = \\ & \quad (conAuthPurse\ name).balance \\ & \quad + sumValue\ ((maybeLost \setminus chosenLost) \\ & \quad \cap \{ pd : PayDetails \mid pd.to = name \}) \end{aligned} $	

□

10.2.2 Hiding $chosenLost$

Original schema 10.3 (Hiding $chosenLost$)

$$Rab \triangleq \exists chosenLost : \mathbb{P} PayDetails \bullet RabCl$$

□

10.2.3 Exposing $pdThis$

Original schema 10.4 (Exposing $pdThis$)

$RabClPd$	_____
$RabCl$	
$pdThis : PayDetails$	
$true$	

□

Original schema 10.5 (Other purses in the retrieve state)

$OtherPursesRab$	_____
$AbstractBetween$	
$chosenLost : \mathbb{P} PayDetails$	
$pdThis : PayDetails$	
$\forall name : \text{dom } conAuthPurse \mid name \notin \{pdThis.from, pdThis.to\} \bullet$	
$(abAuthPurse name).lost =$	
$\quad sumValue((definitelyLost \cup chosenLost)$	
$\quad \cap \{pd : PayDetails \mid pd.from = name\})$	
$\wedge (abAuthPurse name).balance =$	
$\quad (conAuthPurse name).balance$	
$\quad + sumValue((maybeLost \setminus chosenLost)$	
$\quad \cap \{pd : PayDetails \mid pd.to = name\})$	

□

Use this intermediary lemma to avoid expansion of *BetweenWorld* with corresponding (huge) substitutions for *maybeLost*, which lead **Z/Eves** to the brink of crashing.

Added lemma 10.1 ($maybeLost \setminus chosenLost$ member purses $\in \text{dom } conAuthPurse$)

theorem lRabOkayClPdInDomConAuthPurse
 $\forall AuxWorld; pdThis : PayDetails; chosenLost : \mathbb{P} PayDetails \mid$
 $chosenLost \subseteq maybeLost$
 $\wedge pdThis \in maybeLost \setminus chosenLost \bullet$
 $pdThis.from \in \text{dom } conAuthPurse$
 $\wedge pdThis.to \in \text{dom } conAuthPurse$

□

Original schema 10.6 (Retrieve state of $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ for *Okay* operations)

RabOkayClPd

AbstractBetween

chosenLost : $\mathbb{P}$ *PayDetails*

pdThis : *PayDetails*

chosenLost $\subseteq$ *maybeLost*

pdThis $\in$ *maybeLost* $\setminus$ *chosenLost*

$(abAuthPurse\ pdThis.from).balance =$
 $(conAuthPurse\ pdThis.from).balance$
 $+ \text{sumValue } (((maybeLost \setminus chosenLost)$
 $\cap \{pd : PayDetails \mid pd.to = pdThis.from\})$
 $\setminus \{pdThis\})$

$(abAuthPurse\ pdThis.to).balance =$
pdThis.value
 $+ (conAuthPurse\ pdThis.to).balance$
 $+ \text{sumValue } (((maybeLost \setminus chosenLost)$
 $\cap \{pd : PayDetails \mid pd.to = pdThis.to\})$
 $\setminus \{pdThis\})$

$\forall name : \{pdThis.from, pdThis.to\} \bullet$

$(abAuthPurse\ name).lost =$
 $\text{sumValue } (((definitelyLost \cup chosenLost)$
 $\cap \{pd : PayDetails \mid pd.from = name\})$
 $\setminus \{pdThis\})$

OtherPursesRab

□

Added lemma 10.2 (*chosenLost* member purses $\in$ dom *conAuthPurse*)

theorem lRabWillBeLostClPdInDomConAuthPurse

$\forall AuxWorld; pdThis : PayDetails; chosenLost : \mathbb{P} PayDetails \mid$

chosenLost $\subseteq$ *maybeLost*

$\wedge pdThis \in chosenLost \bullet$

pdThis.from $\in$ dom *conAuthPurse*

$\wedge pdThis.to \in$ dom *conAuthPurse*

□

Original schema 10.7 (Retrieve state of $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ for future *Lost* operations)

RabWillBeLostClPd

AbstractBetween

chosenLost : $\mathbb{P}$ *PayDetails*

pdThis : *PayDetails*

chosenLost $\subseteq$ *maybeLost*

pdThis $\in$ *chosenLost*

$(abAuthPurse\ pdThis.from).lost =$
 $\quad pdThis.value$
 $\quad + sumValue\ (((definitelyLost \cup chosenLost)$
 $\quad \cap \{pd : PayDetails \mid pd.from = pdThis.from\})$
 $\quad \setminus \{pdThis\})$

$(abAuthPurse\ pdThis.to).lost =$
 $\quad sumValue\ (((definitelyLost \cup chosenLost)$
 $\quad \cap \{pd : PayDetails \mid pd.from = pdThis.to\})$
 $\quad \setminus \{pdThis\})$

$\forall name : \{pdThis.from, pdThis.to\} \bullet$

$(abAuthPurse\ name).balance =$
 $(conAuthPurse\ name).balance$
 $\quad + sumValue\ (((maybeLost \setminus chosenLost)$
 $\quad \cap \{pd : PayDetails \mid pd.to = name\})$
 $\quad \setminus \{pdThis\})$

OtherPursesRab

□

Added lemma 10.3 (*definitelyLost* member purses $\in$ dom *conAuthPurse*)

theorem lRabHasBeenLostClPdInDomConAuthPurse

$\forall AuxWorld; pdThis : PayDetails; chosenLost : \mathbb{P}\ PayDetails \mid$
 $\quad chosenLost \subseteq maybeLost$
 $\quad \wedge pdThis \in definitelyLost \bullet$
 $\quad \quad pdThis.from \in \text{dom } conAuthPurse$
 $\quad \quad \wedge pdThis.to \in \text{dom } conAuthPurse$

□

Original schema 10.8 (Retrieve state of $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ for past *Lost* operations)

<i>RabHasBeenLostClPd</i> <i>AbstractBetween</i> <i>chosenLost</i> : $\mathbb{P} \text{ PayDetails}$ <i>pdThis</i> : <i>PayDetails</i> <i>chosenLost</i> $\subseteq$ <i>maybeLost</i> <i>pdThis</i> $\in$ <i>definitelyLost</i> $(\text{abAuthPurse } \text{pdThis}.\text{from}).\text{lost} =$ $\text{pdThis}.\text{value}$ $+ \text{sumValue } (((\text{definitelyLost} \cup \text{chosenLost})$ $\cap \{ \text{pd} : \text{PayDetails} \mid \text{pd}.\text{from} = \text{pdThis}.\text{from} \})$ $\setminus \{ \text{pdThis} \})$ $(\text{abAuthPurse } \text{pdThis}.\text{to}).\text{lost} =$ $\text{sumValue } (((\text{definitelyLost} \cup \text{chosenLost})$ $\cap \{ \text{pd} : \text{PayDetails} \mid \text{pd}.\text{from} = \text{pdThis}.\text{to} \})$ $\setminus \{ \text{pdThis} \})$ $\forall \text{name} : \{ \text{pdThis}.\text{from}, \text{pdThis}.\text{to} \} \bullet$ $(\text{abAuthPurse } \text{name}).\text{balance} =$ $(\text{conAuthPurse } \text{name}).\text{balance}$ $+ \text{sumValue } (((\text{maybeLost} \setminus \text{chosenLost})$ $\cap \{ \text{pd} : \text{PayDetails} \mid \text{pd}.\text{to} = \text{name} \})$ $\setminus \{ \text{pdThis} \})$ <i>OtherPursesRab</i>

□

Original schema 10.9 (Retrieve state of $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$ for finalisation)

<i>RabEndClPd</i> <i>AbstractBetween</i> <i>chosenLost</i> : $\mathbb{P} \text{ PayDetails}$ <i>pdThis</i> : <i>PayDetails</i> <i>chosenLost</i> $\subseteq$ <i>maybeLost</i> <i>pdThis</i> $\notin$ <i>definitelyLost</i> $\cup$ <i>maybeLost</i> $\forall \text{name} : \text{dom } \text{conAuthPurse} \cap \{ \text{pdThis}.\text{from}, \text{pdThis}.\text{to} \} \bullet$ $(\text{abAuthPurse } \text{name}).\text{lost} =$ $\text{sumValue } (((\text{definitelyLost} \cup \text{chosenLost})$ $\cap \{ \text{pd} : \text{PayDetails} \mid \text{pd}.\text{from} = \text{name} \})$ $\setminus \{ \text{pdThis} \})$ $\wedge (\text{abAuthPurse } \text{name}).\text{balance} =$ $(\text{conAuthPurse } \text{name}).\text{balance}$ $+ \text{sumValue } (((\text{maybeLost} \setminus \text{chosenLost})$ $\cap \{ \text{pd} : \text{PayDetails} \mid \text{pd}.\text{to} = \text{name} \})$ $\setminus \{ \text{pdThis} \})$ <i>OtherPursesRab</i>
--

□

10.2.4 Partition

Lemma ‘lost’

In here we introduce a similar expansion of *AuxWorld* but assuming *BetweenWorld*. These are just convenience (repetitive) lemmas.

Added lemma 10.4 ($\mathcal{B}$ *fromLogged* extraction)

theorem lBetweenWorldFromLoggedExpansion
 $\forall \text{BetweenWorld} \bullet \text{fromLogged} = \{ pd : \text{authenticFrom} \mid pd.\text{from} \mapsto pd \in \text{allLogs} \}$

□

Added lemma 10.5 ($\mathcal{B}$ *toLogged* extraction)

theorem lBetweenWorldToLoggedExpansion
 $\forall \text{BetweenWorld} \bullet \text{toLogged} = \{ pd : \text{authenticTo} \mid pd.\text{to} \mapsto pd \in \text{allLogs} \}$

□

Added lemma 10.6 ($\mathcal{B}$ *authenticFrom* extraction)

theorem lBetweenWorldAuthenticFromExpansion
 $\forall \text{BetweenWorld} \bullet \text{authenticFrom} = \{ pd : \text{PayDetails} \mid pd.\text{from} \in \text{dom } \text{conAuthPurse} \}$

□

Added lemma 10.7 ($\mathcal{B}$ *authenticTo* extraction)

theorem lBetweenWorldAuthenticToExpansion
 $\forall \text{BetweenWorld} \bullet \text{authenticTo} = \{ pd : \text{PayDetails} \mid pd.\text{to} \in \text{dom } \text{conAuthPurse} \}$

□

Added lemma 10.8 ($\mathcal{B}$ *toInEpv* extraction)

theorem lBetweenWorldToInEpvExpansion
 $\forall \text{BetweenWorld} \bullet \text{toInEpv} = \{ pd : \text{authenticTo} \mid$
 $(\text{conAuthPurse}pd.\text{to}).\text{status} = \text{epv} \wedge (\text{conAuthPurse}pd.\text{to}).pdAuth = pd \}$

□

Added lemma 10.9 ($\mathcal{B}$ *toInEapayee* extraction)

theorem lBetweenWorldToInEapayeeExpansion
 $\forall \text{BetweenWorld} \bullet \text{toInEapayee} = \{ pd : \text{authenticTo} \mid$
 $(\text{conAuthPurse}pd.\text{to}).\text{status} = \text{eaTo} \wedge (\text{conAuthPurse}pd.\text{to}).pdAuth = pd \}$

□

Added lemma 10.10 ($\mathcal{B}$ *fromInEpr* extraction)

theorem lBetweenWorldFromInEprExpansion
 $\forall \text{BetweenWorld} \bullet \text{fromInEpr} = \{ pd : \text{authenticFrom} \mid$
 $(\text{conAuthPurse}pd.\text{from}).\text{status} = \text{epr} \wedge (\text{conAuthPurse}pd.\text{from}).pdAuth = pd \}$

□

Added lemma 10.11 ($\mathcal{B}$ *fromInEpa* extraction)

theorem lBetweenWorldFromInEpaExpansion
 $\forall \text{BetweenWorld} \bullet \text{fromInEpa} = \{ pd : \text{authenticFrom} \mid$
 $(\text{conAuthPurse}pd.\text{from}).\text{status} = \text{epa} \wedge (\text{conAuthPurse}pd.\text{from}).pdAuth = pd \}$

□

Added lemma 10.12 ($\mathcal{B}$ *allLogs* extraction)

theorem lBetweenWorldAllLogsExpansion
 $\forall \text{BetweenWorld} \bullet \text{allLogs} = \text{archive} \cup$
 $\{ n : \text{dom } \text{conAuthPurse}; pd : \text{PayDetails} \mid pd \in (\text{conAuthPurse } n).\text{exLog} \}$

□

Added lemma 10.13 ($\mathcal{B}$ *definitelyLost* extraction)

theorem lBetweenWorldDefinitelyLostExpansion
 $\forall \text{BetweenWorld} \bullet \text{definitelyLost} = \text{toLogged} \cap (\text{fromLogged} \cup \text{fromInEpa})$

□

Added lemma 10.14 ($\mathcal{B}$ *maybeLost* extraction)

theorem lBetweenWorldMaybeLostExpansion
 $\forall \text{BetweenWorld} \bullet \text{maybeLost} = (\text{fromInEpa} \cup \text{fromLogged}) \cap \text{toInEpr}$

□

Added lemma 10.15 ($\mathcal{B}$ *archive* extraction)

theorem lBetweenWorldArchiveExpansion
 $\forall \text{BetweenWorld} \bullet \text{archive} \in \mathbb{P}(\{ \text{PayDetails} \bullet \text{from} \mapsto \theta \text{PayDetails} \} \cup \{ \text{PayDetails} \bullet \text{to} \mapsto \theta \text{PayDetails} \})$

□

Original lemma 10.1 (Lemma ‘notLoggedAndIn’ (C.12))

theorem lNotLoggedAndInC12
 $\forall \text{BetweenWorld} \bullet (\text{fromInEpr} \cup \text{fromInEpa}) \cap \text{fromLogged} = \emptyset \wedge (\text{toInEpr} \cup \text{toInEpayee}) \cap \text{toLogged} = \emptyset$

□

Original lemma 10.2 (Lemma ‘lost’ (C.13))

theorem lLostC13
 $\forall \text{BetweenWorld} \bullet \text{definitelyLost} \cap \text{maybeLost} = \emptyset$

□

Original theorem 10.1 (Retrieve state partition)

theorem tRetrievePartition
 $\forall \text{RabClPd} \bullet \text{RabClPd} \Leftrightarrow (\text{RabOkayClPd} \vee \text{RabWillBeLostClPd} \vee \text{RabHasBeenLostClPd} \vee \text{RabEndClPd})$

□

10.2.5 Quantified forms

Original theorem 10.2 (Retrieve state partition $\forall$ format)

theorem tRetrieveForAllRabClPd
 $\forall \text{RabCl} \bullet \text{RabCl} \Leftrightarrow (\forall \text{pdThis} : \text{PayDetails} \bullet \text{RabClPd})$

□

There is one problem with the existential equivalence. It assumes that there is at least one payment done. This might not be necessarily true if no payment *from-to* a purse has taken place. This is fixed by the structured way we redefined names in Section 3.1.1, where we axiomatically state that there exists at least two different names we can refer to, so that a payment can be made.

Original theorem 10.3 (Retrieve state partition $\exists$ format)

theorem tRetrieveExistsRabClPd
 $\forall \text{RabCl} \bullet \text{RabCl} \Leftrightarrow (\exists \text{pdThis} : \text{PayDetails} \bullet \text{RabClPd})$

□

As we approached Chapter 14, it became clear that the unproved goal from this predicate

$\exists \text{pd} : \text{PayDetail} \bullet \text{true}$

is later discharged, where *true* varies according to the operation being refined.

10.2.6 The full Retrieve state relation

Original schema 10.10 (Complete retrieve state for *Okay* operations)

$$RabOkay \hat{=} RabOkayClPd \setminus (pdThis, chosenLost)$$

□

Original schema 10.11 (Complete retrieve state for future *Lost* operations)

$$RabWillBeLost \hat{=} RabWillBeLostClPd \setminus (pdThis, chosenLost)$$

□

Original schema 10.12 (Complete retrieve state for past *Lost* operations)

$$RabHasBeenLost \hat{=} RabHasBeenLostClPd \setminus (pdThis, chosenLost)$$

□

Original schema 10.13 (Complete retrieve state for finalisation operations)

$$RabEnd \hat{=} RabEndClPd \setminus (pdThis, chosenLost)$$

□

10.3 Retrieve inputs

Original schema 10.14 (Complete retrieve state inputs)

$$RabIn \hat{=} BetwInitIn[a?/g?]$$

□

10.4 Retrieve outputs

Original schema 10.15 (Complete retrieve state outputs)

$$RabOut \hat{=} BetwFinOut[a!/g!]$$

□

10.5 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	15	105
Total	15	114

Table 10.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	2	2
Theorems	3	10
Total	5	12

Table 10.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 10.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	15	73
Theorems	0	68
Axiomatic consistency	0	5
Total	15	146

Table 10.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	17
Forward rules	0	19
Total	0	60

Table 10.5: Summary of *Z/Eves* rulees

Chapter 11

$\mathcal{A}$ to $\mathcal{B}$ initialisation proof

11.1 Proof obligations

For simulation proofs we use the convention to include in theorem names the suffixes *SimAppl* and *SimCorr* for the applicability and correctness theorems.

Original theorem 11.1 ($\mathcal{B}$ world init. refines $\mathcal{A}$ world init.)

theorem tBetwAbsInitSimAppl
 $\forall \textit{BetweenInitState}; \textit{Rab}' \bullet \textit{AbInitState}$

□

Original theorem 11.2 ($\mathcal{B}$ world init. refines $\mathcal{A}$ world init.)

theorem tBetwAbsInputSimAppl
 $\forall \textit{BetwInitIn}; \textit{RabIn} \bullet \textit{AbInitIn}$

□

11.2 Proof of initial inputs

Expand *RabIn* and *AbInitIn*.

Original theorem 11.3 ($\mathcal{B}$ and $\mathcal{A}$ world initial inputs)

theorem tBetwAbsInitInputs
 $\forall \textit{BetwInitIn}; \textit{BetwInitIn}[a?/g?] \bullet a? = g?$

□

11.3 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 11.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	2
Theorems	3	13
Total	3	15

Table 11.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 11.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	73
Theorems	0	68
Axiomatic consistency	0	5
Total	0	146

Table 11.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	17
Forward rules	0	19
Total	0	60

Table 11.5: Summary of *Z/Eves* rulees

Chapter 12

$\mathcal{A}$ to $\mathcal{B}$ finalisation proof

12.1 Proof obligations

The requirement is to prove that the between final state correctly refines the abstract final state, and the between outputs correctly refine the abstract outputs. That is,

$$\begin{aligned} \text{BetwFinOut} &\vdash \exists a! : AOUT \bullet RabOut \wedge AbFinOut \\ \text{BetwFinState} &\vdash \exists AbWorld \bullet Rab \wedge AbFinState \end{aligned}$$

12.2 Output proofs

Original theorem 12.1 ($\mathcal{B}$ world outputs refines $\mathcal{A}$ world outputs.)

$$\begin{aligned} \text{theorem } t\text{BetwAbsFinalOutput} \\ \forall \text{BetwFinOut} \bullet \exists a! : AOUT \bullet RabOut \wedge AbFinOut \end{aligned}$$

□

12.3 State proof

From inference rule *B.5*, we can $[cut]$ $t\text{BetwAbsFinalState}$ to introduce an $AbWorld$ witness, where

$$\begin{aligned} R &\hat{=} \text{BetwFinState} \\ P &\hat{=} (\exists AbWorld \bullet Rab \wedge AbFinState) \\ D &\hat{=} AbWorld \\ Q &\hat{=} abAuthPurse = gAuthPurse \end{aligned}$$

In **Z/Eves**, this means instantiating $AbWorld.abAuthPurse$ with $gAuthPurse$. Nevertheless, as **Z/Eves** uses sequence calculus, we would not be able to apply case analysis this time because we would have two conjoined negations on the assumptions rather than two conjoined predicates on the conclusions. Therefore, we add the cases separately as in the PRG, but with a different “ $[cut]$ ”!

12.3.1 Case $AbFinState$

This lemma is similar to that of *mondex*, with substitution directly.

Original lemma 12.1 (Case: *cut* on $AbWorld$ for the $AbFinState$ (12.3.1))

$$\begin{aligned} \text{theorem } l\text{BetwAbsFinalStateAbWorldCutAbFinStateCase} \\ \forall \text{BetwFinState} \bullet AbFinState[gAuthPurse/abAuthPurse] \end{aligned}$$

□

12.3.2 Case *Rab*

Again the explanation of the proof of the next case was quite helpful and made the automation straightforward. We just needed to add additional (obvious) lemmas about the finiteness of the *chosenLost* witness as *maybeLost*, and some time information about *ConPurse balance*.

Rewriting rule 12.1 (Expanded abstract purse *balance* type)

theorem rule rConPurseBalanceTypeOpen
 $\forall \text{purse} : \text{NAMES} \multimap \text{ConPurse}; \text{name} : \text{NAMES} \mid \text{name} \in \text{dom purse} \bullet$
 $(\text{purse name}).\text{balance} \in \mathbb{N}$

□

Added lemma 12.1 (*maybeLost* is a finite set)

theorem lBetweenWorldMaybeLostIsFinite
 $\forall \text{BetweenWorld} \bullet \text{maybeLost} \in \mathbb{F} \text{PayDetails}$

□

Original lemma 12.2 (Case: *cut* on *AbWorld* for the *Rab* (12.3.2))

theorem lBetwAbsFinalStateAbWorldCutRabCase
 $\forall \text{BetwFinState} \bullet \text{Rab}[g\text{AuthPurse}/ab\text{AuthPurse}]$

□

12.3.3 Actual state related theorem

Original theorem 12.2 (*B* world final. refines *A* world final.)

theorem tBetwAbsFinalState
 $\forall \text{BetwFinState} \bullet \exists \text{AbWorld} \bullet \text{Rab} \wedge \text{AbFinState}$

□

12.4 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 12.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	2	4
Theorems	2	15
Total	4	19

Table 12.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 12.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	1	74
Theorems	0	68
Axiomatic consistency	0	5
Total	1	147

Table 12.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	1	18
Forward rules	0	19
Total	1	61

Table 12.5: Summary of *Z/Eves* rulees

Chapter 13

$\mathcal{A}$ to $\mathcal{B}$ applicability proofs

Comes directly from Section 8.6.3

13.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 13.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	4
Theorems	0	15
Total	0	19

Table 13.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 13.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 13.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 13.5: Summary of *Z/Eves* rulees

Chapter 14

Lemmas for the $\mathcal{A}$ to $\mathcal{B}$ correctness proofs

In this Chapter a series of lemmas for the backward refinement proofs are explained. Nevertheless, by the nature of the inference system of **Z/Eves** many of these are trivially characterised if not already present via some of the (*prenex*) tactics to deal with quantifiers.

Moreover, some of the lemmas are in fact a proof strategy as meta-theorems for a general operation *BOp*, instead of the actual operation name. Therefore, we **DO NOT** include the lemmas as they are presented because: (i) they are an informal argument with meta-lemmas; (ii) they are only suitable for proving the actual simulation theorems by hand.

Instead, we provide lemmas/definitions that are similar to those of Mondex, but that suits **Z/Eves** needs. In this path, we **provide the actual proof steps** to reach to such **Z/Eves** goal from the original Mondex text. Basically, we “expand” the meta-theorems into their actual forms for each *Okay* operation in $\mathcal{B}$, as well as the special cases for *Ignore* and *Abort*. In the end, because we prove the same backward refinement theorems with a quite similar/close proof strategy, it does not matter which particular route (of lemmas) we have chosen to follow.

14.1 Introduction

14.2 Lemma ‘multiple refinement’

In this section, there is no additional lemma for **Z/Eves**.

14.3 Lemma ‘ignore’: separating the branches

Each between operation *BOp* is promoted from *BOpPurseOkay*, disjoined with *Ignore*, and sometimes with *Abort*. Call the first disjunction *BOpOkay*:

$$BOpOkay \triangleq \exists \Delta ConPurse \bullet PhiBOp \wedge BOpPurseOkay$$

Here the *BOpPurseOkay* are one of the following:

<i>AbortPurseOkay</i>	<i>IncreasePurseOkay</i>	<i>StartFromPurseOkay</i>
<i>StartToPurseOkay</i>	<i>ReqPurseOkay</i>	<i>ValPurseOkay</i>
<i>AckPurseOkay</i>	<i>ReadExceptionLogPurseOkay</i>	<i>ClearExceptionLogPurseOkay</i>
<i>AuthoriseExLogClearOkay</i>	<i>Archive</i>	

For this, we added new definitions for those operations separating the disjunction with *Ignore* and/or *Abort*. These disjuncts come from Section 5.5 onwards. As they are needed for the precondition proofs we must include them in Chapter 8 rather than here. They are given in Sections 8.5.3 and 8.5.4.

14.4 Lemma ‘deterministic’: simplifying the *Okay* branch

In here, although we do not use the lemmas of the first two sections to choose an input and $\Delta ConPurse$ because **Z/Eves** prenexing will expose this directly, the way *pdThis* is exposed is quite interesting and demands closer inspection, as we do in the following subsection 14.4.3.

14.4.1 Choosing an input

That is, in Mondex, we had the goal

$$\begin{array}{l} BOpOkay; RabOut; Rab'; a? : AIN \mid RabIn \\ \vdash \\ RabIn \wedge (\exists AbWorld \bullet Rab \wedge AOp) \end{array}$$

In **Z/Eves** that means a rule or lemma that will expose (or weaken) such goal via what in the PRG is called *[cut]*. If we assume the operation for *BOpOkay* as *Increase* without *Ignore*, and the *AOp* to be *AbTransferOkay*, we would have

$$\begin{array}{l} IncreaseOkay; RabOut; Rab'; a? : AIN \mid RabIn \\ \vdash \\ RabIn \wedge (\exists AbWorld \bullet Rab \wedge AbTransferOkay) \end{array}$$

For **Z/Eves** this is state as

$$\begin{array}{l} \forall IncreaseOkay; RabOut; Rab'; a? : AIN \mid RabIn \bullet \\ RabIn \wedge (\exists AbWorld \bullet Rab \wedge AbTransferOkay) \end{array}$$

As pointed out in Mondex, because *RabIn* already contains $a? \in AIN$, it can be further simplified in **Z/Eves** after expanding *RabIn* to

$$\begin{array}{l} IncreaseOkay \wedge RabOut \wedge Rab' \wedge a? \in AIN \wedge BetwInitIn[a?/g?] \\ \Rightarrow \\ (\exists AbWorld \bullet Rab \wedge AbTransferOkay) \end{array}$$

with invoke *RabIn* and simplify.

This is not proved in Mondex, but just pointed out as a proof strategy. So, nothing need to be added here.

14.4.2 Cutting in $\Delta ConPurse$

Mondex follows the previous result and simplifies it once more by expanding the promoted operation. For our running example in **Z/Eves**, we need to expand *IncreaseOkay* with invoke *IncreaseOkay* in

$$\begin{array}{l} IncreaseOkay \wedge RabOut \wedge Rab' \wedge RabIn \\ \Rightarrow \\ (\exists AbWorld \bullet Rab \wedge AbTransferOkay) \end{array}$$

and reach

$$\begin{array}{l} (\exists \Delta ConPurse \bullet PhiBOp \wedge IncreasePurseOkay) \wedge \\ RabOut \wedge Rab' \wedge RabIn \\ \Rightarrow \\ (\exists AbWorld \bullet Rab \wedge AbTransferOkay) \end{array}$$

which we can further prenex to

$$\begin{array}{l} \Delta ConPurse \wedge PhiBOp \wedge IncreasePurseOkay \\ \wedge RabOut \wedge Rab' \wedge RabIn \\ \Rightarrow \\ (\exists AbWorld \bullet Rab \wedge AbTransferOkay) \end{array}$$

14.4.3 Exposing *chosenLost* and *pdThis*

From the previous section, we now expand *Rab'* with invoke predicate *Rab'*, and get rid of the quantifier with *prenex* to reach

$$\begin{aligned}
& \Delta \text{ConPurse} \wedge \text{PhiBOP} \wedge \text{IncreasePurseOkay} \\
& \wedge \text{RabOut} \wedge \text{chosenLost} \in \mathbb{P} \text{PayDetails} \\
& \wedge \text{RabCl}'[\text{chosenLost}/\text{chosenLost}'] \wedge \text{RabIn} \\
& \Rightarrow \\
& (\exists \text{AbWorld} \bullet \text{Rab} \wedge \text{AbTransferOkay})
\end{aligned}$$

Next, Mondex declares a general (meta-) lemmas about every *BOPOkay* operation as

$$\begin{aligned}
& \text{PhiBOP}; \text{BOPPurseOkay}; \text{RabOut}; \text{RabCl}'; \text{RabIn} \\
& \vdash \\
& \exists \text{pdThis} : \text{PayDetails} \bullet \mathcal{P}
\end{aligned}$$

which is to be *[cut]* into the goals. For now, let us assume that $\mathcal{P}$ is just *true*. In **Z/Eves** that means to use the lemma. For our running example, the lemma is

$$\forall \text{PhiBOP}; \text{IncreasePurseOkay}; \text{RabOut}; \text{RabCl}'; \text{RabIn} \bullet (\exists \text{pdThis} : \text{PayDetails} \bullet \mathcal{P})$$

where $\mathcal{P}$ is to be defined earlier depending on the context. If we call this lemma

$$\text{IncreaseOkayPdThisLemma}$$

we can transform the goal to be

$$\begin{aligned}
& \Delta \text{ConPurse} \wedge \text{PhiBOP} \wedge \text{IncreasePurseOkay} \\
& \wedge \text{RabOut} \wedge \text{chosenLost} \in \mathbb{P} \text{PayDetails} \\
& \wedge \text{RabCl}'[\text{chosenLost}/\text{chosenLost}'] \wedge \text{RabIn} \\
& \wedge \text{pdThis} \in \text{PayDetails} \wedge \mathcal{P} \\
& \Rightarrow \\
& (\exists \text{AbWorld} \bullet \text{Rab} \wedge \text{AbTransferOkay})
\end{aligned}$$

after the proof commands

proof
 use *IncreaseOkayPdThisLemma*[*chosenLost'* := *chosenLost*];
 rearrange;
 simplify;
 prenex;
 ■

After that, we use the universally quantified version of *RabCl'* (see Section 10.2.5) and transform the goal into

$$\begin{aligned}
& \Delta \text{ConPurse} \wedge \text{PhiBOP} \wedge \text{IncreasePurseOkay} \\
& \wedge \text{RabOut} \wedge \text{chosenLost} \in \mathbb{P} \text{PayDetails} \\
& \wedge \text{RabCl}'[\text{chosenLost}/\text{chosenLost}'] \wedge \text{RabIn} \\
& \wedge \text{pdThis} \in \text{PayDetails} \wedge \mathcal{P} \\
& \wedge (\forall \text{pdThis}__0 : \text{PayDetails} \bullet \\
& \quad \text{RabClPd}'[\text{chosenLost}/\text{chosenLost}', \text{pdThis}__0/\text{pdThis}']) \\
& \Rightarrow \\
& (\exists \text{AbWorld} \bullet \text{Rab} \wedge \text{AbTransferOkay})
\end{aligned}$$

with the following proof commands

proof
 use *tRetrieveForAllRabClPd'*[*chosenLost'* := *chosenLost*];
 rearrange;
 simplify;
 simplify;
 ■

In **Z/Eves**, when we use a theorem name with a dash, all variables in its declaration are dashed. That is, for a theorem T , $T' \Leftrightarrow T'[-]$.

The next simplification is to instantiate quantified $pdThis_0$ with the known $pdThis$ from the lemma involving $\mathcal{P}$. This changes the goal to

$$\begin{aligned}
& \Delta ConPurse \wedge PhiBOP \wedge IncreasePurseOkay \\
& \wedge RabOut \wedge chosenLost \in \mathbb{P} PayDetails \\
& \wedge RabCl'[chosenLost/chosenLost'] \wedge RabIn \\
& \wedge pdThis \in PayDetails \wedge \mathcal{P} \\
& \wedge RabClPd'[chosenLost/chosenLost', pdThis/pdThis'] \wedge (\forall pdThis_0 : PayDetails \bullet \\
& \quad RabClPd'[chosenLost/chosenLost', pdThis_0/pdThis']) \\
& \Rightarrow \\
& (\exists AbWorld \bullet Rab \wedge AbTransferOkay)
\end{aligned}$$

after the proof steps below.

proof

instantiate $pdThis_0 == pdThis$;
rewrite;

■

Mondex then removes elements from the assumptions to simplify the goal. We cannot do this in **Z/Eves** and it is irrelevant for further proofs, since the assumptions are being removed for clarification of the goal only.

The next change, which uses the existentially quantified version of Rab in the consequent cannot be applied once more. That is because in **Z/Eves**, after expanding the goal to

$$\begin{aligned}
& \Delta ConPurse \wedge PhiBOP \wedge IncreasePurseOkay \\
& \wedge RabOut \wedge chosenLost \in \mathbb{P} PayDetails \\
& \wedge RabCl'[chosenLost/chosenLost'] \wedge RabIn \\
& \wedge pdThis \in PayDetails \wedge \mathcal{P} \\
& \wedge RabClPd'[chosenLost/chosenLost', pdThis/pdThis'] \wedge (\forall pdThis_0 : PayDetails \bullet \\
& \quad RabClPd'[chosenLost/chosenLost', pdThis_0/pdThis']) \\
& \Rightarrow \\
& (\exists AbWorld \bullet \\
& \quad (\exists chosenLost_0 : \mathbb{P} PayDetails \\
& \quad \bullet RabCl[chosenLost_0/chosenLost]) \\
& \quad \wedge AbTransferOkay)
\end{aligned}$$

with `invoke Rab`, we cannot `use tRetrieveExistsRabClPd` directly because $chosenLost_0$ is not available to use in the `use` proof step. What we can do is to say that $chosenLost_0$ is $chosenLost$. It leads to the following goal

$$\begin{aligned}
& \Delta ConPurse \wedge PhiBOP \wedge IncreasePurseOkay \\
& \wedge RabOut \wedge chosenLost \in \mathbb{P} PayDetails \\
& \wedge RabCl'[chosenLost/chosenLost'] \wedge RabIn \\
& \wedge pdThis \in PayDetails \wedge \mathcal{P} \\
& \wedge RabClPd'[chosenLost/chosenLost', pdThis/pdThis'] \wedge (\forall pdThis_0 : PayDetails \bullet \\
& \quad RabClPd'[chosenLost/chosenLost', pdThis_0/pdThis']) \\
& \Rightarrow \\
& (\exists AbWorld \bullet \\
& \quad (RabCl \vee \\
& \quad \quad (\exists chosenLost_0 : \mathbb{P} PayDetails \\
& \quad \quad \bullet RabCl[chosenLost_0/chosenLost]) \\
& \quad \wedge AbTransferOkay)
\end{aligned}$$

with proof steps

proof

invoke Rab ;
instantiate $chosenLost_0 == chosenLost$;
rewrite;

■

As **Z/Eves** does not allow application/usage of theorems in the consequent, we need to use a simplified version of the given goal. By fiddling with the original it is clear that it is just a matter of variable renaming.

For our running example for *IncreasePurseOkay*, we need to include another lemma relating the actual (*Increase*) operation and $\mathcal{Q}$ as

$$\begin{aligned} & \forall \text{PhiBOP}; \text{IncreasePurseOkay}; \text{RabOut}; \text{RabIn}; \text{pdThis} : \text{PayDetails}; \\ & \text{RabClPd}'[\text{pdThis}/\text{pdThis}'] \mid \mathcal{P} \bullet \\ & \exists \text{chosenLost} : \mathbb{P} \text{PayDetails} \bullet \mathcal{Q} \wedge \text{chosenLost} \subseteq \text{maybeLost} \end{aligned}$$

In the derivation of this section, the two lemmas from Section 10.2.5 are used. And here is the problem: the universally quantified equivalence (*tRetrieveForAllRabClPd*) holds, but the existentially quantified one (*tRetrieveExistsRabClPd*) does not! The proof is not possible because for *tRetrieveExistsRabClPd* we reach the predicate

$$\exists \text{pd} : \text{PayDetails} \bullet \text{true}$$

This is exactly the side condition to be proven below in Section 14.4.5 for each *BOPOkay* operation when

$$\mathcal{P} \Leftrightarrow \text{true}$$

This sheds light on the nature of the proofs being done at this stage. That is, because of the use of the existential equivalence via *tRetrieveExistsRabClPd* in order to expose *chosenLost* as required, we need to prove

$$\exists \text{pd} : \text{PayDetails} \bullet \mathcal{P}$$

and something similar for *chosenLost* (in $\mathcal{Q}$).

14.4.4 The existence of *AbWorld*

We also need another lemma from Appendix C as

Original lemma 14.1 (*AbWorld is unique within a choice in Betweenworld*)

theorem lAbWorldUniqueC15

$$\begin{aligned} & \forall \text{RabCl}; \text{BetweenWorld}; \text{chosenLost} : \mathbb{P} \text{PayDetails}; \text{pdThis} : \text{PayDetails} \mid \\ & \text{chosenLost} \subseteq \text{maybeLost} \bullet \exists_1 \text{AbWorld} \bullet \text{RabClPd} \end{aligned}$$

□

Proof

proof[lAbWorldUniqueC15]

■

Each element of *AbWorld* has an explicit equation in *Rab* defining it uniquely in terms of *BetweenWorld* components. The components are entirely independent, and the only constraint that ties any together is that on *chosenLost* and *maybeLost*, which we have directly in the hypothesis.

The constraints required of any *AbWorld* can be shown to hold as follows:

- *abAuthPurse* : *NAME* $\leftrightarrow$ *AbPurse*

conAuthPurse is a finite function. From the retrieve *AbstractBetween* its domain equals the domain of *conAuthPurse*, and so is finite, too.

14.4.5 Statement of lemma ‘deterministic’

TODO: CONTINUE TEXT FROM HERE

We summarise the results that Section 14.4 has developed as a lemma, for each *BOPurseOkay* operation. The correctness proof for a general *Okay* branch consists of the following three proof obligations:

exists-pd:

$$\begin{array}{l} \text{PhiBOP}; \text{BOPurseOkay}; \text{RabOut}; \text{RabCl}'; \text{RabIn} \\ \vdash \\ \exists \text{pdThis} : \text{PayDetails} \bullet \mathcal{P} \end{array}$$

exists-chosenLost:

$$\begin{array}{l} \text{PhiBOP}; \text{BOPurseOkay}; \text{RabOut}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \text{RabIn} \mid \\ \mathcal{P} \\ \vdash \\ \exists \text{chosenLost} : \mathbb{P} \text{PayDetails} \bullet \mathcal{Q} \wedge \text{chosenLost} \subseteq \text{maybeLost} \end{array}$$

check-operation:

$$\begin{array}{l} \text{PhiBOP}; \text{BOPurseOkay}; \text{RabOut}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \\ \text{AbWorld}; \text{RabClPd}; \text{RabIn} \mid \\ \mathcal{P} \wedge \mathcal{Q} \\ \vdash \\ \text{AOp} \end{array}$$

□ 14.4

14.5 Lemma ‘lost unchanged’

Many operations do not change *maybeLost* or *definitelyLost*. We call a general such operation *BOPΞLost*.

For *BOPΞLost* operations, where *maybeLost* = *maybeLost'* and *definitelyLost'* = *definitelyLost*, the proof obligations **exists-pd** and **exists-chosenLost** are satisfied automatically by the instantiation of the predicates $\mathcal{P}$ and $\mathcal{Q}$ as:

$$\begin{array}{l} \mathcal{P} \Leftrightarrow \text{true} \\ \mathcal{Q} \Leftrightarrow \text{chosenLost} = \text{chosenLost}' \end{array}$$

leaving the remaining **check-operation** proof obligation as

$$\begin{array}{l} \text{PhiBOP}; \text{BOPΞLostPurseOkay}; \text{RabOut}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \\ \text{AbWorld}; \text{RabClPd}; \text{RabIn} \mid \\ \text{chosenLost} = \text{chosenLost}' \\ \wedge \text{maybeLost} = \text{maybeLost}' \\ \wedge \text{definitelyLost}' = \text{definitelyLost} \\ \vdash \\ \text{AOp} \end{array}$$

14.5.1 Proof

We add the hypotheses *maybeLost* = *maybeLost'* and *definitelyLost'* = *definitelyLost* to the proof obligations for these *BOPΞLost* operations.

exists-pd

$$\begin{array}{l}
\text{PhiBOP}; \text{BOP} \exists \text{LostPurseOkay}; \text{RabOut}; \text{RabCl}'; \text{RabIn} \mid \\
\text{maybeLost}' = \text{maybeLost} \\
\wedge \text{definitelyLost}' = \text{definitelyLost} \\
\vdash \\
\exists \text{pdThis} : \text{PayDetails} \bullet \text{true}
\end{array}$$

This is trivially true.

□ 14.5.1

exists-chosenLost

$$\begin{array}{l}
\text{PhiBOP}; \text{BOP} \exists \text{LostPurseOkay}; \text{RabOut}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \\
\text{RabIn} \mid \\
\text{maybeLost}' = \text{maybeLost} \\
\wedge \text{definitelyLost}' = \text{definitelyLost} \\
\vdash \\
\exists \text{chosenLost} : \mathbb{P} \text{PayDetails} \bullet \\
\text{chosenLost} = \text{chosenLost}' \wedge \text{chosenLost} \subseteq \text{maybeLost}
\end{array}$$

We apply the **onept** rule to remove the existential quantifier in the consequent, substitute for *maybeLost*, and **thin**.

$$\text{RabClPd}'[\text{pdThis}/\text{pdThis}'] \vdash \text{chosenLost}' \subseteq \text{maybeLost}'$$

The hypothesis $\text{RabClPd}'[\text{pdThis}/\text{pdThis}']$ has $\text{chosenLost}' \subseteq \text{maybeLost}'$.

□ 14.5.1

□ 14.5

14.5.2 Sufficient conditions for invoking lemma ‘lost unchanged’

Since *PhiBOP* gives us that *archive* is unchanged, sufficient conditions for invoking lemma ‘lost unchanged’ are that the operation in question changes neither the purse’s status (hence no movement into or out of *epv* or *epa*) nor its exception log (hence no change to *from* logs or *to* logs).

14.6 Lemma ‘AbIgnore’: Operations that refine AbIgnore

As shown in section 14.2, to prove the refinement of the abstract identity operation *AbIgnore*, we can separately prove correctness for each of the between operations *StartFrom*, *StartTo*, *Val*, *Ack*, *ReadExceptionLog*, *ClearExceptionLog*, *AuthoriseExLogClear*, *Archive*, *Ignore*, *Increase*, and *Abort*.

For those which are structured as promoted operations (that is, all except *Archive* and *Ignore*), consider a general such operation, call it *BOPig*. We note that all *BOPig* operations have the properties:

- *BOPig* is a promoted operation, and thus alters only one concrete purse. It has the form

$$\exists \Delta \text{ConPurse} \bullet \text{PhiBOP} \wedge \text{BOPigPurse}$$

- for any purse, the *name* is unchanged (by definition of the single purse operations)
- the domain of *conAuthPurse* is unchanged (by construction of the promotion)
- for any purse, either *nextSeqNo* is unchanged, or increased.

$$\forall \text{BOPigPurse} \bullet \text{nextSeqNo} \leq \text{nextSeqNo}'$$

We use these properties to simplify the proof obligation for the *BOPig* operations.

We invoke lemma ‘deterministic’ (section 14.4) to reduce the *BOPig* proof obligation to **exists-pd**, **exists-chosenLost** and **check-operation**:

$$\begin{array}{l}
\text{PhiBOP}; \text{BOPigPurse}; \text{RabOut}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \\
\text{AbWorld}; \text{RabClPd}; \text{RabIn} \mid \\
\mathcal{P} \wedge \mathcal{Q} \\
\vdash \\
\text{AbIgnore}
\end{array}$$

For a *BOPig* operation, the **check-operation** proof obligation reduces to **check-operation-ignore**¹:

$$\begin{array}{l}
\text{PhiBOP}; \text{BOPigPurse}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \text{AbWorld}; \text{RabClPd} \mid \\
\mathcal{P} \wedge \mathcal{Q} \\
\vdash \\
\forall n : \text{dom } \text{abAuthPurse} \bullet \\
(\text{abAuthPurse}' n).lost = (\text{abAuthPurse } n).lost \\
\wedge (\text{abAuthPurse}' n).balance = (\text{abAuthPurse } n).balance
\end{array}$$

Proof:

We take the **check-operation** proof obligation, and expand *AbIgnore*. The *BOPigPurse* operations have certain properties in common; we explicitly state these in the hypothesis.

$$\begin{array}{l}
\text{PhiBOP}; \text{BOPigPurse}; \text{RabOut}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \\
\text{AbWorld}; \text{RabClPd}; \text{RabIn} \mid \\
\mathcal{P} \wedge \mathcal{Q} \\
\wedge \text{name}' = \text{name} \\
\wedge \text{nextSeqNo}' \geq \text{nextSeqNo} \\
\vdash \\
\text{AbOp} \wedge \text{abAuthPurse}' = \text{abAuthPurse}
\end{array}$$

We use **consqconj** to split this proof into two parts. The *AbOp* part is trivial: there are no constraints. This leaves the other conjunct to be proven, which is rewritten as follows:

$$\begin{array}{l}
\text{PhiBOP}; \text{BOPigPurse}; \text{RabOut}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \\
\text{AbWorld}; \text{RabClPd}; \text{RabIn} \mid \\
\mathcal{P} \wedge \mathcal{Q} \\
\wedge \text{name}' = \text{name} \\
\wedge \text{nextSeqNo}' \geq \text{nextSeqNo} \\
\vdash \\
\forall n : \text{dom } \text{abAuthPurse} \bullet \text{abAuthPurse}' n = \text{abAuthPurse } n
\end{array}$$

We prove this component by component. From *PhiBOP* in the hypothesis, all concrete purses other than purse *name?* remain unchanged. For the purse *name?*, we also have the equality of the pre and post states of *name*. This leaves the components *balance* and *lost*. We use this with **consqconj** to reduce our proof requirement to the following:

$$\begin{array}{l}
\text{PhiBOP}; \text{BOPigPurse}; \text{RabOut}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \\
\text{AbWorld}; \text{RabClPd}; \text{RabIn} \mid \\
\mathcal{P} \wedge \mathcal{Q} \\
\wedge \text{name}' = \text{name} \\
\wedge \text{nextSeqNo}' \geq \text{nextSeqNo} \\
\vdash \\
\forall n : \text{dom } \text{abAuthPurse} \bullet \\
(\text{abAuthPurse}' n).balance = (\text{abAuthPurse } n).balance \\
\wedge (\text{abAuthPurse}' n).lost = (\text{abAuthPurse } n).lost
\end{array}$$

¹Used in: *Ignore*, 14.7.2.

We then **thin** the hypothesis to get the following, which proves the *AbIgnore* lemma.

$$\begin{array}{l}
\text{PhiBOP}; \text{BOpIgPurse}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \text{AbWorld}; \text{RabClPd} \mid \\
\mathcal{P} \wedge \mathcal{Q} \\
\vdash \\
\forall n : \text{dom } \text{abAuthPurse} \bullet \\
(\text{abAuthPurse}' n).balance = (\text{abAuthPurse } n).balance \\
\wedge (\text{abAuthPurse}' n).lost = (\text{abAuthPurse } n).lost
\end{array}$$

□ 14.6

14.7 Ignore refines AbIgnore

Original theorem 14.1 (Backward simulation correctness: *AbIgnore* $\sqsubseteq$ *Ignore*)

$$\begin{array}{l}
\text{theorem } \text{tBetwIgnoreBackardCorrectness} \\
\forall \text{Ignore}; \text{Rab}'; \text{RabOut} \bullet \exists \text{AbWorld}; a? : \text{AIN} \bullet \text{Rab} \wedge \text{RabIn} \wedge \text{AbIgnore}
\end{array}$$

□

Original theorem 14.2 (Backward simulation correctness: *AbIgnore* $\sqsubseteq$ *Abort*)

$$\begin{array}{l}
\text{theorem } \text{tBetwAbortBackardCorrectness} \\
\forall \text{Abort}; \text{Rab}'; \text{RabOut} \bullet \exists \text{AbWorld}; a? : \text{AIN} \bullet \text{Rab} \wedge \text{RabIn} \wedge \text{AbIgnore}
\end{array}$$

□

Proofs

proof[*tBetwIgnoreBackardCorrectness*]

■

proof[*tBetwAbortBackardCorrectness*]

■

As we saw at the end of section 14.3, by splitting up promoted operations, we have generated a requirement to prove the correctness of the *Ignore* branch once only. We do that here.

14.7.1 Invoking lemma ‘deterministic’

Lemma ‘deterministic’ (section 14.4.5) cannot be applied as-is, because *Ignore* is not written as a promotion (in order to ensure it is total). However, the arguments to split the proof obligation into three parts follow in exactly the same manner even if the unpromoted purse is not exposed. The proof obligations simply have *BOpOkay* in the hypothesis, instead of *PhiBOP*; *BOpPurseOkay*. We use that form to simplify the *Ignore* proof obligation to three parts, and then invoke lemma ‘lost unchanged’ to discharge the first two obligations. We similarly use lemma ‘AbIgnore’ to simplify the third proof obligation to **check-operation-ignore**.

14.7.2 check-operation-ignore

$$\begin{array}{l}
\text{Ignore}; \text{RabClPd}'[\text{pdThis}/\text{pdThis}']; \text{AbWorld}; \text{RabClPd} \mid \\
\text{chosenLost} = \text{chosenLost}' \\
\wedge \text{maybeLost} = \text{maybeLost}' \\
\wedge \text{definitelyLost} = \text{definitelyLost}' \\
\vdash \\
\forall n : \text{dom } \text{abAuthPurse} \bullet \\
(\text{abAuthPurse}' n).balance = (\text{abAuthPurse } n).balance \\
\wedge (\text{abAuthPurse}' n).lost = (\text{abAuthPurse } n).lost
\end{array}$$

The proof of this is immediate: *Ignore* changes no values, *definitelyLost*, *maybeLost* and *chosenLost* do not change, from the hypothesis; so the abstract *balance* and *lost*, which depend only on these unchanging values, are unchanged.

□ 14.7.2

□ 14.7

14.8 *Abort* refines *AbIgnore*

As we saw at the end of section 14.3, by splitting up promoted operations, we have generated a requirement to prove the correctness of the *Abort* branch once only. We do that here. We cast it as a lemma, because we also use it to simplify the proofs of operations that first abort (lemma ‘abort backward’). Concrete *Abort* refines abstract *Ignore*.²

$$Abort; Rab'; RabOut \vdash \exists AbWorld; a? : AIN \bullet Rab \wedge RabIn \wedge AbIgnore$$

Proof:

Abort is written as a disjunction between *Ignore* and a promoted *AbortPurseOkay*. We use lemma ‘ignore’ (section 14.3) to simplify the proof obligation to the correctness of *Ignore* (which we discharge in section 14.7), and the *Okay* branch, which we prove here.

14.8.1 Invoking lemma ‘deterministic’

We use lemma ‘deterministic’ (section 14.4.5) to simplify the proof obligations and then lemma ‘AbIgnore’ (section 14.6) to simplify the **check-operation** step.

We have to instantiate the predicates $\mathcal{P}$ and $\mathcal{Q}$.

$\mathcal{P}$ is a predicate identifying the *pdThis* involved in the transaction. This is the *pdAuth* stored in the aborting purse, unless the aborting purse is in *eaFrom*, in which case we don’t have a defined transaction. We cater for the case of no transaction in the $\mathcal{Q}$ predicate, so $\mathcal{P}$ can safely be defined as

$$\mathcal{P} \Leftrightarrow pdThis = pdAuth$$

$\mathcal{Q}$ is a predicate on *chosenLost*. The after set *chosenLost'* either has *pdThis* removed (if the transaction moves it from *chosenLost* to *definitelyLost*), or is unchanged (because *pdThis* was not in *chosenLost* to start with) or is unchanged because there was no transaction to abort. Hence

$$\begin{aligned} \mathcal{Q} \Leftrightarrow & (pdThis \in maybeLost \wedge chosenLost = chosenLost' \cup \{pdThis\}) \\ & \vee (pdThis \notin maybeLost \wedge status \neq eaFrom \wedge \\ & \quad chosenLost = chosenLost') \\ & \vee (status = eaFrom \wedge chosenLost = chosenLost') \end{aligned}$$

14.8.2 exists-pd

The unpromoted operation *AbortPurseOkay* is incomplete. The output, $m! = \perp$, is not provided until promotion.

$$\begin{aligned} & PhiBOP; AbortPurseOkay; RabOut; RabCl'; RabIn \mid m! = \perp \\ & \vdash \\ & \exists pdThis : PayDetails \bullet pdThis = pdAuth \end{aligned}$$

This is immediate by the one point rule.

□ 14.8.2

²Used in proof of lemma abort, 14.9

14.8.3 Three cases

We split the remaining two proofs, of **exists-chosenLost** and **check-operation**, into three cases each, for each of the three disjuncts of $\mathcal{Q}$. We start by arguing the behaviour of *maybeLost* and *definitelyLost* in the three cases.

- **Case 1: aborted transaction in ‘limbo’:** The aborting purse is the *to* purse in *epv*; the corresponding *from* purse is in *epa* or has logged. Hence aborting the transaction will definitely lose the value.

$$pdThis \in maybeLost$$

- **Case 2: aborted transaction not in ‘limbo’:** The aborting purse is not the *to* purse in *epv*, or the corresponding *from* purse is not in *epa* and has not logged. The transaction has either not got far enough to lose anything, or has progressed sufficiently far that the value was already either successfully transferred or definitely lost.

$$pdThis \notin maybeLost \wedge status \neq eaFrom$$

- **Case 3: no transaction to abort:** The aborting purse is in *eaFrom*, so has no defined transaction. Nothing is aborted, so no value is lost.

$$status = eaFrom$$

Case 1: old transaction in limbo

$$pdThis \in (fromInEpa \cup fromLogged) \cap toInEpv$$

We argue about the behaviour of *maybeLost* and *definitelyLost* using the fact that the purse is the *to* purse initially in *epv* in the aborting transaction, and it logs the old transaction and moves to *eaFrom*. We argue that the transaction *pdThis*, initially in *maybeLost* by construction, is moved into *definitelyLost'* by this case of the *Abort* operation. The transaction was far enough progressed that value may be lost, and it is lost in this case.

Behaviour of *fromInEpa* and *fromLogged* *pdThis* is in *toInEpv* (by our case assumption), so the only purse undergoing any change (*name?*) is the *to* purse; hence there can be no change to the status or logs of any *from* purse. Hence

$$\begin{aligned} fromInEpa &= fromInEpa' \\ fromLogged &= fromLogged' \end{aligned}$$

Behaviour of *toInEpv* *pdThis* is in *toInEpv* (by our case assumption); *pdThis* is not in *toInEpv'* (*Abort* puts the purse into *eaFrom*); all other purses and transactions remain unchanged. So

$$toInEpv = toInEpv' \cup \{pdThis\}$$

Behaviour of *toLogged* *pdThis* is not in *toLogged* (using lemma ‘notLoggedAndIn’ with *pdThis* $\in$ *toInEpv*); *pdThis* is in *toLogged'* (the purse makes a *to* log when it aborts from *epv*); all other purses and transactions remain unchanged. So

$$toLogged = toLogged' \setminus \{pdThis\}$$

Behaviour of *definitelyLost*

$$\begin{aligned} definitelyLost &= toLogged \cap (fromLogged \cup fromInEpa) && [\text{defn } definitelyLost] \\ &= (toLogged' \setminus \{pdThis\}) \cap (fromLogged' \cup fromInEpa') && [\text{above}] \\ &= (toLogged' \cap (fromLogged' \cup fromInEpa')) \setminus \{pdThis\} && [\text{rearrange}] \\ &= definitelyLost' \setminus \{pdThis\} && [\text{defn } definitelyLost'] \end{aligned}$$

Behaviour of *maybeLost*

$$\begin{aligned}
& maybeLost \\
&= (fromInEpa \cup fromLogged) \cap toInEpv && [\text{defn } maybeLost] \\
&= (fromInEpa' \cup fromLogged') \cap (toInEpv' \cup \{pdThis\}) && [\text{above}] \\
&= ((fromInEpa' \cup fromLogged') \cap toInEpv') \\
&\quad \cup ((fromInEpa' \cup fromLogged') \cap \{pdThis\}) && [\text{Spivey}] \\
&= ((fromInEpa' \cup fromLogged') \cap toInEpv') \\
&\quad \cup \{pdThis\} && [\text{case assumption}] \\
&= maybeLost' \cup \{pdThis\} && [\text{defn } maybeLost']
\end{aligned}$$

Case 2: old transaction not in limbo

$$pdThis \notin (fromInEpa \cup fromLogged) \cap toInEpv \wedge status \neq eaFrom$$

We argue that the transaction *pdThis* is not moved into or out of *maybeLost* or *definitelyLost* by this case of the *Abort* operation.

Behaviour of $fromInEpa \cup fromLogged$ If *pdThis* is in *fromInEpa* it is also in *fromLogged'* (the purse is in *epa*, so it makes a *from* log when it aborts); if *pdThis* is in *fromLogged* it is also in *fromLogged'* (logs cannot be removed); if *pdThis* is not in *fromInEpa* $\cup$ *fromLogged* it is not in *fromLogged'* (the purse is not in *epa*, so does not make a *from* log when it aborts), and not in *fromInEpa'* (because it ends in *eaFrom*); all the other purses and transactions remain unchanged. So

$$fromInEpa \cup fromLogged = fromInEpa' \cup fromLogged'$$

Behaviour of *definitelyLost* The cases allowed by our case assumption are:

- *pdThis* refers to the *to* purse in *epv*, hence is not in

$$fromInEpa \cup fromLogged$$

and hence not in *definitelyLost*.

Also it is not in $fromInEpa' \cup fromLogged'$, and hence not in *definitelyLost'*. So *definitelyLost* is unchanged.

- *pdThis* refers to the *to* purse, but not in *epv*, or *pdThis* refers to the *from* purse. Hence *toLogged* is unchanged, since no *to* log is written, and logs cannot be lost.

Also $fromInEpa \cup fromLogged$ is unchanged, and so *definitelyLost* is unchanged.

So

$$definitelyLost' = definitelyLost$$

Behaviour of *maybeLost* The cases allowed by our case assumption are:

- *pdThis* refers to the *to* purse in *epv*, hence is not in

$$fromInEpa \cup fromLogged$$

and hence not in *maybeLost*. Also it is not in $fromInEpa' \cup fromLogged'$, and hence not in *maybeLost'*, so *maybeLost* is unchanged.

- *pdThis* refers to the *to* purse, but not in *epv*, or *pdThis* refers to the *from* purse. Hence *toInEpv* is unchanged, since no purse moves out of or in to *epv*. Also $fromInEpa \cup fromLogged$ is unchanged, so *maybeLost* is unchanged.

So

$$maybeLost' = maybeLost$$

Case 3: no transaction to abort

$$status = eaFrom$$

From *AbortPurseOkay*, no purses change state and no logs are written. Therefore, *definitelyLost* and *maybeLost* don't change.

$$definitelyLost' = definitelyLost$$

$$maybeLost' = maybeLost$$

14.8.4 exists-chosenLost

We now use the behaviour of *maybeLost* and *definitelyLost* in the three cases to prove **exists-chosenLost**.

$$\begin{aligned}
& \text{PhiBOP}; \text{AbortPurseOkay}; \text{RabOut}; \text{RabClPd}'[pdThis/pdThis']; \text{RabIn} \mid \\
& m! = \perp \\
& \wedge pdThis = pdAuth \\
& \vdash \\
& \exists chosenLost : \mathbb{P} \text{PayDetails} \bullet \\
& \quad (pdThis \in maybeLost \wedge chosenLost = chosenLost' \cup \{pdThis\} \\
& \quad \vee pdThis \notin maybeLost \wedge status \neq eaFrom \\
& \quad \quad \wedge chosenLost = chosenLost' \\
& \quad \vee status = eaFrom \wedge chosenLost = chosenLost') \\
& \wedge chosenLost \subseteq maybeLost
\end{aligned}$$

We push the existential quantifier in the consequent into the predicates:

$$\begin{aligned}
& \text{PhiBOP}; \text{AbortPurseOkay}; \text{RabOut}; \text{RabClPd}'[pdThis/pdThis']; \text{RabIn} \mid \\
& m! = \perp \\
& \wedge pdThis = pdAuth \\
& \vdash \\
& pdThis \in maybeLost \\
& \quad \wedge (\exists chosenLost : \mathbb{P} \text{PayDetails} \bullet \\
& \quad \quad chosenLost = chosenLost' \cup \{pdThis\} \\
& \quad \quad \wedge chosenLost \subseteq maybeLost) \\
& \vee pdThis \notin maybeLost \wedge status \neq eaFrom \\
& \quad \wedge (\exists chosenLost : \mathbb{P} \text{PayDetails} \bullet \\
& \quad \quad chosenLost = chosenLost' \\
& \quad \quad \wedge chosenLost \subseteq maybeLost) \\
& \vee status = eaFrom \\
& \quad \wedge (\exists chosenLost : \mathbb{P} \text{PayDetails} \bullet \\
& \quad \quad chosenLost = chosenLost' \\
& \quad \quad \wedge chosenLost \subseteq maybeLost)
\end{aligned}$$

In each case, we **onept** away the *chosenLost* because the predicate includes an explicit definition for it.

$$\begin{aligned}
& \text{PhiBOP}; \text{AbortPurseOkay}; \text{RabOut}; \text{RabClPd}'[pdThis/pdThis']; \text{RabIn} \mid \\
& m! = \perp \\
& \wedge pdThis = pdAuth \\
& \vdash \\
& pdThis \in maybeLost \\
& \quad \wedge chosenLost' \cup \{pdThis\} \subseteq maybeLost \\
& \vee pdThis \notin maybeLost \wedge status \neq eaFrom \\
& \quad \wedge chosenLost' \subseteq maybeLost \\
& \vee status = eaFrom \\
& \quad \wedge chosenLost' \subseteq maybeLost
\end{aligned}$$

In each case, the predicate is of the form $(a \wedge b)$, and we argue below that $a \Rightarrow b$. This allows us to replace $(a \wedge b)$ with a . If we do this, we obtain

$$\begin{aligned}
& \text{PhiBOP}; \text{AbortPurseOkay}; \text{RabOut}; \text{RabClPd}'[pdThis/pdThis']; \text{RabIn} \mid \\
& \quad m! = \perp \\
& \quad \wedge pdThis = pdAuth \\
& \vdash \\
& pdThis \in maybeLost \\
& \vee pdThis \notin maybeLost \wedge status \neq eaFrom \\
& \vee status = eaFrom
\end{aligned}$$

which is *true*.

We now carry out the argument as described above for each of the three disjuncts.

Case 1: old transaction in limbo

We must show that under the assumptions of this lemma and in this case

$$\begin{aligned}
& pdThis \in maybeLost \Rightarrow \\
& \quad chosenLost' \cup \{pdThis\} \subseteq maybeLost
\end{aligned}$$

This follows by:

$$\begin{aligned}
& chosenLost' \cup \{pdThis\} \\
& \subseteq maybeLost' \cup \{pdThis\} \quad [\text{hypothesis}] \\
& \subseteq maybeLost \quad [\text{previous argument for case 1}]
\end{aligned}$$

□ 14.8.4

Case 2: old transaction not in limbo

We must show that under the assumptions of this lemma and in this case

$$\begin{aligned}
& pdThis \notin maybeLost \wedge status \neq eaFrom \Rightarrow \\
& \quad chosenLost' \subseteq maybeLost
\end{aligned}$$

This follows by

$$\begin{aligned}
& chosenLost' \subseteq maybeLost' \quad [\text{hypothesis}] \\
& \Rightarrow chosenLost' \subseteq maybeLost \quad [\text{previous argument for case 2}]
\end{aligned}$$

□ 14.8.4

Case 3: no transaction to abort

We must show that under the assumptions of this lemma and in this case

$$\begin{aligned}
& status = eaFrom \Rightarrow \\
& \quad chosenLost' \subseteq maybeLost
\end{aligned}$$

This follows by

$$\begin{aligned}
& chosenLost' \subseteq maybeLost' \quad [\text{hypothesis}] \\
& \Rightarrow chosenLost' \subseteq maybeLost \quad [\text{previous argument for case 3}]
\end{aligned}$$

□ 14.8.4

□ 14.8.4

14.8.5 check-operation-ignore

We now use the behaviour of *maybeLost* and *definitelyLost* in the three cases to prove **check-operation-ignore**.

$$\begin{aligned}
& \text{PhiBOP}; \text{AbortPurseOkay}; \text{RabClPd}'[pdThis/pdThis']; \\
& \text{AbWorld}; \text{RabClPd} \mid \\
& pdThis = pdAuth \\
& \wedge (pdThis \in maybeLost \wedge chosenLost = chosenLost' \cup \{pdThis\} \\
& \quad \vee pdThis \notin maybeLost \wedge status \neq eaFrom \\
& \quad \quad \wedge chosenLost = chosenLost' \\
& \quad \vee status = eaFrom \wedge chosenLost = chosenLost') \\
& \vdash \\
& \forall n : \text{dom } abAuthPurse \bullet \\
& \quad (abAuthPurse' n).balance = (abAuthPurse n).balance \\
& \quad \wedge (abAuthPurse' n).lost = (abAuthPurse n).lost
\end{aligned}$$

We can prove this for each of the three disjuncts in the hypothesis by **hypdisj**.

Case 1: old transaction in limbo

lost is a function of

$$definitelyLost \cup chosenLost$$

. The *pdThis* moves from *chosenLost* to *definitelyLost'*, so the union is unchanged.

balance is a function of *maybeLost* \ *chosenLost*. The *pdThis* moves from *chosenLost*, and hence from *maybeLost*, so the difference is unchanged.

□ 14.8.5

Case 2+3: old transaction not in limbo or no transaction

From *chosenLost* = *chosenLost'* and the arguments above, all the relevant sets are unchanging, so *lost* and *balance* are unchanging.

□ 14.8.5

□ 14.8.5

□ 14.8

14.9 Lemma ‘abort backward’: operations that first abort

Some of the concrete operations are written as a composition of *AbortPurseOkay* with a simpler operation starting from *eaFrom* (*StartFrom*, *StartTo*, *ReadExceptionLog*, *ExceptionLogClear*).

Where a concrete operation is written as a composition of *AbortPurseOkay* and a simpler operation starting from *eaFrom*, it is sufficient to prove that the promotion of the simpler operation alone refines the relevant abstract operation.

$$\begin{aligned}
& \exists \Delta \text{ConPurse} \bullet \text{PhiBOP} \wedge (\text{AbortPurseOkay} \circ \text{BOPurseEafromOkay}); \\
& \text{Rab}'; \text{RabOut}; \\
& (\forall \text{BOPurseEafromOkay}; \text{Rab}'; \text{RabOut} \bullet \\
& \quad \exists \text{AbWorld}; a? : \text{AIN} \bullet \text{Rab} \wedge \text{RabIn} \wedge \text{AOp}) \\
& \vdash \\
& \exists \text{AbWorld}; a? : \text{AIN} \bullet \text{Rab} \wedge \text{RabIn} \wedge \text{AOp}
\end{aligned}$$

Proof

- Use lemma ‘promoted composition’ (section A.6) to rewrite the promotion of the composition to a composition of promotions, yielding

$$\begin{aligned}
& (\text{AbortOkay} \circ \text{BOPurseEafromOkay}); \\
& \text{Rab}'; \text{RabOut}; \\
& (\forall \text{BOPurseEafromOkay}; \text{Rab}'; \text{RabOut} \bullet \\
& \quad \exists \text{AbWorld}; a? : \text{AIN} \bullet \text{Rab} \wedge \text{RabIn} \wedge \text{AOp}) \\
& \vdash \\
& \exists \text{AbWorld}; a? : \text{AIN} \bullet \text{Rab} \wedge \text{RabIn} \wedge \text{AOp}
\end{aligned}$$

- If $BOp1$ refines $AOp1$ and $BOp2$ refines $AOp2$, then $BOp1 \text{ ; } BOp2$ refines $AOp1 \text{ ; } AOp2$ (invoke lemma ‘compose backward’, section A.4).
- Take $BOp1 = AbortOkay$, $AOp1 = AbIgnore$, and invoke lemma ‘Abort refines AbIgnore’ (section 14.8), to discharge this proof.
- Take $BOp2 = BOpEafromOkay$, $AOp2 = AOp$, and note that we have that BOp refines AOp in the hypothesis.
- Note that $AbIgnore \text{ ; } AOp = AOp$, to reduce this expression in the consequent.

□ 14.9

14.10 Summary of lemmas

In section 25.2.5 we reduced the refinement correctness proof for an operation to:

$$BOp; Rab'; RabOut \vdash \exists AbWorld; a? : AIN \bullet Rab \wedge RabIn \wedge AOp$$

We then built up a set of lemmas which may be used to simplify this proof requirement.

AOp and BOp are often disjunctions of simpler operations, and lemmas ‘multiple refinement’ (section 14.2) and ‘ignore’ (section 14.3) are used to prove that any *Ignore* or *Abort* branches of BOp need be proved once only for all BOp s. These two branches are proved in lemmas later on, after further simplification for a general disjunct (*Ignore*, *Abort* or *Okay*) of BOp . This simplification starts with lemma ‘deterministic’ (section 14.4) which removes the $\exists AbWorld$ in the consequent of the correctness obligation. In doing so, it requires us to prove three side-lemmas (**exists-pd**, **exists-chosenLost**, **check-operation**). Lemma ‘lost unchanged’ (section 14.5) allows the side-lemmas **exists-pd** and **exists-chosenLost** to be discharged immediately given certain conditions. Lemma ‘AbIgnore’ (section 14.6) then provides a simplification of the side-lemma **check-operation** when AOp is *AbIgnore*.

We can now prove that the *Ignore* and *Abort* branches of BOp are correct with respect to AOp . Section 14.7 proves that *Ignore* refines *AbIgnore*, and lemma ‘Abort refines AbIgnore’ (section 14.8) handles the *Abort* branch. With lemmas ‘multiple refinement’ and ‘ignore’, this has now proved the correctness of the *Ignore* and *Abort* branches of all BOp .

Where the *Okay* branch of an operation is composed of *Abort* followed by the ‘active’ operation, lemma ‘abort backward’ gives us that we only need to prove the ‘active’ part.

Returning to the proof obligation written above, any of the *Ignore* or *Abort* branches of a BOp operation are dealt with by the lemmas. This leaves the *Okay* branch (if this contains an initial *Abort*, this can be ignored — from lemma ‘abort backward’ we need only prove the non-aborting part). Usually, we then apply lemma ‘deterministic’ yielding a number of side-lemmas. These may sometimes be further simplified using lemmas ‘lost unchanged’ and ‘AbIgnore’. The remaining proof is then particular to the BOp .

14.11 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 14.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	1	5
Theorems	2	17
Total	3	22

Table 14.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 14.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 14.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 14.5: Summary of *Z/Eves* rulees

Chapter 15

Correctness of *Increase*

15.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 15.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 15.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 15.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 15.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 15.5: Summary of *Z/Eves* rulees

Chapter 16

Correctness of *StartFrom*

16.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 16.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 16.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 16.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 16.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 16.5: Summary of *Z/Eves* rulees

Chapter 17

Correctness of *StartTo*

17.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 17.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 17.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 17.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 17.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 17.5: Summary of *Z/Eves* rulees

Chapter 18

Correctness of *Req*

18.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 18.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 18.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 18.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 18.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 18.5: Summary of Z/Eves rulees

Chapter 19

Correctness of *Val*

19.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 19.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 19.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 19.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 19.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 19.5: Summary of Z/Eves rulees

Chapter 20

Correctness of *Ack*

20.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 20.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 20.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 20.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 20.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 20.5: Summary of Z/Eves rulees

Chapter 21

Correctness of *ReadExceptionLog*

21.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 21.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 21.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 21.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 21.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 21.5: Summary of Z/Eves rulees

Chapter 22

Correctness of *ClearExceptionLog*

22.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 22.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 22.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 22.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 22.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 22.5: Summary of *Z/Eves* rulees

Chapter 23

Correctness of *AuthoriseExLogClear*

23.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 23.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 23.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 23.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 23.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 23.5: Summary of *Z/Eves* rulees

Chapter 24

Correctness of *Archive*

24.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 24.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 24.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 24.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 24.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 24.5: Summary of Z/Eves rulees

Part V

Second refinement: $\mathcal{B}$ to $\mathcal{C}$

Chapter 25

Refinement Proof Rules

Like in Chapters 9, this chapter introduces meta-theorems for the refinement rules, but for $\mathcal{B}$ and $\mathcal{C}$ instead of $\mathcal{A}$ and $\mathcal{B}$.

The only formal part here for **Z/Eves** is the simplification of the precondition in the general simulation rules.

25.1 Security of the implementation

We prove the concrete model $\mathcal{C}$ is secure with respect to the between model $\mathcal{B}$ by showing that every concrete operation correctly refines a between operation. The concrete and between operations are similarly-named. The full list of refinements is:

$$\begin{aligned} StartTo &\sqsubseteq CStartTo \\ StartFrom &\sqsubseteq CStartFrom \\ Req &\sqsubseteq CReq \\ Val &\sqsubseteq CVal \\ Ack &\sqsubseteq CAck \\ ReadExceptionLog &\sqsubseteq CReadExceptionLog \\ ClearExceptionLog &\sqsubseteq CClearExceptionLog \\ AuthoriseExLogClear &\sqsubseteq CAuthoriseExLogClear \\ Archive &\sqsubseteq CArchive \\ Abort &\sqsubseteq CAbort \\ Increase &\sqsubseteq CIncrease \\ Ignore &\sqsubseteq CIgnore \end{aligned}$$

Commented in the original PRG, these theorems are given as

$$\forall StartTo; Rbc; Rbc' \bullet CStartTo$$

which is a simplified version of the correctness theorem for forward simulation with functional retrieve relation where $pre\ StartTo = true$.

25.2 Forwards rules proof obligations

Each of these refinements must be proved correct.

25.2.1 Retrieve

The retrieve relation has one part that links the abstract and concrete states.

25.2.2 Initialisation

$$CInit \vdash \exists B' \bullet BInit \wedge Rbc'$$

25.2.3 Finalisation

$$Rbc; CFin \vdash BFin$$

25.2.4 Applicability

$$Rbc; BIn \mid \text{pre } BOp \vdash \text{pre } COp$$

25.2.5 Correctness

$$Rbc; COp \mid \text{pre } BOp \vdash \exists B' \bullet Rbc' \wedge BOp$$

We can simplify the correctness condition because we know that all the between operations are total, i.e.

$$\text{pre } BOp = \text{true}$$

This was proved earlier, in section 8.5.2. This is true because of the disjunction with *Ignore*.

We can therefore simplify the correctness condition to

$$Rbc; COp \vdash \exists B' \bullet Rbc' \wedge BOp$$

25.3 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	105
Total	0	114

Table 25.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 25.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 25.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 25.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 25.5: Summary of *Z/Eves* rulees

Chapter 26

$\mathcal{B}$ to $\mathcal{C}$ retrieve relation

26.1 Retrieve state

The $\mathcal{B}$ and $\mathcal{C}$ worlds are identical, except that the $\mathcal{C}$ world can ‘lose’ *ether* messages.

Original schema 26.1 ($\mathcal{B}$ and $\mathcal{C}$ (functional) retrieve relation)

Rbc
$BetweenWorld$
$ConWorldCh7$
$conAuthPurseCh7 = conAuthPurse$
$etherCh7 \subseteq ether$
$archiveCh7 = archive$

□

26.2 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	1	106
Total	1	115

Table 26.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	17
Total	0	22

Table 26.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 26.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	74
Theorems	0	68
Axiomatic consistency	0	5
Total	0	147

Table 26.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 26.5: Summary of *Z/Eves* rulees

Chapter 27

Initialisation, Finalisation, and Applicability

27.1 Initialisation proof

Original theorem 27.1 ($\mathcal{B}$ to $\mathcal{C}$ initialisation)

theorem tBetwConcInit
 $\forall \text{ ConInitState} \bullet \exists \text{ BetweenWorld}' \bullet \text{ BetweenInitState} \wedge \text{Rbc}'$

□

27.2 Finalisation proof

Upon finalisation, it is possible identify the $\mathcal{C}$ *definitelyLostCh7* and *maybeLostCh7* with the correspondent $\mathcal{B}$ components.

Added lemma 27.1 (Concrete *AuxWorld* equivalence at finalisation)

theorem lCAuxWorldFinalEquiv
 $\forall \text{ AuxWorldCh7}; \text{ BetweenWorld} \mid$
 $\text{conAuthPurseCh7} = \text{conAuthPurse} \wedge \text{archiveCh7} = \text{archive} \bullet$
 $\text{definitelyLostCh7} = \text{definitelyLost} \wedge \text{maybeLostCh7} = \text{maybeLost}$

□

This is used in the next proof, as mentioned by the original Mondex explanation below.

Original theorem 27.2 ($\mathcal{B}$ to $\mathcal{C}$ finalisation)

theorem tBetwConcFinal
 $\forall \text{ Rbc}; \text{ ConFinState} \bullet \text{ BetwFinState}$

□

We have defined *ConFinState* and *BetwFinState* to have the same mathematical form.

Rbc in the hypothesis requires the concrete and between purse states and archives to be identical, and allows the between *ether* to be bigger than the concrete *ether*.

Finalisation of the purses depends only on the purse states (identical by hypothesis) and on the sets *definitelyLost* and *maybeLost*. These sets themselves depend only on purse states and on the archive (also identical for concrete and between worlds by the retrieve in the hypothesis). As result, *gAuthPurse* for between finalisation is identical to that for concrete finalisation.

27.3 Applicability proofs

Applicability follows automatically from the totality of the concrete operations as shown in section 8.6.

27.4 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	106
Total	0	115

Table 27.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	2	19
Total	2	24

Table 27.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	70
Total	0	82

Table 27.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	1	75
Theorems	0	68
Axiomatic consistency	0	5
Total	1	148

Table 27.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 27.5: Summary of *Z/Eves* rulees

Chapter 28

Lemmas for the $\mathcal{B}$ to $\mathcal{C}$ correctness proofs

28.1 Specialising the proof rules

For each concrete operation COp and corresponding between operation BOp we have to show

$$Rbc; COp \vdash \exists \text{BetweenWorld}' \bullet Rbc' \wedge BOp$$

Many operations are defined as the disjunction of other operations. A COp will have the same branches as a corresponding BOp : a $CIgnore$ branch, and either a $CAbort$ or $COpOkay$ branch, or both. We split the proof obligation into $CIgnore$, $CAbort$ and $COpOkay$ branches, as we did in section 14.3. This gives some or all of the following proof requirements, depending on which branches are in COp :

$$Rbc; CIgnore \vdash \exists \text{BetweenWorld}' \bullet Rbc' \wedge Ignore$$

$$Rbc; CAbort \vdash \exists \text{BetweenWorld}' \bullet Rbc' \wedge Abort$$

$$Rbc; COpOkay \vdash \exists \text{BetweenWorld}' \bullet Rbc' \wedge BOpOkay$$

The correctness of the $CIgnore$ branch is dealt with below in section 28.2. We then develop the correctness proof for the $CAbort$ and $COpOkay$ branches, and introduce a lemma applicable to certain operations. Following this, we present the proof of correctness of two common branches — $CIncrease$ and $CAbort$.

28.2 Correctness of $CIgnore$

The correctness of the $CIgnore$ branch follows trivially by choosing

Added theorem 28.1 ($\mathcal{B}$ to $\mathcal{C}$ $CIgnore$ correct)

$$\begin{aligned} &\text{theorem tBetwConcCIgnoreCorrect} \\ &\quad \forall Rbc; CIgnore \bullet \exists \text{BetweenWorld}' \bullet Rbc' \wedge Ignore \end{aligned}$$

□

Like $PhiBOPBranchesSig$ (see 8.5.3), but with $ConPurse'$ from $AbortPurseOkay$.

Added schema 28.1 ($Abort \sqsubseteq CAbort$ correctness signature)

$$\text{BetwCAbortCorrectSig} \hat{=} \text{PhiBOPBranchesSig} \wedge \text{AbortPurseOkay}$$

□

Added lemma 28.1 ($Abort \sqsubseteq CAbort$ correct $AuxWorld$ type)

$$\begin{aligned} &\text{theorem lAuxWorldCAbortType} \\ &\quad \forall \text{BetwCAbortCorrectSig} \bullet \text{AuxWorld}[conAuthPurse := conAuthPurse \oplus \\ &\quad \quad \{ (name?, \theta ConPurse'[status' := eaFrom]) \}, ether := ether \cup \{ \perp \}] \end{aligned}$$

□

Added lemma 28.2 (*Abort* $\sqsubseteq$ *CAbort* **correct** *BetweenWorld* **type**)

theorem lBetweenWorldCAbortType
 $\forall \text{BetwCAbortCorrectSig} \bullet \text{BetweenWorld}[\text{conAuthPurse} := \text{conAuthPurse} \oplus$
 $\{ (name?, \theta \text{ConPurse}'[\text{status}' := \text{eaFrom}]) \}, \text{ether} := \text{ether} \cup \{ \perp \}]$

□

Added theorem 28.2 (**B C** **correctness** for *Abort*)

theorem tBetwConcCAbortCorrect
 $\forall \text{Rbc}; \text{CAbort} \bullet \exists \text{BetweenWorld}' \bullet \text{Rbc}' \wedge \text{Abort}$

□

28.3 Correctness of a branch of the operation

Like in Section 14.3, in here we need to consider each *BOpOkay* branch separately. The formal declaration of these branches were given in Sections 8.5.3 and 8.5.4.

As occurred in Chapter 14, the proof strategy described in Mondex uses meta-theorems to generalise the proof. For mechanisation, we need to state those theorems explicitly.

28.3.1 Theorems for each branch from *BOpOkay* operation

As *CIncrease* and *CAbort* are originally dealt with separately in their own sections, we just add here the remaining cases.

Added theorem 28.3 (**B** to **C** *CStartFrom* **correct**)

theorem tBetwConcCStartFromCorrect
 $\forall \text{Rbc}; \text{CStartFrom} \bullet \exists \text{BetweenWorld}' \bullet \text{Rbc}' \wedge \text{StartFrom}$

□

Added theorem 28.4 (**B** to **C** *CStartTo* **correct**)

theorem tBetwConcCStartToCorrect
 $\forall \text{Rbc}; \text{CStartTo} \bullet \exists \text{BetweenWorld}' \bullet \text{Rbc}' \wedge \text{StartTo}$

□

Added schema 28.2 (*Req* $\sqsubseteq$ *CReq* **signature**)

$\text{BetwCReqCorrectSig} \hat{=} \text{PhiBOPBranchesSig} \wedge \text{ReqPurseOkay}$

□

theorem lAuxWorldCReqType
 $\forall \text{BetwCReqCorrectSig} \bullet \text{AuxWorld}[\text{conAuthPurse} := \text{conAuthPurse} \oplus$
 $\{ (name?, \theta \text{ConPurse}'[\text{status}' := \text{epa}]) \}, \text{ether} := \text{ether} \cup \{ m! \}]$

theorem lBetweenWorldCReqType
 $\forall \text{BetwCReqCorrectSig} \bullet \text{BetweenWorld}[\text{conAuthPurse} := \text{conAuthPurse} \oplus$
 $\{ (name?, \theta \text{ConPurse}'[\text{status}' := \text{epa}]) \}, \text{ether} := \text{ether} \cup \{ m! \}]$

Added theorem 28.5 (**B** to **C** *CReq* **correct**)

theorem tBetwConcCReqCorrect
 $\forall \text{Rbc}; \text{CReq} \bullet \exists \text{BetweenWorld}' \bullet \text{Rbc}' \wedge \text{Req}$

□

Added theorem 28.6 ($\mathcal{B}$ to $\mathcal{C}$ $CVal$ correct)

theorem tBetwConcCValCorrect
 $\forall Rbc; CVal \bullet \exists BetweenWorld' \bullet Rbc' \wedge Val$

□

Added theorem 28.7 ($\mathcal{B}$ to $\mathcal{C}$ $CAck$ correct)

theorem tBetwConcCAckCorrect
 $\forall Rbc; CAck \bullet \exists BetweenWorld' \bullet Rbc' \wedge Ack$

□

Added theorem 28.8 ($\mathcal{B}$ to $\mathcal{C}$ $CReadExceptionLog$ correct)

theorem tBetwConcCReadExceptionLogCorrect
 $\forall Rbc; CReadExceptionLog \bullet \exists BetweenWorld' \bullet Rbc' \wedge ReadExceptionLog$

□

Added theorem 28.9 ($\mathcal{B}$ to $\mathcal{C}$ $CClearExceptionLog$ correct)

theorem tBetwConcCClearExceptionLogCorrect
 $\forall Rbc; CClearExceptionLog \bullet \exists BetweenWorld' \bullet Rbc' \wedge ClearExceptionLog$

□

Added theorem 28.10 ($\mathcal{B}$ to $\mathcal{C}$ $CAuthoriseExLogClear$ correct)

theorem tBetwConcCAuthoriseExLogClearCorrect
 $\forall Rbc; CAuthoriseExLogClear \bullet \exists BetweenWorld' \bullet Rbc' \wedge AuthoriseExLogClear$

□

Added theorem 28.11 ($\mathcal{B}$ to $\mathcal{C}$ $CArchive$ correct)

theorem tBetwConcCArchiveCorrect
 $\forall Rbc; CArchive \bullet \exists BetweenWorld' \bullet Rbc' \wedge CArchive$

□

Added theorem 28.12 ($\mathcal{B}$ to $\mathcal{C}$ $CIncrease$ correct)

theorem tBetwConcCIncreaseCorrect
 $\forall Rbc; CIncrease \bullet \exists BetweenWorld' \bullet Rbc' \wedge Increase$

□

Added theorem 28.13 ($\mathcal{B}$ to $\mathcal{C}$ branching is correct)

theorem tBetwConcBranchingCorrect
 $(Rbc \wedge CIncrease \Rightarrow (\exists BetweenWorld' \bullet Rbc' \wedge Increase))$
 $\Leftrightarrow$
 $((Rbc \wedge CIgnore \Rightarrow (\exists BetweenWorld' \bullet Rbc' \wedge Ignore)) \wedge$
 $(Rbc \wedge CAbort \Rightarrow (\exists BetweenWorld' \bullet Rbc' \wedge Abort)) \wedge$
 $(Rbc \wedge (\exists \Delta ConPurse \bullet PhiCOp \wedge IncreasePurseOkay)$
 $\Rightarrow (\exists BetweenWorld' \bullet Rbc' \wedge IncreaseOkay)))$

□

28.3.2 Choosing *BetweenWorld'*

TODO: CONTINUE TEXT FROM HERE

In choosing *BetweenWorld'*, we base our choice of the *conAuthPurse'* and *archive'* components on *Rbc'*, and our choice of the *ether'* component on *BOpOkay'*.

We have *conAuthPurse'_0* and *archive'_0* in the hypothesis, and we use this to provide the value for *conAuthPurse'* and *archive'*, respectively (this satisfies the constraint on *conAuthPurse'* and *archive'* in *Rbc'*).

$$\begin{aligned} \text{conAuthPurse}' &= \text{conAuthPurse}'_0 \\ \text{archive}' &= \text{archive}'_0 \end{aligned}$$

m! and *ether* are declared in the hypothesis, and *ether'* can be constructed deterministically from these (note that the following construction satisfies the relevant constraint in *BOpOkay* — either in *PhiBOp* or explicitly as in *Archive*).

$$\text{ether}' = \text{ether} \cup \{m!\}$$

We need to show that the chosen *BetweenWorld'* and *m!* satisfy each of the conjuncts in the consequent (retrieve *Rbc'* and operation *BOpOkay*).

We also need to show that this choice is indeed an after *BetweenWorld'* (that it satisfies the constraints on *BetweenWorld* specified in section 5.3).

28.3.3 Case *BOpOkay*

From the choice of *ether'* above, the relevant constraint on *ether'* in *BOpOkay* is satisfied by construction.

At most one purse changes in *COpOkay*. Let us call this new purse value *p*. This gives

$$\begin{aligned} \text{conAuthPurse}'_0 &= \text{conAuthPurse}_0 \oplus \{p\} \\ \text{conAuthPurse}'_0 &= \text{conAuthPurse} \oplus \{p\} && [Rbc] \\ \text{conAuthPurse}' &= \text{conAuthPurse} \oplus \{p\} && [\text{choice of } \text{conAuthPurse}'] \end{aligned}$$

This satisfies the constraint on *conAuthPurse'* in *BOpOkay* (where at most one purse changes in an identical manner to *COpOkay*).

archive' is a function of *archive* and *m!*, defined in *BOpOkay*. Call this function *f*:

$$\mid f : \text{Logbook} \times \text{MESSAGE} \rightarrow \text{Logbook}$$

Because *COpOkay* is defined in an analogous way, *f* also relates *archive'_0* to *archive_0* and *m!*.

From the hypothesis we have *COpOkay* and *Rbc*, and with our choice of *archive'* we have, respectively

$$\begin{aligned} \text{archive}'_0 &= f(\text{archive}_0, m!) \\ \wedge \text{archive} &= \text{archive}_0 \\ \wedge \text{archive}' &= \text{archive}'_0 \end{aligned}$$

Substituting the latter two equations into the first gives the predicate in *BOpOkay*.

Thus, the *BOpOkay* constraints on all the components of our chosen *BetweenWorld'* are satisfied under the correctness hypothesis and choice of *BetweenWorld'*.

□ 28.3.3

28.3.4 Case *Rbc'*

Both the *conAuthPurse'* and *archive'* components of *BetweenWorld'* satisfy *Rbc'* from the choice of *BetweenWorld'*.

All *COpOkay* operations constrain *ether'* as

$$\text{ether}'_0 \subseteq \text{ether}_0 \cup \{m!\}$$

either through *PhiCOp*, or explicitly in *CArchive*. Hence for *ether'* we have

$$\begin{aligned}
& ether' \\
&= ether \cup \{m!\} && [\text{choice of } ether'] \\
&\supseteq ether_0 \cup \{m!\} && [Rbc] \\
&\supseteq ether'_0 && [COpOkay]
\end{aligned}$$

This satisfies the constraint on *ether'* in *Rbc'*.

28.3.5 Case ‘obey constraints’

We know from the hypothesis that the before *BetweenWorld* satisfies the constraints, so we need check only that the chosen message *m!*, and any change of purse state during the operation, maintains this constraint.

If an operation obeys the following properties, then it preserves the *BetweenWorld* constraints:

- it does not change purse status or current transaction details (*pdAuth*)
- it does not change *allLogs*
- it does not change the payment detail messages, exception log read messages or exception log clear messages in the *ether* (either by not emitting such a message, or by emitting an already existing message)
- no sequence number decreases (all concrete operations have the property, so it is automatically satisfied)

Proof:

The *BetweenWorld* constraints refer only to certain *ether* messages, namely,

req, *val*, *ack*, *exceptionLogResult*, and *exceptionLogClear*

and relate their presence or absence to purse status (*status*, *pdAuth* and *nextSeqNo*) and *allLogs*. From the hypothesis we can invoke lemma ‘logs unchanged’ (section A.3) to say that, as *allLogs* does not change, not does *allLogs*. So operations that do not change the purse status, do not change *allLogs*, and do not emit any relevant new messages, will automatically preserve the constraints.

□ 28.3.5

Even when lemma ‘constraint’ does not apply, we know from the form of the operation that at most one purse changes, and one message is emitted. As at most one purse changes, the proof that the *BetweenWorld* constraints are preserved need refer only to this purse; the constraints hold on the other purses before the operation by hypothesis, and so they hold afterward, too.

28.3.6 Summary of *ConOkay* proof obligation

For each operation, we have to show that either lemma ‘constraint’ holds or that the choice of *BetweenWorld'* obeys the constraints (see section 5.3).

28.4 Correctness of *CIncrease*

CIncrease does not change *status* or *pdAuth*, does not log, and no relevant message is emitted to the *ether*, so lemma ‘constraint’ (section A.2) is applicable.

□ 28.4

28.5 Correctness of *CAbort*

Lemma ‘constraint’ is not applicable, because *CAbort* moves one purse into *eaFrom*, and it may not have been in this state before, and it may log a pending transaction. Therefore we have to show that our chosen *BetweenWorld'* obeys the constraints.

One $\perp$ message is emitted, and (possibly) one log is recorded.

B-1 $req \Rightarrow$ authentic to purse. No new req messages.

B-2 No future $reqs$. No new req messages.

B-3 No future $vals$. No new val messages.

B-4 No future $acks$. No new ack messages.

B-5 No future $from$ logs. The purse moves into $eaFrom$, possibly logging a transaction, and possibly increasing $nextSeqNo$. This does not invalidate this constraint for any previous logs. To create a new $from$ log, the purse would have had to have been in epa (from *LogIfNecessary*). Hence, using *ConPurse* constraint P-ref[p-epr], we have

$$pdAuth.fromSeqNo < nextSeqNo$$

From *AbortPurse*, we also have

$$nextSeqNo \leq nextSeqNo'$$

This gives

$$pdAuth.fromSeqNo < nextSeqNo'$$

The $pdAuth$ is logged when the pre-state purse is in epa , and thus the new log obeys the constraint.

B-6 No future to logs. The purse moves into $eaFrom$, possibly logging a transaction, and possibly increasing $nextSeqNo$. This does not invalidate this constraint for any previous logs. To create a new to log, the purse would have had to have been in epv (from *LogIfNecessary*); hence, using *ConPurse* constraint P-ref[p-epr-name], we have

$$pdAuth.toSeqNo < nextSeqNo$$

From *AbortPurse*, we also have

$$nextSeqNo \leq nextSeqNo'$$

This gives

$$pdAuth.toSeqNo < nextSeqNo'$$

The $pdAuth$ is logged when the pre-state purse is in epv , and thus the new log obeys the constraint.

B-7 $from$ in $\{epr, epa\}$, so no future $from$ logs. The purse moves into $eaFrom$, so no new purses in epr or epa .

B-8 to in $\{epv, eaTo\}$, so no future to logs. The purse moves into $eaFrom$, so no new purses in epv or $eaTo$.

B-9 $epr \Rightarrow \neg val \wedge \neg ack$. The purse moves into $eaFrom$, and so does not move into epr .

B-10 $req \wedge \neg ack \Leftrightarrow toInEpv \vee toLogged$.

- case $\Rightarrow$:

No new req messages; no ack messages removed from the *ether*.

The purse may have moved out of epv , but in such a case *LogIfNecessary* says that it logs, hence re-establishing the condition.

- case $\Leftarrow$:

No purses newly in epv .

There might be a new to log, in which case we must show there was a req , but no ack before. A to log can be made only by a purse moving out of epv . Then the *BetweenWorld* constraint B-ref[b-req-no-ack], on $toInEpv$, before the operation gives us the required req and lack of ack .

B-11 $epv \wedge val \Rightarrow fromInEpa \vee fromLogged$. No purses newly in epv ; no new val messages.

The purse may have moved out of epa . But in such a case *LogIfNecessary* says that it logs, hence re-establishing the condition.

B-12 $fromInEpa \vee fromLogged \Rightarrow req$. No purses newly in epa .

There might be a new $from$ log, in which case we must show there was a req before. A $from$ log can be made only by a purse moving out of epa . Then the *BetweenWorld* constraint B-ref[b-is-req], on $fromInEpa$, before the operation gives us the required req .

B-13 $toLogged$ finite. At most one to log written, so finite before gives finite after.

B-14 $exceptionLogResults$ in $allLogs$. No new exception log result messages.

B-15 Cleared logs archived. No $exceptionLogClear$ messages are added, and the archive is unchanged.

B-16 req for each log. If there are no new logs, then the constraint holds from the pre-state.

If a transaction exception is logged, then the purse status must have been either epv or epa . From constraints B-ref[b-req-no-ack] and B-ref[b-is-req], there was a req in the pre-state $ether$ for the transaction which was logged. This req will also be in the post-state $ether$.

□ 28.5

28.6 Lemma ‘logs unchanged’

When the *archive* and the individual purse logs do not change, and when no new req messages are added to the *ether*, the set of *PayDetails* representing all the logs does not change either.

$$\begin{aligned}
& BOpOkay \mid archive' = archive \\
& \quad \wedge req \triangleright ether' = req \triangleright ether \\
& \quad \wedge \forall n : \text{dom } conAuthPurse \bullet \\
& \quad \quad (conAuthPurse' n).exLog = (conAuthPurse n).exLog \\
& \vdash \\
& allLogs' = allLogs \\
& \wedge toLogged' = toLogged \\
& \wedge fromLogged' = fromLogged
\end{aligned}$$

Proof:

$$\begin{aligned}
allLogs &= archive \\
& \cup \{ n : \text{dom } conAuthPurse; ld : PayDetails \mid \\
& \quad ld \in (conAuthPurse n).exLog \} \\
& \hspace{20em} [\text{defn}] \\
&= archive' \\
& \cup \{ n : \text{dom } conAuthPurse'; ld : PayDetails \mid \\
& \quad ld \in (conAuthPurse' n).exLog \} \\
& \hspace{15em} [\text{assumption and } PhiBOp] \\
&= allLogs' \\
& \hspace{20em} [\text{defn}] \\
allLogs &= \{ n : \text{dom } conAuthPurse; pd : PayDetails \mid \\
& \quad n \mapsto pd \in allLogs \wedge req pd \in ether \} \\
& \hspace{20em} [\text{defn}] \\
&= \{ n : \text{dom } conAuthPurse'; pd : PayDetails \mid \\
& \quad n \mapsto pd \in allLogs' \wedge req pd \in ether' \} \\
& \hspace{15em} [\text{assumption and above}] \\
&= allLogs' \\
& \hspace{20em} [\text{defn}]
\end{aligned}$$

The arguments for $toLogged$ and $fromLogged$ follow in exactly the same way.

□ 28.6

28.7 Lemma ‘abort forward’: operations that first abort

Some concrete operations are written as a composition of *Abort* and a simpler operation starting from *eaFrom* (*StartFrom*, *StartTo*, *ReadExceptionLog*, *ClearExceptionLog*, etc.).

Where a $\mathcal{C}$ operation is written as a composition of *CAbort* and a simpler operation starting from *eaFrom*, and the corresponding $\mathcal{B}$ operation is structured analogously, it is sufficient to prove that the simpler $\mathcal{C}$ operation refines the corresponding $\mathcal{B}$ operation.

$$\begin{array}{l}
(CAbort \circ COpEafrom); Rbc; \\
(\forall COpEafrom; Rbc \bullet \exists BetweenWorld' \bullet Rbc' \wedge BOpEafrom) \\
\vdash \\
\exists BetweenWorld' \bullet Rbc' \wedge (Abort \circ BOpEafrom)
\end{array}$$

Proof We have already proved in section 28.5 that *CAbort* refines *Abort*. Adding this to our hypothesis, we get

$$\begin{array}{l}
(CAbort \circ COpEafrom); Rbc; \\
(\forall CAbort; Rbc \bullet \exists BetweenWorld' \bullet Rbc' \wedge Abort); \\
(\forall COpEafrom; Rbc \bullet \exists BetweenWorld' \bullet Rbc' \wedge BOpEafrom) \\
\vdash \\
\exists BetweenWorld' \bullet Rbc' \wedge (Abort \circ BOpEafrom)
\end{array}$$

The hypothesis is now in precisely the form required to use lemma ‘compose forward’, (section A.5) and we do so to prove the consequent.

□ 28.7

28.8 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	106
Total	0	115

Table 28.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	19
Total	0	24

Table 28.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	2	72
Total	2	84

Table 28.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	2	77
Theorems	13	81
Axiomatic consistency	0	5
Total	15	163

Table 28.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 28.5: Summary of *Z/Eves* rulees

Chapter 29

Correctness proofs

29.1 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	3
Generic definitions	0	0
Schemas	0	106
Total	0	115

Table 29.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	19
Total	0	24

Table 29.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	72
Total	0	84

Table 29.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	77
Theorems	0	81
Axiomatic consistency	0	5
Total	0	163

Table 29.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table 29.5: Summary of Z/Eves rulees

Chapter 30

Summary

Part VI

Appendices

Appendix A

Lemmas and their proofs

A.1 Lemma ‘deterministic’

A.2 Lemma ‘constraint’

A.3 Lemma ‘logs unchanged’

A.4 Lemma ‘compose backward’

A.5 Lemma ‘compose forward’

A.6 Lemma ‘promoted composition’

Appendix B

Auxiliary toolkit definitions

Originally Appendix D (p.225). In this document it appears after the original Chapter 4 because we need the definitions of *NAME* and *AbPurse* from Chapter 3 and *PayDetails* from chapter 4. These functions were used in conjectures in the original Chapter 2.

B.1 Total abstract balance

Original axiomatic Definition B.1 (Total balance of abstract purses)

$totalAbBalance : (NAME \multimap AbPurse) \rightarrow \mathbb{N}$
<i>Use braces for empty set to allow better automation</i>
$\langle\langle rule\ dTotalAbBalanceBase \rangle\rangle$ $totalAbBalance\ \{\} = 0$
$\langle\langle rule\ dTotalAbBalanceInduc \rangle\rangle$ $\forall w : (NAME \multimap AbPurse); n : NAME; AbPurse \mid n \notin \text{dom } w \bullet$ $totalAbBalance\ (\{ n \mapsto \theta AbPurse \} \cup w) =$ $balance + totalAbBalance\ w$

□

Proofs

proof[*totalAbBalance*\$*domainCheck*]
 prove;

■

B.2 Total lost value

Original axiomatic Definition B.2 (Total lost of abstract purses)

$totalLost : (NAME \multimap AbPurse) \rightarrow \mathbb{N}$
$\langle\langle rule\ dTotalLostBase \rangle\rangle$ $totalAbBalance\ \{\} = 0$
$\langle\langle rule\ dTotalLostInduc \rangle\rangle$ $\forall w : (NAME \multimap AbPurse); n : NAME; AbPurse \mid n \notin \text{dom } w \bullet$ $totalLost\ (\{ n \mapsto \theta AbPurse \} \cup w) =$ $balance + totalLost\ w$

□

Proofs

proof[*totalLost* $\$$ *domainCheck*]
 prove;

■

B.3 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	2	5
Generic definitions	0	0
Schemas	0	106
Total	2	117

Table B.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	19
Total	0	24

Table B.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	72
Total	0	84

Table B.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	77
Theorems	0	81
Axiomatic consistency	0	5
Total	0	163

Table B.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table B.5: Summary of Z/Eves rulees

Appendix C

Security Properties

C.1 Abstract model SPs

C.1.1 No value creation

Original schema C.1 (No value is created)

<i>No Value Creation</i>	_____
$\Delta AbWorld$	
$totalAbBalance_{abAuthPurse}' \leq totalAbBalance_{abAuthPurse}$	

□

Proofs

Maybe I should include a frule for *AbWorld* for the type of *abAuthPurse*. Let us see.

proof[*No Value Creation*\$domainCheck]
prove by reduce;
 ■

C.1.2 All value accounted

Original schema C.2 (All values are accounted for)

<i>All Value Accounted</i>	_____
$\Delta AbWorld$	
$totalAbBalance_{abAuthPurse}' + totalLost_{abAuthPurse}' =$ $totalAbBalance_{abAuthPurse} + totalLost_{abAuthPurse} =$ $totalAbBalance_{abAuthPurse} + totalLost_{abAuthPurse}$	

□

Proofs

proof[*All Value Accounted*\$domainCheck]
prove by reduce;
 ■

C.1.3 Authentic purses

Already included in 3.

C.2 Concrete model SPs

Already included in 4.

C.3 SPs and the models

The

C.4 Summary

Original declarations	This chapter	Total
Abbreviations	0	0
Given sets	0	2
Free types	0	4
Axiomatic definitions	0	5
Generic definitions	0	0
Schemas	2	108
Total	2	119

Table C.1: Summary of original declarations

Original theorems	This chapter	Total
Lemmas	0	5
Theorems	0	19
Total	0	24

Table C.2: Summary of original declarations

Added declarations	This chapter	Total
Axiomatic definitions	0	12
Schema	0	72
Total	0	84

Table C.3: Summary of added declarations

Added theorems	This chapter	Total
Lemmas	0	77
Theorems	0	81
Axiomatic consistency	0	5
Total	0	163

Table C.4: Summary of added theorems

<i>Z/Eves</i> declarations	This chapter	Total
Toolkit extensions	0	6
Assumption rules	0	18
Rewriting rules	0	18
Forward rules	0	19
Total	0	61

Table C.5: Summary of Z/Eves rulees

Part VII

Proofs

Appendix D

Chapter 3 proofs

D.1 A theory for purse names

For declarations in Section 3.1.1

```
proof[tNamesExistentialConsistency]  
  instantiate NAMES == {name1, name2};  
  instantiate n1 == name1, n2 == name2;  
  apply dExistsDistinctNames to predicate name1 = name2;  
  rewrite;
```

■

```
proof[tNamesUniversalConsistency]  
  instantiate NAMES == {name1, name2};  
  prenex;  
  rewrite;  
  split n1 = name1;  
  simplify;  
  cases;  
  instantiate n2 == name2;  
  apply dExistsDistinctNames to predicate name1 = name2;  
  rewrite;  
  next;  
  instantiate n2__1 == name1;  
  rewrite;  
  next;
```

■

```
proof[NAMES$domainCheck]  
  rewrite;
```

■

```
proof[rInNames]  
  rewrite;
```

■

```
proof[rFiniteMember]  
  rewrite;
```

■

```

proof[lExistsName]
  use NAMES$declaration;
  apply inFinset1;
  apply extensionality;
  prenex;
  rewrite;
  instantiate name == x;
  simplify;
  ■

```

```

proof[lExistsDifferentNames]
  use dNamesElem;
  simplify;
  ■

```

```

proof[lExistsDifferentNameFromGivenName]
  use dNamesDistinctElem[n1__0 := n1];
  prenex;
  instantiate n2__0 == n2;
  rearrange;
  simplify;
  ■

```

D.2 Finite space of numbers

See Section 3.1.2.

```

proof[tNATBoundaryConsistency]
  instantiate MAX_NAT == 0;
  prove by rewrite;
  ■

```

```

proof[tNATDefinitionConsistency]
  instantiate NAT == 0 .. MAX_NAT;
  prove by rewrite;
  ■

```

```

proof[gMaxNatMaxType]
  use MAX_NAT$declaration;
  rewrite;
  ■

```

```

proof[gMaxNatNatType]
  use MAX_NAT$declaration;
  simplify;
  ■

```

```

proof[rMaxNatType]
  use MAX_NAT$declaration;
  apply inNat ;
  simplify;
  ■

```

```

proof[rMaxNatType2]
  use MAX_NAT$declaration;
  apply inNat ;
  simplify;
  ■

```

```

proof[tNATConsistency]
  instantiate n == 0;
  invoke NAT;
  rewrite;
  use rMaxNatType;
  simplify;
  ■

```

```

proof[rInNAT]
  split x ∈ NAT;
  simplify;
  invoke NAT;
  cases;
  rewrite;
  next;
  split x ≤ MAX_NAT;
  simplify;
  split  $\neg x \geq 0$ ;
  simplify;
  rewrite;
  next;
  ■

```

D.3 Transfer details

See Section 3.1.3.

```

proof[fTransferDetailsValueType]
  prove by reduce;
  ■

```

```

proof[fTransferDetailsMemberMaxType]
  with enabled (TransferDetails$member, TransferDetails$inSet) prove by reduce;
  ■

```

```

proof[fTransferDetailsMember]
  with enabled (TransferDetails$member, TransferDetails$inSet) prove by reduce;
  ■

```

```

proof[fTransferDetailsInSetMember]
  with enabled (TransferDetails$member, TransferDetails$inSet) prove by reduce;
  ■

```

D.4 Security properties

See Section 3.2.

```

proof[fAuthenticNameType]
  prove by reduce;
  ■

```

Proofs

```

proof[SufficientFundsProperty$domainCheck]
  prove by reduce;
  ■

```

D.5 Secure operations

See Section 3.3.

```

proof[gAINTrasferPFunMaxType]
  use transfer$declaration;
  invoke ( $\_ \rightarrow \_$ );
  rewrite;
  ■

```

```

proof[gAINTrasferRelType]
  use transfer$declaration;
  invoke ( $\_ \rightarrow \_$ );
  invoke ( $\_ \leftrightarrow \_$ );
  invoke ( $\_ \leftrightarrow \_$ );
  prove by rewrite;
  apply inPower;
  prove by rewrite;
  instantiate  $e\_0 == e$ ;
  apply inCross2;
  prove by rewrite;
  ■

```

```

proof[gAINTrasferPFunType]
  use transfer$declaration;
  invoke ( $\_ \rightarrow \_$ );
  invoke ( $\_ \mapsto \_$ );
  prove by rewrite;
  instantiate  $x\_0 == x, y1\_0 == y1, y2\_0 == y2$ ;
  prove by rewrite;
  invoke ( $\_ \leftrightarrow \_$ );
  apply inPower;
  instantiate  $e == (x, y1)$ ;
  prove by rewrite;
  ■

```

```

proof[gAINTransferType]
  invoke ( $\_ \mapsto \_$ );
  rewrite;
  invoke ( $\_ \mapsto \_$ );
  prove by rewrite;
  use transfer$injective[ $x\$ := y1, x\$' := y2$ ];
  use pairInFunction[ $\lfloor$  from : NAME; to : NAME; value :  $\mathbb{Z}$   $\rfloor$  , AIN]
    [ $f := transfer, x := y1, y := x$ ];
  use pairInFunction[ $\lfloor$  from : NAME; to : NAME; value :  $\mathbb{Z}$   $\rfloor$  , AIN]
    [ $f := transfer, x := y2, y := x$ ];
  rearrange;
  simplify;
  use transfer$declaration;
  invoke ( $\_ \rightarrow \_$ );
  invoke ( $\_ \mapsto \_$ );
  invoke ( $\_ \leftrightarrow \_$ );
  prove by rewrite;
  apply inPower;
  instantiate  $e == (y1, transfer\ y2)$ ;
  instantiate  $e == (y2, transfer\ y2)$ ;
  prove by rewrite;
  ■

```

```

proof[rAINTransferResult]
  prove by rewrite;
  ■

```

```

proof[rAINTransferIsTotal]
  prove by rewrite;
  ■

```

```

proof[rAINTransferDomRanRel]
  prove by rewrite;
  ■

```

Proofs

```

proof[AbWorldSecureOp$domainCheck]
  prove by rewrite;
  ■

```

```

proof[rAbPurseLostTypeOpen]
  apply inDom;
  prenex;
  use pairInFunction[NAME,  $\langle \text{balance} : \mathbb{Z}; \text{lost} : \mathbb{Z} \rangle$ ]
    [f := purse, x := name, y := y];
  apply AbPurse$inSet;
  prove by rewrite;
  equality substitute purse name;
  invoke ( $- \rightsquigarrow -$ );
  invoke ( $- \leftrightarrow -$ );
  invoke ( $- \leftrightarrow -$ );
  prove by rewrite;
  apply inPower;
  instantiate e == (name, y);
  apply tupleInCross2;
  apply AbPurse$member;
  prove by reduce;
  ■

```

```

proof[rAbPurseBalanceTypeOpen]
  apply inDom;
  prenex;
  use pairInFunction[NAME,  $\langle \text{balance} : \mathbb{Z}; \text{lost} : \mathbb{Z} \rangle$ ]
    [f := purse, x := name, y := y];
  apply AbPurse$inSet;
  prove by rewrite;
  equality substitute purse name;
  invoke ( $- \rightsquigarrow -$ );
  invoke ( $- \leftrightarrow -$ );
  invoke ( $- \leftrightarrow -$ );
  prove by rewrite;
  apply inPower;
  instantiate e == (name, y);
  apply tupleInCross2;
  apply AbPurse$member;
  prove by reduce;
  ■

```

```

proof[rThetaPurseInstantiated]
  apply inDom;
  prenex;
  use pairInFunction[NAME,  $\langle \text{balance} : \mathbb{Z}; \text{lost} : \mathbb{Z} \rangle$ ]
    [f := purse, x := name, y := y];
  apply AbPurse$inSet;
  prove by rewrite;
  ■

```

```

proof[lAbAuthPurseFromTransferOkayMuEquivalence]
  apply muValue1;
  prove by rewrite;
  invoke;
  prove by rewrite;
  The values for balance0 and lost0 do not matter.

```


We just need to find an existent one.

```
instantiate balance == (abAuthPurse from?).balance,  
  lost == (abAuthPurse from?).lost,  
  balance__0 == 1, lost__0 == 1;  
prove by rewrite;  
invoke;  
prove by rewrite;
```

■

proof[lAbAuthPurseToTransferOkayMuEquivalence]

```
apply muValue1;  
prove by rewrite;  
invoke;  
prove by rewrite;
```

The values for balance0 and lost0 do not matter.

We just need to find an existent one.

```
instantiate balance == (abAuthPurse to?).balance,  
  lost == (abAuthPurse to?).lost,  
  balance__0 == 1, lost__0 == 1;  
prove by rewrite;  
invoke;  
prove by rewrite;
```

Could not understand why the rule was not

applied directly as before. So did it by hand!

```
use rAbPurseBalanceTypeOpen[name := to?, purse := abAuthPurse];
```

Due to the way Z/Eves rearranges predicates

"prove" is not helpful. So we do it ourselves.

```
rearrange;  
simplify;  
apply inNat;  
split (abAuthPurse to?).balance ≥ 0;  
simplify;
```

■

proof[lAbAuthPurseFromTransferLostMuEquivalence]

```
apply muValue1;  
prove by rewrite;  
invoke;  
prove by rewrite;
```

The values for balance0 and lost0 do not matter.

We just need to find an existent one.

```
instantiate balance == (abAuthPurse from?).balance,  
  lost == (abAuthPurse from?).lost,  
  balance__0 == 1, lost__0 == 1;  
prove by rewrite;  
invoke;  
prove by rewrite;
```

Could not understand why the rule was not

applied directly as before. So did it by hand!

*use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
Due to the way Z/Eves rearranges predicates*

”prove” is not helpful. So we do it ourselves.

*rearrange;
simplify;
apply inNat;
split (abAuthPurse from?).lost ≥ 0;
simplify;*

■

proof[AbTransferOkayTD\$domainCheck]

*prove by rewrite;
cases;
instantiate balance == (abAuthPurse from?).balance,
lost == (abAuthPurse from?).lost;
prove by reduce;
instantiate balance == 0, lost == 0;
simplify;
next;
instantiate balance__0 == (abAuthPurse to?).balance,
lost__0 == (abAuthPurse to?).lost;
prove by reduce;
instantiate balance == 0, lost == 0;*

I couldn't figure out why this automatic rule is not being applied

*use rAbPurseBalanceTypeOpen[purse := abAuthPurse, name := to?];
rearrange;
apply inNat;
simplify;
next;*

■

proof[AbTransferLostTD\$domainCheck]

prove by rewrite;

■

D.6 Summary

Declarations	This chapter	Total
Domain checks	5	9
Proof scripts	33	36
Total proofs	38	45
Total explained	0	0

Table D.1: Summary of proofs

Appendix E

Chapter 4 proofs

E.1 Message details

See Section 4.2.

```
proof[lExistsPayDetails]  
  use lExistsDifferentNames;  
  prenex;  
  instantiate pd == θ PayDetails[from := n1,  
    fromSeqNo := 0, to := n2, toSeqNo := 0, value := 0];  
  prove by reduce;
```

■

E.2 Value transfer operations

See Section 4.7.

```
proof[fCounterPartyDetailsNameType]  
  with enabled (CounterPartyDetails$member) prove by reduce ;
```

■

```
proof[fCounterPartyDetailsValueUpperBound]  
  with enabled (CounterPartyDetails$member) prove by reduce ;
```

■

```
proof[fCounterPartyDetailsValueLowerBound]  
  with enabled (CounterPartyDetails$member) prove by reduce ;
```

■

```
proof[fCounterPartyDetailsNextSeqNoUpperBound]  
  with enabled (CounterPartyDetails$member) prove by reduce ;
```

■

```
proof[fCounterPartyDetailsNextSeqNoLowerBound]  
  with enabled (CounterPartyDetails$member) prove by reduce ;
```

■

proof[*fCounterPartyDetailsMember*]
 with enabled (*CounterPartyDetails\$member*,
CounterPartyDetails\$inSet) prove by reduce;

■

proof[*fCounterPartyDetailsInSetMember*]
 with enabled (*CounterPartyDetails\$member*,
CounterPartyDetails\$inSet) prove by reduce;

■

E.2.1 *StartFrom*

proof[*gMESSAGEStartFromRelType*]
 use *startFrom\$declaration*;
 invoke ($- \rightarrow -$);
 invoke ($- \leftrightarrow -$);
 invoke ($- \leftrightarrow -$);
At this point fCounterPartyDetailsMember and

fCounterPartyDetailsInSetMember are used

rewrite;

Trivial rewriting is needed in order

not to loose information about power

type of startFrom.

trivial rewrite;

prenex;

rearrange;

equality substitute *r*;

apply *inPower*;

prenex;

instantiate $e_0 == e$;

with enabled (*inCross2*) prove by rewrite;

■

proof[*gMESSAGEStartFromPFunType*]
 use *startFrom\$declaration*;
 invoke ($- \rightarrow -$);
 invoke ($- \leftrightarrow -$);
At this point gMESSAGEStartFromRelType is used

prove by rewrite;

instantiate $x_0 == x$, $y1_0 == y1$, $y2_0 == y2$;

At this point fCounterPartyDetailsMember and

fCounterPartyDetailsInSetMember are used

prove by rewrite;

invoke ($- \leftrightarrow -$);

apply *inPower*;

instantiate $e == (x, y1)$;

prove by rewrite;

■

proof[*gMESSAGEStartFromPInjType*]
 invoke ($\neg \leftrightarrow \neg$);
 rewrite;
 invoke ($\neg \leftrightarrow \neg$);
At this point gMESSAGEStartFromRelType is used

 prove by rewrite;
At this point gMESSAGEStartFromPFunType is used

 use startFrom\$injective[$x\$:= y1, x\$' := y2$];
 use pairInFunction[$\langle \text{name} : \text{NAME}; \text{nextSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle, \text{MESSAGE}$]
 [$f := \text{startFrom}, x := y1, y := x$];
 use pairInFunction[$\langle \text{name} : \text{NAME}; \text{nextSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle, \text{MESSAGE}$]
 [$f := \text{startFrom}, x := y2, y := x$];
At this point fCounterPartyDetailsMember and

fCounterPartyDetailsInSetMember are used

 prove by rewrite;
 use startFrom\$declaration;
 invoke ($\neg \rightarrow \neg$);
 invoke ($\neg \leftrightarrow \neg$);
 invoke ($\neg \leftrightarrow \neg$);
 prove by rewrite;
 apply inPower;
 instantiate $e == (y1, \text{startFrom } y2)$;
 instantiate $e == (y2, \text{startFrom } y2)$;
 prove by rewrite;
 ■

proof[*ValidStartFrom\$domainCheck*]
 prove by rewrite;
 ■

proof[*StartFromPurseEafromOkay\$domainCheck*]
 with enabled (*CounterPartyDetails\$member*) prove by reduce ;
 ■

E.2.2 StartTo

proof[*gMESSAGEStartToRelType*]
 use startTo\$declaration;
 invoke ($\neg \rightarrow \neg$);
 invoke ($\neg \leftrightarrow \neg$);
 invoke ($\neg \leftrightarrow \neg$);
 rewrite;
 trivial rewrite;
 prenex;
 rearrange;
 equality substitute r ;
 apply inPower;
 prenex;
 instantiate $e_0 == e$;
 with enabled (*inCross2*) prove by rewrite;
 ■

```

proof[gMESSAGEstartToPFunType]
  use startTo$declaration;
  invoke ( $\rightarrow$ );
  invoke ( $\leftrightarrow$ );
  prove by rewrite;
  instantiate  $x\_0 == x, y1\_0 == y1, y2\_0 == y2$ ;
  prove by rewrite;
  invoke ( $\leftrightarrow$ );
  apply inPower;
  instantiate  $e == (x, y1)$ ;
  prove by rewrite;

```

■

```

proof[gMESSAGEstartToPInjType]
  invoke ( $\leftrightarrow$ );
  rewrite;
  invoke ( $\leftrightarrow$ );
  prove by rewrite;
  use startTo$injective[ $x\$ := y1, x\$' := y2$ ];
  use pairInFunction[ $\langle name : NAME; nextSeqNo : \mathbb{N}; value : \mathbb{N} \rangle, MESSAGE$ ]
    [ $f := startTo, x := y1, y := x$ ];
  use pairInFunction[ $\langle name : NAME; nextSeqNo : \mathbb{N}; value : \mathbb{N} \rangle, MESSAGE$ ]
    [ $f := startTo, x := y2, y := x$ ];
  prove by rewrite;
  use startTo$declaration;
  invoke ( $\rightarrow$ );
  invoke ( $\leftrightarrow$ );
  invoke ( $\leftrightarrow$ );
  prove by rewrite;
  apply inPower;
  instantiate  $e == (y1, startTo y2)$ ;
  instantiate  $e == (y2, startTo y2)$ ;
  prove by rewrite;

```

■

```

proof[ValidStartTo$domainCheck]
  prove by rewrite;

```

■

```

proof[StartToPurseEafromOkay$domainCheck]
  with enabled (CounterPartyDetails$member) prove by reduce ;

```

■

E.2.3 ReqPurseOkay

```

proof[AuthenticReqMessage$domainCheck]
  prove by rewrite;

```

■

```

proof[ReqPurseOkay$domainCheck]
  prove by rewrite;

```

■

E.2.4 *ValPurse*

proof[*AuthenticValMessage\$domainCheck*]
 prove by rewrite;

■

proof[*ValPurseOkay\$domainCheck*]
 prove by rewrite;

■

E.2.5 *AckPurse*

proof[*AuthenticAckMessage\$domainCheck*]
 prove by rewrite;

■

E.3 Exception logging operations

See Section 4.8.

E.3.1 *ReadExceptionLogPurse*

proof[*ReadExceptionLogPurseEafromOkay\$domainCheck*]
 prove by rewrite;

■

E.3.2 *ClearExceptionLogPurse*

proof[*ClearExceptionLogPurseEafromOkay\$domainCheck*]
 prove by rewrite;

■

E.4 Summary

Declarations	This chapter	Total
Domain checks	11	20
Proof scripts	14	50
Total proofs	25	70
Total explained	0	0

Table E.1: Summary of proofs

Appendix F

Chapter 5 proofs

F.1 The world

```
proof[tLogbookConsistency]  
  prove by rewrite;  
  invoke ( $\_ \leftrightarrow \_$ );  
  apply cupSubset;  
  apply inPower;  
  prove by rewrite;  
  apply mapDef;  
  prove by reduce;  
  ■
```

```
proof[ConWorld$domainCheck]  
  prove by rewrite;  
  ■
```

F.2 *AuxWorld*

See Section 5.2.

```
proof[AuxWorld$domainCheck]  
  prove by rewrite;  
  ■
```

```
proof[tAuxWorldIdentity]  
  invoke AuxWorld;  
  equality substitute definitelyLost;  
  equality substitute maybeLost;  
  apply cupCommutates to expression  $\text{fromLogged} \cup \text{fromInEpa}$ ;  
  apply capCommutates to expression  
     $(\text{fromInEpa} \cup \text{fromLogged}) \cap (\text{toInEpa} \cup \text{toLogged})$ ;  
  apply distributeCapOverCupLeft to expression  
     $(\text{toInEpa} \cup \text{toLogged}) \cap (\text{fromInEpa} \cup \text{fromLogged})$ ;  
  rewrite;  
  ■
```

F.2.1 Finiteness properties

```
proof[rCrossFinite]
  rewrite;
```

■

```
proof[lFinsetSubset]
  rewrite;
```

■

```
proof[lPayDetailsSpaceFiniteMaxType]
  reduce;
```

■

```
proof[lPayDetailsBindingsSubsetPayDetailsSpace]
  apply inPower;
  apply inDom;
  prove by reduce;
  invoke ( $\_ \leftrightarrow \_$ );
  with normalization simplify;
  prove by rewrite;
  apply inPower;
  prove by reduce;
```

■

```
proof[lPayDetailsBindingsFiniteMaxType]
  use lFinsetSubset [ $X := \text{dom } \text{PayDetailsBindings}, Y := \text{PayDetailsSpace},$ 
     $Z := ((\text{NAME} \times \text{NAME}) \times (\mathbb{Z} \times (\mathbb{Z} \times \mathbb{Z})))$ ];
  use lPayDetailsBindingsSubsetPayDetailsSpace;
  use lPayDetailsSpaceFiniteMaxType;
  simplify;
```

■

```
proof[gPayDetailsBindingsPfun]
  invoke ( $\_ \leftrightarrow \_$ );
  invoke ( $\_ \leftrightarrow \_$ );
  prove by reduce;
```

■

```
proof[gPayDetailsBindingsFfun]
  use functionFinite [ $((\text{NAME} \times \text{NAME}) \times (\mathbb{Z} \times (\mathbb{Z} \times \mathbb{Z})))$ ,
     $\langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle$ ]
    [ $A := ((\text{NAME} \times \text{NAME}) \times (\mathbb{Z} \times (\mathbb{Z} \times \mathbb{Z})))$ ,
       $B := \text{PayDetails}, f := \text{PayDetailsBindings}$ ];
  use gPayDetailsBindingsPfun;
  use lPayDetailsBindingsFiniteMaxType;
  rewrite;
```

■

```

proof[lPayDetailsBindinsRange]
  apply extensionality2;
  apply inPower;
  apply inRan;
  prove by rewrite;
  apply PayDetails$member;
  prove by reduce;

```

■

```

proof[lPayDetailsIsFinite]
  use finiteFunction[((NAME × NAME) × (ℤ × (ℤ × ℤ))),
    ⟨ from : NAME; fromSeqNo : ℤ; to : NAME; toSeqNo : ℤ; value : ℤ ⟩]
  [f := PayDetailsBindings];
  rewrite;

```

■

```

proof[lPayDetailsFromPurseNameFinite]
  use lPayDetailsIsFinite;
  use lFinsetSubset[X := { pd : PayDetails | pd.from = name },
    Y := PayDetails, Z := ⟨ from : NAME; fromSeqNo : ℤ; to :
      NAME; toSeqNo : ℤ; value : ℤ ⟩];
  rearrange;
  rewrite;
  invoke (ℱ _);
  trivial rewrite;
  rearrange;
  rewrite;
  prenex;
  instantiate n__0 == n;
  rewrite;
  instantiate f__0 == f;
  rewrite;
  apply ranDefinition;
  rewrite;
  apply extensionality;
  prenex;
  rewrite;
  prenex;
  rewrite;
  instantiate x__1 == x;
  instantiate y__0 == y;
  prenex;
  rewrite;
  instantiate x__2 == x__0;
  instantiate x__5 == x__0;
  prove by rewrite;

```

■

```

proof[lPayDetailsToPurseNameFinite]
  use lPayDetailsIsFinite;
  use lFinsetSubset[X := { pd : PayDetails | pd.to = name },
    Y := PayDetails, Z := { from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :  $\mathbb{Z}$ ; value :  $\mathbb{Z}$  } ];
  rearrange;
  rewrite;
  invoke ( $\mathbb{F}$  _);
  trivial rewrite;
  rearrange;
  rewrite;
  prenex;
  instantiate n__0 == n;
  rewrite;
  instantiate f__0 == f;
  rewrite;
  apply ranDefinition;
  rewrite;
  apply extensionality;
  prenex;
  rewrite;
  prenex;
  rewrite;
  instantiate x__1 == x;
  instantiate y__0 == y;
  prenex;
  rewrite;
  instantiate x__2 == x__0;
  instantiate x__5 == x__0;
  prove by rewrite;
  ■

```

```

proof[lPayDetailsFromPurseDomainFinite]
  use lPayDetailsIsFinite;
  use lFinsetSubset[ $X := \{ \text{pd} : \text{PayDetails} \mid \text{pd.from} \in \text{dom purse} \},$ 
     $Y := \text{PayDetails}, Z := \langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle$ ];
  rearrange;
  rewrite;
  invoke ( $\mathbb{F}$  _);
  trivial rewrite;
  rearrange;
  rewrite;
  prenex;
  instantiate  $n\_0 == n$ ;
  instantiate  $f\_0 == f$ ;
  rewrite;
  apply ranDefinition;
  rewrite;
  apply extensionality;
  prenex;
  rewrite;
  prenex;
  rewrite;
  instantiate  $x\_1 == x$ ;
  instantiate  $y\_0 == y$ ;
  prenex;
  rewrite;
  instantiate  $x\_2 == x\_0$ ;
  instantiate  $x\_5 == x\_0$ ;
  rearrange;
  rewrite;
  rearrange;
  rewrite;

```

■

```

proof[lPayDetailsToPurseDomainFinite]
  use lPayDetailsIsFinite;
  use lFinsetSubset[ $X := \{ \text{pd} : \text{PayDetails} \mid \text{pd.to} \in \text{dom purse} \},$ 
     $Y := \text{PayDetails}, Z := \langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle$ ];
  rearrange;
  rewrite;
  invoke ( $\mathbb{F}$  _);
  trivial rewrite;
  rearrange;
  rewrite;
  prenex;
  instantiate  $n\_0 == n$ ;
  instantiate  $f\_0 == f$ ;
  rewrite;
  apply ranDefinition;
  rewrite;
  apply extensionality;
  prenex;
  rewrite;
  prenex;
  rewrite;
  instantiate  $x\_1 == x$ ;
  instantiate  $y\_0 == y$ ;
  prenex;
  rewrite;
  instantiate  $x\_2 == x\_0$ ;
  instantiate  $x\_5 == x\_0$ ;
  rearrange;
  rewrite;
  rearrange;
  rewrite;
  ■

```

```

proof[fAuxWorldAuthenticFromIsFinite]
  invoke AuxWorld;
  equality substitute authenticFrom;
  rewrite;
  ■

```

```

proof[fAuxWorldAuthenticToIsFinite]
  invoke AuxWorld;
  equality substitute authenticTo;
  rewrite;
  ■

```

F.2.2 *AuxWorld* does not add constraints

```

proof[PayDetailsFromDomEPRPurse$domainCheck]
  rewrite;
  ■

```

```

proof[PayDetailsFromDomEPAPurse$domainCheck]
  rewrite;

```

■

```

proof[PayDetailsToDomEPVPurse$domainCheck]
  rewrite;

```

■

```

proof[PayDetailsToDomEATOPurse$domainCheck]
  rewrite;

```

■

```

proof[lPayDetailsFromPurseEPRIsFinite]
  use lPayDetailsIsFinite;
  use lFinsetSubset[X := PayDetailsFromDomEPRPurse,
    Y := PayDetails, Z :=  $\langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle$  ];
  rearrange;
  simplify;
  invoke ( $\mathbb{F}$   $-$ );
  trivial rewrite;
  rearrange;
  rewrite;
  prenex;
  instantiate n_0 == n;
  instantiate f_0 == f;
  rewrite;
  apply ranDefinition;
  rewrite;
  apply extensionality;
  prenex;
  rewrite;
  prenex;
  rewrite;
  instantiate x_1 == x;
  instantiate y_0 == y;
  prenex;
  rewrite;
  rewrite;
  instantiate x_2 == x_0;
  instantiate x_5 == x_0;
  rearrange;
  rewrite;
  rearrange;
  rewrite;

```

■


```

proof[lPayDetailsFromPurseEPAIsFinite]
  use lPayDetailsIsFinite;
  use lFinsetSubset[X := PayDetailsFromDomEPAPurse,
    Y := PayDetails, Z :=  $\langle \mid$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to :
      NAME; toSeqNo :  $\mathbb{Z}$ ; value :  $\mathbb{Z} \mid \rangle$  ];
  rearrange;
  simplify;
  invoke ( $\mathbb{F}$  _);
  trivial rewrite;
  rearrange;
  rewrite;
  prenex;
  instantiate n__0 == n;
  instantiate f__0 == f;
  rewrite;
  apply ranDefinition;
  rewrite;
  apply extensionality;
  prenex;
  rewrite;
  prenex;
  rewrite;
  instantiate x__1 == x;
  instantiate y__0 == y;
  prenex;
  rewrite;
  rewrite;
  instantiate x__2 == x__0;
  instantiate x__5 == x__0;
  rearrange;
  rewrite;
  rearrange;
  rewrite;

```

■

```

proof[lPayDetailsToPurseEPVIsFinite]
  use lPayDetailsIsFinite;
  use lFinsetSubset[X := PayDetailsToDomEPVPurse,
    Y := PayDetails, Z :=  $\langle \langle$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to :
      NAME; toSeqNo :  $\mathbb{Z}$ ; value :  $\mathbb{Z}$   $\rangle \rangle$  ];
  rearrange;
  simplify;
  invoke ( $\mathbb{F}$  _);
  trivial rewrite;
  rearrange;
  rewrite;
  prenex;
  instantiate n__0 == n;
  instantiate f__0 == f;
  rewrite;
  apply ranDefinition;
  rewrite;
  apply extensionality;
  prenex;
  rewrite;
  prenex;
  rewrite;
  instantiate x__1 == x;
  instantiate y__0 == y;
  prenex;
  rewrite;
  rewrite;
  instantiate x__2 == x__0;
  instantiate x__5 == x__0;
  rearrange;
  rewrite;
  rearrange;
  rewrite;

```

■

```

proof[lPayDetailsToPurseEATOfinite]
  use lPayDetailsIsFinite;
  use lFinsetSubset[X := PayDetailsToDomEATOPurse,
    Y := PayDetails, Z := { from : NAME; fromSeqNo : ℤ; to :
      NAME; toSeqNo : ℤ; value : ℤ }];
  rearrange;
  simplify;
  invoke (ℱ _);
  trivial rewrite;
  rearrange;
  rewrite;
  prenex;
  instantiate n__0 == n;
  instantiate f__0 == f;
  rewrite;
  apply ranDefinition;
  rewrite;
  apply extensionality;
  prenex;
  rewrite;
  prenex;
  rewrite;
  instantiate x__1 == x;
  instantiate y__0 == y;
  prenex;
  rewrite;
  rewrite;
  instantiate x__2 == x__0;
  instantiate x__5 == x__0;
  rearrange;
  rewrite;
  rearrange;
  rewrite;

```

■

```

proof[tAuxWorldDoesNotAddConstraints]
  rewrite;
  invoke AuxWorld;
  simplify;
  invoke NewVariables;
  instantiate
    conAuthPurse__0 == conAuthPurse, archive__0 == archive,
    ether__0 == ether;
  invoke AuxWorld;
  simplify;
  invoke (_ ↔ _);
  rewrite;
  use lPayDetailsFromPurseEPRIIsFinite[purse := conAuthPurse];
  use lPayDetailsFromPurseEPAIsFinite[purse := conAuthPurse];
  use lPayDetailsToPurseEPVIsFinite[purse := conAuthPurse];
  use lPayDetailsToPurseEATOfinite[purse := conAuthPurse];
  prove by rewrite;

```

■

F.3 Constraints on the ether

See Section 5.3.

proof[*BetweenWorldB1\$domainCheck*]
 prove by rewrite;

■

proof[*BetweenWorldB2\$domainCheck*]
 Do not prove by reduce because it will apply

 too much equality substitution on authenticTo

 prove by rewrite;
 invoke BetweenWorldB1;
 instantiate pd__0 == pd;
 Now we have everything exposed, it is fine

 prove by reduce;

■

proof[*BetweenWorldNEW1\$domainCheck*]
 prove by rewrite;

■

proof[*BetweenWorldNEW2\$domainCheck*]
 prove by rewrite;

■

proof[*BetweenWorldB3\$domainCheck*]
 prove by rewrite;
 invoke BetweenWorldNEW1;
 invoke BetweenWorldNEW2;
 instantiate pd__0 == pd;
 instantiate pd__1 == pd;
 prove by reduce ;

■

proof[*BetweenWorldNEW3\$domainCheck*]
 prove by rewrite;

■

proof[*BetweenWorldNEW4\$domainCheck*]
 prove by rewrite;

■

proof[*BetweenWorldB4\$domainCheck*]

prove by rewrite;
invoke BetweenWorldNEW3;
invoke BetweenWorldNEW4;
instantiate pd__0 == pd;
instantiate pd__1 == pd;
prove by reduce ;

■

proof[*BetweenWorldB5\$domainCheck*]

As there is no need for instantiation this time,

prove by reduce is a better (and simpler) choice.

prove by reduce ;

■

proof[*BetweenWorldB6\$domainCheck*]

prove by reduce ;

■

proof[*BetweenWorldB7\$domainCheck*]

prove by reduce ;

■

proof[*BetweenWorldB8\$domainCheck*]

prove by reduce ;

■

proof[*BetweenWorldB9\$domainCheck*]

prove by rewrite;

■

proof[*BetweenWorldB10\$domainCheck*]

prove by rewrite;

■

proof[*BetweenWorldB11\$domainCheck*]

prove by rewrite;

■

proof[*BetweenWorldB12\$domainCheck*]

Reduction is needed because of the disjunction

at the assumptions generated by the set union.

prove by reduce;

■

proof[*BetweenWorldB15\$domainCheck*]
prove by rewrite;

■

proof[*BetweenWorldB16\$domainCheck*]
Similar to domain check of BetweenWorldB12

prove by reduce;

■

proof[*lBetweenWorldExpansion*]
invoke BetweenWorld;
invoke BetweenWorldB16;
invoke BetweenWorldB15;
invoke BetweenWorldB14;
invoke BetweenWorldB13;
invoke BetweenWorldB12;
invoke BetweenWorldB11;
invoke BetweenWorldB10;
invoke BetweenWorldB9;
invoke BetweenWorldB8;
invoke BetweenWorldB7;
invoke BetweenWorldB6;
invoke BetweenWorldB5;
invoke BetweenWorldB4;
invoke BetweenWorldNEW4;
invoke BetweenWorldNEW3;
invoke BetweenWorldB3;
invoke BetweenWorldNEW2;
invoke BetweenWorldNEW1;
invoke BetweenWorldB2;
invoke BetweenWorldB1;
simplify;

■

proof[*lBetweenWorldAuxWorldExpansion*]
use lBetweenWorldExpansion;
rearrange;
simplify;
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
simplify;

■

proof[*lBetweenWorldFullExpansion*]
use lBetweenWorldAuxWorldExpansion;
rearrange;
simplify;
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
invoke ConWorld;
simplify;

■

F.4 Framing schema

See Section 5.4.

proof[*PhiBOp\$domainCheck*]
prove by rewrite;

■

F.5 Operations at the world level only

See Section 5.7.

proof[*AuthoriseExLogClearOkay\$domainCheck*]
prove by rewrite;

■

proof[*Archive\$domainCheck*]
prove by rewrite;

■

F.6 Summary

Declarations	This chapter	Total
Domain checks	27	47
Proof scripts	25	75
Total proofs	52	122
Total explained	0	0

Table F.1: Summary of proofs

Appendix G

Chapter 6 proofs

G.1 Summing values

See Section 6.1.

```
proof[sumValue$domainCheck]  
  prove by rewrite;  
  ■
```

G.2 Initialisation

See Section 6.2.

```
proof[BetweenInitState$domainCheck]  
  prove by rewrite;  
  ■
```

```
proof[fPayDetailsFromType]  
  with enabled (PayDetails$member) prove by reduce ;  
  ■
```

```
proof[fPayDetailsToType]  
  with enabled (PayDetails$member) prove by reduce ;  
  ■
```

```
proof[fPayDetailsValueType]  
  with enabled (PayDetails$member) prove by reduce ;  
  ■
```

```
proof[fPayDetailsMember]  
  with enabled (PayDetails$member, PayDetails$inSet) prove by reduce;  
  ■
```

```

proof[gMESSAGEReqRelType]
  use req$declaration;
  invoke ( _  $\rightarrow$  _ );
  invoke ( _  $\leftrightarrow$  _ );
  invoke ( _  $\leftrightarrow$  _ );
  At this point fCounterPartyDetailsMember and
    fCounterPartyDetailsInSetMember are used

  rewrite;
  Trivial rewriting is needed in order
    not to loose information about power
    type of req.

  trivial rewrite;
  prenex;
  rearrange;
  equality substitute r;
  apply inPower;
  prenex;
  instantiate e__0 == e;
  with enabled (inCross2) prove by rewrite;
  ■

```

```

proof[gMESSAGEReqPFunType]
  use req$declaration;
  invoke ( _  $\rightarrow$  _ );
  invoke ( _  $\leftrightarrow$  _ );
  At this point gMESSAGEReqRelType is used

  prove by rewrite;
  instantiate x__0 == x, y1__0 == y1, y2__0 == y2;
  At this point fPayDetailsMember is used

  prove by rewrite;
  Small difference from other similar proofs
    as there is no fPayDetailsInSetMember

  invoke ( _  $\leftrightarrow$  _ );
  apply inPower;
  instantiate e == (x, y1);
  with enabled (inCross2) prove by rewrite;
  ■

```

```

proof[gMESSAGEReqPayDetailsRelType]
  use req$declaration;
  invoke ( _  $\rightarrow$  _ );
  invoke ( _  $\leftrightarrow$  _ );
  prove by rewrite;
  ■

```

```

proof[gMESSAGEReqPInjType]
  invoke ( $\_ \rightsquigarrow \_$ );
  rewrite;
  invoke ( $\_ \rightarrow \_$ );
  At this point gMESSAGEReqRelType is used

  prove by rewrite;
  At this point gMESSAGEReqPFunType is used

  use pairInFunction[ $\langle \_ \text{ from} : \text{NAME}; \text{fromSeqNo} : \mathbb{N}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle, \text{MESSAGE}$ ]
    [f := req, x := y1, y := x];
  use pairInFunction[ $\langle \_ \text{ from} : \text{NAME}; \text{fromSeqNo} : \mathbb{N}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{N}; \text{value} : \mathbb{N} \rangle, \text{MESSAGE}$ ]
    [f := req, x := y2, y := x];
  At this point fPayDetailsMember is used

  prove by rewrite;
  Small difference from other similar proofs

  use gMESSAGEReqPayDetailsRelType;
  invoke ( $\_ \leftrightarrow \_$ );
  apply inPower;
  instantiate e == (y2, req y2);
  instantiate e == (y1, req y2);
  with enabled (inCross2) prove by rewrite;
  ■

```

```

proof[rMESSAGEReqInvMaxType]
  apply inRan;
  prove by rewrite;
  use pairInFunction[ $\langle \_ \text{ from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle, \text{MESSAGE}$ ]
    [f := req, x := x, y := m];
  prove by rewrite;
  use memberFirstInDom[ $\langle \_ \text{ from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle, \text{MESSAGE}$ ]
    [R := req, x := x, y := req x];
  rearrange;
  with predicate (req  $\in \langle \_ \text{ from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle \leftrightarrow \text{MESSAGE}$ ) rewrite;
  simplify;
  use applyInverse[ $\langle \_ \text{ from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle, \text{MESSAGE}$ ]
    [A :=  $\langle \_ \text{ from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle$ ,
     B := MESSAGE, f := req, x := x];
  with enabled (PayDetails$member) prove by reduce;
  ■

```

```

proof[BetwInitIn$domainCheck]
  cases;
  prove by rewrite;
  next;
  prove by reduce;
  use rMESSAGEReqInvMaxType[m := m?];
  apply PayDetails$inSet;
  prove by rewrite;
  next;
  use muInSet[{ TransferDetails[from__1/from, to__1/to, value__1/value]
    | from__1 = (req ~ m?).from  $\wedge$  to__1 = (req ~ m?).to
     $\wedge$  value__1 = (req ~ m?).value }];
  prove by reduce;
  instantiate a ==  $\theta$  TransferDetails[from := (req ~ m?).from,
    to := (req ~ m?).to, value := (req ~ m?).value];
  use rMESSAGEReqInvMaxType[m := m?];
  apply PayDetails$inSet;
  prove by reduce;
  equality substitute
    ( $\mu$  m : { TransferDetails[from__1/from, to__1/to, value__1/value]
      | from__1 = (req ~ m?).from  $\wedge$  to__1 = (req ~ m?).to
       $\wedge$  value__1 = (req ~ m?).value });
  apply TransferDetails$thetaMember;
  prove by reduce;
  next;
  ■

```

G.3 Finalisation

See Section 6.3.

```

proof[BetwFinState$domainCheck]
  prove by rewrite;
  ■

```

G.4 Summary

Declarations	This chapter	Total
Domain checks	4	51
Proof scripts	9	84
Total proofs	13	135
Total explained	0	0

Table G.1: Summary of proofs

Appendix H

Chapter 7 proofs

See Section 7.2.

proof[*PhiCOp\$domainCheck*]
 prove by rewrite;

■

proof[*CPhiCOpExpLogClearNEW\$domainCheck*]
 prove by rewrite;

■

proof[*CArchive\$domainCheck*]
 prove by rewrite;

■

proof[*ConFinState\$domainCheck*]
 prove by rewrite;

■

H.1 Summary

Declarations	This chapter	Total
Domain checks	4	55
Proof scripts	0	84
Total proofs	4	139
Total explained	0	0

Table H.1: Summary of proofs

Appendix I

Chapter 8 proofs

```
proof[fConPurseNextSeqNoMaxType]  
  invoke ConPurse;  
  rewrite;  
  ■
```

I.1 Abstract model consistency

See Section 8.2.

```
proof[tExistsInitialAbWorld]  
  prove by reduce ;  
  instantiate abAuthPurse' == {};  
  prove by reduce ;  
  ■
```

```
proof[tAbOpIsTotal]  
  instantiate a! == aNullOut, abAuthPurse' == abAuthPurse;  
  prove by reduce;  
  ■
```

```
proof[tAbIgnoreIsTotal]  
  prove by reduce ;  
  ■
```

```
proof[tAbTransferIsTotal]  
  instantiate a! == aNullOut, abAuthPurse' == abAuthPurse;  
  with disabled (AbTransferOkay, AbTransferLost) prove by reduce;  
  ■
```

```
proof[tAbWorldSecureOpPRE]  
  instantiate a! == aNullOut, abAuthPurse' == abAuthPurse;  
  prove by reduce;  
  ■
```


proof[*AbTransferOpSig\$domainCheck*]
prove by rewrite;

■

proof[*lAbTransferOkayPREAuthPurseUpdateMaxType*]
apply overrideInPowerCross;
apply cupSubset;
invoke AbWorld;
rewrite;
use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
apply inNat;
prove by rewrite;

■

proof[*lAbTransferOkayPREAuthPurseMaxType*]
apply overrideInPowerCross;
apply cupInRel;
invoke AbWorld;
rewrite;
use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
apply inNat;
prove by rewrite;

■

```

proof[lAbTransferOkayPREAbWorld]
  invoke AbWorld;
  invoke ( $- \leftrightarrow -$ );
  rewrite;
  cases;
    apply overrideInPfun;
    apply cupInPfun;
    rewrite;
    use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
    use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
    apply inNat;
    invoke ( $- \leftrightarrow -$ );
    rearrange;
    rewrite;
    invoke AbPurse;
    invoke SufficientFundsProperty;
    prove by rewrite;
    use rAbPurseBalanceTypeOpen[name := to?, purse := abAuthPurse];
    apply inNat;
    invoke ( $- \leftrightarrow -$ );
    invoke Authentic;
    invoke TransferDetails;
    rearrange;
    rewrite;
  next;
    apply oplusDef;
    invoke ( $- \leftrightarrow -$ );
    apply cupSubset;
    prove by rewrite;
    use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
    use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
    invoke ( $- \leftrightarrow -$ );
    apply inNat;
    rearrange;
    rewrite;
    invoke AbPurse;
    invoke TransferDetails;
    rewrite;
    use rAbPurseBalanceTypeOpen[name := from?, purse := abAuthPurse];
    use rAbPurseBalanceTypeOpen[name := to?, purse := abAuthPurse];
    rearrange;
    apply inNat;
    invoke ( $- \leftrightarrow -$ );
    invoke SufficientFundsProperty;
    rewrite;
  next;

```

■

proof[*lAbTransferOkayPRENDResEquiv*]
apply extensionality to predicate ($\{from?\} \cup \{to?\} \triangleleft$
 $(abAuthPurse \oplus (\{from?, \theta \ AbPurse[balance := -1 * value?$
 $+ (abAuthPurse from?).balance, lost := (abAuthPurse from?).lost]\})$
 $\cup \{(to?, \theta \ AbPurse[balance := value? +$
 $(abAuthPurse to?).balance, lost := (abAuthPurse to?).lost]\})\}$
 $= (\{from?\} \cup \{to?\}) \triangleleft abAuthPurse;$
use lAbTransferOkayPREAuthPurseMaxType;
use lAbTransferOkayPREAuthPurseUpdateMaxType;
use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
apply ndresDef;
apply oplusDef;
apply inNat;
rearrange;
simplify;
prove by rewrite;
apply inCup;
invoke AbWorld;
prove by rewrite;
■

proof[*lAbTransferOkayPREAuthentic*]
use lAbTransferOkayPREAbWorld;
use lAbTransferOkayPREAuthPurseUpdateMaxType;
use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
apply inNat;
invoke Authentic;
rearrange;
simplify;
apply inDom;
prove by rewrite;
apply oplusDef;
prove by rewrite;
apply inCup;
invoke AbWorld;
prove by rewrite;
■

proof[*lAbTransferOkayPREFromOPlusEquiv*]
apply applyOverride1;
apply applyCupLeft;
apply applyUnit;
invoke ($- \leftrightarrow -$);
apply cupInPfun;
use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
apply lAbTransferOkayPREAuthPurseUpdateMaxType;
apply inNat;
invoke AbWorld;
invoke Authentic;
rearrange;
rewrite;
■

```

proof[lAbTransferOkayPREToOPlusEquiv]
  apply applyOverride1;
  apply applyCupLeft;
  apply applyUnit;
  invoke ( $- \leftrightarrow -$ );
  apply cupInPfun;
  use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
  use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
  apply lAbTransferOkayPREAuthPurseUpdateMaxType;
  apply inNat;
  invoke AbWorld;
  invoke Authentic;
  rearrange;
  rewrite;

```

■

```

proof[tAbTransferOkayTDPRE]
  instantiate a! == aNullOut, abAuthPurse' == abAuthPurse
    ⊕ ({(from?,  $\theta$  AbPurse[balance := - 1 * value? +
      (abAuthPurse from?).balance,
      lost := (abAuthPurse from?).lost]))}
    ∪ {(to?,  $\theta$  AbPurse[balance := value? +
      (abAuthPurse to?).balance,
      lost := (abAuthPurse to?).lost]))});
  with disabled (AbWorldSecureOp, TransferDetails, Authentic,
    SufficientFundsProperty) prove by reduce;
  use lAbTransferOkayPREAuthPurseMaxType;
  rearrange;
  simplify;
  with disabled (AbWorld, Authentic, SufficientFundsProperty,
    TransferDetails) prove by reduce;
  use lAbTransferOkayPREAbWorld;
  use lAbTransferOkayPRENDResEquiv;
  use lAbTransferOkayPREAuthentic;
  use lAbTransferOkayPREFromOPlusEquiv;
  use lAbTransferOkayPREToOPlusEquiv;
  rearrange;
  simplify;
  apply muValue2;
  prove by rewrite;
  instantiate balance == (abAuthPurse from?).balance,
    lost == (abAuthPurse from?).lost,
    balance__0 == 1, lost__0 == 1;
  instantiate balance__3 == (abAuthPurse to?).balance,
    lost__3 == (abAuthPurse to?).lost,
    balance__4 == 1, lost__4 == 1;
  invoke  $\Delta$  AbPurse;
  invoke AbPurse;
  rearrange;
  rewrite;
  use rAbPurseBalanceTypeOpen[name := from?, purse := abAuthPurse];
  use rAbPurseBalanceTypeOpen[name := to?, purse := abAuthPurse];
  invoke TransferDetails;
  apply inNat;
  rearrange;
  invoke Authentic;
  invoke SufficientFundsProperty;
  rewrite;

```

■

```

proof[tAbTransferLostTDPRE]
  instantiate a! == aNullOut,
    abAuthPurse' == abAuthPurse  $\oplus$  {(from?,
       $\theta$  AbPurse'[balance' := (abAuthPurse from?).balance - value?,
      lost' := (abAuthPurse from?).lost + value?])};
  with disabled (AbWorldSecureOp, TransferDetails, Authentic,
    SufficientFundsProperty) prove by reduce;
  use lAbTransferOkayPREAuthPurseMaxType;
  rearrange;
  simplify;
  with disabled (AbWorld, Authentic, SufficientFundsProperty,
    TransferDetails) prove by reduce;
  As this theorem is simpler, and we know

  the path to follow already from OkayTD,

  hence we tackle case by directly.

  case: Theta equivalence to function application

  instantiate
    balance == (abAuthPurse from?).balance,
    lost == (abAuthPurse from?).lost,
    balance__0 == 1, lost__0 == 1;
  invoke  $\Delta$  AbPurse;
  invoke AbPurse;
  use rAbPurseBalanceTypeOpen[name := from?, purse := abAuthPurse];
  use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
  apply inNat;
  invoke TransferDetails;
  invoke SufficientFundsProperty;
  prove by rewrite;
  case: Authentic from purse

  invoke Authentic;
  invoke AbWorld;
  invoke ( $- \leftrightarrow -$ );
  prove by rewrite;
  case: NDRes equivalence

  apply extensionality to predicate ( $\{ \text{from?} \} \cup \{ \text{to?} \}$ )
     $\Leftarrow$  (abAuthPurse  $\oplus$  {(from?,
       $\theta$  AbPurse[balance :=  $-1 * \text{value?}$ 
      + (abAuthPurse from?).balance,
      lost := value? + (abAuthPurse from?).lost])})
    = ( $\{ \text{from?} \} \cup \{ \text{to?} \}) \Leftarrow \text{abAuthPurse}$ ;
  use rAbPurseLostTypeOpen[name := to?, purse := abAuthPurse];
  use rAbPurseLostTypeOpen[name := from?, purse := abAuthPurse];
  apply ndresDef;
  apply oplusDef;
  apply inNat;
  rearrange;
  simplify;
  invoke ( $- \leftrightarrow -$ );
  prove by rewrite;
  case: final type checking

```

invoke AbPurse;
prove by rewrite;

■

I.2 Totality of between model for purses

See Section 8.3.

proof[*tIncreasePurseOkayIsTotal*]
prove by reduce;
instantiate nextSeqNo' == nextSeqNo, nextSeqNo__0 == nextSeqNo;
prove by rewrite;
cases;
prove by rewrite;
next;
prove by rewrite;
next;
prove by rewrite;
next;
prove by rewrite;
next;
prove by rewrite;
next;
prove by rewrite;
next;
prove by rewrite;
next;
prove by rewrite;
next;
prove by rewrite;
next;
prove by rewrite;
next;

■

```

proof[tAbortPurseOkayIsTotal]
  invoke AbortPurseOkay;
  invoke BetwPurseOpSig;
  invoke  $\Xi$  ConPurseAbort;
  invoke LogIfNecessary;
  invoke  $\Delta$  ConPurse;
  invoke ConPurse;
  rearrange;
  rewrite;
  rearrange;
  rewrite;
  split  $\neg$  status = epv  $\wedge$   $\neg$  status = epa;
  simplify;
  rearrange;
  cases;
  instantiate m! == m?, pdAuth' == pdAuth, nextSeqNo' == nextSeqNo;
  prove by rewrite;
  invoke ConPurseAbort;
  instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
  invoke ConPurse;
  prove by rewrite;
  cases;
  instantiate pd__0 == pd;
  prove by rewrite;
  next;
  prove by rewrite;
  next;
  prove by rewrite;
  next;
  prove by rewrite;
  split status = epv;
  simplify;
  cases;
  instantiate m! == m?, pdAuth' == pdAuth, nextSeqNo' == nextSeqNo;
  prenex;
  rewrite;
  split pd  $\in$  exLog;
  rearrange;
  simplify;
  cases;
  instantiate pd__0 == pd;
  simplify;
  invoke ConPurseAbort;
  instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
  invoke ConPurse;
  rewrite;
  next;
  rewrite;
  invoke ConPurseAbort;
  instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
  invoke ConPurse;
  rewrite;
  next;
  simplify;

```



```

instantiate m! == m?, pdAuth' == pdAuth, nextSeqNo' == nextSeqNo;
prenex;
rewrite;
split pd ∈ exLog;
rearrange;
simplify;
cases;
instantiate pd__0 == pd;
simplify;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;
rewrite;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;

```

■

I.3 Precondition of between model purses (Chapter 4)

See Section 8.4.

```

proof[tReqPurseOkayPRE]
  invoke ReqPurseOkay;
  invoke AuthenticReqMessage;
  invoke BetwPurseOpSig;
  invoke  $\Xi$  ConPurseReq;
  invoke  $\Delta$  ConPurse;
  invoke ConPurse;
  rearrange;
  rewrite;
  invoke ConPurseReq;
  invoke ConPurse;
  instantiate balance__0 == balance, status__0 == status;
  rewrite;

```

■

proof[*tValPurseOkayPRE*]
invoke ValPurseOkay;
invoke AuthenticValMessage;
invoke BetwPurseOpSig;
invoke Ξ ConPurseVal;
invoke Δ ConPurse;
invoke ConPurse;
rearrange;
rewrite;
invoke ConPurseVal;
invoke ConPurse;
instantiate balance__0 == balance, status__0 == status;
rewrite;
apply PayDetails\$member;
prove by reduce;

■

proof[*tAckPurseOkayPRE*]
invoke AckPurseOkay;
invoke AuthenticAckMessage;
invoke BetwPurseOpSig;
invoke Ξ ConPurseAck;
invoke Δ ConPurse;
invoke ConPurse;
rearrange;
rewrite;
instantiate pdAuth' == pdAuth;
rewrite;
invoke ConPurseAck;
invoke ConPurse;
instantiate status__0 == status, pdAuth__0 == pdAuth;
rewrite;

■

proof[*tReadExceptionLogPurseEafromOkayPRE*]
invoke ReadExceptionLogPurseEafromOkay;
invoke BetwPurseOpSig;
invoke Ξ ConPurse;
rewrite;
instantiate m! == $\perp$;
invoke ConPurse;
rewrite;

■

proof[*tClearExceptionLogPurseEafromOkayPRE*]
 invoke *ClearExceptionLogPurseEafromOkay*;
 invoke *BetwPurseOpSig*;
 invoke Ξ *ConPurseClear*;
 invoke Δ *ConPurse*;
 invoke *ConPurse*;
 rearrange;
 rewrite;
 invoke *ConPurseClear*;
 invoke *ConPurse*;
 instantiate *exLog__0* == *exLog*;
 rewrite;

■

proof[*tReadExceptionLogEafromOkayPRE*]
I tried reusing tAbortPurseOkayPRE

but it did not help much.

invoke *ReadExceptionLogPurseOkay*;
 invoke *BetwPurseOpSig*;
 invoke *ReadExceptionLogPurseEafromOkay*;
 invoke *AbortPurseOkay*;
 invoke *LogIfNecessary*;
 invoke Ξ *ConPurse*;
 invoke Ξ *ConPurseAbort*;
 invoke Δ *ConPurse*;
 invoke *ConPurse*;
 rearrange;
 rewrite;
 rearrange;
 rewrite;
 split $\neg$ *status* = *epv* $\wedge$ $\neg$ *status* = *epa*;
 simplify;
 rearrange;
 cases;
 instantiate *m!* == $\perp$, *pdAuth'* == *pdAuth*, *nextSeqNo'* == *nextSeqNo*;
 prove by rewrite;
 invoke *ConPurseAbort*;
 instantiate
nextSeqNo__0 == *nextSeqNo*, *exLog__0* == *exLog*, *pdAuth__0* == *pdAuth*,
status__0 == *status*;
 invoke *ConPurse*;
 prove by rewrite;
 cases;
 instantiate *pd__0* == *pd*;
 prove by rewrite;
 next;
 prove by rewrite;
 next;
 prove by rewrite;
 next;
 prove by rewrite;
 next;
 split *status* = *epv*;
 simplify;

```

cases;
instantiate m! ==  $\perp$ , pdAuth' == pdAuth, nextSeqNo' == nextSeqNo;
prenex;
rewrite;
split pd  $\in$  exLog;
simplify;
cases;
instantiate pd__0 == pd;
simplify;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;
rewrite;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;
simplify;
instantiate m! ==  $\perp$ , pdAuth' == pdAuth, nextSeqNo' == nextSeqNo;
prenex;
rewrite;
split pd  $\in$  exLog;
rearrange;
simplify;
cases;
instantiate pd__0 == pd;
simplify;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;
rewrite;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;

```

■

```

proof[tClearExceptionLogEafromOkayPRE]
  invoke ClearExceptionLogPurseOkay;
  invoke BetwPurseOpSig;
  invoke ClearExceptionLogPurseEafromOkay;
  invoke AbortPurseOkay;
  invoke LogIfNecessary;
  invoke  $\Xi$  ConPurseAbort;
  invoke  $\Xi$  ConPurseClear;
  invoke  $\Delta$  ConPurse;
  invoke ConPurse;
  rearrange;
  rewrite;
  rearrange;
  rewrite;
  invoke ConPurseClear;
  invoke ConPurse;
  instantiate exLog__1 == exLog;
  rearrange;
  rewrite;
  split  $\neg$  status = epv  $\wedge$   $\neg$  status = epa;
  simplify;
  rearrange;
  cases;
  instantiate pdAuth' == pdAuth, nextSeqNo' == nextSeqNo;
  prove by rewrite;
  cases;
  invoke ConPurseAbort;
  instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
  invoke ConPurse;
  prove by rewrite;
  instantiate pd__0 == pd;
  prove by rewrite;
  next;
  prove by rewrite;
  next;
  prove by rewrite;
  next;
  split status = epv;
  simplify;
  cases;
  instantiate pdAuth' == pdAuth, nextSeqNo' == nextSeqNo;
  prenex;
  rewrite;
  split pd  $\in$  exLog;
  simplify;
  cases;
  instantiate pd__0 == pd;
  simplify;
  invoke ConPurseAbort;
  instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
  invoke ConPurse;
  rewrite;
  next;

```

```

rewrite;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;
simplify;
instantiate pdAuth' == pdAuth, nextSeqNo' == nextSeqNo;
prenex;
rewrite;
split pd ∈ exLog;
rearrange;
simplify;
cases;
instantiate pd__0 == pd;
simplify;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;
rewrite;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
rewrite;
next;

```

■

Luckily, the few available **Z/Eves** rules includes the necessary information to discharge the proof of *rMuPayDetailsValue* establishing the relationship between these particular μ and θ expressions. For this we have used rule *muValue1* (**Z/Eves** toolkit p.8). The tricky bit is to define generic actual S as the set m involved in the μ expressions. In most **Z/Eves** lemmas (99% of them), you should always pass the maximal type (carrier set) as the generic actual. Otherwise, the typechecker will include the given generic actual around all related expressions, which becomes a pain for automation. Fortunately, the equality above does not involve any expression, such as $\cup$, or application, such as $f x$, hence the use of non-maximal generic actuals does not incur any proof burden. This is the second time in our experience with the **Z/Eves** toolkit where a non-maximal generic actual is precisely what you need.

```

proof[lStartFromMuPayDetailsEquivalence]
  with enabled (CounterPartyDetails$member) prove by reduce;
  apply muValue1 ;
  prove by reduce;

```

■

```

proof[lStartToMuPayDetailsEquivalence]
  with enabled (CounterPartyDetails$member) prove by reduce;
  apply muValue1 ;
  prove by reduce;

```

■

```

proof[tStartFromPurseEafromOkayPRE]
  invoke StartFromPurseEafromOkay;
  invoke BetwPurseStartOpSig;
  invoke  $\Xi$  ConPurseStart;
  invoke BetwPurseOpSig;
  invoke ValidStartFrom;
  invoke  $\Delta$  ConPurse;
  invoke ConPurse;
  rearrange;
  rewrite;
  instantiate nextSeqNo' == nextSeqNo + 1;
  rewrite;
  invoke PayDetails;
  invoke TransferDetails;
  rewrite;
  invoke ConPurseStart;
  invoke ConPurse;
  instantiate nextSeqNo__0 == nextSeqNo,
    pdAuth__0 == pdAuth, status__0 == status;
  rearrange;
  rewrite;

```

■

```

proof[tStartToPurseEafromOkayPRE]
  invoke StartToPurseEafromOkay;
  invoke BetwPurseStartOpSig;
  invoke  $\Xi$  ConPurseStart;
  invoke BetwPurseOpSig;
  invoke ValidStartTo;
  invoke  $\Delta$  ConPurse;
  invoke ConPurse;
  rearrange;
  rewrite;
  rearrange;
  instantiate nextSeqNo' == nextSeqNo + 1;
  rewrite;
  invoke PayDetails;
  invoke TransferDetails;
  rewrite;
  invoke ConPurseStart;
  invoke ConPurse;
  rewrite;
  instantiate nextSeqNo__0 == nextSeqNo,
    pdAuth__0 == pdAuth, status__0 == status;
  rewrite;

```

■

```

proof[tStartFromEafromOkayPRE]
  invoke StartFromPurseOkay;
  invoke StartFromPurseEafromOkay;
  invoke BetwPurseStartOpSig;
  invoke AbortPurseOkay;
  invoke LogIfNecessary;
  invoke  $\Xi$  ConPurseAbort;
  invoke  $\Xi$  ConPurseStart;
  invoke BetwPurseOpSig;
  invoke ValidStartFrom;
  invoke  $\Delta$  ConPurse;
  rearrange;
  rewrite;
  rearrange;
  rewrite;
  invoke ConPurse;
  rearrange;
  rewrite;
  invoke PayDetails;
  invoke TransferDetails;
  equality substitute startFrom  $\sim m?$ ;
  rearrange;
  rewrite;
  split  $\neg \text{status} = \text{epv} \wedge \neg \text{status} = \text{epa}$ ;
  simplify;
  cases;
  rearrange;
  prove by rewrite;
  instantiate nextSeqNo' == nextSeqNo + 1, nextSeqNo__0 == nextSeqNo,
    pdAuth__0 == pdAuth;
  prove by rewrite;
  cases;
  invoke ConPurseAbort;
  instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
  invoke ConPurse;
  prove by rewrite;
  instantiate pd__0 == pd;
  prove by rewrite;
  next;
  invoke ConPurseStart;
  instantiate nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth,
    status__0 == status;
  invoke ConPurse;
  prove by rewrite;
  instantiate pd__0 == pd;
  prove by rewrite;
  next;
  split status = epv;
  rewrite;
  cases;
  instantiate nextSeqNo' == nextSeqNo + 1, nextSeqNo__0 == nextSeqNo,
    pdAuth__0 == pdAuth;
  rewrite;
  prove by rewrite;

```



```

cases;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
prove by rewrite;
instantiate pd__0 == pd;
prove by rewrite;
next;
invoke ConPurseStart;
invoke ConPurse;
rewrite;
instantiate
nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth, status__0 == status;
rewrite;
prenex;
rewrite;
split pd ∈ exLog;
simplify;
cases;
instantiate pd__0 == pd;
rewrite;
next;
prove by rewrite;
next;
split pd ∈ exLog;
cases;
instantiate pd__0 == pd;
prove by rewrite;
next;
prove by rewrite;
next;
simplify;
instantiate
nextSeqNo' == nextSeqNo + 1, nextSeqNo__0 == nextSeqNo,
pdAuth__0 == pdAuth;
rewrite;
prove by rewrite;
cases;
invoke ConPurseAbort;
instantiate
nextSeqNo__0 == nextSeqNo, exLog__0 == exLog, pdAuth__0 == pdAuth,
status__0 == status;
invoke ConPurse;
prove by rewrite;
instantiate pd__0 == pd;
prove by rewrite;
next;
invoke ConPurseStart;
invoke ConPurse;
rewrite;
instantiate
nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth, status__0 == status;
rewrite;
prenex;
rewrite;

```

```

split pd ∈ exLog;
simplify;
cases;
instantiate pd__0 == pd;
rewrite;
next;
prove by rewrite;
next;
split pd ∈ exLog;
cases;
instantiate pd__0 == pd;
prove by rewrite;
next;
prove by rewrite;
next;

```

■

proof[*rReqStartToPayDetailsInMessage*]
 prove by reduce;

■

proof[*tStartToEafromOkayPRE*]
 invoke *StartToPurseOkay*;
 invoke *StartToPurseEafromOkay*;
 invoke *BetwPurseStartOpSig*;
 invoke *AbortPurseOkay*;
 invoke *LogIfNecessary*;
 invoke Ξ *ConPurseAbort*;
 invoke Ξ *ConPurseStart*;
 invoke *BetwPurseOpSig*;
 invoke *ValidStartTo*;
 invoke Δ *ConPurse*;
 rearrange;
 rewrite;
 rearrange;
 rewrite;
 invoke *ConPurse*;
 rearrange;
 rewrite;
 invoke *PayDetails*;
 invoke *TransferDetails*;
 equality substitute *startTo* $\sim m?$;
 rearrange;
 rewrite;
 split $\neg$ *status* = *epv* $\wedge$ $\neg$ *status* = *epa*;
 simplify;
 cases;
 rearrange;
 rewrite;
 instantiate *nextSeqNo'* == *nextSeqNo* + 1, *nextSeqNo__0* == *nextSeqNo*,
 pdAuth__0 == *pdAuth*;
 rewrite;

```

cases;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
prove by rewrite;
instantiate pd__0 == pd;
prove by rewrite;
next;
invoke ConPurseStart;
instantiate nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth,
  status__0 == status;
invoke ConPurse;
prove by rewrite;
instantiate pd__0 == pd;
prove by rewrite;
next;
rearrange;
rewrite;
split status = epv;
rewrite;
cases;
instantiate nextSeqNo' == nextSeqNo + 1, nextSeqNo__0 == nextSeqNo,
  pdAuth__0 == pdAuth;
rewrite;
prove by rewrite;
cases;
invoke ConPurseAbort;
instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
  pdAuth__0 == pdAuth, status__0 == status;
invoke ConPurse;
prove by rewrite;
instantiate pd__0 == pd;
prove by rewrite;
next;
invoke ConPurseStart;
invoke ConPurse;
rewrite;
instantiate nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth,
  status__0 == status;
rewrite;
prenex;
rewrite;
split pd ∈ exLog;
simplify;
cases;
instantiate pd__0 == pd;
rewrite;
next;
prove by rewrite;
next;
split pd ∈ exLog;
cases;
instantiate pd__0 == pd;
prove by rewrite;
next;

```

```

    prove by rewrite;
    next;
    simplify;
    instantiate nextSeqNo' == nextSeqNo + 1,
      nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth;
    rewrite;
    prove by rewrite;
    cases;
    invoke ConPurseAbort;
    instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
      pdAuth__0 == pdAuth, status__0 == status;
    invoke ConPurse;
    prove by rewrite;
    instantiate pd__0 == pd;
    prove by rewrite;
    next;
    invoke ConPurseStart;
    invoke ConPurse;
    rewrite;
    instantiate nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth,
      status__0 == status;
    rewrite;
    prenex;
    rewrite;
    split pd ∈ exLog;
    simplify;
    cases;
    instantiate pd__0 == pd;
    rewrite;
    next;
    prove by rewrite;
    next;
    split pd ∈ exLog;
    cases;
    instantiate pd__0 == pd;
    prove by rewrite;
    next;
    prove by rewrite;
    next;

```

■

I.4 $\mathcal{B}$ consistency proofs

See Section 8.5.

```

proof[tExistsInitialBetweenWorld]
  prove by reduce;
  instantiate
    ether'
    == {readExceptionLog,  $\perp$ }
        $\cup \cup$ 
       { cpd : CounterPartyDetails • {startFrom cpd, startTo cpd} };
  apply inPower;
  prove by reduce;
  instantiate
    conAuthPurse' == {}, archive' == {}, allLogs' == {},
    authenticFrom' == {}, authenticTo' == {}, fromLogged' == {},
    toLogged' == {}, toInEpr' == {}, toInEapayee' == {},
    fromInEpr' == {}, fromInEpa' == {}, definitelyLost' == {},
    maybeLost' == {};
  prove by reduce;
  apply dLogbook ;
  apply extensionality;
  prove by rewrite;

```

■

```

proof[rPFunElement]
  with enabled (inDom) prove by rewrite;
  use pairInFunction[X, Y][f := f, x := x, y := y__0];
  prove by rewrite;

```

■

```

proof[rPFunSubsetOplusRel]
  Expose at once:

  1-proof goals from =

  2-definition of override

  3-definition of dom

  with enabled (extensionality, oplusDef, inDom) prove by rewrite;
  Special trick to avoid disjunction

  split y ∈ f;
  simplify;
  Special trick to avoid negation

  split  $\neg$  (y.1, y__0) ∈ g;
  simplify;
  rearrange;
  Exposing the set types of f and g

  invoke ( $\neg \leftrightarrow$ );
  invoke ( $\neg \leftrightarrow$ );
  prove by rewrite;
  Expose at once all type information about f and g

  with enabled (inPower, inCross2) prove by rewrite;
  Type of element from f

```

```

instantiate e == y;
  Type of element from g

instantiate e__0 == (y.1, y__0);
  Elements of g are elements of f

instantiate e__1 == (y.1, y__0);
  f is functional

instantiate x__3 == y.1, y1 == y__0, y2 == y.2;
  g is functional

instantiate x__4 == y.1, y1__0 == y__0, y2__0 == y.2;
prove by rewrite;

```

■

```

proof[lPFunSubsetOplusUnitRel]
  with enabled (inDom, dresDef, rPFunSubsetOplusRel, rPFunElement)
    prove by rewrite;
  apply oplusDef;
  prove by rewrite;
  cases;
    Case where the type is correct

    apply extensionality to predicate f =
      { p__0 : f | p__0.1 ∈ {x} } ∪ [X × Y]
      dom [X, Y] { p__1 : f | p__1.1 ∈ {x} } ≪ [X, Y] f;
    with enabled (inDom) prove by rewrite;
  next;
    Case where the type is contradictory

    invoke (− ↔ −);
    prove by rewrite;
  next;

```

■

```

proof[PhiBOpSig$domainCheck]
  prove by rewrite;

```

■

```

proof[lPromotedAuxWorldType]
  use lPFunSubsetOplusUnitRel[NAME, ⟨ balance : ℤ; exLog :
    ℙ ⟨ from : NAME; fromSeqNo : ℤ; to : NAME; toSeqNo :
    ℤ; value : ℤ ⟩; name : NAME; nextSeqNo : ℤ; pdAuth :
    ⟨ from : NAME; fromSeqNo : ℤ; to : NAME; toSeqNo :
    ℤ; value : ℤ ⟩; status : STATUS ⟩]
  [f := conAuthPurse, x := name?, y := θ ConPurse];
  rearrange;
  invoke AuxWorld;
  invoke ConWorld;
  invoke ConPurse;
  rewrite;

```

■

```

proof[lPromotedBetweenWorldType]
  use lPromotedAuxWorldType;
  use lPFunSubsetOplusUnitRel[NAME,  $\langle \text{balance} : \mathbb{Z}; \text{exLog} : \mathbb{P} \langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle; \text{name} : \text{NAME}; \text{nextSeqNo} : \mathbb{Z}; \text{pdAuth} : \langle \text{from} : \text{NAME}; \text{fromSeqNo} : \mathbb{Z}; \text{to} : \text{NAME}; \text{toSeqNo} : \mathbb{Z}; \text{value} : \mathbb{Z} \rangle; \text{status} : \text{STATUS} \rangle$ ];
  [f := conAuthPurse, x := name?, y :=  $\theta$  ConPurse];
  rearrange;
  invoke PhiBOPSig;
  invoke BetweenOpSig;
  rewrite;
  invoke BetweenWorld;
  cases;
  invoke BetweenWorldB1;
  rewrite;
  next;
  invoke BetweenWorldB2;
  invoke BetweenWorldB1;
  rewrite;
  next;
  invoke BetweenWorldB3;
  invoke BetweenWorldNEW1;
  invoke BetweenWorldNEW2;
  rewrite;
  next;
  invoke BetweenWorldB4;
  invoke BetweenWorldNEW3;
  invoke BetweenWorldNEW4;
  rewrite;
  next;
  invoke BetweenWorldB5;
  rewrite;
  next;
  invoke BetweenWorldB6;
  rewrite;
  next;
  invoke BetweenWorldB7;
  rewrite;
  next;
  invoke BetweenWorldB8;
  rewrite;
  next;
  invoke BetweenWorldB9;
  rewrite;
  prenex;
  instantiate pd__0 == pd;
  with enabled (disjointCat, disjointDef) prove by rewrite;
  apply extensionality to predicate
    ether  $\cap (\{\text{val } \text{pd}\} \cup \{\text{ack } \text{pd}\}) = \{\}$ ;
  apply extensionality to predicate
    (ether  $\cup \{m!\}$ )  $\cap (\{\text{val } \text{pd}\} \cup \{\text{ack } \text{pd}\}) = \{\}$ ;
  prenex;
  rewrite;
  rewrite;
  instantiate x__0 == x;
  rearrange;

```

```

rewrite;
next;
invoke BetweenWorldB10;
rewrite;
next;
invoke BetweenWorldB11;
rewrite;
next;
invoke BetweenWorldB12;
rewrite;
next;
invoke BetweenWorldB13;
rewrite;
next;
invoke BetweenWorldB14;
rewrite;
prenex;
instantiate pd__0 == pd;
rearrange;
rewrite;
next;
invoke BetweenWorldB15;
rewrite;
next;
invoke BetweenWorldB16;
rewrite;
next;

```

■

Firstly, we expand the relevant schemas to prepare for instantiation. Next, we instantiate the variables of the local and global states. For $status'$, $exLog'$, $pdAuth'$, $nextSeqNo'$, $name$ we just use the values from the invariant from $ConPurse$. For $balance'$ any natural number would do, but we just have chosen the one from $ConPurse$ as well. For $m!$, we need a message from $ether$, which in here we can pick $m?$ because $m? \in ether$. After that, we discharge each property of between world.

proof[$tBetwPhiBOPRE$]

Gather lemma information

```

use lPromotedBetweenWorldType[m! := m?];
use lPFunSubsetOplusUnitRel[NAME,  $\Downarrow$  balance :  $\mathbb{Z}$ ; exLog :
 $\mathbb{P} \Downarrow$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :
 $\mathbb{Z}$ ; value :  $\mathbb{Z} \Downarrow$ ; name : NAME; nextSeqNo :  $\mathbb{Z}$ ; pdAuth :
 $\Downarrow$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :
 $\mathbb{Z}$ ; value :  $\mathbb{Z} \Downarrow$ ; status : STATUS  $\Downarrow$ ]
[f := conAuthPurse, x := name?, y :=  $\theta$  ConPurse];

```

Expand schemas in the goal

```

invoke PhiBOP;
invoke  $\Delta$  BetweenWorld;
invoke  $\Delta$  ConPurse;

```

Expand schema from the invariant

```

invoke PhiBOPSig;
Invoke earlier to expose ConPurse' variables

```



```

invoke ConPurse;
rearrange;
rewrite;
  Properly build up the list of variables

  existentially quantified

rearrange;
simplify;
instantiate
  Instantiate the variables from

  the local state on ConPurse

  balance' == balance, exLog' == exLog, name' == name,
  nextSeqNo' == nextSeqNo, pdAuth' == pdAuth, status' == status,
  Instantiate the variables from

  the global state on Betweenworld

  allLogs' == allLogs, authenticFrom' == authenticFrom,
  authenticTo' == authenticTo, definitelyLost' == definitelyLost,
  fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
  fromLogged' == fromLogged, maybeLost' == maybeLost,
  toInEapayee' == toInEapayee, toInEpv' == toInEpv,
  toLogged' == toLogged, m! == m?;
rewrite;
  Deal with each case for status

cases;
rearrange;
rewrite;
next;
rearrange;
rewrite;
next;
rearrange;
rewrite;
next;
rearrange;
rewrite;
next;
rearrange;
rewrite;
next;
rearrange;
rewrite;
next;
rearrange;
rewrite;
next;

```

■

Here *prove by reduce* is not the best choice as it does too much (too slowly). In general, in proofs involving (nested) schema inclusions, it is usually not a good idea to use *prove by reduce*. Instead, one should use specific instantiations and should pick the right schema to *invoke*.

```

proof[tBetwIgnoreIsTotal]
  invoke Ignore;
  invoke  $\Xi$  BetweenWorld;
  rewrite;

```

■

In the next four proofs we take advantage on the fact the promoted operations are disjoined with *Ignore*, hence their proof follows directly as we have already proved pre *Ignore*. The instantiation take into account that the two important facts about *Ignore*, which are $\exists \text{BetweenWorld} \wedge m! = \perp$.

proof[*tBetwIncreaseIsTotal*]

```

instantiate
  allLogs' == allLogs, archive' == archive,
  authenticFrom' == authenticFrom, authenticTo' == authenticTo,
  conAuthPurse' == conAuthPurse, definitelyLost' == definitelyLost,
  fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
  fromLogged' == fromLogged, maybeLost' == maybeLost,
  toInEapayee' == toInEapayee, toInEpv' == toInEpv,
  toLogged' == toLogged, m! ==  $\perp$ , ether' == ether;
rewrite;
invoke Increase;
invoke Ignore;
invoke  $\exists \text{BetweenWorld}$ ;
simplify;

```

■

proof[*tBetwAbortIsTotal*]

```

instantiate
  allLogs' == allLogs, archive' == archive,
  authenticFrom' == authenticFrom, authenticTo' == authenticTo,
  conAuthPurse' == conAuthPurse, definitelyLost' == definitelyLost,
  fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
  fromLogged' == fromLogged, maybeLost' == maybeLost,
  toInEapayee' == toInEapayee, toInEpv' == toInEpv,
  toLogged' == toLogged, m! ==  $\perp$ , ether' == ether;
rewrite;
invoke Abort;
invoke Ignore;
invoke  $\exists \text{BetweenWorld}$ ;
simplify;

```

■

Again, for these promoted operations, the precondition calculation is trivial because of the disjunction with *Ignore*.

proof[*tBetwStartFromIsTotal*]

```

instantiate
  allLogs' == allLogs, archive' == archive,
  authenticFrom' == authenticFrom, authenticTo' == authenticTo,
  conAuthPurse' == conAuthPurse, definitelyLost' == definitelyLost,
  fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
  fromLogged' == fromLogged, maybeLost' == maybeLost,
  toInEapayee' == toInEapayee, toInEpv' == toInEpv,
  toLogged' == toLogged, m! ==  $\perp$ , ether' == ether;
rewrite;
invoke StartFrom;
invoke Ignore;
invoke  $\exists \text{BetweenWorld}$ ;
simplify;

```

■

proof[*tBetwStartToIsTotal*]

instantiate

allLogs' == *allLogs*, *archive'* == *archive*,
authenticFrom' == *authenticFrom*, *authenticTo'* == *authenticTo*,
conAuthPurse' == *conAuthPurse*, *definitelyLost'* == *definitelyLost*,
fromInEpa' == *fromInEpa*, *fromInEpr'* == *fromInEpr*,
fromLogged' == *fromLogged*, *maybeLost'* == *maybeLost*,
toInEapayee' == *toInEapayee*, *toInEpv'* == *toInEpv*,
toLogged' == *toLogged*, *m!* == $\perp$, *ether'* == *ether*;

rewrite;

invoke StartTo;

invoke Ignore;

invoke Ξ *BetweenWorld*;

simplify;

■

proof[*tBetwReqIsTotal*]

instantiate

allLogs' == *allLogs*, *archive'* == *archive*,
authenticFrom' == *authenticFrom*, *authenticTo'* == *authenticTo*,
conAuthPurse' == *conAuthPurse*, *definitelyLost'* == *definitelyLost*,
fromInEpa' == *fromInEpa*, *fromInEpr'* == *fromInEpr*,
fromLogged' == *fromLogged*, *maybeLost'* == *maybeLost*,
toInEapayee' == *toInEapayee*, *toInEpv'* == *toInEpv*,
toLogged' == *toLogged*, *m!* == $\perp$, *ether'* == *ether*;

rewrite;

invoke Req;

invoke Ignore;

invoke Ξ *BetweenWorld*;

simplify;

■

proof[*tBetwValIsTotal*]

instantiate

allLogs' == *allLogs*, *archive'* == *archive*,
authenticFrom' == *authenticFrom*, *authenticTo'* == *authenticTo*,
conAuthPurse' == *conAuthPurse*, *definitelyLost'* == *definitelyLost*,
fromInEpa' == *fromInEpa*, *fromInEpr'* == *fromInEpr*,
fromLogged' == *fromLogged*, *maybeLost'* == *maybeLost*,
toInEapayee' == *toInEapayee*, *toInEpv'* == *toInEpv*,
toLogged' == *toLogged*, *m!* == $\perp$, *ether'* == *ether*;

rewrite;

invoke Val;

invoke Ignore;

invoke Ξ *BetweenWorld*;

simplify;

■

proof[*tBetwAckIsTotal*]

```

instantiate
  allLogs' == allLogs, archive' == archive,
  authenticFrom' == authenticFrom, authenticTo' == authenticTo,
  conAuthPurse' == conAuthPurse, definitelyLost' == definitelyLost,
  fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
  fromLogged' == fromLogged, maybeLost' == maybeLost,
  toInEapayee' == toInEapayee, toInEpv' == toInEpv,
  toLogged' == toLogged, m! ==  $\perp$ , ether' == ether;
rewrite;
invoke Ack;
invoke Ignore;
invoke  $\Xi$  BetweenWorld;
simplify;

```

■

Once more, for these promoted operations, the precondition calculation is trivial because of the disjunction with *Ignore*.

proof[*tBetwReadExceptionLogIsTotal*]

```

instantiate
  allLogs' == allLogs, archive' == archive,
  authenticFrom' == authenticFrom, authenticTo' == authenticTo,
  conAuthPurse' == conAuthPurse, definitelyLost' == definitelyLost,
  fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
  fromLogged' == fromLogged, maybeLost' == maybeLost,
  toInEapayee' == toInEapayee, toInEpv' == toInEpv,
  toLogged' == toLogged, m! ==  $\perp$ , ether' == ether;
rewrite;
invoke ReadExceptionLog;
invoke Ignore;
invoke  $\Xi$  BetweenWorld;
simplify;

```

■

proof[*tBetwClearExceptionLogIsTotal*]

```

instantiate
  allLogs' == allLogs, archive' == archive,
  authenticFrom' == authenticFrom, authenticTo' == authenticTo,
  conAuthPurse' == conAuthPurse, definitelyLost' == definitelyLost,
  fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
  fromLogged' == fromLogged, maybeLost' == maybeLost,
  toInEapayee' == toInEapayee, toInEpv' == toInEpv,
  toLogged' == toLogged, m! ==  $\perp$ , ether' == ether;
rewrite;
invoke ClearExceptionLog;
invoke Ignore;
invoke  $\Xi$  BetweenWorld;
simplify;

```

■

```

proof[tBetwAuthoriseExLogClearIsTotal]
  instantiate
    allLogs' == allLogs, archive' == archive,
    authenticFrom' == authenticFrom, authenticTo' == authenticTo,
    conAuthPurse' == conAuthPurse, definitelyLost' == definitelyLost,
    fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
    fromLogged' == fromLogged, maybeLost' == maybeLost,
    toInEapayee' == toInEapayee, toInEpv' == toInEpv,
    toLogged' == toLogged, m! ==  $\perp$ , ether' == ether;
  rewrite;
  invoke AuthoriseExLogClear;
  invoke Ignore;
  invoke  $\Xi$  BetweenWorld;
  simplify;
  ■

```

```

proof[tBetwArchiveIsTotal]
  invoke Archive;
  invoke  $\Delta$  BetweenWorld;
  invoke BetweenOpSig;
  rewrite;
  instantiate allLogs' == allLogs, archive' == archive,
    authenticFrom' == authenticFrom, authenticTo' == authenticTo,
    definitelyLost' == definitelyLost, fromInEpa' == fromInEpa,
    fromInEpr' == fromInEpr, fromLogged' == fromLogged,
    maybeLost' == maybeLost, toInEapayee' == toInEapayee,
    toInEpv' == toInEpv, toLogged' == toLogged;
  rewrite;
  ■

```

```

proof[lAuxWorldFromLoggedExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldToLoggedExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldAuthenticFromExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldAuthenticToExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldToInEprExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldToInEapayeeExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldFromInEprExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldFromInEpaExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldAllLogsExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldDefinitelyLostExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldMaybeLostExpansion]
  invoke AuxWorld;
  simplify;
  ■

```

```

proof[lAuxWorldArchiveExpansion]
  invoke AuxWorld;
  invoke ConWorld;
  apply dLogbook;
  simplify;
  ■

```

```

proof[PhiBOpBranchesSig$domainCheck]
  rewrite;
  ■

```

```

proof[tBetwIncreaseOkayPRE]
  use lPromotedBetweenWorldType[m! :=  $\perp$ ];
  invoke IncreaseOkay;
  invoke IncreasePurseOkay;
  invoke PhiBOPBranchesSig;
  invoke PhiBOPSig;
  invoke BetweenOpSig;
  invoke PhiBOP;
  invoke  $\Xi$  ConPurseIncrease;
  invoke  $\Delta$  BetweenWorld;
  invoke  $\Delta$  ConPurse;
  rearrange;
  rewrite;
  rearrange;
  rewrite;
  instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,
    authenticTo' == authenticTo, definitelyLost' == definitelyLost,
    fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
    fromLogged' == fromLogged, maybeLost' == maybeLost,
    toInEapayee' == toInEapayee, toInEpv' == toInEpv,
    toLogged' == toLogged;
  rewrite;
  instantiate balance__0 == balance, exLog__0 == exLog,
    name__0 == name, nextSeqNo__0 == nextSeqNo,
    pdAuth__0 == pdAuth, status__0 == status,
    nextSeqNo' == nextSeqNo;
  rewrite;
  invoke  $\Delta$  ConPurse;
  invoke ConPurseIncrease;
  rewrite;
  instantiate nextSeqNo__0 == nextSeqNo;
  rewrite;

```

■

```

proof[PhiBOPAbortSig$domainCheck]
  invoke PhiBOPBranchesSig;
  prove by rewrite;

```

■

```

proof[lPromotedConAuthPurseInjectivity]
  use lPFunSubsetOplusUnitRel[NAME,  $\langle \langle$  balance :  $\mathbb{Z}$ ; exLog :
     $\mathbb{P} \langle \langle$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :
       $\mathbb{Z}$ ; value :  $\mathbb{Z} \rangle \rangle$ ; name : NAME; nextSeqNo :  $\mathbb{Z}$ ; pdAuth :
         $\langle \langle$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :
           $\mathbb{Z}$ ; value :  $\mathbb{Z} \rangle \rangle$ ; status : STATUS  $\rangle \rangle$ ]
  [f := conAuthPurse, x := name?, y :=  $\theta$  ConPurse];
  prove by rewrite;

```

■

proof[*lPromotedConAuthNewPurseInjectivity*]

Expand definitions

invoke *ConWorld*;
invoke ($_ \rightsquigarrow _$);
invoke ($_ \rightsquigarrow _$);
invoke ($_ \rightsquigarrow _$);

Because oplus, avoid partially expansion too early

prove by rewrite;
invoke ($_ \leftrightarrow _$);
prove by rewrite;
cases;

Finiteness case

apply oplusDef;
prove by rewrite;
next;

Injectivity case

rearrange;
instantiate $x_1 == x$, $y1_1 == y1$, $y2_1 == y2$;

Carefully expand only the injectivity oplus expression

apply oplusDef to expression $conAuthPurse \oplus \{(name?, \theta \ ConPurse')\}$;
prove by rewrite;

Expand the crucial assumption for the injectivity

apply inRan;
prove by rewrite;

Reshape the goal to a more intuitive proof

split ($y1, x$) $\in conAuthPurse \wedge \neg y1 = name?$;
split ($y2, x$) $\in conAuthPurse \wedge \neg y2 = name?$;
rewrite;
cases;

rearrange;
split ($y1, x$) $\in conAuthPurse$;
rewrite;
instantiate $x_2 == y2$;
apply oplusDef;
prove by rewrite;

next;

rearrange;
split ($y2, x$) $\in conAuthPurse$;
rewrite;
instantiate $x_2 == y1$;
apply oplusDef;
prove by rewrite;

next;

■


```

proof[lPromotedAuxWorldAbortType]
  Expand signatures

  invoke PhiBOpAbortSig;
  invoke PhiBOpBranchesSig;
  invoke PhiBOpSig;
  invoke BetweenOpSig;
  Expose AuxWorld

  invoke BetweenWorld;
  invoke BetweenWorldB1;
  invoke AuxWorld;
  Remove trivial cases

  rewrite;
  As name? is in conAuthPurse, simplify result of domain over oplus

  apply cupSubsetLeft to expression { name? }  $\cup$  dom conAuthPurse;
  rewrite;
  authenticTo and From appear in many of the cases, early substitute them

  equality substitute authenticTo;
  equality substitute authenticFrom;
  Allow the first case to spread throughout the assumptions

  split status = epv  $\vee$  status = epa;
  simplify;
  Case 1: status = epv or epa

  cases;
  Each part in AxuWorld generates a sub-goal

  cases;
  Case 1.1: ConWorld + conAuthPurse type check

  invoke ConWorld;
  rewrite;
  prenex;
  instantiate n__0 == n;
  apply applyOverride1;
  rewrite;
  rearrange;
  rewrite;
  Now only injectivity type checking remains

  use lPromotedConAuthNewPurseInjectivity[balance' := balance,
    exLog' := exLog  $\cup$  { pdAuth }, name' := name,
    nextSeqNo' := nextSeqNo, pdAuth' := pdAuth,
    status' := eaFrom];
  rearrange;
  invoke ConWorld;
  Simplify prior to expansion of ConPurse for better results

  simplify;
  invoke ConPurse;
  prenex;
  rearrange;
  simplify;
  Now two subsub-goals: archive from ConWorld

```

and pd from ConPurse with substitutions

instantiate nld__0 == nld;
instantiate pd__0 == pd;
rewrite;

Trivial cases on update of exLog'

split pd ∈ exLog;
split name = pdAuth.from ∨ name = pdAuth.to;
rewrite;

next;

Case 1.2: by extensionality on each AuxWorld

equality, firstly at toInEpv

equality substitute toInEpv;

apply extensionality to predicate

$$\begin{aligned} & \{ \text{pd_76} : \{ \text{pd_77} : \text{PayDetails} \mid \text{pd_77.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid (\text{conAuthPurse } \text{pd_76.to}).\text{status} = \text{epv} \\ & \quad \wedge (\text{conAuthPurse } \text{pd_76.to}).\text{pdAuth} = \text{pd_76} \} = \\ & \{ \text{pd_78} : \{ \text{pd_79} : \text{PayDetails} \mid \text{pd_79.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \}, \\ & \quad \text{status} := \text{eaFrom}])\}) \text{pd_78.to}).\text{status} = \text{epv} \\ & \quad \wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \}, \\ & \quad \text{status} := \text{eaFrom}])\}) \text{pd_78.to}).\text{pdAuth} = \text{pd_78} \}; \end{aligned}$$

prenex;

rewrite;

From the information in the assumption, we just need to apply oplus

apply applyOverride;

rewrite;

next;

Case 1.3: by extensionality on toInEapayee

equality substitute toInEapayee;

apply extensionality to predicate

$$\begin{aligned} & \{ \text{pd_74} : \{ \text{pd_75} : \text{PayDetails} \mid \text{pd_75.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid (\text{conAuthPurse } \text{pd_74.to}).\text{status} = \text{eaTo} \\ & \quad \wedge (\text{conAuthPurse } \text{pd_74.to}).\text{pdAuth} = \text{pd_74} \} = \\ & \{ \text{pd_76} : \{ \text{pd_77} : \text{PayDetails} \mid \text{pd_77.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \}, \\ & \quad \text{status} := \text{eaFrom}])\}) \text{pd_76.to}).\text{status} = \text{eaTo} \\ & \quad \wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \}, \\ & \quad \text{status} := \text{eaFrom}])\}) \text{pd_76.to}).\text{pdAuth} = \text{pd_76} \}; \end{aligned}$$

prenex;

rewrite;

apply applyOverride;

rewrite;

next;

Case 1.4: by extensionality on fromInEpr

equality substitute fromInEpr;

apply extensionality to predicate

$$\{ \text{pd_74} : \{ \text{pd_75} : \text{PayDetails} \mid \text{pd_75.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_74.from}).\text{status} = \text{epr}$$

$$\wedge (\text{conAuthPurse } \text{pd_74.from}).\text{pdAuth} = \text{pd_74} \} =$$

$$\{ \text{pd_76} : \{ \text{pd_77} : \text{PayDetails} \mid \text{pd_77.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \},$$

$$\text{status} := \text{eaFrom}])) \text{pd_76.from}).\text{status} = \text{epr}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \},$$

$$\text{status} := \text{eaFrom}])) \text{pd_76.from}).\text{pdAuth} = \text{pd_76} \};$$
 prenex;
 rewrite;
 apply applyOverride;
 rewrite;
 next;

Case 1.5: by extensionality on fromInEpa

equality substitute fromInEpa;
 apply extensionality to predicate

$$\{ \text{pd_78} : \{ \text{pd_79} : \text{PayDetails} \mid \text{pd_79.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_78.from}).\text{status} = \text{epa}$$

$$\wedge (\text{conAuthPurse } \text{pd_78.from}).\text{pdAuth} = \text{pd_78} \}$$

$$= \{ \text{pd_80} : \{ \text{pd_81} : \text{PayDetails} \mid \text{pd_81.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \},$$

$$\text{status} := \text{eaFrom}])) \text{pd_80.from}).\text{status} = \text{epa}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \},$$

$$\text{status} := \text{eaFrom}])) \text{pd_80.from}).\text{pdAuth} = \text{pd_80} \};$$
 prenex;
 rewrite;
 apply applyOverride1;
 rewrite;
 next;

Case 1.5: by extensionality on allLogs

equality substitute allLogs;
 apply extensionality to predicate

$$\text{archive} \cup \{ \text{n_15} : \text{dom conAuthPurse}; \text{pd_59} : \text{PayDetails} \mid$$

$$\text{pd_59} \in (\text{conAuthPurse } \text{n_15}).\text{exLog} \} =$$

$$\text{archive} \cup \{ \text{n_16} : \text{dom conAuthPurse}; \text{pd_60} : \text{PayDetails} \mid$$

$$\text{pd_60} \in ((\text{conAuthPurse} \oplus \{(\text{name?},$$

$$\theta \text{ ConPurse}[\text{exLog} := \text{exLog} \cup \{ \text{pdAuth} \},$$

$$\text{status} := \text{eaFrom}])) \text{n_16}).\text{exLog} \};$$
 prenex;
 rewrite;
 prenex;
 cases;
 instantiate n__1 == n, pd__2 == pd;
 rearrange;
 apply applyOverride1;
 rewrite;
 next;
 instantiate n__2 == n__0, pd__3 == pd__0;
 rearrange;
 apply applyOverride1;
 rewrite;
 split $\neg n = \text{name?}$;
 simplify;
 next;

Case 2: status not = epv or epa

```

cases;
  invoke ConWorld;
  rewrite;
  prenex;
  instantiate n__0 == n;
  apply applyOverride1;
  rewrite;
  rearrange;
  rewrite;
  use lPromotedConAuthNewPurseInjectivity[balance' := balance,
    exLog' := exLog, name' := name, nextSeqNo' := nextSeqNo,
    pdAuth' := pdAuth, status' := eaFrom];
  rearrange;
  invoke ConWorld;
  simplify;
  invoke ConPurse;
  prenex;
  rearrange;
  simplify;
  instantiate nld__0 == nld;
  instantiate pd__0 == pd;
  rewrite;
next;
  equality substitute toInEpv;
  apply extensionality to predicate
  { pd__76 : { pd__77 : PayDetails | pd__77.to ∈ dom conAuthPurse }
    | (conAuthPurse pd__76.to).status = epv

    ∧ (conAuthPurse pd__76.to).pdAuth = pd__76 }
= { pd__78 : { pd__79 : PayDetails | pd__79.to ∈ dom conAuthPurse }
  | ((conAuthPurse
    ⊕ {(name?, θ ConPurse[status := eaFrom])})
    pd__78.to).status
    = epv

    ∧ ((conAuthPurse
    ⊕ {(name?, θ ConPurse[status := eaFrom])})
    pd__78.to).pdAuth
    = pd__78 } };
  prenex;
  rewrite;
  apply applyOverride;
  rewrite;
next;
  equality substitute toInEapayee;

```

apply extensionality to predicate

$$\{ \text{pd_74} : \{ \text{pd_75} : \text{PayDetails} \mid \text{pd_75.to} \in \text{dom conAuthPurse} \} \\ \mid (\text{conAuthPurse } \text{pd_74.to}).\text{status} = \text{eaTo} \\$$

$$\wedge (\text{conAuthPurse } \text{pd_74.to}).\text{pdAuth} = \text{pd_74} \}$$

$$= \{ \text{pd_76} : \{ \text{pd_77} : \text{PayDetails} \mid \text{pd_77.to} \in \text{dom conAuthPurse} \} \\ \mid ((\text{conAuthPurse} \\ \oplus \{(name?, \theta \text{ ConPurse}[status := \text{eaFrom}])\}) \\ \text{pd_76.to}).\text{status} \\ = \text{eaTo} \\$$

$$\wedge ((\text{conAuthPurse} \\ \oplus \{(name?, \theta \text{ ConPurse}[status := \text{eaFrom}])\}) \\ \text{pd_76.to}).\text{pdAuth} \\ = \text{pd_76} \};$$

prenex;
rewrite;
apply applyOverride;
rewrite;
next;
equality substitute fromInEpr;
apply extensionality to predicate

$$\{ \text{pd_74} : \{ \text{pd_75} : \text{PayDetails} \mid \text{pd_75.from} \in \text{dom conAuthPurse} \} \\ \mid (\text{conAuthPurse } \text{pd_74.from}).\text{status} = \text{epr} \\$$

$$\wedge (\text{conAuthPurse } \text{pd_74.from}).\text{pdAuth} = \text{pd_74} \}$$

$$= \{ \text{pd_76} : \{ \text{pd_77} : \text{PayDetails} \mid \text{pd_77.from} \in \text{dom conAuthPurse} \} \\ \mid ((\text{conAuthPurse} \\ \oplus \{(name?, \theta \text{ ConPurse}[status := \text{eaFrom}])\}) \\ \text{pd_76.from}).\text{status} \\ = \text{epr} \\$$

$$\wedge ((\text{conAuthPurse} \\ \oplus \{(name?, \theta \text{ ConPurse}[status := \text{eaFrom}])\}) \\ \text{pd_76.from}).\text{pdAuth} \\ = \text{pd_76} \};$$

prenex;
rewrite;
apply applyOverride;
rewrite;
next;
equality substitute fromInEpa;

```

    apply extensionality to predicate
    { pd__78 : { pd__79 : PayDetails | pd__79.from ∈ dom conAuthPurse }
      | (conAuthPurse pd__78.from).status = epa

      ∧ (conAuthPurse pd__78.from).pdAuth = pd__78 }
  = { pd__80 :
      { pd__81 : PayDetails | pd__81.from ∈ dom conAuthPurse }
      | ((conAuthPurse
          ⊕ {(name?, θ ConPurse[status := eaFrom])})
          pd__80.from).status
          = epa

          ∧ ((conAuthPurse
              ⊕ {(name?, θ ConPurse[status := eaFrom])})
              pd__80.from).pdAuth
              = pd__80 } };
  prenex;
  rewrite;
  apply applyOverride1;
  rewrite;
next;
  equality substitute allLogs;
  apply extensionality to predicate
  archive
  ∪ { n__15 : dom conAuthPurse; pd__59 : PayDetails
      | pd__59 ∈ (conAuthPurse n__15).exLog }
  = archive
  ∪ { n__16 : dom conAuthPurse; pd__60 : PayDetails
      | pd__60
        ∈ ((conAuthPurse
            ⊕ {(name?, θ ConPurse[status := eaFrom])})
            n__16).exLog };
  prenex;
  rewrite;
  prenex;
  cases;
    instantiate n__1 == n, pd__2 == pd;
    rearrange;
    apply applyOverride1;
    rewrite;
  next;
    instantiate n__2 == n__0, pd__3 == pd__0;
    rearrange;
    apply applyOverride1;
    rewrite;
    split ¬ n = name?;
    simplify;
  next;

```

■

In the next proof we could have expanded *BetweenWorld* at the beginning, hence reusing most proof commands and ending up with a considerably smaller proof. Nevertheless, if by any means some change is needed, rerunning the proof would require new insight. By leaving it very fine-grained, it is easier for further maintenance.

proof[*lPromotedBetweenWorldAbortType*]

Start by assuming AuxWorld properly typed

use lPromotedAuxWorldAbortType;
rearrange;
simplify;

Expand the signatures

invoke PhiBOpAbortSig;
invoke PhiBOpBranchesSig;
invoke PhiBOpSig;
invoke BetweenOpSig;

The proof from now on is equivalent for both cases.

We split it for the sake of clarity in case we need

to modify the proof later on.

split status = epv $\vee$ status = epa;
rewrite;
cases;
 invoke BetweenWorld;
 cases;
 invoke BetweenWorldB1;
 rewrite;
next;

For each B property, it follows directly by expanding

the appropriate AuxWorld component, providing the trivial

instantiations, as well as applying override application.

```

    invoke BetweenWorldB2;
    invoke BetweenWorldB1;
    rewrite;
    prenex;
    instantiate pd__1 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB3;
    invoke BetweenWorldNEW1;
    invoke BetweenWorldNEW2;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB4;
    invoke BetweenWorldNEW3;
    invoke BetweenWorldNEW4;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB5;
    rewrite;
    prenex;
    instantiate pd__0 == pd;
    apply applyOverride;
    prove by rewrite;
    use lAuxWorldFromLoggedExpansion;
    use lAuxWorldAuthenticFromExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB6;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldToLoggedExpansion;
    use lAuxWorldAuthenticToExpansion;
    apply applyOverride;
    prove by rewrite;
next;
    invoke BetweenWorldB7;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldFromLoggedExpansion;
    apply applyOverride;
    prove by rewrite;

```



```

    invoke BetweenWorldB9;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB10;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB11;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB12;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB13;
    rewrite;
next;
    invoke BetweenWorldB14;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB15;
    prenex;
    instantiate pds__0 == pds, name__1 == name__0;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB16;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    For the second (ELSE) status case, the proof is just the same.

    As we know the shape, we minimise some of the proof steps.

```

```

invoke BetweenWorld;
cases;
  invoke BetweenWorldB1;
  rewrite;
next;
  invoke BetweenWorldB2;
  invoke BetweenWorldB1;
  rewrite;
  prenex;
  instantiate pd__1 == pd;
  instantiate pd__0 == pd;
  use lAuxWorldAuthenticToExpansion;
  apply applyOverride;
  prove by rewrite;
next;
  invoke BetweenWorldB3;
  invoke BetweenWorldNEW1;
  invoke BetweenWorldNEW2;
  rewrite;
  prenex;
  instantiate pd__2 == pd;
  instantiate pd__0 == pd;
  instantiate pd__1 == pd;
  use lAuxWorldAuthenticFromExpansion;
  use lAuxWorldAuthenticToExpansion;
  apply applyOverride;
  prove by rewrite;
next;
  invoke BetweenWorldB4;
  invoke BetweenWorldNEW3;
  invoke BetweenWorldNEW4;
  rewrite;
  prenex;
  instantiate pd__0 == pd;
  instantiate pd__1 == pd;
  instantiate pd__2 == pd;
  use lAuxWorldAuthenticFromExpansion;
  use lAuxWorldAuthenticToExpansion;
  apply applyOverride;
  prove by rewrite;
next;
  invoke BetweenWorldB5;
  rewrite;
  prenex;
  instantiate pd__0 == pd;
  use lAuxWorldFromLoggedExpansion;
  use lAuxWorldAuthenticFromExpansion;
  apply applyOverride;
  prove by rewrite;
next;
  invoke BetweenWorldB6;
  prenex;
  instantiate pd__0 == pd;
  use lAuxWorldToLoggedExpansion;
  use lAuxWorldAuthenticToExpansion;
  apply applyOverride;
  prove by rewrite;
next;
  invoke BetweenWorldB7;
  prenex;
  instantiate pd__0 == pd;
  use lAuxWorldAuthenticFromExpansion;
  use lAuxWorldFromLoggedExpansion;
  apply applyOverride;

```

Although the next proof is now small, do not let it be misleading. It took quite some time and effort to reach it. The difficulty lies beneath the various quantified variables and how they can be blown away through the various interactions hidden beneath the schema inclusions. Finding the right granularity for reduction/expansion, as well as the appropriate place where instantiations would be suitable took around 2 days. Also, finding the right lemma for *BetweenWorld/AuxWorld* took most of this time. Fortunately, this will be reused in other proofs.

proof[*tBetwAbortOkayPRE*]

We know about the BetweenWorld instantiations type

use lPromotedBetweenWorldAbortType;

Surgically reduce

with disabled (BetweenWorld, ConPurse) prove by reduce;

Instantiate the trivial variables from ConPurseAbort

instantiate exLog__0 == exLog, nextSeqNo__0 == nextSeqNo,
pdAuth__0 == pdAuth, status__0 == status;

Instantiate remaining Delta ConPurse variables

instantiate nextSeqNo' == nextSeqNo, pdAuth' == pdAuth,
exLog' == exLog $\cup$
(if status = epv $\vee$ status = epa then { pdAuth } else {});

invoke Δ ConPurse;

rearrange;

rewrite;

Instantiate BetweenWorld, and we are done

instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,
authenticTo' == authenticTo, definitelyLost' == definitelyLost,
fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
fromLogged' == fromLogged, maybeLost' == maybeLost,
toInEapayee' == toInEapayee, toInEpr' == toInEpr,
toLogged' == toLogged;

ConPurse with altered exLog

invoke ConPurse;

prenex;

rearrange;

Handle each Abort case

split status = epv $\vee$ status = epa;

rewrite;

cases;

instantiate pd__1 == pd;

split pd $\in$ exLog;

rewrite;

next;

instantiate pd__3 == pd__0;

rewrite;

next;

■

proof[*lPromotedAuxWorldStartFromType*]

Expand signatures

```

invoke PhiBOpStartFromSig;
invoke ValidStartFrom;
invoke PhiBOpBranchesSig;
invoke PhiBOpSig;
invoke BetweenOpSig;
  Expose AuxWorld

invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
  Remove trivial cases

rewrite;
  As name? is in conAuthPurse, simplify result of domain over oplus

apply cupSubsetLeft to expression { name? }  $\cup$  dom conAuthPurse;
rewrite;
  authenticTo and From appear in many of the cases, early substitute them

equality substitute authenticTo;
equality substitute authenticFrom;
  Each part in AxuWorld generates a sub-goal

cases;
  Case 1.1: ConWorld + conAuthPurse type check

invoke ConWorld;
rewrite;
prenex;
instantiate n__0 == n;
apply applyOverride1;
rewrite;
rearrange;
rewrite;
  Now only injectivity type checking remains

use lPromotedConAuthNewPurseInjectivity[balance' := balance,
  exLog' := exLog, name' := name, nextSeqNo' := 1 + nextSeqNo,
  pdAuth' :=  $\theta$  PayDetails[name/from, nextSeqNo/fromSeqNo,
    to := (startFrom  $\sim$  m?).name,
    toSeqNo := (startFrom  $\sim$  m?).nextSeqNo,
    value := (startFrom  $\sim$  m?).value],
  status' := epr];
rearrange;
invoke ConWorld;
  Simplify prior to expansion of ConPurse

  for better results

simplify;
invoke ConPurse;
prenex;
rearrange;
simplify;
  Now two subsub-goals: archive from ConWorld

  and pd from ConPurse with substitutions

```

```

instantiate nld__0 == nld;
instantiate pd__0 == pd;
rewrite;
  Trivial cases: type check on updated pdAuth

invoke PayDetails;
invoke TransferDetails;
rewrite;
next;
  Case 1.2: by extensionality on each AuxWorld

  equality, firstly at toInEpv

equality substitute toInEpv;
apply extensionality to predicate
{ pd__77 : { pd__78 : PayDetails | pd__78.to ∈ dom conAuthPurse }
  | (conAuthPurse pd__77.to).status = epv
  ∧ (conAuthPurse pd__77.to).pdAuth = pd__77 }
= { pd__79 : { pd__80 : PayDetails | pd__80.to ∈ dom conAuthPurse }
  | ((conAuthPurse ⊕ {(name?, θ ConPurse[nextSeqNo := 1 + nextSeqNo,
  pdAuth := θ PayDetails[name/from, nextSeqNo/fromSeqNo,
  to := (startFrom ~ m?).name,
  toSeqNo := (startFrom ~ m?).nextSeqNo,
  value := (startFrom ~ m?).value],
  status := epr]})) pd__79.to).status = epv
  ∧ ((conAuthPurse ⊕ {(name?, θ ConPurse[nextSeqNo := 1 + nextSeqNo,
  pdAuth := θ PayDetails[name/from, nextSeqNo/fromSeqNo,
  to := (startFrom ~ m?).name,
  toSeqNo := (startFrom ~ m?).nextSeqNo,
  value := (startFrom ~ m?).value],
  status := epr]})) pd__79.to).pdAuth = pd__79 } };
prenex;
rewrite;
  From the information in the assumption, we just need to apply oplus

apply applyOverride;
rewrite;
next;
  Case 1.3: by extensionality on toInEapayee

```

equality substitute toInEapayee;
apply extensionality to predicate

$$\{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.to} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_75.to}).\text{status} = \text{eaTo}$$

$$\wedge (\text{conAuthPurse } \text{pd_75.to}).\text{pdAuth} = \text{pd_75} \}$$

$$= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name},$$

$$\text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}],$$

$$\text{status} := \text{epr}])) \text{pd_77.to}).\text{status}$$

$$= \text{eaTo}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name},$$

$$\text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}],$$

$$\text{status} := \text{epr}])) \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \};$$
prenex;
rewrite;
apply applyOverride;
rewrite;
next;

Case 1.4: by extensionality on fromInEpr

equality substitute fromInEpr;
apply extensionality to predicate

$$\{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_75.from}).\text{status} = \text{epr}$$

$$\wedge (\text{conAuthPurse } \text{pd_75.from}).\text{pdAuth} = \text{pd_75} \}$$

$$= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name},$$

$$\text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}],$$

$$\text{status} := \text{epr}])) \text{pd_77.from}).\text{status} = \text{epr}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name},$$

$$\text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}],$$

$$\text{status} := \text{epr}])) \text{pd_77.from}).\text{pdAuth} = \text{pd_77} \};$$
prenex;
rewrite;
apply applyOverride;
rewrite;
next;

Case 1.5: by extensionality on fromInEpa

equality substitute fromInEpa;
apply extensionality to predicate

$$\{ \text{pd_79} : \{ \text{pd_80} : \text{PayDetails} \mid \text{pd_80.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_79.from}).\text{status} = \text{epa}$$

$$\wedge (\text{conAuthPurse } \text{pd_79.from}).\text{pdAuth} = \text{pd_79} \}$$

$$= \{ \text{pd_81} : \{ \text{pd_82} : \text{PayDetails} \mid \text{pd_82.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name},$$

$$\text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}],$$

$$\text{status} := \text{epr}])) \text{pd_81.from}).\text{status} = \text{epa}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name},$$

$$\text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}],$$

$$\text{status} := \text{epr}])) \text{pd_81.from}).\text{pdAuth} = \text{pd_81} \};$$
prenex;
rewrite;
apply applyOverride1;
rewrite;
next;

Case 1.5: by extensionality on allLogs

equality substitute allLogs;
apply extensionality to predicate

$$\text{archive} \cup \{ n_15 : \text{dom conAuthPurse}; \text{pd_60} : \text{PayDetails}$$

$$\mid \text{pd_60} \in (\text{conAuthPurse } n_15).\text{exLog} \}$$

$$= \text{archive} \cup \{ n_16 : \text{dom conAuthPurse}; \text{pd_61} : \text{PayDetails}$$

$$\mid \text{pd_61} \in ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{name}/\text{from}, \text{nextSeqNo}/\text{fromSeqNo},$$

$$\text{to} := (\text{startFrom} \sim m?).\text{name},$$

$$\text{toSeqNo} := (\text{startFrom} \sim m?).\text{nextSeqNo},$$

$$\text{value} := (\text{startFrom} \sim m?).\text{value}],$$

$$\text{status} := \text{epr}])) n_16).\text{exLog} \};$$
prenex;
rewrite;
prenex;
cases;

$$\text{instantiate } n_1 == n, \text{pd_3} == \text{pd};$$

$$\text{rearrange;}$$

$$\text{apply applyOverride1;}$$

$$\text{rewrite;}$$
next;

$$\text{instantiate } n_2 == n_0, \text{pd_4} == \text{pd_0};$$

$$\text{rearrange;}$$

$$\text{apply applyOverride1;}$$

$$\text{rewrite;}$$

$$\text{split } \neg n = \text{name?};$$

$$\text{simplify;}$$
next;

■

proof[*lPromotedBetweenWorldStartFromType*]

Start by assuming AuxWorld properly typed

use lPromotedAuxWorldStartFromType;
rearrange;
simplify;

Expand the signatures

invoke PhiBOpStartFromSig;
invoke PhiBOpBranchesSig;
invoke ValidStartFrom;
invoke PhiBOpSig;
invoke BetweenOpSig;
invoke BetweenWorld;
cases;
 invoke BetweenWorldB1;
 rewrite;
next;

For each B property, it follows directly by expanding

the appropriate AuxWorld component, providing the trivial

instantiations, as well as applying override application.


```

    invoke BetweenWorldB2;
    invoke BetweenWorldB1;
    rewrite;
    prenex;
    instantiate pd__1 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB3;
    invoke BetweenWorldNEW1;
    invoke BetweenWorldNEW2;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB4;
    invoke BetweenWorldNEW3;
    invoke BetweenWorldNEW4;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB5;
    rewrite;
    prenex;
    instantiate pd__0 == pd;
    apply applyOverride;
    prove by rewrite;
    use lAuxWorldFromLoggedExpansion;
    use lAuxWorldAuthenticFromExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB6;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldToLoggedExpansion;
    use lAuxWorldAuthenticToExpansion;
    apply applyOverride;
    prove by rewrite;
next;
    invoke BetweenWorldB7;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldFromLoggedExpansion;
    apply applyOverride;
    prove by rewrite;

```

```

    invoke BetweenWorldB9;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB10;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB11;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB12;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB13;
    rewrite;
next;
    invoke BetweenWorldB14;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB15;
    prenex;
    instantiate pds__0 == pds, name__1 == name__0;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;
    invoke BetweenWorldB16;
    prenex;
    instantiate pd__0 == pd;
    apply cupSubsetRight to expression ether  $\cup \{ \perp \}$ ;
    prove by rewrite;
next;

```

■

proof[*tBetwStartFromEafromOkayPRE*]

use *lPromotedBetweenWorldStartFromType*;
 with disabled (*BetweenWorld*) prove by reduce;

ConPurse: local ConPurseStart, and Delta ConPurse

instantiate status__0 == *eaFrom*, nextSeqNo__0 == nextSeqNo,
 pdAuth__0 == pdAuth, nextSeqNo' == nextSeqNo + 1;
 rewrite;

BetweenWorld

instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,
authenticTo' == authenticTo, definitelyLost' == definitelyLost,
fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
fromLogged' == fromLogged, maybeLost' == maybeLost,
toInEapayee' == toInEapayee, toInEpr' == toInEpr,
toLogged' == toLogged;
with disabled (BetweenWorld) prove by reduce;

■

proof[*tBetwStartFromOkayPRE*]

use lPromotedBetweenWorldStartFromType;
with disabled (BetweenWorld) prove by reduce;
ConPurse: local ConPurseStart, and Delta ConPurse

instantiate pdAuth__0 == pdAuth, nextSeqNo__0 == nextSeqNo,
nextSeqNo' == nextSeqNo + 1;
rearrange;
rewrite;

BetweenWorld

instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,
authenticTo' == authenticTo, definitelyLost' == definitelyLost,
fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
fromLogged' == fromLogged, maybeLost' == maybeLost,
toInEapayee' == toInEapayee, toInEpr' == toInEpr,
toLogged' == toLogged;
invoke Δ ConPurse;
invoke ConPurse;
rewrite;

instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
pdAuth__0 == pdAuth, status__0 == status;
simplify;
instantiate nextSeqNo__1 == nextSeqNo, pdAuth__1 == pdAuth,
status__1 == status;
simplify;
invoke PayDetails;
invoke TransferDetails;
rewrite;

■

proof[*PhiBOPStartToSig\$domainCheck*]

prove by rewrite;

■

proof[*lPromotedAuxWorldStartToType*]

Expand signatures

invoke PhiBOPStartToSig;
invoke ValidStartTo;
invoke PhiBOPBranchesSig;
invoke PhiBOPSig;
invoke BetweenOpSig;

Expose AuxWorld

invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
Remove trivial cases

rewrite;
As name? is in conAuthPurse, simplify result of domain over oplus

apply cupSubsetLeft to expression { name? } $\cup$ dom conAuthPurse;
rewrite;
authenticTo and From appear in many of the cases, early substitute them

equality substitute authenticTo;
equality substitute authenticFrom;
Each part in AxuWorld generates a sub-goal

cases;
Case 1.1: ConWorld + conAuthPurse type check

invoke ConWorld;
rewrite;
prenex;
instantiate n__0 == n;
apply applyOverride1;
rewrite;
rearrange;
rewrite;
Now only injectivity type checking remains

use lPromotedConAuthNewPurseInjectivity[balance' := balance,
exLog' := exLog, name' := name, nextSeqNo' := 1 + nextSeqNo,
pdAuth' := θ PayDetails[from := (startTo $\sim$ m?).name,
fromSeqNo := (startTo $\sim$ m?).nextSeqNo, name/to,
nextSeqNo/toSeqNo, value := (startTo $\sim$ m?).value],
status' := epv];
rearrange;
invoke ConWorld;
Simplify prior to expansion of ConPurse

for better results

simplify;
invoke ConPurse;
prenex;
rearrange;
simplify;
Now two subsub-goals: archive from ConWorld

and pd from ConPurse with substitutions

instantiate nld__0 == nld;
instantiate pd__0 == pd;
rewrite;

next;
Case 1.2: by extensionality on each AuxWorld

equality, firstly at toInEpv

equality substitute toInEpr;
 apply extensionality to predicate

$$\{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_77.to}).\text{status} = \text{epv}$$

$$\wedge (\text{conAuthPurse } \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \}$$

$$= \{ \text{pd_79} : \{ \text{pd_80} : \text{PayDetails} \mid \text{pd_80.to} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$$

$$\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name/to},$$

$$\text{nextSeqNo/toSeqNo}, \text{value} := (\text{startTo} \sim m?).\text{value}],$$

$$\text{status} := \text{epv}])) \text{pd_79.to}).\text{status} = \text{epv}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$$

$$\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name/to},$$

$$\text{nextSeqNo/toSeqNo}, \text{value} := (\text{startTo} \sim m?).\text{value}],$$

$$\text{status} := \text{epv}])) \text{pd_79.to}).\text{pdAuth} = \text{pd_79} \};$$
 prenex;
 rewrite;
From the information in the assumption, we just need to apply oplus

 apply applyOverride;
 rewrite;
 next;

Case 1.3: by extensionality on toInEapayee

equality substitute toInEapayee;
Name (numbers) here are different but it does not matter because the shape is right

 apply extensionality to predicate

$$\{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_77.to}).\text{status} = \text{eaTo}$$

$$\wedge (\text{conAuthPurse } \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \}$$

$$= \{ \text{pd_79} : \{ \text{pd_80} : \text{PayDetails} \mid \text{pd_80.to} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$$

$$\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name/to},$$

$$\text{nextSeqNo/toSeqNo}, \text{value} := (\text{startTo} \sim m?).\text{value}],$$

$$\text{status} := \text{epv}])) \text{pd_79.to}).\text{status} = \text{eaTo}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$$

$$\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name/to},$$

$$\text{nextSeqNo/toSeqNo}, \text{value} := (\text{startTo} \sim m?).\text{value}],$$

$$\text{status} := \text{epv}])) \text{pd_79.to}).\text{pdAuth} = \text{pd_79} \};$$
 prenex;
 rewrite;
 apply applyOverride;
 rewrite;
 next;

Case 1.4: by extensionality on fromInEpr

equality substitute fromInEpr;
apply extensionality to predicate

$$\{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_75.from}).\text{status} = \text{epr}$$

$$\wedge (\text{conAuthPurse } \text{pd_75.from}).\text{pdAuth} = \text{pd_75} \}$$

$$= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$$

$$\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name/to},$$

$$\text{nextSeqNo/toSeqNo}, \text{value} := (\text{startTo} \sim m?).\text{value}],$$

$$\text{status} := \text{epv}])) \text{pd_77.from}).\text{status} = \text{epr}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$$

$$\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name/to},$$

$$\text{nextSeqNo/toSeqNo}, \text{value} := (\text{startTo} \sim m?).\text{value}],$$

$$\text{status} := \text{epv}])) \text{pd_77.from}).\text{pdAuth} = \text{pd_77} \};$$
prenex;
rewrite;
apply applyOverride;
rewrite;

next;

Case 1.5: by extensionality on fromInEpa

equality substitute fromInEpa;
apply extensionality to predicate

$$\{ \text{pd_79} : \{ \text{pd_80} : \text{PayDetails} \mid \text{pd_80.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_79.from}).\text{status} = \text{epa}$$

$$\wedge (\text{conAuthPurse } \text{pd_79.from}).\text{pdAuth} = \text{pd_79} \}$$

$$= \{ \text{pd_81} : \{ \text{pd_82} : \text{PayDetails} \mid \text{pd_82.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$$

$$\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name/to},$$

$$\text{nextSeqNo/toSeqNo}, \text{value} := (\text{startTo} \sim m?).\text{value}],$$

$$\text{status} := \text{epv}])) \text{pd_81.from}).\text{status} = \text{epa}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{nextSeqNo} := 1 + \text{nextSeqNo},$$

$$\text{pdAuth} := \theta \text{ PayDetails}[\text{from} := (\text{startTo} \sim m?).\text{name},$$

$$\text{fromSeqNo} := (\text{startTo} \sim m?).\text{nextSeqNo}, \text{name/to},$$

$$\text{nextSeqNo/toSeqNo}, \text{value} := (\text{startTo} \sim m?).\text{value}],$$

$$\text{status} := \text{epv}])) \text{pd_81.from}).\text{pdAuth} = \text{pd_81} \};$$
prenex;
rewrite;
apply applyOverride1;
rewrite;

next;

Case 1.5: by extensionality on allLogs

```

equality substitute allLogs;
apply extensionality to predicate
  archive  $\cup$  {  $n\_15$  : dom conAuthPurse;  $pd\_60$  : PayDetails
    |  $pd\_60 \in (conAuthPurse\ n\_15).exLog$  }
= archive  $\cup$  {  $n\_16$  : dom conAuthPurse;  $pd\_61$  : PayDetails
  |  $pd\_61 \in ((conAuthPurse \oplus \{(name?, \theta\} ConPurse[
    nextSeqNo := 1 + nextSeqNo,
    pdAuth := \theta\ PayDetails[from := (startTo \sim m?).name,
      fromSeqNo := (startTo \sim m?).nextSeqNo, name/to,
      nextSeqNo/toSeqNo, value := (startTo \sim m?).value],
    status := epv]))\ n\_16).exLog$  };
prenex;
rewrite;
prenex;
cases;
  instantiate  $n\_1 == n$ ,  $pd\_3 == pd$ ;
  rearrange;
  apply applyOverride1;
  rewrite;
next;
  instantiate  $n\_2 == n\_0$ ,  $pd\_4 == pd\_0$ ;
  rearrange;
  apply applyOverride1;
  rewrite;
  split  $\neg n = name?$ ;
  simplify;
next;

```

■

proof[lPromotedBetweenWorldStartToType]

Start by assuming AuxWorld properly typed

```

use lPromotedAuxWorldStartToType;
rearrange;
simplify;

```

Expand the signatures

```

invoke PhiBOPStartToSig;
invoke PhiBOPBranchesSig;
invoke ValidStartTo;
invoke PhiBOPSig;
invoke BetweenOpSig;
invoke BetweenWorld;
cases;

```

```

  invoke BetweenWorldB1;

```

```

prenex;

```

```

instantiate  $pd\_0 == pd$ ;

```

As req is injective and $req\ pd = req\ pd'$, then $pd = pd'$

```

use pinjApplicationsEqual[ $\langle \downarrow from : NAME; fromSeqNo : \mathbb{Z}; to :
  NAME; toSeqNo : \mathbb{Z}; value : \mathbb{Z} \downarrow, MESSAGE \rangle$ ]
[A :=  $\langle \downarrow from : NAME; fromSeqNo : \mathbb{Z}; to :
  NAME; toSeqNo : \mathbb{Z}; value : \mathbb{Z} \downarrow,$ 
  B := MESSAGE, f := req, x := pd, y := pd'];
pd' must be in authenticTo

```

```

use lAuxWorldAuthenticToExpansion;

```

pd' is either pd or the update, split earlier then

```

split req pd ∈ ether;
prove by rewrite;
next;
  For each B property, it follows directly by expanding

  the appropriate AuxWorld component, providing the trivial

  instantiations, as well as applying override application.

  invoke BetweenWorldB2;
  invoke BetweenWorldB1;
  prenex;
    As req is injective and req pd = req pd', then pd = pd'

    Use it earlier to avoid loosing pd' via equality substitution

  use pinjApplicationsEqual[⟦ from : NAME; fromSeqNo : ℤ; to :
    NAME; toSeqNo : ℤ; value : ℤ ⟧, MESSAGE]
  [A := ⟦ from : NAME; fromSeqNo : ℤ; to :
    NAME; toSeqNo : ℤ; value : ℤ ⟧,
    B := MESSAGE, f := req, x := pd, y := pd'];
  use pinjApplicationsEqual[⟦ from : NAME; fromSeqNo : ℤ; to :
    NAME; toSeqNo : ℤ; value : ℤ ⟧, MESSAGE]
  [A := ⟦ from : NAME; fromSeqNo : ℤ; to :
    NAME; toSeqNo : ℤ; value : ℤ ⟧,
    B := MESSAGE, f := req, x := pd_0, y := pd'];
  Get rid of the AuxWorld type checking case

  simplify;
  pd' must be in authenticTo

  use lAuxWorldAuthenticToExpansion;
  cases;
    Property B1 within B2

    instantiate pd_1 == pd;
    prove by rewrite;
  next;
    Property B2

```



```

    instantiate pd__1 == pd__0;
    apply applyOverride;
    prove by rewrite;
    split req pd ∈ ether;
    prove by rewrite;
    instantiate pd__1 == pd;
    rewrite;
next;
    invoke BetweenWorldB3;
    invoke BetweenWorldNEW1;
    invoke BetweenWorldNEW2;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB4;
    invoke BetweenWorldNEW3;
    invoke BetweenWorldNEW4;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB5;
    rewrite;
    prenex;
    instantiate pd__0 == pd;
    apply applyOverride;
    prove by rewrite;
    use lAuxWorldFromLoggedExpansion;
    use lAuxWorldAuthenticFromExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB6;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldToLoggedExpansion;
    use lAuxWorldAuthenticToExpansion;
    apply applyOverride;
    prove by rewrite;
next;
    invoke BetweenWorldB7;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldFromLoggedExpansion;
    apply applyOverride;
    prove by rewrite;
next;
    invoke BetweenWorldB8;
    prenex;
    instantiate pd__0 == pd;

```

proof[*tBetwStartToEafromOkayPRE*]
 use *lPromotedBetweenWorldStartToType*;
 with disabled (*BetweenWorld*) prove by reduce;
 instantiate *status__0* == *eaFrom*, *nextSeqNo__0* == *nextSeqNo*,
 pdAuth__0 == *pdAuth*, *nextSeqNo'* == *nextSeqNo* + 1;
 rewrite;
 instantiate *allLogs'* == *allLogs*, *authenticFrom'* == *authenticFrom*,
 authenticTo' == *authenticTo*, *definitelyLost'* == *definitelyLost*,
 fromInEpa' == *fromInEpa*, *fromInEpr'* == *fromInEpr*,
 fromLogged' == *fromLogged*, *maybeLost'* == *maybeLost*,
 toInEapayee' == *toInEapayee*, *toInEprv'* == *toInEprv*,
 toLogged' == *toLogged*;
 invoke Δ *ConPurse*;
 invoke *ConPurse*;
 rewrite;
 invoke *PayDetails*;
 invoke *TransferDetails*;
 rewrite;

■

proof[*tBetwStartToOkayPRE*]
 use *lPromotedBetweenWorldStartToType*;
 with disabled (*BetweenWorld*) prove by reduce;
 instantiate *pdAuth__0* == *pdAuth*, *nextSeqNo__0* == *nextSeqNo*,
 nextSeqNo' == *nextSeqNo* + 1;
 rearrange;
 rewrite;
 instantiate *allLogs'* == *allLogs*, *authenticFrom'* == *authenticFrom*,
 authenticTo' == *authenticTo*, *definitelyLost'* == *definitelyLost*,
 fromInEpa' == *fromInEpa*, *fromInEpr'* == *fromInEpr*,
 fromLogged' == *fromLogged*, *maybeLost'* == *maybeLost*,
 toInEapayee' == *toInEapayee*, *toInEprv'* == *toInEprv*,
 toLogged' == *toLogged*;
 invoke Δ *ConPurse*;
 invoke *ConPurse*;
 rewrite;
 instantiate *nextSeqNo__0* == *nextSeqNo*, *exLog__0* == *exLog*,
 pdAuth__0 == *pdAuth*, *status__0* == *status*;
 simplify;
 instantiate *nextSeqNo__1* == *nextSeqNo*, *pdAuth__1* == *pdAuth*,
 status__1 == *status*;
 simplify;
 invoke *PayDetails*;
 invoke *TransferDetails*;
 rewrite;

■

proof[*PhiBOPReqSig\$domainCheck*]
 prove by rewrite;

■

proof[*lPromotedAuxWorldReqType*]
 Expand signatures

```

invoke PhiBOPReqSig;
invoke PhiBOPBranchesSig;
invoke PhiBOPSig;
invoke BetweenOpSig;
  Expose AuxWorld

invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
  Remove trivial cases

rewrite;
  As name? is in conAuthPurse, simplify result of domain over oplus

apply cupSubsetLeft to expression { name? }  $\cup$  dom conAuthPurse;
rewrite;
  authenticTo and From appear in many of the cases, early substitute them

equality substitute authenticTo;
equality substitute authenticFrom;
  Each part in AxuWorld generates a sub-goal

cases;
  Case 1.1: ConWorld + conAuthPurse type check

  invoke ConWorld;
  rewrite;
  prenex;
  instantiate n__0 == n;
  apply applyOverride1;
  rewrite;
  rearrange;
  rewrite;
    Now injectivity type checking

    use lPromotedConAuthNewPurseInjectivity[balance' := balance - pdAuth.value,
      exLog' := exLog, name' := pdAuth.from, nextSeqNo' := nextSeqNo,
      pdAuth' := pdAuth, status' := epa];
    rearrange;
    invoke ConWorld;
    simplify;
    invoke ConPurse;
    prenex;
    rearrange;
    simplify;
    instantiate nld__0 == nld;
    instantiate pd__0 == pd;
    rearrange;
    rewrite;
  next;
    Case 1.2: by extensionality on each AuxWorld

    equality, firstly at toInEpv

```

equality substitute toInEpr;
apply extensionality to predicate

$$\{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_77.to}).\text{status} = \text{epv}$$

$$\wedge (\text{conAuthPurse } \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \}$$

$$= \{ \text{pd_79} : \{ \text{pd_80} : \text{PayDetails} \mid \text{pd_80.to} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pd_79.to}.\text{pdAuth.value}, \text{name} := \text{pd_79.to}.\text{pdAuth.from},$$

$$\text{status} := \text{epa}])\}) \text{pd_79.to}).\text{status} = \text{epv}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pd_79.to}.\text{pdAuth.value}, \text{name} := \text{pd_79.to}.\text{pdAuth.from},$$

$$\text{status} := \text{epa}])\}) \text{pd_79.to}).\text{pdAuth} = \text{pd_79} \};$$
prenex;
rewrite;
From the information in the assumption, we just need to apply oplus

apply applyOverride;
rewrite;
next;

Case 1.3: by extensionality on toInEapayee

equality substitute toInEapayee;
apply extensionality to predicate

$$\{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.to} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_75.to}).\text{status} = \text{eaTo}$$

$$\wedge (\text{conAuthPurse } \text{pd_75.to}).\text{pdAuth} = \text{pd_75} \}$$

$$= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pd_77.to}.\text{pdAuth.value}, \text{name} := \text{pd_77.to}.\text{pdAuth.from},$$

$$\text{status} := \text{epa}])\}) \text{pd_77.to}).\text{status} = \text{eaTo}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pd_77.to}.\text{pdAuth.value}, \text{name} := \text{pd_77.to}.\text{pdAuth.from},$$

$$\text{status} := \text{epa}])\}) \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \};$$
prenex;
rewrite;
apply applyOverride;
rewrite;
next;

Case 1.4: by extensionality on fromInEpr

equality substitute fromInEpr;
apply extensionality to predicate

$$\{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_75.from}).\text{status} = \text{epr}$$

$$\wedge (\text{conAuthPurse } \text{pd_75.from}).\text{pdAuth} = \text{pd_75} \}$$

$$= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pd_77.from}.\text{pdAuth.value}, \text{name} := \text{pd_77.from}.\text{pdAuth.from},$$

$$\text{status} := \text{epa}])\}) \text{pd_77.from}).\text{status} = \text{epr}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pd_77.from}.\text{pdAuth.value}, \text{name} := \text{pd_77.from}.\text{pdAuth.from},$$

$$\text{status} := \text{epa}])\}) \text{pd_77.from}).\text{pdAuth} = \text{pd_77} \};$$
prenex;
rewrite;
apply applyOverride;
rewrite;
next;

Case 1.5: by extensionality on fromInEpa

```

equality substitute fromInEpa;
apply extensionality to predicate
  { pd_79 : { pd_80 : PayDetails | pd_80.from ∈ dom conAuthPurse }
    | (conAuthPurse pd_79.from).status = epa
    ∧ (conAuthPurse pd_79.from).pdAuth = pd_79 }
= { pd_81 : { pd_82 : PayDetails | pd_82.from ∈ dom conAuthPurse }
  | ((conAuthPurse ⊕ {(name?, θ ConPurse[balance :=
    balance + 1 * pdAuth.value, name := pdAuth.from,
    status := epa]})) pd_81.from).status = epa
    ∧ ((conAuthPurse ⊕ {(name?, θ ConPurse[balance :=
    balance + 1 * pdAuth.value, name := pdAuth.from,
    status := epa]})) pd_81.from).pdAuth = pd_81 };
prenex;
rewrite;
apply applyOverride1;
rewrite;
next;

```

Case 1.5: by extensionality on allLogs

```

equality substitute allLogs;
apply extensionality to predicate
  archive ∪ { n_15 : dom conAuthPurse; pd_60 : PayDetails
    | pd_60 ∈ (conAuthPurse n_15).exLog }
= archive ∪ { n_16 : dom conAuthPurse; pd_61 : PayDetails
  | pd_61 ∈ ((conAuthPurse ⊕ {(name?, θ ConPurse[balance :=
    balance + 1 * pdAuth.value, name := pdAuth.from,
    status := epa]})) n_16).exLog };
prenex;
rewrite;
prenex;
cases;
  instantiate n_1 == n, pd_2 == pd;
  rearrange;
  apply applyOverride1;
  rewrite;
next;
  instantiate n_2 == n_0, pd_3 == pd_0;
  rearrange;
  apply applyOverride1;
  rewrite;
  split ⊃ n = name?;
  simplify;
next;

```

■

proof[lPromotedBetweenWorldReqType]

Start by assuming AuxWorld properly typed

```

use lPromotedAuxWorldReqType;
rearrange;
simplify;
Expand the signatures

```

```

invoke PhiBOpReqSig;
invoke PhiBOpBranchesSig;
invoke PhiBOpSig;
invoke BetweenOpSig;
invoke BetweenWorld;
cases;
  invoke BetweenWorldB1;
  rewrite;
next;
  For each  $B$  property, it follows directly by expanding

  the appropriate  $AuxWorld$  component, providing the trivial

  instantiations, as well as applying override application.

invoke BetweenWorldB2;
invoke BetweenWorldB1;
rewrite;
prenex;
instantiate  $pd\_1 == pd$ ;
apply applyOverride;
prove by rewrite;
instantiate  $pd\_0 == pd$ ;
use  $\lambda AuxWorldAuthenticToExpansion$ ;
prove by rewrite;
next;
  invoke BetweenWorldB3;
  invoke BetweenWorldNEW1;
  invoke BetweenWorldNEW2;
  rewrite;
  prenex;
  use  $\lambda AuxWorldAuthenticFromExpansion$ ;
  use  $\lambda AuxWorldAuthenticToExpansion$ ;
  apply applyOverride;
  prove by rewrite;
  cases;
     $pd$  in authenticTo

    instantiate  $pd\_2 == pd$ ;
    prove by rewrite;
    instantiate  $pd\_0 == pd$ ;
    rearrange;
    simplify;
  next;
     $pd$  in authenticFrom

```

```

    instantiate pd__2 == pd;
    prove by rewrite;
    instantiate pd__1 == pd;
    rearrange;
    simplify;
next;
    invoke BetweenWorldB4;
    invoke BetweenWorldNEW3;
    invoke BetweenWorldNEW4;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB5;
    rewrite;
    prenex;
    instantiate pd__0 == pd;
    apply applyOverride;
    prove by rewrite;
    use lAuxWorldFromLoggedExpansion;
    use lAuxWorldAuthenticFromExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB6;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldToLoggedExpansion;
    use lAuxWorldAuthenticToExpansion;
    apply applyOverride;
    prove by rewrite;
next;
    invoke BetweenWorldB7;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldFromLoggedExpansion;
    apply applyOverride;
    prove by rewrite;
    split (conAuthPurse pd.from).status = epr
      ∨ (conAuthPurse pd.from).status = epa;
    simplify;
next;
    invoke BetweenWorldB8;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticToExpansion;
    use lAuxWorldToLoggedExpansion;
    apply applyOverride;
    prove by rewrite;
    split (conAuthPurse pd.to).status = eaTo
      ∨ (conAuthPurse pd.to).status = epv;
    simplify;
next;
    invoke BetweenWorldB9;
    prenex;
    instantiate pd__0 == pd;
    with enabled (disjointCat, disjointDef) prove by rewrite;

```

proof[*tBetwReqOkayPRE*]
 use *lPromotedBetweenWorldReqType*;
 with disabled (*BetweenWorld*) prove by reduce;
 instantiate *status__0 == epr*, *balance__0 == balance*;
 rewrite;
 instantiate *allLogs' == allLogs*, *authenticFrom' == authenticFrom*,
 authenticTo' == authenticTo, *definitelyLost' == definitelyLost*,
 fromInEpa' == fromInEpa, *fromInEpr' == fromInEpr*,
 fromLogged' == fromLogged, *maybeLost' == maybeLost*,
 toInEapayee' == toInEapayee, *toInEpr' == toInEpr*,
 toLogged' == toLogged;
 rewrite;

■

proof[*lPromotedAuxWorldValType*]

Expand signatures

invoke *PhiBOpValSig*;
 invoke *PhiBOpBranchesSig*;
 invoke *PhiBOpSig*;
 invoke *BetweenOpSig*;

Expose AuxWorld

invoke *BetweenWorld*;
 invoke *BetweenWorldB1*;
 invoke *AuxWorld*;

Remove trivial cases

rewrite;

As name? is in conAuthPurse, simplify result of domain over oplus

apply *cupSubsetLeft* to expression { *name?* } $\cup$ dom *conAuthPurse*;
 rewrite;

authenticTo and From appear in many of the cases, early substitute them

equality substitute *authenticTo*;
 equality substitute *authenticFrom*;

Each part in AuxWorld generates a sub-goal

cases;

Case 1.1: ConWorld + conAuthPurse type check

invoke *ConWorld*;
 rewrite;
 prenex;
 instantiate *n__0 == n*;
 apply *applyOverride1*;
 rewrite;
 rearrange;
 rewrite;

Now only injectivity type checking remains


```

use lPromotedConAuthNewPurseInjectivity[balance' := balance + pdAuth.value,
  exLog' := exLog, name' := name, nextSeqNo' := nextSeqNo,
  pdAuth' := pdAuth, status' := eaTo];
rearrange;
invoke ConWorld;
  Simplify prior to expansion of ConPurse

  for better results

simplify;
invoke ConPurse;
prenex;
rearrange;
simplify;
  Now two subsub-goals: archive from ConWorld

  and pd from ConPurse with substitutions

instantiate nld__0 == nld;
instantiate pd__0 == pd;
rearrange;
rewrite;
use fPayDetailsValueType[x := pdAuth];
rearrange;
apply rInNAT;
rewrite;
next;
  Case 1.2: by extensionality on each AuxWorld

  equality, firstly at toInEpv

equality substitute toInEpv;
apply extensionality to predicate
{ pd__77 : { pd__78 : PayDetails | pd__78.to ∈ dom conAuthPurse }
  | (conAuthPurse pd__77.to).status = epv
  ∧ (conAuthPurse pd__77.to).pdAuth = pd__77 }
= { pd__79 : { pd__80 : PayDetails | pd__80.to ∈ dom conAuthPurse }
  | ((conAuthPurse ⊕ {(name?, θ ConPurse[balance :=
    balance + pdAuth.value, status := eaTo])}) pd__79.to).status = epv
  ∧ ((conAuthPurse ⊕ {(name?, θ ConPurse[balance :=
    balance + pdAuth.value, status := eaTo])}) pd__79.to).pdAuth = pd__79 } };
prenex;
rewrite;
  From the information in the assumption, we just need to apply oplus

apply applyOverride;
rewrite;
next;
  Case 1.3: by extensionality on toInEapayee

```

equality substitute toInEapayee;
apply extensionality to predicate

$$\{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.to} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_75.to}).\text{status} = \text{eaTo}$$

$$\wedge (\text{conAuthPurse } \text{pd_75.to}).\text{pdAuth} = \text{pd_75} \}$$

$$= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pdAuth.value}, \text{status} := \text{eaTo}])\})$$

$$\text{pd_77.to}).\text{status} = \text{eaTo}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pdAuth.value}, \text{status} := \text{eaTo}])\}) \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \};$$
prenex;
rewrite;
apply applyOverride;
rewrite;
next;

Case 1.4: by extensionality on fromInEpr

equality substitute fromInEpr;
apply extensionality to predicate

$$\{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_75.from}).\text{status} = \text{epr}$$

$$\wedge (\text{conAuthPurse } \text{pd_75.from}).\text{pdAuth} = \text{pd_75} \}$$

$$= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pdAuth.value}, \text{status} := \text{eaTo}])\}) \text{pd_77.from}).\text{status} = \text{epr}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pdAuth.value}, \text{status} := \text{eaTo}])\}) \text{pd_77.from}).\text{pdAuth} = \text{pd_77} \};$$
prenex;
rewrite;
apply applyOverride;
rewrite;
next;

Case 1.5: by extensionality on fromInEpa

equality substitute fromInEpa;
apply extensionality to predicate

$$\{ \text{pd_79} : \{ \text{pd_80} : \text{PayDetails} \mid \text{pd_80.from} \in \text{dom conAuthPurse} \}$$

$$\mid (\text{conAuthPurse } \text{pd_79.from}).\text{status} = \text{epa}$$

$$\wedge (\text{conAuthPurse } \text{pd_79.from}).\text{pdAuth} = \text{pd_79} \}$$

$$= \{ \text{pd_81} : \{ \text{pd_82} : \text{PayDetails} \mid \text{pd_82.from} \in \text{dom conAuthPurse} \}$$

$$\mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pdAuth.value}, \text{status} := \text{eaTo}])\}) \text{pd_81.from}).\text{status} = \text{epa}$$

$$\wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{balance} :=$$

$$\text{balance} + \text{pdAuth.value}, \text{status} := \text{eaTo}])\})$$

$$\text{pd_81.from}).\text{pdAuth} = \text{pd_81} \};$$
prenex;
rewrite;
apply applyOverride1;
rewrite;
next;

Case 1.5: by extensionality on allLogs

```

equality substitute allLogs;
apply extensionality to predicate
  archive  $\cup$  {  $n\_15$  : dom conAuthPurse;  $pd\_60$  : PayDetails
    |  $pd\_60 \in (conAuthPurse\ n\_15).exLog$  }
= archive  $\cup$  {  $n\_16$  : dom conAuthPurse;  $pd\_61$  : PayDetails
  |  $pd\_61 \in ((conAuthPurse \oplus \{(name?, \theta\} ConPurse[balance :=
    balance + pdAuth.value, status := eaTo]))\ n\_16).exLog$  };
prenex;
rewrite;
prenex;
cases;
  instantiate  $n\_1 == n$ ,  $pd\_2 == pd$ ;
  rearrange;
  apply applyOverride1;
  rewrite;
next;
  instantiate  $n\_2 == n\_0$ ,  $pd\_3 == pd\_0$ ;
  rearrange;
  apply applyOverride1;
  rewrite;
  split  $\neg n = name?$ ;
  simplify;
next;

```

■

proof[lPromotedBetweenWorldValType]

Start by assuming AuxWorld properly typed

```

use lPromotedAuxWorldValType;
rearrange;
simplify;

```

Expand the signatures

```

invoke PhiBOpValSig;
invoke PhiBOpBranchesSig;
invoke PhiBOpSig;
invoke BetweenOpSig;
invoke BetweenWorld;
cases;

```

```

  invoke BetweenWorldB1;
  rewrite;

```

next;

For each B property, it follows directly by expanding

the appropriate AuxWorld component, providing the trivial

instantiations, as well as applying override application.

```

    invoke BetweenWorldB2;
    invoke BetweenWorldB1;
    rewrite;
    prenex;
    instantiate pd__1 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB3;
    invoke BetweenWorldNEW1;
    invoke BetweenWorldNEW2;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB4;
    invoke BetweenWorldNEW3;
    invoke BetweenWorldNEW4;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB5;
    rewrite;
    prenex;
    instantiate pd__0 == pd;
    apply applyOverride;
    prove by rewrite;
    use lAuxWorldFromLoggedExpansion;
    use lAuxWorldAuthenticFromExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB6;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldToLoggedExpansion;
    use lAuxWorldAuthenticToExpansion;
    apply applyOverride;
    prove by rewrite;
next;
    invoke BetweenWorldB7;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldFromLoggedExpansion;
    apply applyOverride;
    prove by rewrite;

```

proof[*PhiBOpValSig\$domainCheck*]
 prove by rewrite;

■

proof[*tBetwValOkayPRE*]
 use *lPromotedBetweenWorldValType*;
 with disabled (*BetweenWorld*) prove by reduce;
 instantiate *status_0 == epv, balance_0 == balance*;
 rewrite;
 instantiate *allLogs' == allLogs, authenticFrom' == authenticFrom,*
 authenticTo' == authenticTo, definitelyLost' == definitelyLost,
 fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
 fromLogged' == fromLogged, maybeLost' == maybeLost,
 toInEapayee' == toInEapayee, toInEpr' == toInEpr,
 toLogged' == toLogged;
 rewrite;
 use *fPayDetailsValueType[x := pdAuth]*;
 rearrange;
 apply *rInNAT*;
 rewrite;

■

proof[*lPromotedAuxWorldAckType*]

Expand signatures

invoke PhiBOpAckSig;
invoke PhiBOpBranchesSig;
invoke PhiBOpSig;
invoke BetweenOpSig;

Expose AuxWorld

invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;

Remove trivial cases

rewrite;

As name? is in conAuthPurse, simplify result of domain over oplus

apply cupSubsetLeft to expression { name? } ∪ dom conAuthPurse;
rewrite;

authenticTo and From appear in many of the cases, early substitute them

equality substitute authenticTo;
equality substitute authenticFrom;

Each part in AxuWorld generates a sub-goal

cases;

Case 1.1: ConWorld + conAuthPurse type check

```

invoke ConWorld;
rewrite;
prenex;
instantiate n__0 == n;
apply applyOverride1;
rewrite;
rearrange;
rewrite;
  Now only injectivity type checking remains

use lPromotedConAuthNewPurseInjectivity[balance' := balance,
  exLog' := exLog, name' := name, nextSeqNo' := nextSeqNo,
  pdAuth' := pdAuth, status' := eaFrom];
rearrange;
invoke ConWorld;
  Simplify prior to expansion of ConPurse

  for better results

simplify;
invoke ConPurse;
prenex;
rearrange;
simplify;
  Now two subsub-goals: archive from ConWorld

  and pd from ConPurse with substitutions

instantiate nld__0 == nld;
instantiate pd__0 == pd;
rewrite;
next;
  Case 1.2: by extensionality on each AuxWorld

  equality, firstly at toInEpv

equality substitute toInEpv;
apply extensionality to predicate
  { pd__77 : { pd__78 : PayDetails | pd__78.to ∈ dom conAuthPurse }
    | (conAuthPurse pd__77.to).status = epv
    ∧ (conAuthPurse pd__77.to).pdAuth = pd__77 }
= { pd__79 : { pd__80 : PayDetails | pd__80.to ∈ dom conAuthPurse }
  | ((conAuthPurse ⊕ {(name?, θ ConPurse[
    status := eaFrom])}) pd__79.to).status = epv
    ∧ ((conAuthPurse ⊕ {(name?, θ ConPurse[
    status := eaFrom])}) pd__79.to).pdAuth = pd__79 };
prenex;
rewrite;
  From the information in the assumption, we just need to apply oplus

apply applyOverride;
rewrite;
next;
  Case 1.3: by extensionality on toInEapayee

```

equality substitute toInEapayee;
apply extensionality to predicate

$$\begin{aligned} & \{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid (\text{conAuthPurse } \text{pd_75.to}).\text{status} = \text{eaTo} \\ & \quad \wedge (\text{conAuthPurse } \text{pd_75.to}).\text{pdAuth} = \text{pd_75} \} \\ &= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid ((\text{conAuthPurse} \oplus \{(name?, \theta \text{ ConPurse}[status := eaFrom])\}) \\ & \quad \quad \text{pd_77.to}).\text{status} = \text{eaTo} \\ & \quad \wedge ((\text{conAuthPurse} \oplus \{(name?, \theta \text{ ConPurse}[\\ & \quad \quad \text{status} := eaFrom])\}) \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \}; \end{aligned}$$
prenex;
rewrite;
apply applyOverride;
rewrite;
next;

Case 1.4: by extensionality on fromInEpr

equality substitute fromInEpr;
apply extensionality to predicate

$$\begin{aligned} & \{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.from} \in \text{dom conAuthPurse} \} \\ & \quad \mid (\text{conAuthPurse } \text{pd_75.from}).\text{status} = \text{epr} \\ & \quad \wedge (\text{conAuthPurse } \text{pd_75.from}).\text{pdAuth} = \text{pd_75} \} \\ &= \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.from} \in \text{dom conAuthPurse} \} \\ & \quad \mid ((\text{conAuthPurse} \oplus \{(name?, \theta \text{ ConPurse}[\\ & \quad \quad \text{status} := eaFrom])\}) \text{pd_77.from}).\text{status} = \text{epr} \\ & \quad \wedge ((\text{conAuthPurse} \oplus \{(name?, \theta \text{ ConPurse}[\\ & \quad \quad \text{status} := eaFrom])\}) \text{pd_77.from}).\text{pdAuth} = \text{pd_77} \}; \end{aligned}$$
prenex;
rewrite;
apply applyOverride;
rewrite;
next;

Case 1.5: by extensionality on fromInEpa

equality substitute fromInEpa;
apply extensionality to predicate

$$\begin{aligned} & \{ \text{pd_79} : \{ \text{pd_80} : \text{PayDetails} \mid \text{pd_80.from} \in \text{dom conAuthPurse} \} \\ & \quad \mid (\text{conAuthPurse } \text{pd_79.from}).\text{status} = \text{epa} \\ & \quad \wedge (\text{conAuthPurse } \text{pd_79.from}).\text{pdAuth} = \text{pd_79} \} \\ &= \{ \text{pd_81} : \{ \text{pd_82} : \text{PayDetails} \mid \text{pd_82.from} \in \text{dom conAuthPurse} \} \\ & \quad \mid ((\text{conAuthPurse} \oplus \{(name?, \theta \text{ ConPurse}[\\ & \quad \quad \text{status} := eaFrom])\}) \text{pd_81.from}).\text{status} = \text{epa} \\ & \quad \wedge ((\text{conAuthPurse} \oplus \{(name?, \theta \text{ ConPurse}[\\ & \quad \quad \text{status} := eaFrom])\}) \text{pd_81.from}).\text{pdAuth} = \text{pd_81} \}; \end{aligned}$$
prenex;
rewrite;
apply applyOverride1;
rewrite;
next;

Case 1.5: by extensionality on allLogs

```

equality substitute allLogs;
apply extensionality to predicate
  archive  $\cup$  {  $n\_15$  : dom conAuthPurse;  $pd\_60$  : PayDetails
    |  $pd\_60 \in (conAuthPurse\ n\_15).exLog$  }
= archive  $\cup$  {  $n\_16$  : dom conAuthPurse;  $pd\_61$  : PayDetails
  |  $pd\_61 \in ((conAuthPurse \oplus \{(name?, \theta\} ConPurse[
    status := eaFrom]))\ n\_16).exLog$  };
prenex;
rewrite;
prenex;
cases;
  instantiate  $n\_1 == n$ ,  $pd\_2 == pd$ ;
  rearrange;
  apply applyOverride1;
  rewrite;
next;
  instantiate  $n\_2 == n\_0$ ,  $pd\_3 == pd\_0$ ;
  rearrange;
  apply applyOverride1;
  rewrite;
  split  $\neg n = name?$ ;
  simplify;
next;

```

■

proof[*PhiBOpAckSig\$domainCheck*]
 prove by rewrite;

■

proof[*lPromotedBetweenWorldAckType*]
Start by assuming AuxWorld properly typed

```

use lPromotedAuxWorldAckType;
rearrange;
simplify;

```

Expand the signatures

```

invoke PhiBOpAckSig;
invoke PhiBOpBranchesSig;
invoke PhiBOpSig;
invoke BetweenOpSig;
invoke BetweenWorld;
cases;
  invoke BetweenWorldB1;
  rewrite;
next;

```

For each B property, it follows directly by expanding

the appropriate AuxWorld component, providing the trivial

instantiations, as well as applying override application.


```

    invoke BetweenWorldB2;
    invoke BetweenWorldB1;
    rewrite;
    prenex;
    instantiate pd__1 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB3;
    invoke BetweenWorldNEW1;
    invoke BetweenWorldNEW2;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB4;
    invoke BetweenWorldNEW3;
    invoke BetweenWorldNEW4;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB5;
    rewrite;
    prenex;
    instantiate pd__0 == pd;
    apply applyOverride;
    prove by rewrite;
    use lAuxWorldFromLoggedExpansion;
    use lAuxWorldAuthenticFromExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB6;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldToLoggedExpansion;
    use lAuxWorldAuthenticToExpansion;
    apply applyOverride;
    prove by rewrite;
next;
    invoke BetweenWorldB7;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldFromLoggedExpansion;
    apply applyOverride;
    prove by rewrite;

```

proof[*tBetwAckOkayPRE*]
 use *lPromotedBetweenWorldAckType*;
 with disabled (*BetweenWorld*) prove by reduce;
 instantiate *status__0* == *epa*, *pdAuth__0* == *pdAuth*;
 rewrite;
 instantiate *allLogs'* == *allLogs*, *authenticFrom'* == *authenticFrom*,
 authenticTo' == *authenticTo*, *definitelyLost'* == *definitelyLost*,
 fromInEpa' == *fromInEpa*, *fromInEpr'* == *fromInEpr*,
 fromLogged' == *fromLogged*, *maybeLost'* == *maybeLost*,
 toInEapayee' == *toInEapayee*, *toInEpv'* == *toInEpv*,
 toLogged' == *toLogged*, *pdAuth'* == *pdAuth*;
 rewrite;
 invoke Δ *ConPurse*;
 invoke *ConPurse*;
 rewrite;

■

proof[*tBetwReadExceptionLogEafromOkayPRE*]
 use *lPromotedBetweenWorldType*[*m!* := $\perp$, *status* := *eaFrom*];
 with disabled (*BetweenWorld*) prove by reduce;
 instantiate *allLogs'* == *allLogs*, *authenticFrom'* == *authenticFrom*,
 authenticTo' == *authenticTo*, *definitelyLost'* == *definitelyLost*,
 fromInEpa' == *fromInEpa*, *fromInEpr'* == *fromInEpr*,
 fromLogged' == *fromLogged*, *maybeLost'* == *maybeLost*,
 toInEapayee' == *toInEapayee*, *toInEpv'* == *toInEpv*,
 toLogged' == *toLogged*, *m!* == $\perp$;
 with disabled (*BetweenWorld*) prove by reduce;

■

proof[*tBetwReadExceptionLogOkayPRE*]
 use *lPromotedBetweenWorldType*[*m!* := $\perp$, *status* := *eaFrom*];
 with disabled (*BetweenWorld*) prove by reduce;
 instantiate *nextSeqNo__0* == *nextSeqNo*, *exLog__0* == *exLog*,
 pdAuth__0 == *pdAuth*, *status__0* == *status*,
 nextSeqNo' == *nextSeqNo*, *pdAuth'* == *pdAuth*;
 rewrite;
 instantiate *allLogs'* == *allLogs*, *authenticFrom'* == *authenticFrom*,
 authenticTo' == *authenticTo*, *definitelyLost'* == *definitelyLost*,
 fromInEpa' == *fromInEpa*, *fromInEpr'* == *fromInEpr*,
 fromLogged' == *fromLogged*, *maybeLost'* == *maybeLost*,
 toInEapayee' == *toInEapayee*, *toInEpv'* == *toInEpv*,
 toLogged' == *toLogged*, *m!* == $\perp$;
 with disabled (*BetweenWorld*) prove by reduce;

■

proof[*PhiBOPClearExceptionLogSig\$domainCheck*]
 invoke *PhiBOPBranchesSig*;
 prove by rewrite;

■

proof[*lPromotedAuxWorldClearExceptionLogType*]
 Expand signatures

```

invoke PhiBOPClearExceptionLogSig;
invoke PhiBOPBranchesSig;
invoke PhiBOPSig;
invoke BetweenOpSig;
  Expose AuxWorld

invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
  Remove trivial cases

rewrite;
  As name? is in conAuthPurse, simplify result of domain over oplus

apply cupSubsetLeft to expression { name? }  $\cup$  dom conAuthPurse;
rewrite;
  authenticTo and From appear in many of the cases, early substitute them

equality substitute authenticTo;
equality substitute authenticFrom;
  Each part in AxuWorld generates a sub-goal

cases;
  Case 1.1: ConWorld + conAuthPurse type check

  invoke ConWorld;
  rewrite;
  prenex;
  instantiate n__0 == n;
  apply applyOverride1;
  rewrite;
  rearrange;
  rewrite;
    Now only injectivity type checking remains

  use lPromotedConAuthNewPurseInjectivity[balance' := balance,
    exLog' := {}, name' := name, nextSeqNo' := nextSeqNo,
    pdAuth' := pdAuth, status' := eaFrom];
  rearrange;
  invoke ConWorld;
    Simplify prior to expansion of ConPurse

    for better results

  simplify;
  invoke ConPurse;
  prenex;
  rearrange;
  rewrite;
    Now two subsub-goals: archive from ConWorld

    and pd from ConPurse with substitutions

  instantiate nld__0 == nld;
  rewrite;
next;
  Case 1.2: by extensionality on each AuxWorld

  equality, firstly at toInEpu

```

equality substitute toInEpr;
apply extensionality to predicate

$$\begin{aligned} & \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid (\text{conAuthPurse } \text{pd_77.to}).\text{status} = \text{epv} \\ & \quad \wedge (\text{conAuthPurse } \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \} \\ = & \{ \text{pd_79} : \{ \text{pd_80} : \text{PayDetails} \mid \text{pd_80.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \{\}, \text{status} := \text{eaFrom}])\}) \\ & \quad \quad \text{pd_79.to}).\text{status} = \text{epv} \\ & \quad \wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \{\}, \\ & \quad \quad \text{status} := \text{eaFrom}])\}) \text{pd_79.to}).\text{pdAuth} = \text{pd_79} \}; \\ \text{prenex;} \\ \text{rewrite;} \\ \text{From the information in the assumption, we just need to apply oplus} \\ \\ \text{apply applyOverride;} \\ \text{rewrite;} \\ \text{next;}
\end{aligned}$$

Case 1.3: by extensionality on toInEpayee

equality substitute toInEpayee;
apply extensionality to predicate

$$\begin{aligned} & \{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid (\text{conAuthPurse } \text{pd_75.to}).\text{status} = \text{eaTo} \\ & \quad \wedge (\text{conAuthPurse } \text{pd_75.to}).\text{pdAuth} = \text{pd_75} \} \\ = & \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.to} \in \text{dom conAuthPurse} \} \\ & \quad \mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \{\}, \\ & \quad \quad \text{status} := \text{eaFrom}])\}) \text{pd_77.to}).\text{status} \\ & \quad = \text{eaTo} \\ & \quad \wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \{\}, \\ & \quad \quad \text{status} := \text{eaFrom}])\}) \text{pd_77.to}).\text{pdAuth} = \text{pd_77} \}; \\ \text{prenex;} \\ \text{rewrite;} \\ \text{apply applyOverride;} \\ \text{rewrite;} \\ \text{next;}
\end{aligned}$$

Case 1.4: by extensionality on fromInEpr

equality substitute fromInEpr;
apply extensionality to predicate

$$\begin{aligned} & \{ \text{pd_75} : \{ \text{pd_76} : \text{PayDetails} \mid \text{pd_76.from} \in \text{dom conAuthPurse} \} \\ & \quad \mid (\text{conAuthPurse } \text{pd_75.from}).\text{status} = \text{epr} \\ & \quad \wedge (\text{conAuthPurse } \text{pd_75.from}).\text{pdAuth} = \text{pd_75} \} \\ = & \{ \text{pd_77} : \{ \text{pd_78} : \text{PayDetails} \mid \text{pd_78.from} \in \text{dom conAuthPurse} \} \\ & \quad \mid ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \{\}, \\ & \quad \quad \text{status} := \text{eaFrom}])\}) \text{pd_77.from}).\text{status} = \text{epr} \\ & \quad \wedge ((\text{conAuthPurse} \oplus \{(\text{name?}, \theta \text{ ConPurse}[\text{exLog} := \{\}, \\ & \quad \quad \text{status} := \text{eaFrom}])\}) \text{pd_77.from}).\text{pdAuth} = \text{pd_77} \}; \\ \text{prenex;} \\ \text{rewrite;} \\ \text{apply applyOverride;} \\ \text{rewrite;} \\ \text{next;}
\end{aligned}$$

Case 1.5: by extensionality on fromInEpa

```

equality substitute fromInEpa;
apply extensionality to predicate
  { pd__79 : { pd__80 : PayDetails | pd__80.from ∈ dom conAuthPurse }
    | (conAuthPurse pd__79.from).status = epa
    ∧ (conAuthPurse pd__79.from).pdAuth = pd__79 }
= { pd__81 : { pd__82 : PayDetails | pd__82.from ∈ dom conAuthPurse }
  | ((conAuthPurse ⊕ {(name?, θ ConPurse[exLog := {},
    status := eaFrom])}) pd__81.from).status = epa
    ∧ ((conAuthPurse ⊕ {(name?, θ ConPurse[exLog := {},
    status := eaFrom])}) pd__81.from).pdAuth = pd__81 };
prenex;
rewrite;
apply applyOverride1;
rewrite;
next;
  Case 1.5: by extensionality on allLogs

equality substitute allLogs;
apply extensionality to predicate
  archive ∪ { n__15 : dom conAuthPurse; pd__60 : PayDetails
    | pd__60 ∈ (conAuthPurse n__15).exLog }
= archive ∪ { n__16 : dom conAuthPurse; pd__61 : PayDetails
  | pd__61 ∈ ((conAuthPurse ⊕ {(name?, θ ConPurse[
    exLog := {}, status := eaFrom])}) n__16).exLog };
prenex;
rewrite;
prenex;
cases;
  instantiate n__1 == n, pd__3 == pd;
  rearrange;
  apply applyOverride1;
  rewrite;
  instantiate pd__1 == pd;
  rearrange;
  rewrite;
next;
  instantiate n__2 == n__0, pd__4 == pd__0;
  rearrange;
  apply applyOverride1;
  rewrite;
next;

```

■

proof[*lPromotedBetweenWorldClearExceptionLogType*]

Start by assuming *AuxWorld* properly typed

```

use lPromotedAuxWorldClearExceptionLogType;
rearrange;
simplify;
Expand the signatures

```

```

invoke PhiBOpClearExceptionLogSig;
invoke PhiBOpBranchesSig;
invoke PhiBOpSig;
invoke BetweenOpSig;
invoke BetweenWorld;
cases;
  invoke BetweenWorldB1;
  rewrite;
next;
  For each B property, it follows directly by expanding
the appropriate AuxWorld component, providing the trivial
instantiations, as well as applying override application.

```

```

    invoke BetweenWorldB2;
    invoke BetweenWorldB1;
    rewrite;
    prenex;
    instantiate pd__1 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB3;
    invoke BetweenWorldNEW1;
    invoke BetweenWorldNEW2;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB4;
    invoke BetweenWorldNEW3;
    invoke BetweenWorldNEW4;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB5;
    rewrite;
    prenex;
    instantiate pd__0 == pd;
    apply applyOverride;
    prove by rewrite;
    use lAuxWorldFromLoggedExpansion;
    use lAuxWorldAuthenticFromExpansion;
    prove by rewrite;
next;
    invoke BetweenWorldB6;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldToLoggedExpansion;
    use lAuxWorldAuthenticToExpansion;
    apply applyOverride;
    prove by rewrite;
next;
    invoke BetweenWorldB7;
    prenex;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldFromLoggedExpansion;
    apply applyOverride;
    prove by rewrite;

```

```

proof[tBetwClearExceptionLogEafromOkayPRE]
  use lPromotedBetweenWorldClearExceptionLogType;
  with disabled (BetweenWorld) prove by reduce;
  instantiate exLog__0 == exLog;
  rewrite;
  instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,
    authenticTo' == authenticTo, definitelyLost' == definitelyLost,
    fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
    fromLogged' == fromLogged, maybeLost' == maybeLost,
    toInEapayee' == toInEapayee, toInEpv' == toInEpv,
    toLogged' == toLogged;
  rewrite;
  ■

```

```

proof[tBetwClearExceptionLogOkayPRE]
  use lPromotedBetweenWorldClearExceptionLogType;
  with disabled (BetweenWorld) prove by reduce;
  instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status;
  instantiate exLog__1 == exLog;
  rewrite;
  instantiate nextSeqNo' == nextSeqNo, pdAuth' == pdAuth;
  rewrite;
  instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,
    authenticTo' == authenticTo, definitelyLost' == definitelyLost,
    fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
    fromLogged' == fromLogged, maybeLost' == maybeLost,
    toInEapayee' == toInEapayee, toInEpv' == toInEpv,
    toLogged' == toLogged;
  rewrite;
  invoke  $\Delta$  ConPurse;
  invoke ConPurse;
  rewrite;
  ■

```



```

proof[tBetwAuthoriseExLogClearOkayPRE]
  with disabled (BetweenWorld) prove by reduce;
  instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,
    authenticTo' == authenticTo, definitelyLost' == definitelyLost,
    fromInEpa' == fromInEpa, fromInEpr' == fromInEpr,
    fromLogged' == fromLogged, maybeLost' == maybeLost,
    toInEapayee' == toInEapayee, toInEpv' == toInEpv,
    toLogged' == toLogged;
  rewrite;
  use lBetweenWorldArchiveExpansion;
  rearrange;
  simplify;
  MISSING :
  (∃ pds :  $\mathbb{P}_1$  PayDetails •
    (BetweenWorld[ether := ether ∪ {exceptionLogClear (name?, image pds)}])
    ∧ (∀ e__1 : {name?} × pds • e__1 ∈ archive)))
  instantiate pds == ArchivePaymentsInstance;
  rearrange;
  use dArchivePaymentsInstance[name := name?];
  rearrange;
  rewrite;
  equality substitute ArchivePaymentsInstance;
  apply inPower;
  prenex;
  apply inCross2;
  prenex;
  instantiate e__0 == e;
  rearrange;
  rewrite;
  ■

```

I.5 $\mathcal{C}$ consistency proofs

See Section 8.6.

This proof is a little different from what is on the text.

```

proof[tExistsInitialConWorld]
  prove by reduce;
  instantiate
    etherCh7'
    == {readExceptionLog,  $\perp$ }
        $\cup \cup$ 
       { cpd : CounterPartyDetails • {startFrom cpd, startTo cpd} },
    conAuthPurseCh7' == {}, archiveCh7' == {};
  prove by reduce;
  instantiate
    allLogs' == {}, authenticFrom' == {}, authenticTo' == {},
    fromLogged' == {}, toLogged' == {}, toInEprv' == {},
    toInEpayee' == {}, fromInEpr' == {}, fromInEpa' == {},
    definitelyLost' == {}, maybeLost' == {},
    ether'
    == {readExceptionLog,  $\perp$ }
        $\cup \cup$ 
       { cpd : CounterPartyDetails • {startFrom cpd, startTo cpd} };
  prove by reduce;
  apply dLogbook;
  apply extensionality;
  prove by rewrite;
  apply inPower;
  prove by rewrite;
  ■

```

proof[*tPhiCOpPRE*]

```

  instantiate conAuthPurseCh7' == conAuthPurseCh7
    ⊕ {(name? ↦ θ ConPurse)},
  archiveCh7' == archiveCh7, etherCh7' == etherCh7,
  m! == m?, balance' == (conAuthPurseCh7 name?).balance,
  exLog' == (conAuthPurseCh7 name?).exLog,
  name' == (conAuthPurseCh7 name?).name,
  nextSeqNo' == (conAuthPurseCh7 name?).nextSeqNo,
  pdAuth' == (conAuthPurseCh7 name?).pdAuth,
  status' == (conAuthPurseCh7 name?).status;
  invoke PhiCOp;
  invoke PhiCOpSig;
  invoke Δ ConPurse;
  invoke Δ ConWorldCh7;
  invoke ConOpSig;
  use lPromotedConAuthPurseInjectivity[conAuthPurse := conAuthPurseCh7,
    archive := archiveCh7, ether := etherCh7];
  rearrange;
  simplify;
  invoke ConWorldCh7;
  invoke ConWorld;
  prove by rewrite;
  instantiate nld_0 == nld;
  instantiate n_0 == n;
  prove by rewrite;
  apply applyOverride;
  prove by rewrite;
  invoke ( _ ↗ _ );
  invoke ( _ ↗ _ );
  invoke ( _ ↗ _ );
  prove by rewrite;
  apply oplusDef;
  prove by rewrite;
  split n = name?;
  prove by rewrite;
  ■

```

proof[*tCIgnoreIsTotal*]

```

  prove by reduce;
  ■

```

proof[*tCIncreaseIsTotal*]

```

  instantiate archiveCh7' == archiveCh7,
    conAuthPurseCh7' == conAuthPurseCh7,
    etherCh7' == etherCh7, m! == ⊥;
  with disabled (PhiCOp, ConPurse, IncreasePurseOkay) prove by reduce;
  ■

```

proof[*tCAbortIsTotal*]

```

  instantiate archiveCh7' == archiveCh7,
    conAuthPurseCh7' == conAuthPurseCh7,
    etherCh7' == etherCh7, m! == ⊥;
  with disabled (PhiCOp, ConPurse, IncreasePurseOkay) prove by reduce;
  ■

```

proof[*tCStartFromIsTotal*]
instantiate *archiveCh7'* == *archiveCh7*,
conAuthPurseCh7' == *conAuthPurseCh7*,
etherCh7' == *etherCh7*, *m!* == $\perp$;
with disabled (*PhiCOp*, *ConPurse*, *IncreasePurseOkay*) *prove by reduce*;

■

proof[*tCStartToIsTotal*]
instantiate *archiveCh7'* == *archiveCh7*,
conAuthPurseCh7' == *conAuthPurseCh7*,
etherCh7' == *etherCh7*, *m!* == $\perp$;
with disabled (*PhiCOp*, *ConPurse*, *IncreasePurseOkay*) *prove by reduce*;

■

proof[*tCReqIsTotal*]
instantiate *archiveCh7'* == *archiveCh7*,
conAuthPurseCh7' == *conAuthPurseCh7*,
etherCh7' == *etherCh7*, *m!* == $\perp$;
with disabled (*PhiCOp*, *ConPurse*, *IncreasePurseOkay*) *prove by reduce*;

■

proof[*tCValIsTotal*]
instantiate *archiveCh7'* == *archiveCh7*,
conAuthPurseCh7' == *conAuthPurseCh7*,
etherCh7' == *etherCh7*, *m!* == $\perp$;
with disabled (*PhiCOp*, *ConPurse*, *IncreasePurseOkay*) *prove by reduce*;

■

proof[*tCAckIsTotal*]
instantiate *archiveCh7'* == *archiveCh7*,
conAuthPurseCh7' == *conAuthPurseCh7*,
etherCh7' == *etherCh7*, *m!* == $\perp$;
with disabled (*PhiCOp*, *ConPurse*, *IncreasePurseOkay*) *prove by reduce*;

■

proof[*tCReadExceptionLogIsTotal*]
instantiate *archiveCh7'* == *archiveCh7*,
conAuthPurseCh7' == *conAuthPurseCh7*,
etherCh7' == *etherCh7*, *m!* == $\perp$;
with disabled (*PhiCOp*, *ConPurse*, *IncreasePurseOkay*) *prove by reduce*;

■

proof[*tCClearExceptionLogIsTotal*]
instantiate *archiveCh7'* == *archiveCh7*,
conAuthPurseCh7' == *conAuthPurseCh7*,
etherCh7' == *etherCh7*, *m!* == $\perp$;
with disabled (*PhiCOp*, *ConPurse*, *IncreasePurseOkay*) *prove by reduce*;

■

```

proof[tCAuthoriseExLogClearIsTotal]
  instantiate archiveCh7' == archiveCh7,
    conAuthPurseCh7' == conAuthPurseCh7,
    etherCh7' == etherCh7, m! ==  $\perp$ ;
  with disabled (PhiCOp, ConPurse, IncreasePurseOkay) prove by reduce;
  ■

```

```

proof[tCArchiveIsTotal]
  instantiate archiveCh7' == archiveCh7,
    conAuthPurseCh7' == conAuthPurseCh7,
    etherCh7' == etherCh7, m! ==  $\perp$ ;
  invoke CArchive;
  invoke ConOpSig;
  invoke  $\Delta$  ConWorldCh7;
  rewrite;
  ■

```

```

proof[PhiCOpBranchesSig$domainCheck]
  prove by rewrite;
  ■

```

```

proof[lConAuthPurseInjFinite]
  use lPFunSubsetOplusUnitRel[NAME,  $\langle \langle$  balance :  $\mathbb{Z}$ ; exLog :
     $\mathbb{P}$   $\langle \langle$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :
       $\mathbb{Z}$ ; value :  $\mathbb{Z}$   $\rangle$ ; name : NAME; nextSeqNo :  $\mathbb{Z}$ ; pdAuth :
         $\langle \langle$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :
           $\mathbb{Z}$ ; value :  $\mathbb{Z}$   $\rangle$ ; status : STATUS  $\rangle$ ]
    [f := conAuthPurseCh7, x := name?, y :=  $\theta$  ConPurse];
  use lPromotedConAuthPurseInjectivity[archive := archiveCh7,
    conAuthPurse := conAuthPurseCh7, ether := etherCh7];
  prove by rewrite;
  ■

```

Again, to get the instantiations and expansions right, as well as the (minimal) necessary auxiliary lemmas, took quite a while.

```

proof[tConcIncreaseOkayPRE]
  with disabled (ConWorldCh7, ConPurse) prove by reduce;
  instantiate nextSeqNo__0 == nextSeqNo,
    nextSeqNo' == nextSeqNo, etherCh7' == etherCh7;
  use lConAuthPurseInjFinite;
  with disabled (ConPurse) prove by reduce;
  instantiate n__0 == n;
  split n = name?;
  prove by rewrite;
  ■

```

```

proof[tConcAbortOkayPRE]
  with disabled (ConWorldCh7, ConPurse) prove by reduce;
  instantiate nextSeqNo__0 == nextSeqNo, exLog__0 == exLog,
    pdAuth__0 == pdAuth, status__0 == status,
    nextSeqNo' == nextSeqNo, pdAuth' == pdAuth,
    exLog' == exLog  $\cup$  (if status = epv  $\vee$  status = epa
      then { pdAuth } else {}),
    etherCh7' == etherCh7;
  use lConAuthPurseInjFinite;
  with disabled (ConPurse) prove by reduce;
  split status = epv  $\vee$  status = epa;
  simplify;
  cases;
    invoke ConPurse;
    prove by rewrite;
    apply applyOverride1;
    instantiate pd__0 == pd;
    instantiate n__0 == n;
    split n  $\in$  dom conAuthPurseCh7;
    prove by rewrite;
  cases;
    Now only remains to prove (twice the same) that

    conAuthPurseCh7  $\oplus$  {(name?,  $\theta$  ConPurse[exLog := exLog  $\cup$ 
      {pdAuth}, status := eaFrom])}  $\in$  NAMES  $\rightsquigarrow$  ConPurse
  next;
  next;
  next;
    apply applyOverride1;
    invoke ConPurse;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate n__0 == n;
    prove by rewrite;
    MISSING :
    conAuthPurseCh7  $\oplus$  {(name?,  $\theta$  ConPurse[status := eaFrom])}
       $\in$  NAMES  $\rightsquigarrow$  ConPurse
  next;
  rewrite;

```

■

```

proof[tConcStartFromOkayPRE]
  with disabled (ConWorldCh7, ConPurse) prove by reduce;
  instantiate nextSeqNo__1 == nextSeqNo,
    exLog__0 == exLog, pdAuth__1 == pdAuth,
    status__0 == status;
  instantiate nextSeqNo__2 == nextSeqNo,
    pdAuth__2 == pdAuth, status__1 == status;
  instantiate etherCh7' == etherCh7, nextSeqNo' == nextSeqNo + 1,
    nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth;
  prove by reduce;
  apply applyOverride1;
  instantiate n__0 == n;
  split n ∈ dom conAuthPurseCh7;
  prove by rewrite;
  cases;
  MISSING :
    conAuthPurseCh7 ⊕ {(name?, θ ConPurse[nextSeqNo := 1 + nextSeqNo,
      pdAuth := θ PayDetails[name/from, nextSeqNo/fromSeqNo,
        to := (startFrom ~ m?).name, toSeqNo := (startFrom ~ m?).nextSeqNo,
        value := (startFrom ~ m?).value], status := epr])}
      ∈ NAMES ⇎ ConPurse
  next;
  MISSING :
    conAuthPurseCh7 ⊕ {(name?, θ ConPurse[nextSeqNo := 1 + nextSeqNo,
      pdAuth := θ PayDetails[name/from, nextSeqNo/fromSeqNo,
        to := (startFrom ~ m?).name, toSeqNo := (startFrom ~ m?).nextSeqNo,
        value := (startFrom ~ m?).value], status := epr])}
      ∈ NAMES ⇎ ConPurse
  next;

```

■

proof[*tConcStartToOkayPRE*]
with disabled (ConWorldCh7, ConPurse) prove by reduce;
instantiate nextSeqNo__1 == nextSeqNo,
exLog__0 == exLog, pdAuth__1 == pdAuth,
status__0 == status;
instantiate nextSeqNo__2 == nextSeqNo,
pdAuth__2 == pdAuth, status__1 == status;
instantiate etherCh7' == etherCh7, nextSeqNo' == nextSeqNo + 1,
nextSeqNo__0 == nextSeqNo, pdAuth__0 == pdAuth;
prove by reduce;
apply applyOverride1;
instantiate n__0 == n;
split n ∈ dom conAuthPurseCh7;
prove by rewrite;
cases;
MISSING :
conAuthPurseCh7 ⊕ {(name?, θ ConPurse[nextSeqNo := 1 + nextSeqNo,
pdAuth := θ PayDetails[from := (startTo ~ m?).name,
fromSeqNo := (startTo ~ m?).nextSeqNo, name/to, nextSeqNo/toSeqNo,
value := (startTo ~ m?).value], status := epv)]}
∈ NAMES ⇔ ConPurse
next;
MISSING :
conAuthPurseCh7 ⊕ {(name?, θ ConPurse[nextSeqNo := 1 + nextSeqNo,
pdAuth := θ PayDetails[from := (startTo ~ m?).name,
fromSeqNo := (startTo ~ m?).nextSeqNo, name/to, nextSeqNo/toSeqNo,
value := (startTo ~ m?).value], status := epv)]}
∈ NAMES ⇔ ConPurse
next;

■

proof[*tConcReqOkayPRE*]
with disabled (ConWorldCh7, ConPurse) prove by reduce;
instantiate etherCh7' == etherCh7, balance__0 == balance,
status__0 == status;
prove by reduce;
apply applyOverride1;
instantiate n__0 == n;
split n = name?;
prove by rewrite;
MISSING :
conAuthPurseCh7
⊕ {(name?,
θ
*ConPurse[balance := balance + ⁻1 * pdAuth.value,*
name := pdAuth.from, status := epa])}
∈ NAMES ⇔ ConPurse

■

proof[*tConcValOkayPRE*]
with disabled (ConWorldCh7, ConPurse) prove by reduce;
instantiate etherCh7' == etherCh7, balance__0 == balance,
status__0 == status;
prove by reduce;
apply applyOverride1;
instantiate n__0 == n;
prove by rewrite;
use fPayDetailsValueType[x := pdAuth];
apply rInNAT;
rearrange;
rewrite;
MISSING :
conAuthPurseCh7 $\oplus$ {(name?, θ ConPurse[balance := balance + pdAuth.value,
status := eaTo])} $\in$ NAMES $\rightsquigarrow$ ConPurse

■

proof[*tConcAckOkayPRE*]
with disabled (ConWorldCh7, ConPurse) prove by reduce;
instantiate etherCh7' == etherCh7, pdAuth' == pdAuth,
status__0 == status, pdAuth__0 == pdAuth;
prove by reduce;
apply applyOverride1;
instantiate n__0 == n;
prove by rewrite;
split n = name?;
rewrite;
cases;
MISSING :
conAuthPurseCh7 $\oplus$ {(name?, θ ConPurse[status := eaFrom])} $\in$ NAMES $\rightsquigarrow$ ConPurse

■

proof[*tConcReadExceptionLogOkayPRE*]

■

proof[*PhiCOPClearExceptionLogSig\$domainCheck*]

prove by rewrite;

■

proof[*tConcClearExceptionLogOkayPRE*]

■

proof[*tConcAuthoriseExLogClearOkayPRE*]
prove by reduce;
instantiate etherCh7' == etherCh7, balance__0 == balance,
exLog__0 == exLog, name__0 == name, nextSeqNo__0 == nextSeqNo,
pdAuth__0 == pdAuth, status__0 == status,
balance' == balance, exLog' == exLog, name' == name,
nextSeqNo' == nextSeqNo, pdAuth' == pdAuth,
status' == status;
prove by reduce;

■

I.6 Summary

Declarations	This chapter	Total
Domain checks	9	64
Proof scripts	119	203
Total proofs	128	267
Total explained	0	0

Table I.1: Summary of proofs

Appendix J

Chapter 10 proofs

J.1 Retrieve state

See Section 10.2.

Do not use prove by reduce because of *BetweenWorld*.

```
proof[RabCl$domainCheck]  
  invoke AbstractBetween;  
  prove by rewrite;  
  ■
```

```
proof[OtherPursesRab$domainCheck]  
  invoke AbstractBetween;  
  prove by rewrite;  
  ■
```

We put it here a repeated proof case by case for better performance. Although we believe this domain check *true*, this proof is being a nightmare. Whatever approach we take, the complexity of predicates involved reaches a point where **Z/Eves** starts struggling real hard about available identifiers and possibilities for instantiations. As the source of the problem lies inside sets equalities/substitution of *AuxWorld*, we cannot get rid of them, but instead we need to find different proof trees. To solve this, we used the lemma below. We tried more than 10 variations until we got the final proof.

```

proof[lRabOkayClPdInDomConAuthPurse]
  apply inDiff;
  rewrite;
  invoke AuxWorld;
  equality substitute maybeLost;
  rewrite;
  equality substitute toInEpv;
  rewrite;
  equality substitute authenticTo;
  rewrite;
  split pdThis ∈ fromInEpa;
  simplify;
  cases;
  rearrange;
  equality substitute fromInEpa;
  rewrite;
  equality substitute authenticFrom;
  rewrite;
  next;
  equality substitute fromLogged;
  rewrite;
  equality substitute authenticFrom;
  rewrite;
  next;

```

■

```

proof[RabOkayClPd$domainCheck]
  common to all goals

  prenex;
  rearrange;
  ensures that pdThis in domain of conAuthPurse

  use lRabOkayClPdInDomConAuthPurse;
  links abAuthPurse with conAuthPurse

  invoke AbstractBetween;
  equality substitute dom abAuthPurse;
  rearrange;
  rewrite;
  rearrange;
  two goals for name as pdThis.from, pdThis.to

  split name = pdThis.from;
  simplify;

```

■

proof[*lRabWillBeLostClPdInDomConAuthPurse*]
 rewrite;
 apply *inPower* to predicate *chosenLost* $\in \mathbb{P}$ *maybeLost*;
 instantiate *e* == *pdThis*;
 rearrange;
 rewrite;
 invoke *AuxWorld*;
 equality substitute *maybeLost*;
 rewrite;
 equality substitute *toInEpv*;
 rewrite;
 equality substitute *authenticTo*;
 rewrite;
 split *pdThis* $\in$ *fromInEpa*;
 simplify;
 cases;
 rearrange;
 equality substitute *fromInEpa*;
 rewrite;
 equality substitute *authenticFrom*;
 rewrite;
 next;
 equality substitute *fromLogged*;
 rewrite;
 equality substitute *authenticFrom*;
 rewrite;
 next;

■

proof[*RabWillBeLostClPd\$domainCheck*]
common to all goals

prenex;
rearrange;
ensures that pdThis in domain of conAuthPurse

use lRabWillBeLostClPdInDomConAuthPurse;
links abAuthPurse with conAuthPurse

invoke AbstractBetween;
 equality substitute *dom abAuthPurse*;
 rearrange;
 rewrite;
 rearrange;
two goals for name as pdThis.from, pdThis.to

 split *name* = *pdThis.from*;
 simplify;

■

```

proof[lRabHasBeenLostClPdInDomConAuthPurse]
  invoke AuxWorld;
  equality substitute definitelyLost;
  rewrite;
  equality substitute toLogged;
  rewrite;
  equality substitute authenticTo;
  rewrite;
  split pdThis ∈ fromInEpa;
  simplify;
  cases;
  rearrange;
  equality substitute fromInEpa;
  rewrite;
  equality substitute authenticFrom;
  rewrite;
  next;
  equality substitute fromLogged;
  rewrite;
  equality substitute authenticFrom;
  rewrite;
  next;

```

■

```

proof[RabHasBeenLostClPd$domainCheck]
  common to all goals

  prenex;
  rearrange;
  ensures that pdThis in domain of conAuthPurse

  use lRabHasBeenLostClPdInDomConAuthPurse;
  links abAuthPurse with conAuthPurse

  invoke AbstractBetween;
  equality substitute dom abAuthPurse;
  rearrange;
  rewrite;
  rearrange;
  two goals for name as pdThis.from, pdThis.to

  split name = pdThis.from;
  simplify;

```

■

```

proof[RabEndClPd$domainCheck]
  links abAuthPurse with conAuthPurse

  invoke AbstractBetween;
  rewrite;

```

■

J.2 Partition

See Section 10.2.4.

proof[*lBetweenWorldFromLoggedExpansion*]
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
simplify;
■

proof[*lBetweenWorldToLoggedExpansion*]
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
simplify;
■

proof[*lBetweenWorldAuthenticFromExpansion*]
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
simplify;
■

proof[*lBetweenWorldAuthenticToExpansion*]
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
simplify;
■

proof[*lBetweenWorldToInEpuExpansion*]
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
simplify;
■

proof[*lBetweenWorldToInEapayeeExpansion*]
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
simplify;
■

proof[*lBetweenWorldFromInEprExpansion*]
invoke BetweenWorld;
invoke BetweenWorldB1;
invoke AuxWorld;
simplify;
■

proof[*lBetweenWorldFromInEpaExpansion*]
 invoke BetweenWorld;
 invoke BetweenWorldB1;
 invoke AuxWorld;
 simplify;

■

proof[*lBetweenWorldAllLogsExpansion*]
 invoke BetweenWorld;
 invoke BetweenWorldB1;
 invoke AuxWorld;
 simplify;

■

proof[*lBetweenWorldDefinitelyLostExpansion*]
 invoke BetweenWorld;
 invoke BetweenWorldB1;
 invoke AuxWorld;
 simplify;

■

proof[*lBetweenWorldMaybeLostExpansion*]
 invoke BetweenWorld;
 invoke BetweenWorldB1;
 invoke AuxWorld;
 simplify;

■

proof[*lBetweenWorldArchiveExpansion*]
 invoke BetweenWorld;
 invoke BetweenWorldB1;
 invoke AuxWorld;
 invoke ConWorld;
 apply dLogbook;
 simplify;

■

proof[*lNotLoggedAndInC12*]
 Expand the equalities

apply extensionality;
 prove by rewrite;
 cases;

from case:

firstly, expand what we need


```

use lBetweenWorldFromLoggedExpansion;
use lBetweenWorldFromInEpaExpansion;
use lBetweenWorldFromInEprExpansion;
use lBetweenWorldAuthenticFromExpansion;
rearrange;
simplify;
  follow advice to use B7

invoke BetweenWorld;
invoke BetweenWorldB7;
instantiate pd == x;
  carefully avoid rearrangement in order

  to avoid BetweenWorld substitutions.

equality substitute fromInEpr;
equality substitute fromInEpa;
equality substitute authenticFrom;
rewrite;
rearrange;
rewrite;
  Finished: just need rearrangement

cases;
  epa case from B7

  rearrange;
  rewrite;
  rearrange;
  rewrite;
next;
  epr case from B7

  rearrange;
  rewrite;
next;
  to case: firstly, expand what we need

use lBetweenWorldToLoggedExpansion;
use lBetweenWorldToInEapayeeExpansion;
use lBetweenWorldToInEpvExpansion;
use lBetweenWorldAuthenticToExpansion;
rearrange;
simplify;
  follow advice to use B8

invoke BetweenWorld;
invoke BetweenWorldB8;
instantiate pd == x__0;
  carefully avoid rearrangement in order

  to avoid BetweenWorld substitutions.

equality substitute toInEapayee;
equality substitute toInEpv;
equality substitute authenticTo;
rewrite;
rearrange;
rewrite;
  Finished: just need rearrangement

```

```

    cases;
      eaTo case from B8

      rearrange;
      rewrite;
      rearrange;
      rewrite;
    next;
      epv case from B8

      rearrange;
      rewrite;
    next;
  ■

```

```

proof[lLostC13]
  use lBetweenWorldMaybeLostExpansion;
  use lBetweenWorldDefinitelyLostExpansion;
  rearrange;
  simplify;
  equality substitute definitelyLost;
  equality substitute maybeLost;
  rewrite;
  apply capSubsetLeft to expression (fromInEpa  $\cup$  fromLogged)  $\cap$ 
    (fromInEpa  $\cup$  fromLogged) ;
  rewrite;
  use lNotLoggedAndInC12;
  rearrange;
  rewrite;
  apply extensionality;
  prove by rewrite;
  instantiate x_3 == x;
  prove by rewrite;
  ■

```

```

proof[tRetrievePartition]
  use lRetrievePartitionWillBeLostEquiv;
  simplify;
  ■

```

J.2.1 Quantified forms

See Section 10.2.5.

```

proof[tRetrieveForAllRabClPd]
  invoke RabClPd ;
  prove by rewrite;
  ■

```

With the structured definition of names, it is now possible to proof this theorem, as we know there exists at least two different names so that a payment can be made.

```

proof[tRetrieveExistsRabClPd]
  invoke RabClPd;
  prove by rewrite;
  use lExistsPayDetails;
  prenex;
  instantiate pdThis == pd;
  with disabled (RabCl) prove by reduce;
  ■

```

J.3 Summary

Declarations	This chapter	Total
Domain checks	6	70
Proof scripts	20	223
Total proofs	26	293
Total explained	0	0

Table J.1: Summary of proofs

Appendix K

Chapter 11 proofs

See Chapter 11.

The PRG's textual proof was very helpful here!

```
proof[tBetwAbsInitSimAppl]  
  invoke Rab ;  
  prenex;  
  invoke RabCl ;  
  invoke AbInitState ;  
  simplify;
```

■

```
proof[tBetwAbsInputSimAppl]  
  invoke ;  
  split m? ∈ ran req;  
  rewrite;
```

■

```
proof[tBetwAbsInitInputs]  
  invoke BetwInitIn;  
  rewrite;  
  split m? ∈ ran req;  
  simplify;
```

■

K.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	3	226
Total proofs	3	296
Total explained	0	0

Table K.1: Summary of proofs

Appendix L

Chapter 12 proofs

L.1 Output proofs

See Section 12.2.

The proof of *tBetwAbsFinalOutput* follows directly from the PRG explanation of it.

```
proof[tBetwAbsFinalOutput]  
  invoke RabOut;  
  invoke AbFinOut;  
  prove by rewrite;  
  ■
```

L.2 State proofs

See Section 12.3.

Case *AbFinState*

```
proof[lBetwAbsFinalStateAbWorldCutAbFinStateCase]  
  invoke AbFinState;  
  rewrite;  
  ■
```

Case Rab

```

proof[rConPurseBalanceTypeOpen]
  apply inDom;
  prenex;
  use pairInFunction[NAME,  $\langle$  balance :  $\mathbb{Z}$ ; exLog :  $\mathbb{P}$ 
     $\langle$  from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :  $\mathbb{Z}$ ; value :  $\mathbb{Z}$ 
     $\rangle$ ; name : NAME; nextSeqNo :  $\mathbb{Z}$ ; pdAuth :  $\langle$ 
      from : NAME; fromSeqNo :  $\mathbb{Z}$ ; to : NAME; toSeqNo :  $\mathbb{Z}$ ; value :  $\mathbb{Z}$ 
     $\rangle$ ; status : STATUS  $\rangle$  ]
  [f := purse, x := name, y := y];
  apply ConPurse$inSet;
  prove by rewrite;
  invoke ( _  $\leftrightarrow$  _ );
  invoke ( _  $\leftrightarrow$  _ );
  invoke ( _  $\leftrightarrow$  _ );
  prove by rewrite;
  apply inPower;
  instantiate e == (name, y);
  apply tupleInCross2;
  apply ConPurse$member;
  prove by reduce;

```

■

```

proof[lBetweenWorldMaybeLostIsFinite]
  invoke BetweenWorld;
  invoke BetweenWorldB1;
  invoke AuxWorld;
  simplify;

```

■

```

proof[lBetwAbsFinalStateAbWorldCutRabCase]
  invoke Rab;
  invoke RabCl;
  invoke BetwFinState;
  invoke GlobalWorld;
  invoke AbstractBetween;
  prove by rewrite;
  use lBetweenWorldMaybeLostExpansion;
  use lBetweenWorldMaybeLostIsFinite;
  rearrange;
  simplify;
  instantiate chosenLost == maybeLost;
  prove by rewrite;
  instantiate name__0 == name;
  prove by rewrite;
  apply diffSuperset to expression
    toInEpv  $\cap$  (fromInEpa  $\cup$  fromLogged)
    \ toInEpv  $\cap$  (fromInEpa  $\cup$  fromLogged);
  rewrite;
  apply theIntegerElimination;
  rewrite;
  use rConPurseBalanceTypeOpen[purse := conAuthPurse, name := name];
  apply inNat;
  rearrange;
  rewrite;

```

■


```

proof[tBetwAbsFinalState]
  instantiate abAuthPurse == gAuthPurse;
  use lBetwAbsFinalStateAbWorldCutAbFinStateCase;
  use lBetwAbsFinalStateAbWorldCutRabCase;
  rearrange;
  simplify;
  ■

```

L.3 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	6	232
Total proofs	6	302
Total explained	0	0

Table L.1: Summary of proofs

Appendix M

Chapter 14 proofs

M.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table M.1: Summary of proofs

Appendix N

Chapter 15 proofs

N.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table N.1: Summary of proofs

Appendix O

Chapter 16 proofs

O.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table O.1: Summary of proofs

Appendix P

Chapter 17 proofs

P.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table P.1: Summary of proofs

Appendix Q

Chapter 18 proofs

Q.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table Q.1: Summary of proofs

Appendix R

Chapter 19 proofs

R.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table R.1: Summary of proofs

Appendix S

Chapter 20 proofs

S.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table S.1: Summary of proofs

Appendix T

Chapter 21 proofs

T.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table T.1: Summary of proofs

Appendix U

Chapter 22 proofs

U.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table U.1: Summary of proofs

Appendix V

Chapter 23 proofs

V.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table V.1: Summary of proofs

Appendix W

Chapter 24 proofs

W.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	232
Total proofs	0	302
Total explained	0	0

Table W.1: Summary of proofs

Appendix X

Chapter 27 proofs

X.1 Initialisation proof

See Section 27.1.

```
proof[tBetwConcInit]  
  invoke ConInitState;  
  prenex;  
  instantiate allLogs__0' == allLogs',  
    archive__0' == archive', authenticFrom__0' == authenticFrom',  
    authenticTo__0' == authenticTo', conAuthPurse__0' == conAuthPurse',  
    definitelyLost__0' == definitelyLost', ether__0' == ether',  
    fromInEpa__0' == fromInEpa', fromInEpr__0' == fromInEpr',  
    fromLogged__0' == fromLogged', maybeLost__0' == maybeLost',  
    toInEapayee__0' == toInEapayee', toInEpr__0' == toInEpr',  
    toLogged__0' == toLogged';  
  invoke Rbc;  
  simplify;
```

■

X.2 Finalisation proof

See Section 27.2.

```
proof[lCAuxWorldFinalEquiv]  
  invoke BetweenWorld;  
  invoke BetweenWorldB1;  
  invoke AuxWorldCh7;  
  invoke AuxWorld;  
  prove by rewrite;
```

■

```

proof[tBetwConcFinal]
  invoke BetwFinState;
  invoke ConFinState;
  invoke Rbc;
  prenex;
  equality substitute conAuthPurse;
  instantiate name__0 == name;
  simplify;
  rearrange;
  simplify;
  use lCAuxWorldFinalEquiv;
  rearrange;
  simplify;
  ■

```

X.3 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	3	235
Total proofs	3	305
Total explained	0	0

Table X.1: Summary of proofs

Appendix Y

Chapter 28 proofs

Y.1 Correctness of *CIgnore*

See Section 28.2.

THIS IS BRUTAL FORCE, DEFINITELY THERE MUST BE A BETTER WAY TO PROVE THIS.

```
proof[tBetwConcCIgnoreCorrect]  
  invoke Ignore;  
  invoke CIgnore;  
  invoke  $\Xi$  BetweenWorld;  
  invoke  $\Xi$  ConWorldCh7;  
  invoke Rbc;  
  apply ConWorldCh7 $\$$ thetasEqual;  
  rearrange;  
  equality substitute archiveCh7';  
  equality substitute conAuthPurseCh7';  
  equality substitute etherCh7';  
  simplify;  
  instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,  
    authenticTo' == authenticTo, definitelyLost' == definitelyLost,  
    ether' == ether, fromInEpa' == fromInEpa,  
    fromInEpr' == fromInEpr, fromLogged' == fromLogged,  
    maybeLost' == maybeLost, toInEapayee' == toInEapayee,  
    toInEpv' == toInEpv, toLogged' == toLogged;  
  rewrite;
```

■

```

proof[lAuxWorldCAbortType]
  invoke BetwCAbortCorrectSig;
  invoke PhiBOpBranchesSig;
  invoke PhiBOpSig;
  invoke BetweenOpSig;
  invoke BetweenWorld;
  invoke BetweenWorldB1;
  invoke AuxWorld;
  rewrite;
  apply cupSubsetLeft to expression { name? }  $\cup$  dom conAuthPurse;
  rewrite;
  equality substitute authenticTo;
  equality substitute authenticFrom;
  cases;
  invoke ConWorld;
  rewrite;
  prenex;
  instantiate n__0 == n;
  apply applyOverride1;
  rewrite;
  rearrange;
  rewrite;
  invoke AbortPurseOkay;
  invoke LogIfNecessary;
  invoke  $\Xi$  ConPurseAbort;
  rewrite;
  equality substitute balance';
  equality substitute name';
  use
    lPromotedConAuthNewPurseInjectivity[status' := eaFrom,
      exLog'
      := exLog
       $\cup$  (if status = epv
           $\vee$  status = epa
          then { pdAuth }
          else {}),
      balance' := balance, name' := name];
  rearrange;
  invoke ConWorld;
  simplify;
  invoke  $\Delta$  ConPurse;
  invoke ConPurse;
  prenex;
  rearrange;
  rewrite;
  instantiate nld__0 == nld;
  rewrite;
  split
    status = epv

 $\vee$  status = epa;
  simplify;
  rearrange;
  cases;
  equality substitute exLog';
  instantiate pd__0 == pd;
  split pd  $\in$  exLog;
  simplify;
  cases;
  next;
  next;
  next;

```



```

proof[lBetweenWorldCAbortType]
  use lAuxWorldCAbortType;
  rearrange;
  simplify;
  invoke BetwCAbortCorrectSig;
  invoke PhiBOpBranchesSig;
  invoke PhiBOpSig;
  invoke BetweenOpSig;
  invoke BetweenWorld;
  invoke AbortPurseOkay;
  cases;
    invoke BetweenWorldB1;
    rewrite;
  next;
    invoke BetweenWorldB2;
    invoke BetweenWorldB1;
    rewrite;
    prenex;
    instantiate pd__1 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
  next;
    invoke BetweenWorldB3;
    invoke BetweenWorldNEW1;
    invoke BetweenWorldNEW2;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
    cases;
      instantiate pd__0 == pd;
      rearrange;
      rewrite;
    next;
      instantiate pd__1 == pd;
      rearrange;
      rewrite;
  next;
    invoke BetweenWorldB4;
    invoke BetweenWorldNEW3;
    invoke BetweenWorldNEW4;
    rewrite;
    prenex;
    instantiate pd__2 == pd;
    apply applyOverride;
    prove by rewrite;
    instantiate pd__0 == pd;
    instantiate pd__1 == pd;
    use lAuxWorldAuthenticFromExpansion;
    use lAuxWorldAuthenticToExpansion;
    prove by rewrite;
  next;
    invoke BetweenWorldB5;
    rewrite;
    prenex;
    instantiate pd__0 == pd;
    apply applyOverride;

```

proof[*tBetwConcCAbortCorrect*]

Expand CAbort

invoke CAbort;
invoke Abort;
prenex;

Split available cases

split CIgnore;
simplify;
cases;

Reuse CIgnore case

use tBetwConcCIgnoreCorrect;
prove by rewrite;
instantiate allLogs__0' == allLogs', authenticFrom__0' == authenticFrom',
authenticTo__0' == authenticTo', definitelyLost__0' == definitelyLost',
ether__0' == ether', fromInEpa__0' == fromInEpa',
fromInEpr__0' == fromInEpr', fromLogged__0' == fromLogged',
maybeLost__0' == maybeLost', toInEapayee__0' == toInEapayee',
toInEpv__0' == toInEpv', toLogged__0' == toLogged',
archive__0' == archive', conAuthPurse__0' == conAuthPurse';
simplify;
next;

CAbort case

Clear names

simplify;

Call the lemma for 28.3.1, choosing BetweenWorld

use lBetweenWorldCAbortType;
invoke BetwCAbortCorrectSig;
invoke PhiBOPBranchesSig;
invoke PhiBOPSig;
invoke BetweenOpSig;

Next, expand/simplify PhiBOP and PhiCOP

invoke PhiBOP;
invoke PhiCOP;
invoke Δ BetweenWorld;
rewrite;

Next, let's get rid of the simple Rbc' sub-goal

invoke Rbc;
equality substitute;
rearrange;
rewrite;

Expand AbortPurseOkay, but not too much (yet)

invoke AbortPurseOkay;
invoke ∃ ConPurseAbort;

Don't expand Delta ConPurse yet to know better which instantiations to choose

rewrite;

balance, name are Xi from ConPurseAbort, and status' is given

instantiate $balance_0 == balance, name_0 == name,$
 $exLog_0 == exLog, exLog_0' == exLog',$
 $pdAuth_0 == pdAuth, pdAuth_0' == pdAuth',$
 $status_0 == status, nextSeqNo_0 == nextSeqNo,$
 $nextSeqNo_0' == nextSeqNo';$
Avoid expanding LogIfNecessary and ConPurseAbort for effectiveness of next rewrite

rewrite;

Next, let's get rid of m ? in ether subgoal

apply inPower to predicate $etherCh7 \in \mathbb{P} \text{ ether};$
instantiate $e == m?;$
instantiate $ether' == ether \cup \{ \perp \};$
rearrange;
rewrite;

Now we have (clear) BetweenWorld' instantiations (28.3.1)

left to finish the CAbort case and the entire proof

rearrange;
instantiate $allLogs' == allLogs, authenticFrom' == authenticFrom,$
 $authenticTo' == authenticTo, definitelyLost' == definitelyLost,$
 $fromInEpa' == fromInEpa, fromInEpr' == fromInEpr, fromLogged' == fromLogged,$
 $maybeLost' == maybeLost, toInEapayee' == toInEapayee, toInEpv' == toInEpv,$
 $toLogged' == toLogged;$
rewrite;

Rbc equivalences

equality substitute $archiveCh7';$
equality substitute $archiveCh7;$
equality substitute $conAuthPurseCh7;$
rearrange;

Finally, let's get the sub-goal on the ether (28.3.3)

apply inPower to predicate $etherCh7' \in \mathbb{P} (ether \cup \{ \perp \});$
prenex;
apply inPower to predicate $etherCh7' \in \mathbb{P} (etherCh7 \cup \{ \perp \});$
instantiate $e_0 == e;$
rearrange;
rewrite;
instantiate $e_1 == e;$
rearrange;
simplify;
next;

■

proof[$tBetwConcCReqCorrect$]

Expand CReq

invoke $CReq;$
invoke $Req;$
prenex;

Split available cases

split $CIgnore;$
simplify;
cases;

Reuse CIgnore case

```

use tBetwConcCIgnoreCorrect;
prove by rewrite;
instantiate allLogs__0' == allLogs', authenticFrom__0' == authenticFrom',
  authenticTo__0' == authenticTo', definitelyLost__0' == definitelyLost',
  ether__0' == ether', fromInEpa__0' == fromInEpa',
  fromInEpr__0' == fromInEpr', fromLogged__0' == fromLogged',
  maybeLost__0' == maybeLost', toInEapayee__0' == toInEapayee',
  toInEpv__0' == toInEpv', toLogged__0' == toLogged',
  archive__0' == archive', conAuthPurse__0' == conAuthPurse';
simplify;
next;
CReq case

  Clear names

simplify;
  Call the lemma for 28.3.1, choosing BetweenWorld

use lBetweenWorldCReqType;
invoke BetwCReqCorrectSig;
invoke PhiBOpBranchesSig;
invoke PhiBOpSig;
invoke BetweenOpSig;
  Next, expand/simplify PhiBOp and PhiCOP

invoke PhiBOp;
invoke PhiCOP;
invoke Δ BetweenWorld;
rewrite;
  Next, let's get rid of the simple Rbc' sub-goal

invoke Rbc;
equality substitute;
rearrange;
rewrite;
  Because of rearranging oddity, we need to instantiate Delta

  ConPurse earlier on before expanding Xi ConPurseReq

instantiate balance__0 == balance, balance__0' == balance',
  name__0 == name, name__0' == name, exLog__0 == exLog,
  exLog__0' == exLog, pdAuth__0 == pdAuth, pdAuth__0' == pdAuth,
  status__0 == status, status__0' == status',
  nextSeqNo__0 == nextSeqNo, nextSeqNo__0' == nextSeqNo';
  Expand ReqPurseOkay, but not too much (yet)

invoke ReqPurseOkay;
invoke ∃ ConPurseReq;
rearrange;
  Don't expand Delta ConPurse yet to know better which instantiations to choose

rewrite;
  Next, let's get rid of m? in ether subgoal

```

apply inPower to predicate etherCh7 $\in \mathbb{P}$ ether;
instantiate e == m?;
instantiate ether' == ether $\cup$ { m! };
rearrange;
rewrite;
rearrange;

Now we have (clear) BetweenWorld' instantiations (28.3.1)

left to finish the CAbort case and the entire proof

instantiate allLogs' == allLogs, authenticFrom' == authenticFrom,
authenticTo' == authenticTo, definitelyLost' == definitelyLost,
fromInEpa' == fromInEpa, fromInEpr' == fromInEpr, fromLogged' == fromLogged,
maybeLost' == maybeLost, toInEapayee' == toInEapayee, toInEprv' == toInEprv,
toLogged' == toLogged;
rewrite;

Rbc equivalences

equality substitute archiveCh7';
equality substitute archiveCh7;
equality substitute conAuthPurseCh7;
rearrange;

Finally, let's get the sub-goal on the ether (28.3.3)

apply inPower to predicate etherCh7' $\in \mathbb{P}$ (ether $\cup$ { m! });
prenex;
apply inPower to predicate etherCh7' $\in \mathbb{P}$ (etherCh7 $\cup$ { m! });
instantiate e__0 == e;
rearrange;
rewrite;
instantiate e__1 == e;
rearrange;
simplify;
next;

■

Y.2 Branching

Y.3 Correctness of a branch of the operation

See Section 28.3.

proof[*tBetwConcCStartFromCorrect*]

■

proof[*tBetwConcCStartToCorrect*]

■

proof[*tBetwConcCReqCorrect*]

■

proof[*tBetwConcCValCorrect*]

■

proof[*tBetwConcCAckCorrect*]
■

proof[*tBetwConcCReadExceptionLogCorrect*]
■

proof[*tBetwConcCClearExceptionLogCorrect*]
■

proof[*tBetwConcCAuthoriseExLogClearCorrect*]
■

proof[*tBetwConcCArchiveCorrect*]
■

Appendix Z

Chapter 29 proofs

Z.1 Summary

Declarations	This chapter	Total
Domain checks	0	70
Proof scripts	0	249
Total proofs	0	319
Total explained	0	0

Table Z.1: Summary of proofs